

Macacário: Maratona lập trình v3.3

Viện Công nghệ Hàng không
Lucas França de Oliveira (Mexicano T-18)
lucas.fra.oli18@gmail.com

9 tháng 9 năm 2019

Nguồn gốc: Macacário Maratona de Programação v3.3

Contents

1	Giới thiệu	7
1.1	Các lỗi thiên niên kỷ	7
1.2	1010 điều răn	8
1.3	Mèo bản nhưng hợp lệ	9
1.4	Giới hạn biểu diễn dữ liệu	9
1.5	Số lượng số nguyên tố từ 1 đến 10^n	9
1.6	Tam giác Pascal	9
1.7	Giai thừa	10
1.8	Bảng ASCII	10
1.9	Số nguyên tố tới 10.000	11
2	C++ và thư viện STD	13
2.1	Macros	13
2.2	Trình biên dịch GNU	13
2.3	I/O nhanh	13
2.4	Kiểm tra overflow	14
2.5	C++11	14
2.6	Complex	14
2.7	Pair	14
2.8	List	15
2.9	Vector	15
2.10	Deque	15
2.11	Queue	15
2.12	Stack	15
2.13	Map	15
2.14	Set	15

2.15	Ordered set	15
2.16	Unordered set và map	15
2.17	Priority Queue	15
2.18	Bitset	16
2.19	String	16
2.20	Algorithm và numeric	16
2.21	Algorithm: không sửa đổi	16
2.22	Algorithm: sửa đổi	16
2.23	Algorithm: phân hoạch	16
2.24	Algorithm: sắp xếp	16
2.25	Algorithm: tìm kiếm nhị phân	16
2.26	Algorithm: heap	16
2.27	Algorithm: cực đại và cực tiểu	16
2.28	Algorithm: hoán vị	16
2.29	Numeric: tích lũy	17
2.30	Functional	17
3	Cấu trúc dữ liệu	18
3.1	Max-Queue	18
3.2	Union-Find	21
3.3	Binary Indexed Tree / Fenwick Tree	22
3.4	Binary Indexed Tree / Fenwick Tree với cập nhật và truy vấn đoạn	22
3.5	2D Binary Indexed Tree / Fenwick Tree	24
3.6	Segment Tree	25
3.7	Segment Tree với Lazy Propagation	26
3.8	2D Segment Tree	28
3.9	Persistent Segment Tree	30
3.10	Li Chao Segment Tree	33
3.11	Split-Merge Segment Tree	34
3.12	Segment Tree Beats	38
3.13	Merge Sort Tree	41
3.14	Fractional Cascading Merge Sort Tree	42
3.15	Sparse Table	44
3.16	Pre-Suf Table	44
3.17	AVL Tree	45
3.18	Treap / Cartesian Tree	48
3.19	Treap / Cartesian Tree ngẫu nhiên	50
3.20	Persistent Randomized BST	53
3.21	Deque đệ quy thuần hàm	56
3.22	Splay Tree	60
3.23	Link Cut Tree	62

3.24	Link Cut Tree vô hướng	68
3.25	Wavelet Tree	72
3.26	Heavy-Light Decomposition	76
3.27	Centroid Decomposition	79
3.28	Interval Tree	81
3.29	Xor Trie	83
3.30	Convex Hull Trick	85
3.31	Convex Hull Trick động và tích vô hướng lớn nhất	86
3.32	Van Emde Boas Tree	88
3.33	Lowest Common Ancestor (LCA) và truy vấn đường đi trên cây	91
4	Paradigms	95
4.1	Merge Sort	95
4.2	Quick Sort	95
4.3	Longest Increasing Subsequence (LIS)	96
4.4	Bài toán cặp gần nhất	97
4.5	Tối ưu hai con trỏ	98
4.6	Tối ưu Convex Hull Trick	99
4.7	Tối ưu Slope Trick	100
4.8	Tối ưu chia để trị	101
4.9	Tối ưu Knuth	102
4.10	Tối ưu Lagrange	103
5	Đồ thị	105
5.1	DFS Spanning Tree	105
5.2	Điểm khớp và cầu	105
5.3	Sắp xếp tô pô	106
5.4	Thành phần liên thông mạnh: thuật toán Kosaraju	106
5.5	2-SAT: thuật toán Tarjan	107
5.6	Đường đi ngắn nhất: thuật toán Dijkstra	110
5.7	Đường đi ngắn nhất: thuật toán Floyd-Warshall	111
5.8	Đường đi ngắn nhất: thuật toán Bellman-Ford	111
5.9	Đường đi ngắn nhất thứ k	112
5.10	Đường đi ngắn nhất: Shortest Path Faster Algorithm (SPFA)	113
5.11	Cây khung nhỏ nhất: thuật toán Kruskal	114
5.12	Cây khung nhỏ nhất: thuật toán Prim	115
5.13	Luồng cực đại: thuật toán Edmonds-Karp	116
5.14	Luồng cực đại: thuật toán Dinic	117
5.15	Luồng cực đại: thuật toán Push-Relabel	119
5.16	Cắt toàn cục nhỏ nhất: thuật toán Stoer-Wagner	121
5.17	Min Cost Max Flow	121

5.18	Ghép cặp chi phí nhỏ nhất: thuật toán Hungary	124
5.19	Ghép cặp cực đại: thuật toán Kuhn	125
5.20	Ghép cặp cực đại: thuật toán Hopcroft-Karp	126
5.21	Minimum Vertex Cover	127
5.22	Maximum Matching: thuật toán Blossom	129
5.23	Gomory-Hu Tree: thuật toán Gusfield	132
5.24	Euler Tour: thuật toán Fleury	134
5.25	Dominator Tree	136
5.26	Đồ thị đáng chú ý	139
5.27	Định lý Erdos-Gallai	139
5.28	Định lý và công thức đồ thị	140
6	Toán học	141
6.1	Số học modular	141
6.2	Định lý phần dư Trung Hoa tổng quát	142
6.3	Số nguyên tố	143
6.4	Công thức Legendre	144
6.5	Tổng GCD	145
6.6	Crivo tuyến tính và hàm nhân tính	147
6.7	Nghịch đảo Mobius	148
6.8	Số Catalan	148
6.9	Số Stirling loại một	148
6.10	Số Stirling loại hai	150
6.11	Đẳng thức tổng nhị thức	150
6.12	Bổ đề Burnside và định lý đếm Polya	151
6.13	Kiểm tra nguyên tố Miller-Rabin	151
6.14	Thuật toán Pollard-Rho	152
6.15	Baby-Step Giant-Step cho logarit rời rạc	153
6.16	Trò chơi Nim và định lý Sprague-Grundy	154
6.17	Bộ ba Pythagoras	155
6.18	Ma trận	155
6.19	Lũy thừa ma trận và Fibonacci	156
6.20	Hệ tuyến tính: định thức và khử Gauss	158
6.21	Nhân ma trận thưa	161
6.22	Phương pháp Gauss-Seidel	161
6.23	XOR-SAT	163
6.24	Fast Fourier Transform (FFT)	165
6.25	Number Theoretic Transform (NTT)	167
6.26	Tích chập vòng	169
6.27	Số phức	170
6.28	Chia đa thức	170

6.29	Đánh giá tại nhiều điểm	174
6.30	Nội suy đa thức	174
6.31	Fast Walsh-Hadamard Transform	174
6.32	Tích chập với CRT	176
6.33	Tích chập với phân rã căn	177
6.34	Tích phân bằng quy tắc Simpson	177
6.35	Mã Gray	178
6.36	BigInteger trong Java	178
6.37	Bignum trong C++	178
6.38	Sự phá sản của con bạc	182
6.39	Định lý và công thức	182
7	Xử lý chuỗi	183
7.1	Hàm tiền tố: thuật toán Knuth-Morris-Pratt (KMP)	183
7.2	Rabin-Karp	184
7.3	Repetend: chu kỳ nhỏ nhất của chuỗi	185
7.4	Suffix Array: thuật toán Karp-Miller-Rosenberg (KMR)	186
7.5	Hàm Z và thuật toán Z	189
7.6	Thuật toán Manacher	190
7.7	Aho-Corasick	191
7.8	Automaton hậu tố	192
7.9	Inaccurate String Matching với FFT	194
7.10	Automaton KMP	197
7.11	Palindromic Tree	200
8	Hình học tính toán	203
8.1	Điểm 2D và đoạn thẳng	203
8.2	Đường tròn 2D	206
8.3	Great Circle	209
8.4	Tam giác 2D	210
8.5	Đa giác 2D	211
8.6	Convex Hull	215
8.7	Điểm trong đa giác lồi	218
8.8	Tổng Minkowski	220
8.9	Comparator cực	223
8.10	Tam giác hóa Delaunay	224
8.11	Giao của đa giác	230
8.12	Minimum Enclosing Circle	230
8.13	Giao nửa mặt phẳng	231
8.14	Điểm 3D	235
8.15	Tam giác 3D	236

8.16 Đường thẳng 3D	237
8.17 Hình học giải tích	239
8.18 Tọa độ cực, trụ và cầu	239
8.19 Giải tích vector 2D	239
8.20 Giải tích vector 3D	239
8.21 Vấn đề độ chính xác, tổng ổn định và công thức Bhaskara	239
9 Linh tinh	242
9.1 Thuật toán Mo	242
9.2 Lịch Gregory	242
9.3 Simplex	244
9.4 Duyệt polyomino	247
9.5 Ma phương bậc lẻ	248
9.6 Chu trình trong dãy: thuật toán Floyd	249
9.7 Biểu thức ngoặc sang Ba Lan	249
9.8 Bài toán Josephus	250
9.9 Bài toán histogram	250
9.10 Bài toán hôn nhân ổn định	251
9.11 Mã Huffman	252
9.12 Bài toán quân mã	253
9.13 Giao matroid	253

1 Giới thiệu

1.1 Các lỗi thiên niên kỷ

Lỗi lý thuyết

- Không đọc đề bài một cách bình tĩnh.
- Vội vàng giả định một tính chất nào đó của lời giải.
- Không đọc lại giới hạn trước khi nộp.
- Khi sửa một thuật toán, chú ý mọi chi tiết của cấu trúc thuật toán, phần nào cần hoặc không cần thay đổi.
- Bài có thể là NP, được ngụy trang hoặc không nêu rõ giới hạn; lúc đó lời giải sẽ khó thật sự, không phải lúc cổ đoạt Nobel.

Lỗi về giá trị cực đại của biến

- Tính cẩn thận để xem vô cực có thật sự đủ vô cực không.
- Kiểm tra các phép toán với vô cực có tràn 31 bit không.
- Cẩn thận phép nhân int tràn 32 bit, ví dụ kiểm tra dấu bằng $a*b > 0$.

Lỗi trường hợp biên

- Đã thử $n=0$, $n=1$, $n=MAXN$ chưa? Nhiều bài cần xử lý riêng.
- Nghĩ qua mọi trường hợp có thể là biên hoặc cô lập.
- Trường hợp biên có thể phá cả việc dựng cấu trúc, ví dụ danh sách kề.
- Không quên self-loop hoặc đa cạnh trong đồ thị.
- Bài đường đi Euler cần kiểm tra đồ thị liên thông.

Lỗi lơ đãng khi cài đặt

- Copy/paste sai mã, rất thường gặp.
- Đặt phép gán trong if, ví dụ if ($a = 0$).
- Quên khởi tạo biến.
- Nhầm break và continue.
- Không khởi tạo lại mỗi test: mảng đã xóa chưa, biến cộng dồn đã đưa về đúng giá trị chưa?
- Output đã đúng định dạng chưa?
- Mảng khai báo đủ lớn chưa?
- Cẩn thận modulo số âm: dùng $(x \% n + n) \% n$.

Lỗi cài đặt khác

- Khai báo biến global và local cùng tên.
- Khai báo sai kiểu biến, ví dụ int thay vì double hoặc char.
- Không dùng tên biến max và min.
- Nhớ rằng size() là unsigned.
- Nhớ 1 là int: long long a = 1 « 40 không đúng, nên dùng 1LL « 40.

Lỗi giới hạn

- Độ phức tạp thời gian và bộ nhớ là bao nhiêu? 10^8 là mốc tham khảo cho thời gian; cũng cần kiểm tra bộ nhớ.
- Có thể giảm hằng số bằng thuật toán tốt hơn hoặc phép toán nhanh hơn không?
- Bài có phải trường hợp riêng cần tối ưu thay vì dùng thẳng thư viện không?

Lỗi với số thực

- Tránh float/double nếu không cần hoặc không đơn giản hơn.
- Luôn dùng double, không dùng float, trừ khi đề yêu cầu rõ.
- So sánh bằng với sai số, có thể cả tuyệt đối và tương đối.
- Tránh trừ hai số gần bằng nhau.

Lỗi khác

- Tránh cấp phát động nếu không cần.
- Không dùng STL khi không cần thiết, nhưng dùng khi nó làm lời giải rõ hoặc cần để đạt độ phức tạp.

1.2 1010 điều răn

Theo PUC-RJ.

0. Không chia cho zero.
1. Không cấp phát động.
2. So sánh số thực bằng EPS.
3. Kiểm tra đồ thị có thể không liên thông.
4. Kiểm tra cạnh có thể có trọng số âm.
5. Kiểm tra có thể có nhiều cạnh nối cùng hai đỉnh.
6. Kiểm tra mọi chỉ số của quy hoạch động.
7. Giảm branching factor của DFS.
8. Cắt tỉa mọi thứ có thể trong DFS.
9. Cẩn thận điểm trùng và điểm thẳng hàng.

1.3 Mẹo bản nhưng hợp lệ

- Phương pháp Steve Halim: nếu toàn bộ output khả dĩ vừa trong mã nguồn, brute force trên máy vài phút rồi ghi thẳng đáp án vào code để nộp. Kiểm tra giới hạn kích thước submission.
- Giai thừa tới 10^9 : chạy chương trình trên máy để brute force giai thừa, in mỗi 10^3 hoặc 10^6 bước, dán output vào code và dùng giá trị tiền xử lý.
- Bài có hằng số: nếu một giá trị hữu ích không phụ thuộc input nhưng bạn không biết, brute force trên máy rồi dán vào code.
- Debug bằng assert: có thể đặt assert trong code nộp để biến WA thành RTE. Chỉ dùng khi thật sự bí vì tốn submission.

1.4 Giới hạn biểu diễn dữ liệu

Kiểu	scanf	bit	nhỏ nhất	lớn nhất
char	%c	8	0	255
signed char	%hhd	8	-128	127
unsigned char	%hhu	8	0	255
short	%hd	16	-32.768	32.767
unsigned short	%hu	16	0	65.535
int	%d	32	-2×10^9	2×10^9
unsigned int	%u	32	0	4×10^9
long long	%lld	64	-9×10^{18}	9×10^{18}
unsigned long long	%llu	64	0	18×10^{18}

Kiểu	scanf	số mũ	chữ số thập phân chính xác
float	%f	38	6
double	%lf	308	15
long double	%Lf	19.728	18

1.5 Số lượng số nguyên tố từ 1 đến 10^n

Luôn đúng rằng $n / \ln(n) < \pi(n) < 1.26n / \ln(n)$.

$\pi(10^1) = 4$	$\pi(10^2) = 25$	$\pi(10^3) = 168$
$\pi(10^4) = 1.229$	$\pi(10^5) = 9.592$	$\pi(10^6) = 78.498$
$\pi(10^7) = 664.579$	$\pi(10^8) = 5.761.455$	$\pi(10^9) = 50.847.534$

1.6 Tam giác Pascal

n \ p	0	1	2	3	4	5	6	7	8	9	10
0	1										
1	1	1									
2	1	2	1								
3	1	3	3	1							
4	1	4	6	4	1						
5	1	5	10	10	5	1					
6	1	6	15	20	15	6	1				
7	1	7	21	35	35	21	7	1			
8	1	8	28	56	70	56	28	8	1		
9	1	9	36	84	126	126	84	36	9	1	

10	1	10	45	120	210	252	210	120	45	10	1
----	---	----	----	-----	-----	-----	-----	-----	----	----	---

C(33,16)	1.166.803.110	giới hạn int
C(34,17)	2.333.606.220	giới hạn unsigned int
C(66,33)	7.219.428.434.016.265.740	giới hạn long long
C(67,33)	14.226.520.737.620.288.370	giới hạn unsigned long long

1.7 Giai thừa

Giai thừa tới 20 cùng các mốc giới hạn kiểu.

0!	1	
1!	1	
2!	2	
3!	6	
4!	24	
5!	120	
6!	720	
7!	5.040	
8!	40.320	
9!	362.880	
10!	3.628.800	
11!	39.916.800	
12!	479.001.600	giới hạn unsigned int
13!	6.227.020.800	
14!	87.178.291.200	
15!	1.307.674.368.000	
16!	20.922.789.888.000	
17!	355.687.428.096.000	
18!	6.402.373.705.728.000	
19!	121.645.100.408.832.000	
20!	2.432.902.008.176.640.000	giới hạn unsigned long long

1.8 Bảng ASCII

0 0x00 NUL	1 0x01 SOH	2 0x02 STX	3 0x03 ETX
4 0x04 EOT	5 0x05 ENQ	6 0x06 ACK	7 0x07 BEL
8 0x08 BS	9 0x09 TAB	10 0x0A LF	11 0x0B VT
12 0x0C FF	13 0x0D CR	14 0x0E SO	15 0x0F SI
16 0x10 DLE	17 0x11 DC1	18 0x12 DC2	19 0x13 DC3
20 0x14 DC4	21 0x15 NAK	22 0x16 SYN	23 0x17 ETB
24 0x18 CAN	25 0x19 EM	26 0x1A SUB	27 0x1B ESC
28 0x1C FS	29 0x1D GS	30 0x1E RS	31 0x1F US
32 0x20	33 0x21 !	34 0x22 "	35 0x23 #
36 0x24 \$	37 0x25 %	38 0x26 &	39 0x27 '
40 0x28 (	41 0x29)	42 0x2A *	43 0x2B +
44 0x2C ,	45 0x2D -	46 0x2E .	47 0x2F /
48 0x30 0	49 0x31 1	50 0x32 2	51 0x33 3
52 0x34 4	53 0x35 5	54 0x36 6	55 0x37 7
56 0x38 8	57 0x39 9	58 0x3A :	59 0x3B ;
60 0x3C <	61 0x3D =	62 0x3E >	63 0x3F ?
64 0x40 @	65 0x41 A	66 0x42 B	67 0x43 C
68 0x44 D	69 0x45 E	70 0x46 F	71 0x47 G
72 0x48 H	73 0x49 I	74 0x4A J	75 0x4B K
76 0x4C L	77 0x4D M	78 0x4E N	79 0x4F O
80 0x50 P	81 0x51 Q	82 0x52 R	83 0x53 S
84 0x54 T	85 0x55 U	86 0x56 V	87 0x57 W
88 0x58 X	89 0x59 Y	90 0x5A Z	91 0x5B [
92 0x5C \	93 0x5D]	94 0x5E ^	95 0x5F _
96 0x60 `	97 0x61 a	98 0x62 b	99 0x63 c
100 0x64 d	101 0x65 e	102 0x66 f	103 0x67 g
104 0x68 h	105 0x69 i	106 0x6A j	107 0x6B k
108 0x6C l	109 0x6D m	110 0x6E n	111 0x6F o
112 0x70 p	113 0x71 q	114 0x72 r	115 0x73 s
116 0x74 t	117 0x75 u	118 0x76 v	119 0x77 w

120 0x78	x	121 0x79	y	122 0x7A	z	123 0x7B	{
124 0x7C		125 0x7D	}	126 0x7E	~	127 0x7F	DEL

1.9 Số nguyên tố tới 10.000

Có 1229 số nguyên tố tới 10.000.

2	3	5	7	11	13	17	19	23	29	31	37	41	43	47
53	59	61	67	71	73	79	83	89	97	101	103	107	109	113
127	131	137	139	149	151	157	163	167	173	179	181	191	193	197
199	211	223	227	229	233	239	241	251	257	263	269	271	277	281
283	293	307	311	313	317	331	337	347	349	353	359	367	373	379
383	389	397	401	409	419	421	431	433	439	443	449	457	461	463
467	479	487	491	499	503	509	521	523	541	547	557	563	569	571
577	587	593	599	601	607	613	617	619	631	641	643	647	653	659
661	673	677	683	691	701	709	719	727	733	739	743	751	757	761
769	773	787	797	809	811	821	823	827	829	839	853	857	859	863
877	881	883	887	907	911	919	929	937	941	947	953	967	971	977
983	991	997	1009	1013	1019	1021	1031	1033	1039	1049	1051	1061	1063	1069
1087	1091	1093	1097	1103	1109	1117	1123	1129	1151	1153	1163	1171	1181	1187
1193	1201	1213	1217	1223	1229	1231	1237	1249	1259	1277	1279	1283	1289	1291
1297	1301	1303	1307	1319	1321	1327	1361	1367	1373	1381	1399	1409	1423	1427
1429	1433	1439	1447	1451	1453	1459	1471	1481	1483	1487	1489	1493	1499	1511
1523	1531	1543	1549	1553	1559	1567	1571	1579	1583	1597	1601	1607	1609	1613
1619	1621	1627	1637	1657	1663	1667	1669	1693	1697	1699	1709	1721	1723	1733
1741	1747	1753	1759	1777	1783	1787	1789	1801	1811	1823	1831	1847	1861	1867
1871	1873	1877	1879	1889	1901	1907	1913	1931	1933	1949	1951	1973	1979	1987
1993	1997	1999	2003	2011	2017	2027	2029	2039	2053	2063	2069	2081	2083	2087
2089	2099	2111	2113	2129	2131	2137	2141	2143	2153	2161	2179	2203	2207	2213
2221	2237	2239	2243	2251	2267	2269	2273	2281	2287	2293	2297	2309	2311	2333
2339	2341	2347	2351	2357	2371	2377	2381	2383	2389	2393	2399	2411	2417	2423
2437	2441	2447	2459	2467	2473	2477	2503	2521	2531	2539	2543	2549	2551	2557
2579	2591	2593	2609	2617	2621	2633	2647	2657	2659	2663	2671	2677	2683	2687
2689	2693	2699	2707	2711	2713	2719	2729	2731	2741	2749	2753	2767	2777	2789
2791	2797	2801	2803	2819	2833	2837	2843	2851	2857	2861	2879	2887	2897	2903
2909	2917	2927	2939	2953	2957	2963	2969	2971	2999	3001	3011	3019	3023	3037
3041	3049	3061	3067	3079	3083	3089	3109	3119	3121	3137	3163	3167	3169	3181
3187	3191	3203	3209	3217	3221	3229	3251	3253	3257	3259	3271	3299	3301	3307
3313	3319	3323	3329	3331	3343	3347	3359	3361	3371	3373	3389	3391	3407	3413
3433	3449	3457	3461	3463	3467	3469	3491	3499	3511	3517	3527	3529	3533	3539
3541	3547	3557	3559	3571	3581	3583	3593	3607	3613	3617	3623	3631	3637	3643
3659	3671	3673	3677	3691	3697	3701	3709	3719	3727	3733	3739	3761	3767	3769
3779	3793	3797	3803	3821	3823	3833	3847	3851	3853	3863	3877	3881	3889	3907
3911	3917	3919	3923	3929	3931	3943	3947	3967	3989	4001	4003	4007	4013	4019
4021	4027	4049	4051	4057	4073	4079	4091	4093	4099	4111	4127	4129	4133	4139
4153	4157	4159	4177	4201	4211	4217	4219	4229	4231	4241	4243	4253	4259	4261
4271	4273	4283	4289	4297	4327	4337	4339	4349	4357	4363	4373	4391	4397	4409
4421	4423	4441	4447	4451	4457	4463	4481	4483	4493	4507	4513	4517	4519	4523
4547	4549	4561	4567	4583	4591	4597	4603	4621	4637	4639	4643	4649	4651	4657
4663	4673	4679	4691	4703	4721	4723	4729	4733	4751	4759	4783	4787	4789	4793
4799	4801	4813	4817	4831	4861	4871	4877	4889	4903	4909	4919	4931	4933	4937
4943	4951	4957	4967	4969	4973	4987	4993	4999	5003	5009	5011	5021	5023	5039
5051	5059	5077	5081	5087	5099	5101	5107	5113	5119	5147	5153	5167	5171	5179
5189	5197	5209	5227	5231	5233	5237	5261	5273	5279	5281	5297	5303	5309	5323
5333	5347	5351	5381	5387	5393	5399	5407	5413	5417	5419	5431	5437	5441	5443
5449	5471	5477	5479	5483	5501	5503	5507	5519	5521	5527	5531	5557	5563	5569
5573	5581	5591	5623	5639	5641	5647	5651	5653	5657	5659	5669	5683	5689	5693
5701	5711	5717	5737	5741	5743	5749	5779	5783	5791	5801	5807	5813	5821	5827
5839	5843	5849	5851	5857	5861	5867	5869	5879	5881	5897	5903	5923	5927	5939
5953	5981	5987	6007	6011	6029	6037	6043	6047	6053	6067	6073	6079	6089	6091
6101	6113	6121	6131	6133	6143	6151	6163	6173	6197	6199	6203	6211	6217	6221
6229	6247	6257	6263	6269	6271	6277	6287	6299	6301	6311	6317	6323	6329	6337
6343	6353	6359	6361	6367	6373	6379	6389	6397	6421	6427	6449	6451	6469	6473
6481	6491	6521	6529	6547	6551	6553	6563	6569	6571	6577	6581	6599	6607	6619
6637	6653	6659	6661	6673	6679	6689	6691	6701	6703	6709	6719	6733	6737	6761
6763	6779	6781	6791	6793	6803	6823	6827	6829	6833	6841	6857	6863	6869	6871
6883	6899	6907	6911	6917	6947	6949	6959	6961	6967	6971	6977	6983	6991	6997

7001	7013	7019	7027	7039	7043	7057	7069	7079	7103	7109	7121	7127	7129	7151
7159	7177	7187	7193	7207	7211	7213	7219	7229	7237	7243	7247	7253	7283	7297
7307	7309	7321	7331	7333	7349	7351	7369	7393	7411	7417	7433	7451	7457	7459
7477	7481	7487	7489	7499	7507	7517	7523	7529	7537	7541	7547	7549	7559	7561
7573	7577	7583	7589	7591	7603	7607	7621	7639	7643	7649	7669	7673	7681	7687
7691	7699	7703	7717	7723	7727	7741	7753	7757	7759	7789	7793	7817	7823	7829
7841	7853	7867	7873	7877	7879	7883	7901	7907	7919	7927	7933	7937	7949	7951
7963	7993	8009	8011	8017	8039	8053	8059	8069	8081	8087	8089	8093	8101	8111
8117	8123	8147	8161	8167	8171	8179	8191	8209	8219	8221	8231	8233	8237	8243
8263	8269	8273	8287	8291	8293	8297	8311	8317	8329	8353	8363	8369	8377	8387
8389	8419	8423	8429	8431	8443	8447	8461	8467	8501	8513	8521	8527	8537	8539
8543	8563	8573	8581	8597	8599	8609	8623	8627	8629	8641	8647	8663	8669	8677
8681	8689	8693	8699	8707	8713	8719	8731	8737	8741	8747	8753	8761	8779	8783
8803	8807	8819	8821	8831	8837	8839	8849	8861	8863	8867	8887	8893	8923	8929
8933	8941	8951	8963	8969	8971	8999	9001	9007	9011	9013	9029	9041	9043	9049
9059	9067	9091	9103	9109	9127	9133	9137	9151	9157	9161	9173	9181	9187	9199
9203	9209	9221	9227	9239	9241	9257	9277	9281	9283	9293	9311	9319	9323	9337
9341	9343	9349	9371	9377	9391	9397	9403	9413	9419	9421	9431	9433	9437	9439
9461	9463	9467	9473	9479	9491	9497	9511	9521	9533	9539	9547	9551	9587	9601
9613	9619	9623	9629	9631	9643	9649	9661	9677	9679	9689	9697	9719	9721	9733
9739	9743	9749	9767	9769	9781	9787	9791	9803	9811	9817	9829	9833	9839	9851
9857	9859	9871	9883	9887	9901	9907	9923	9929	9931	9941	9949	9967	9973	

2 C++ và thư viện STD

2.1 Macros

Các macro và alias thường dùng trong template. Phần mã được giữ nguyên.

Mã nguồn mẫu: header.cpp

```
#include <bits/stdc++.h>
#define DEBUG false
#define debugf if (DEBUG) printf
#define MAXN 200309
#define MAXM 900009
#define MAXLOGN 20
#define ALFA 256
#define MOD 1000000007
#define INF 0x3f3f3f3f
#define INFL 0x3f3f3f3f3f3f3f3f
#define EPS 1e-9
#define PI 3.141592653589793238462643383279502884
#define FOR(x,n) for(int x=0; (x)<int(n); (x)++)
#define FORl(x,n) for(int x=1; (x)<=int(n); (x)++)
#define REP(x,n) for(int x=int(n)-1; (x)>=0; (x)--)
#define REPl(x,n) for(int x=(n); (x)>0; (x)--)
#define pb push_back
#define pf push_front
#define fi first
#define se second
#define mp make_pair
#define sz(x) int(x.size())
#define all(x) x.begin(), x.end()
#define mset(x,y) memset(&x, (y), sizeof(x))
using namespace std;
typedef long long ll;
typedef unsigned long long ull;
typedef long double ld;
typedef unsigned int uint;
typedef vector<int> vi;
typedef pair<int, int> ii;

int main() {}
```

2.2 Trình biên dịch GNU

Một số builtin của GNU dịch trực tiếp thành lệnh máy: `__builtin_ffs`, `__builtin_clz`, `__builtin_ctz`, `__builtin_popcount`, `__builtin_parity` và các biến thể `ll`. Không gọi các hàm đếm zero đầu/cuối với đối số 0.

2.3 I/O nhanh

Để tăng tốc cin/cout, dùng `ios_base::sync_with_stdio(0)`. `gets` nhanh hơn `scanf`, `scanf` nhanh hơn cin; `puts` nhanh hơn `printf`, `printf` nhanh hơn cout. Không dùng `gets` ngoài lập trình thi đấu. Mẫu đọc nhanh giữ nguyên bên dưới.

Mã nguồn mẫu: Miscellaneous/fastio.cpp

```
#include <cstdio>
#define MAXI 1000009

/*
 * Pre-read all input fast
 */
```

```

char input[MAXI];
void readInput() {
    input[fread(input, 1, sizeof(input)-4, stdin)] = 0;
}

/*
 * a bit faster than scanf("%d", &x)
 */

int readInt() {
    bool minus = false;
    int ans = 0;
    char c;
    while (c = getchar()) {
        if (c == '-') break;
        if (c >= '0' && c <= '9') break;
    }
    if (c == '-') minus = true;
    else ans = c - '0';
    while (c = getchar()) {
        if (c < '0' || c > '9') break;
        ans = ans*10 + (c - '0');
    }
    return minus ? -ans : ans;
}

/*
 * TEST MATRIX
 */

int main() {
    while(true) {
        int x = readInt();
        if (x < -10000) return 0;
        printf("x = %d\n", x);
    }
}

```

2.4 Kiểm tra overflow

Trước khi cộng, trừ, nhân với biên lớn, kiểm tra dấu và so với INFL; ví dụ $b > 0 \ \&\& \ a > \text{INFL} - b$ báo $a+b$ overflow, $b < 0 \ \&\& \ a < -\text{INFL} - b$ báo underflow.

2.5 C++11

`auto a = b` lấy kiểu của `b`; `auto a = b()` lấy kiểu trả về; `for (T a : b)` duyệt phần tử; `for (T &a : b)` duyệt tham chiếu; lambda có dạng `[] (params) -> type { body }`.

2.6 Complex

Ví dụ: `#include <complex>, complex<double> point`. Các hàm hữu ích: `real`, `imag`, `abs`, `arg`, `norm`, `conj`, `polar`.

2.7 Pair

`#include <utility>, pair<tipo1, tipo2> P`, với `first` và `second`.

2.8 List

Các constructor và thao tác chính: `begin`, `end`, `rbegin`, `rend`, `size`, `empty`, `clear`, `swap`, `front`, `back`, `push_back`, `pop_back`, `push_front`, `pop_front`, `erase`, `insert`.

2.9 Vector

Các thao tác chính: `begin`, `end`, `size`, `empty`, `clear`, `reserve`, `front`, `back`, `erase`, `pop_back`, `push_back`, `swap`.

2.10 Deque

`#include <queue>`, `deque<tipo> Q`; truy cập ngẫu nhiên được bằng `Q[i]`; hỗ trợ `front`, `back`, `pop_front`, `pop_back`, `push_front`, `push_back`.

2.11 Queue

`#include <queue>`, `queue<tipo> Q`; các thao tác: `back`, `empty`, `front`, `pop`, `push`, `size`.

2.12 Stack

`#include <stack>`, `stack<tipo> P`; các thao tác: `empty`, `pop`, `push`, `size`, `top`.

2.13 Map

`#include <map>`, ví dụ `map<string, int> si`; dùng `find`, `lower_bound`, `upper_bound`, `erase`, `count`. Khi duyệt map, `it->first` là khóa và `it->second` là giá trị.

2.14 Set

`#include <set>`, dùng `find`, `insert`, `erase`, `lower_bound`, `upper_bound`. Với comparator tùy chỉnh, định nghĩa struct `cmp` có `bool operator()(tipo, tipo) const`.

2.15 Ordered set

Dùng GNU PBDS tree để có `find_by_order(p)` và `order_by_key(v)`, tức set có thống kê thứ tự.

2.16 Unordered set và map

Tương tự set và map, nhưng dùng bảng băm, thường nhanh hơn; cần C++11: `unordered_set<tipo>`, `unordered_map<chave, valor>`.

2.17 Priority Queue

`#include <queue>`, `priority_queue<tipo> pq`; thao tác: `empty`, `size`, `top`, `push`, `pop`. Mặc định phần tử lớn hơn đứng trước.

2.18 Bitset

#include <bitset>, bitset<MAXN> bs; hỗ trợ count, to_string, to_ulong, to_ullong, set, reset, flip và các toán tử bit.

2.19 String

#include <string>, hỗ trợ nối bằng +, +=, append, thêm/xóa bằng push_back, erase, replace, tìm bằng find, lấy con bằng substr.

2.20 Algorithm và numeric

Trong các mô tả sau, beg và end là con trỏ hoặc iterator. Comparator là hàm bool comp(T a, T b), evaluator là bool eval(T a), accumulator là T add(T a, T b). Có thể viết bằng lambda C++11.

2.21 Algorithm: không sửa đổi

any_of, all_of, none_of, for_each, count, count_if duyệt đoạn [beg, end) mà không thay đổi nội dung.

2.22 Algorithm: sửa đổi

copy, fill, generate, remove, replace, swap, reverse, rotate, random_shuffle, unique thao tác trực tiếp trên đoạn hoặc container.

2.23 Algorithm: phân hoạch

partition đưa phần tử thỏa evaluator lên trước; stable_partition giữ thứ tự tương đối trong từng phần.

2.24 Algorithm: sắp xếp

is_sorted, sort, stable_sort, nth_element dùng toán tử < hoặc comparator.

2.25 Algorithm: tìm kiếm nhị phân

binary_search, lower_bound, upper_bound, equal_range dùng trên đoạn đã sắp xếp.

2.26 Algorithm: heap

make_heap, push_heap, pop_heap, sort_heap thao tác trên heap trong một đoạn.

2.27 Algorithm: cực đại và cực tiểu

min, max, minmax, min_element, max_element, minmax_element, lexicographical_compare.

2.28 Algorithm: hoán vị

next_permutation và prev_permutation sinh hoán vị kế tiếp hoặc trước đó theo thứ tự từ điển.

2.29 Numeric: tích lũy

`accumulate`, `partial_sum`, `adjacent_difference`, `inner_product`, `iota`.

2.30 Functional

Các function object chuẩn gồm `plus`, `minus`, `multiplies`, `divides`, `modulus`, `negate`, `equal_to`, `less`, `greater`, `logical_and`, `bit_and` và các biến thể liên quan.

3 Cấu trúc dữ liệu

3.1 Max-Queue

Hàng đợi hỗ trợ lấy giá trị lớn nhất trong $O(1)$ khâu hao.

Mã nguồn mẫu: Data Structures/maxqueue.cpp

```
#include <queue>
#include <list>
using namespace std;
typedef pair<int, int> ii;

/*
 * MAX QUEUE
 */

class MaxQueue {
    list<ii> q, l;
    int cnt = 0;
public:
    MaxQueue() : cnt(0) {}
    void push(int x) {
        ii cur = ii(x, cnt++);
        while(!l.empty() && l.back() <= cur) l.pop_back();
        q.push_back(cur);
        l.push_back(cur);
    }
    int front() { return q.front().first; }
    void pop() {
        if (q.front().second == l.front().second) l.pop_front();
        q.pop_front();
    }
    int max() { return l.front().first; }
    int size() { return q.size(); }
};

/*
 * Codeforces 980F
 */

#include <bits/stdc++.h>
#define DEBUG false
#define debugf if (DEBUG) printf
#define MAXN 500309
#define MAXM 300009
#define MOD 1000000007
#define INF 0x3f3f3f3f
#define EPS 1e-9
#define FOR(x,n) for(int x=0; (x)<(n); (x)++)
#define FOR1(x,n) for(int x=1; (x)<=(n); (x)++)
#define ROF(x,n) for(int x=(n)-1; (x)>=0; (x)--)
#define ROF1(x,n) for(int x=(n); (x)>0; (x)--)
#define pb push_back
#define pf push_front
#define fi first
#define se second
#define all(x) x.begin(), x.end()
typedef vector<int> vi;
typedef long long ll;

int n, m;
vi g[MAXN], st;
int cycle[MAXN], num[MAXN], pos[MAXN], par[MAXN];
int cnt = 0;
vi cycles[MAXN];
int down[MAXN], up[MAXN];
```

```

void buildcycles(int u, int p) {
    num[u] = 1;
    pos[u] = st.size();
    st.pb(u);
    for(int v : g[u]) {
        if (v == p) continue;
        else if (num[v] == 1 && cycle[u] == -1) {
            cycles[cnt].clear();
            for(int i = pos[v]; i <= pos[u]; i++) {
                cycle[st[i]] = cnt;
                cycles[cnt].pb(st[i]);
            }
            cnt++;
        }
        else if (num[v] == 0) {
            buildcycles(v, u);
            if (cycle[u] == -1 || cycle[v] != cycle[u]) par[v] = u;
        }
    }
    num[u] = 2;
    st.pop_back();
}

int find(vi & v, int u) {
    FOR(i, int(v.size())) {
        if (v[i] == u) return i;
    }
    return -1;
}

int nodedown(int u);

int cycledown(int u) {
    if (down[u] != -1) return down[u];
    int c = cycle[u];
    if (c == -1) return nodedown(u);
    vi & arr = cycles[c];
    int sz = arr.size();
    nodedown(u);
    int j = find(arr, u);
    assert(j != -1);
    FOR(i, sz) {
        int d = abs(i-j);
        if (i == j) continue;
        down[u] = max(down[u], min(d, sz-d) + nodedown(arr[i]));
    }
    return down[u];
}

int nodedown(int u) {
    if (down[u] != -1) return down[u];
    down[u] = 0;
    for(int v : g[u]) {
        if (v == par[u]) continue;
        if (cycle[u] != -1 && cycle[u] == cycle[v]) continue;
        down[u] = max(down[u], 1 + cycledown(v));
    }
    return down[u];
}

void solve(vi & L, vi & Z) {
    int sz = L.size();
    int k = sz/2;
    FOR(i, sz) L.pb(L[i]);
    vi left = L, right = L;
    FOR(i, 2*sz) left[i] += i;
    FOR(i, 2*sz) right[i] += 2*sz-i-1;
    Z.assign(sz, 0);
    MaxQueue q;

```

```

    for(int i = 0; i < sz; i++) {
        q.push(right[i]);
    }
    while(q.size() > k) q.pop();
    for(int i = sz; i < 2*sz; i++) {
        Z[i-sz] = max(Z[i-sz], q.max() - (2*sz-i-1));
        q.push(right[i]);
        q.pop();
    }
    q = MaxQueue();
    for(int i = 2*sz-1; i >= sz; i--) {
        q.push(left[i]);
    }
    while(q.size() > k) q.pop();
    for(int i = sz-1; i >= 0; i--) {
        Z[i] = max(Z[i], q.max() - i);
        q.push(left[i]);
        q.pop();
    }
}

void nodeup(int u, int d);

void cycleup(int u, int d) {
    int c = cycle[u];
    if (c == -1) {
        nodeup(u, d);
        return;
    }
    cycledown(u);
    vi & arr = cycles[c];
    int sz = arr.size();
    up[u] = d;
    int j = find(arr, u);
    assert(j != -1);
    vi L(sz), Z;
    int daux = d;
    for(int v : g[u]) {
        if (v == par[u]) continue;
        if (cycle[u] != -1 && cycle[u] == cycle[v]) continue;
        daux = max(daux, 1 + nodedown(v));
    }
    FOR(i, sz) {
        if (i == j) L[i] = daux;
        else L[i] = nodedown(arr[i]);
    }
    solve(L, Z);
    FOR(i, sz) {
        if (i == j) nodeup(arr[i], max(Z[i], d));
        else nodeup(arr[i], Z[i]);
    }
}

void nodeup(int u, int d) {
    up[u] = d;
    nodedown(u);
    int sz = g[u].size();
    int x = d, y = -1;
    for(int v : g[u]) {
        if (v == par[u]) continue;
        if (cycle[u] != -1 && cycle[u] == cycle[v]) continue;
        if (x >= y && 1+nodedown(v) > y) y = 1+down[v];
        else if (x < y && 1+nodedown(v) > x) x = 1+down[v];
    }
    for(int v : g[u]) {
        if (v == par[u]) continue;
        if (cycle[u] != -1 && cycle[u] == cycle[v]) continue;
        if (1+nodedown(v) == max(x, y))
            cycleup(v, min(x, y)+1);
    }
}

```

```

        else cycleup(v, max(x, y)+1);
    }
}

int main() {
    scanf("%d %d", &n, &m);
    FOR(j, m) {
        int u, v;
        scanf("%d %d", &u, &v);
        g[u].pb(v);
        g[v].pb(u);
    }
    memset(&cycle, -1, sizeof cycle);
    memset(&down, -1, sizeof down);
    memset(&up, -1, sizeof up);
    memset(&num, 0, sizeof num);
    buildcycles(1, -1);
    cycleup(1, 0);
    FORl(u, n) {
        printf("%d ", max(up[u], down[u]));
    }
    printf("\n");
    return 0;
}

```

3.2 Union-Find

Disjoint Set Union với nén đường đi và union theo kích thước/hạng.

Mã nguồn mẫu: Data Structures/unionfind.cpp

```

#include <cstdio>
#include <vector>
using namespace std;

class UnionFind {
private:
    vector<int> parent, rank;
public:
    UnionFind(int N) {
        rank.assign(N+1, 0);
        parent.assign(N+1, 0);
        for (int i = 0; i <= N; i++) parent[i] = i;
    }
    int find(int i) {
        while(i != parent[i]) i = parent[i];
        return i;
    }
    bool isSameSet(int i, int j) {
        return find(i) == find(j);
    }
    void unionSet (int i, int j) {
        if (isSameSet(i, j)) return;
        int x = find(i), y = find(j);
        if (rank[x] > rank[y]) parent[y] = x;
        else {
            parent[x] = y;
            if (rank[x] == rank[y]) rank[y]++;
        }
    }
};

int main() {
    int a, b;
    char c;
    UnionFind uf(1000009);
    while(scanf(" %c %d %d", &c, &a, &b)!=EOF) {

```

```

        if (c=='U' || c=='u') {
            uf.unionSet(a,b);
        }
        else if (c=='i' || c=='I') {
            if (uf.isSameSet(a,b)) printf("yes\n");
            else printf("no\n");
        }
        else break;
    }
    return 0;
}

```

3.3 Binary Indexed Tree / Fenwick Tree

Fenwick tree cho cộng điểm và truy vấn tổng tiền tố.

Mã nguồn mẫu: Trees/bit.cpp

```

#include <cstdio>
#include <vector>
using namespace std;

const int neutral = 0;
#define comp(a, b) ((a)+(b))

class FenwickTree {
private:
    vector<int> ft;
public:
    FenwickTree(int n) { ft.assign(n + 1, 0); }
    int rsq(int i) { // returns RSQ(1, i)
        int sum = neutral;
        for(; i; i -= (i & -i))
            sum = comp(sum, ft[i]);
        return sum;
    }
    int rsq(int i, int j) {
        return rsq(j) - rsq(i - 1);
    }
    void update(int i, int v) {
        for(; i > 0 && i < (int)ft.size(); i += (i & -i))
            ft[i] = comp(v, ft[i]);
    }
};

int main() {}

```

3.4 Binary Indexed Tree / Fenwick Tree với cập nhật và truy vấn đoạn

Biến thể Fenwick hỗ trợ cập nhật đoạn và truy vấn đoạn.

Mã nguồn mẫu: Trees/rangebit.cpp

```

#include <cstdio>
#include <vector>
#include <ctime>
#include <algorithm>
using namespace std;

/*
 * 2D BIT
 */

class FenwickTree {
private:

```

```

vector<int> ft1, ft2;
int rsq(vector<int> & ft, int i) {
    int sum = 0;
    for(; i; i -= (i & -i)) sum += ft[i];
    return sum;
}
void update(vector<int> & ft, int i, int v) {
    for(; i < (int)ft.size(); i += (i & -i))
        ft[i] += v;
}
public:
    FenwickTree(int n) {
        ft1.assign(n + 1, 0); //1-indexed
        ft2.assign(n + 1, 0); //1-indexed
    }
    void update(int i, int j, int v) {
        update(ft1, i, v);
        update(ft1, j+1, -v);
        update(ft2, i, v*(i-1));
        update(ft2, j+1, -v*j);
    }
    int rsq(int i) {
        return rsq(ft1, i)*i - rsq(ft2, i);
    }
    int rsq(int i, int j) {
        return rsq(j) - rsq(i-1);
    }
};

/*
 * TEST MATRIX
 */

int vet[10009];
int sum(int a, int b) {
    int ans = vet[a];
    for(int i=a+1; i<=b; i++) ans += vet[i];
    return ans;
}

void update(int a, int b, int k) {
    for(int i=a; i<=b; i++) vet[i] += k;
}

int main() {
    int N = 10000, a, b, k;
    srand(time(NULL));
    FenwickTree ft(N);
    for(int i=1; i<=N; i++) {
        vet[i] = rand()%N;
        ft.update(i, i, vet[i]);
    }
    for(int q=0; q<100; q++) {

        printf("test #%d:", q+1);
        a = (rand()%N) + 1;
        b = (rand()%N) + 1;
        if (a>b) swap(a, b);
        k = rand()%100;
        printf(" %d to %d plus %d\n", a, b, k);

        update(a, b, k);
        ft.update(a, b, k);

        a = (rand()%N) + 1;
        b = (rand()%N) + 1;
        if (a>b) swap(a, b);
    }
}

```

```

        if (sum(a, b) != ft.rsq(a, b)) printf("test failed\n");
    }
    return 0;
}

```

3.5 2D Binary Indexed Tree / Fenwick Tree

Fenwick hai chiều.

Mã nguồn mẫu: Trees/2dbit.cpp

```

#include <vector>
using namespace std;

const int neutral = 0;
#define comp(a, b) ((a)+(b))

class FenwickTree2D {
private:
    vector< vector<int> > ft;
public:
    FenwickTree2D(int n, int m) {
        ft.assign(n + 1, vector<int>(m + 1, 0));    //1-indexed
    }
    int rsq(int i, int j) { // returns RSQ((1,1), (i,j))
        int sum = 0, _j = j;
        while(i > 0) {
            j = _j;
            while(j > 0) {
                sum = comp(sum, ft[i][j]);
                j -= (j & -j);
            }
            i -= (i & -i);
        }
        return sum;
    }
    void update(int i, int j, int v) {
        int _j = j;
        while(i < (int)ft.size()) {
            j = _j;
            while(j < (int)ft[i].size()) {
                ft[i][j] = comp(v, ft[i][j]);
                j += (j & -j);
            }
            i += (i & -i);
        }
    }
};

/*
 *   TEST MATRIX
 */
#include <cstdio>
#include <algorithm>

vector< vector<int> > A;

int rsq(int x, int y) {
    int ans = neutral;
    for(int i=1; i<=x; i++) {
        for(int j=1; j<=y; j++) {
            ans = comp(ans, A[i][j]);
        }
    }
    return ans;
}

```



```

void update(int x, int y, int v) {
    A[x][y] += v;
}

bool test() {
    int N = 500, nTests = 10000;
    int x, y, v;
    A.resize(N+1);
    for(int i=1; i<=N; i++) A[i].resize(N+1);
    FenwickTree2D ft(N, N);
    for(int i=1; i<=N; i++) {
        for(int j=1; j<=N; j++) {
            A[i][j] = rand() % N;
            ft.update(i, j, A[i][j]);
        }
    }
    printf("starting tests...\n");
    for(int q=1; q<=nTests; q++) {
        x = rand()%N + 1;
        y = rand()%N + 1;
        v = rand()%N;
        ft.update(x, y, v);
        update(x, y, v);
        x = rand()%N + 1;
        y = rand()%N + 1;
        int q1 = rsq(x, y);
        int q2 = ft.rsq(x, y);
        if (q1 != q2) {
            printf("Failed test %d, q1 = %d, q2 = %d\n", q, q1, q2);
            return false;
        }
    }
    printf("all tests passed\n");
    return true;
}

int main() {
    test();
    return 0;
}

```

3.6 Segment Tree

Cây đoạn cơ bản.

Mã nguồn mẫu: Trees/segmenttree.cpp

```

#include <cstdio>
#include <vector>
#include <algorithm>
#define INF (1<<30)
using namespace std;

/*
 * Segment Tree
 */

const int neutral = 0;
#define comp(a, b) ((a)+(b))

class SegmentTree {
    vector<int> a;
    int n;
public:
    SegmentTree(int* st, int* en) {
        int sz = int(en-st);
        for (n = 1; n < sz; n <= 1);
    }

```

```

        a.assign(n << 1, neutral);
        for(int i=0; i<sz; i++) a[i+n] = st[i];
        for(int i=n+sz-1; i; i--)
            a[i>>1] = comp(a[i>>1], a[i]);
    }
    void update(int i, int x) {
        a[i += n] = x; //substitui
        for (i >>= 1; i; i >>= 1)
            a[i] = comp(a[i<<1], a[1+(i<<1)]);
    }
    int query(int l, int r) {
        int ans = neutral;
        for (l += n, r += n+1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) ans = comp(ans, a[l++]);
            if (r & 1) ans = comp(ans, a[--r]);
        }
        return ans;
    }
};

/*
 * TEST MATRIX
 */

int vet[16];

int main() {
    int n;
    scanf("%d", &n);
    for(int i=0; i<n; i++) scanf("%d", &vet[i]);
    char c;
    int a, b;
    SegmentTree st(vet, vet+n);
    while(scanf(" %c", &c), c=='u' || c=='s' ) {
        if (c == 'u') {
            scanf("%d %d", &a, &b);
            st.update(a, b);
        }
        else {
            scanf("%d %d", &a, &b);
            printf("soma(%d,%d)=%d\n", a, b, st.query(a, b));
        }
    }
    return 0;
}

```

3.7 Segment Tree với Lazy Propagation

Cây đoạn có lazy propagation.

Mã nguồn mẫu: Trees/lazysegmenttree.cpp

```

#include <cstdio>
#include <vector>
#include <algorithm>
#define INF (1<<30)
using namespace std;

/*
 * Lazy Propagation Segment Tree
 */

const int neutral = 0; //comp(x, neutral) = x
#define comp(a, b) ((a)+(b))

class SegmentTree {
private:

```

```

    vector<int> st, lazy;
    int size;
#define left(p) (p << 1)
#define right(p) ((p << 1) + 1)
    void build(int p, int l, int r, int* A) {
        if (l == r) { st[p] = A[l]; return; }
        int m = (l + r) / 2;
        build(left(p), l, m, A);
        build(right(p), m+1, r, A);
        st[p] = comp(st[left(p)], st[right(p)]);
    }
    void push(int p, int l, int r) {
        st[p] += (r - l + 1)*lazy[p];    //Caso RSQ
        //st[p] += lazy[p];            //Caso RMQ
        if (l != r) {
            lazy[right(p)] += lazy[p];
            lazy[left(p)] += lazy[p];
        }
        lazy[p] = 0;
    }
    void update(int p, int l, int r, int a, int b, int k) {
        push(p, l, r);
        if (a > r || b < l) return;
        else if (l >= a && r <= b) {
            lazy[p] = k; push(p, l, r); return;
        }
        int m = (l + r) / 2;
        update(left(p), l, m, a, b, k);
        update(right(p), m+1, r, a, b, k);
        st[p] = comp(st[left(p)], st[right(p)]);
    }
    int query(int p, int l, int r, int a, int b) {
        push(p, l, r);
        if (a > r || b < l) return neutral;
        if (l >= a && r <= b) return st[p];
        int m = (l + r) / 2;
        int p1 = query(left(p), l, m, a, b);
        int p2 = query(right(p), m+1, r, a, b);
        return comp(p1, p2);
    }
public:
    SegmentTree(int* bg, int* en) {
        size = (int)(en - bg);
        st.assign(4 * size, neutral);
        lazy.assign(4 * size, 0);
        build(1, 0, size - 1, bg);
    }
    int query(int a, int b) { return query(1, 0, size - 1, a, b); }
    void update(int a, int b, int k) { update(1, 0, size - 1, a, b, k); }
};

/*
 * TEST MATRIX
 */

int vet[16];

int main() {
    int n, k;
    scanf("%d", &n);
    for(int i=0; i<n; i++) scanf("%d", &vet[i]);
    char c;
    int a, b;
    SegmentTree st(vet, vet+n);
    while(scanf(" %c", &c), c=='u' || c=='s' ) {
        if (c == 'u') {
            scanf("%d %d %d", &a, &b, &k);
            st.update(a, b, k);
        }
    }
}

```

```

        else {
            scanf("%d %d", &a, &b);
            printf("soma(%d,%d)=%d\n", a, b, st.query(a, b));
        }
    }
    return 0;
}

```

3.8 2D Segment Tree

Cây đoạn hai chiều.

Mã nguồn mẫu: Trees/2dsegmenttree.cpp

```

#include <algorithm>
#include <vector>
using namespace std;
#define MAXM 100000009 //1e8+9
#define INF 0x3f3f3f3f

/*
 * 2D Segment Tree
 */

int rs[MAXM], ls[MAXM], st[MAXM], cnt = 0;

class SegmentTree2D {
    int sizex, sizey, v, root;
    int x, y, ix, jx, iy, jy;
    void updatex(int p, int lx, int rx) {
        if (x < lx || rx < x) return;
        st[p] += v;
        if (lx == rx) return;
        if (!rs[p]) rs[p] = ++cnt, ls[p] = ++cnt;
        int mx = (lx + rx) / 2;
        updatex(ls[p], lx, mx);
        updatex(rs[p], mx + 1, rx);
    }
    void updatey(int p, int ly, int ry) {
        if (y < ly || ry < y) return;
        if (!st[p]) st[p] = ++cnt;
        updatex(st[p], 0, sizex);
        if (ly == ry) return;
        if (!rs[p]) rs[p] = ++cnt, ls[p] = ++cnt;
        int my = (ly + ry) / 2;
        updatey(ls[p], ly, my);
        updatey(rs[p], my + 1, ry);
    }
    int queryx(int p, int lx, int rx) {
        if (!p || jx < lx || ix > rx) return 0;
        if (ix <= lx && rx <= jx) return st[p];
        int mx = (lx + rx) / 2;
        return queryx(ls[p], lx, mx) +
            queryx(rs[p], mx + 1, rx);
    }
    int queryy(int p, int ly, int ry) {
        if (!p || jy < ly || iy > ry) return 0;
        if (iy <= ly && ry <= jy) return queryx(st[p], 0, sizex);
        int my = (ly + ry) / 2;
        return queryy(ls[p], ly, my) +
            queryy(rs[p], my + 1, ry);
    }
public:
    SegmentTree2D(int nx, int ny) : sizex(nx), sizey(ny) {
        root = ++cnt;
    }
    void update(int _x, int _y, int _v) {

```

```

        x = _x; y = _y; v = _v;
        updatey(root, 0, sizey);
    }
    int query(int _ix, int _jx, int _iy, int _jy) {
        ix = _ix; jx = _jx; iy = _iy; jy = _jy;
        return queryy(root, 0, sizey);
    }
};

/*
 *   TEST MATRIX
 */
#include <cstdio>

vector< vector<int> > A;

int query(int lx, int rx, int ly, int ry) {
    int ans = 0;
    for(int i=lx; i<=rx; i++) {
        for(int j=ly; j<=ry; j++) {
            ans += A[i][j];
        }
    }
    return ans;
}

void update(int x, int y, int v) {
    A[x][y] += v;
}

bool test() {
    int N = 500, nTests = 100000;
    int lx, rx, ly, ry, x, y, v;
    A.resize(N);
    for(int i=0; i<N; i++) {
        A[i].assign(N, 0);
    }
    SegmentTree2D st(N, N);
    printf("starting tests...\n");
    for(int q=1; q<=nTests; q++) {
        x = rand()%N;
        y = rand()%N;
        v = rand()%N;
        st.update(x, y, v);
        update(x, y, v);
        lx = rand()%N;
        rx = rand()%N;
        ly = rand()%N;
        ry = rand()%N;
        int q1 = query(lx, rx, ly, ry);
        int q2 = st.query(lx, rx, ly, ry);
        if (q1 != q2) {
            printf("Failed test %d, q1 = %d, q2 = %d\n", q, q1, q2);
            return false;
        }
    }
    printf("all tests passed\n");
    return true;
}

int main() {
    test();
    return 0;
}

```

3.9 Persistent Segment Tree

Cây đoạn persistent; update trả về phiên bản mới, phiên bản 0 là cây ban đầu.

Mã nguồn mẫu: Trees/persistentsegtree.cpp

```
#include <vector>
#include <algorithm>
#define INF (1<<30)
using namespace std;
#define MAXS 2000009
#define MAXN 200009

/*
 * Persistent Segment Tree
 */

const int neutral = 0; //comp(x, neutral) = x
#define comp(a, b) ((a)+(b))

int nds, st[MAXS], ls[MAXS], rs[MAXS];

class PersistentSegmentTree {
private:
    int vroot[MAXN];
    int size, nds, nv;
    int newnode() {
        ls[nds] = rs[nds] = -1;
        st[nds++] = neutral;
        return nds-1;
    }
    void build(int p, int l, int r, int* A) {
        if (l == r) {
            st[p] = A ? A[l] : neutral;
            return;
        }
        int m = (l + r) / 2;
        build(ls[p] = newnode(), l, m, A);
        build(rs[p] = newnode(), m+1, r, A);
        st[p] = comp(st[ls[p]], st[rs[p]]);
    }
    void update(int prv, int p, int l, int r, int i, int k) {
        if (l == r) {
            st[p] = k; //substitui
            //st[p] = st[prv] + k; //agrega
            return;
        }
        int m = (l + r) / 2;
        if (i <= m) {
            rs[p] = rs[prv];
            update(ls[prv], ls[p] = newnode(), l, m, i, k);
        }
        else {
            ls[p] = ls[prv];
            update(rs[prv], rs[p] = newnode(), m+1, r, i, k);
        }
        st[p] = comp(st[ls[p]], st[rs[p]]);
    }
    int query(int p, int l, int r, int a, int b) {
        if (a > r || b < l) return neutral;
        if (l >= a && r <= b) return st[p];
        int m = (l + r) / 2;
        int p1 = query(ls[p], l, m, a, b);
        int p2 = query(rs[p], m + 1, r, a, b);
        return comp(p1, p2);
    }
public:
    PersistentSegmentTree(int* begin, int* end) {
        nds = nv = 0; size = (int)(end-begin);
```

```

        vroot[nv++] = newnode();
        build(vroot[0], 0, size-1, begin);
    }
    PersistentSegmentTree(int _size = 1) {
        nds = nv = 0; size = _size;
        vroot[nv++] = newnode();
        build(vroot[0], 0, size-1, NULL);
    }
    int query(int a, int b, int v) {
        return query(vroot[v], 0, size-1, a, b);
    }
    int update(int i, int v, int k) {
        vroot[nv++] = newnode();
        update(vroot[v], vroot[nv-1], 0, size-1, i, k);
        return nv-1;
    }
    int nver() { return nv; }
};

/*
 * TEST MATRIX
 */

#include <ctime>
#include <cstdio>
#define DEBUG false
const int N = 500;
int arr[N+9][N+9];
int curver;
PersistentSegmentTree pst;
bool test(int NTests) {
    srand(time(NULL));
    for(int nt=0; nt<NTests; nt++) {
        for(int i=0; i<N; i++) arr[0][i] = rand()%N;

        if(DEBUG) printf("building seg tree...\n");
        pst = PersistentSegmentTree(&arr[0][0], &arr[0][0] + N);
        curver = 1;
        if(DEBUG) printf("arr[%d]:", curver-1);
        for(int j=0; j<N; j++) {
            if(DEBUG) printf(" %2d", arr[curver-1][j]);
        }
        if(DEBUG) printf("\n");

        for(int i=0; i<N; i++) {
            if(DEBUG) printf("test %d:\n", i+1);

            // update
            int id = rand()%N;
            int k = rand()%N;
            int v = rand()%curver;
            if(DEBUG) printf("update over version %d: arr[%d] = %d\n", v, id, k);
            for(int j=0; j<N; j++) arr[curver][j] = arr[v][j];
            curver++;
            arr[curver-1][id] = k;
            if(DEBUG) printf("arr[%d]:", curver-1);
            for(int j=0; j<N; j++) {
                if(DEBUG) printf(" %2d", arr[curver-1][j]);
            }
            if(DEBUG) printf("\n");

            if (pst.update(id, v, k) != curver-1) {
                printf("failed to update at test %d\n", i+1);
                return false;
            }

            //queries over recent version
            for(int t=0; t<N; t++) {
                int a = rand()%N;

```

```

        int b = rand()%N;
        if (a > b) swap(a, b);
        int val1 = 0;
        for(int j=a; j<=b; j++) val1 = comp(val1, arr[curver-1][j]);
        int val2 = pst.query(a, b, curver-1);
        if(DEBUG) printf("sum[%d,%d] = %d (ver %d)\n", a, b, val1, curver-1);
        if (val1 != val2) {
            printf("failed at test %d, case %d (%d==%d), ver %d\n", i+1, t+1, val1, val2, curver-1);
            printf("arr[%d]:", curver-1);
            for(int j=0; j<N; j++) {
                printf(" %2d", arr[curver-1][j]);
            }
            printf("\nquery [%d,%d]\n", a, b);
            return false;
        }
    }

    //queries over all versions
    for(int t=0; t<N; t++) {
        int a = rand()%N;
        int b = rand()%N;
        int v = rand()%curver;
        if (a > b) swap(a, b);
        int val1 = 0;
        for(int j=a; j<=b; j++) val1 = comp(val1, arr[v][j]);
        int val2 = pst.query(a, b, v);
        if(DEBUG) printf("sum[%d,%d] = %d (ver %d)\n", a, b, val1, v);
        if (val1 != val2) {
            printf("failed at test %d, case %d (%d==%d), ver %d\n", i+1, t+1, val1, val2, v);
            printf("arr[%d]:", v);
            for(int j=0; j<N; j++) {
                printf(" %2d", arr[v][j]);
            }
            printf("\nquery [%d,%d]\n", a, b);
            return false;
        }
    }
}

printf("all tests passed\n");
return true;
}

int main() {
    test(10);
    return 0;
}

/*
 * SPOJ DQUERY
 */

/*#define MAXN 100000
#include <cstdio>
#include <map>

int a[MAXN], id[MAXN], ver[MAXN];

bool cmp(int i, int j) {
    return a[i] < a[j];
}

PersistentSegmentTree pst(MAXN);

int main() {
    int N, Q, u, v, x;
    scanf("%d", &N);
    map<int, int> last;
    for(int i=0; i<N; i++) {

```



```

        scanf("%d", &x);
        if (last.count(x)) a[i] = last[x];
        else a[i] = -1;
        last[x] = i;
        id[i] = i;
    }
    sort(id, id+N, cmp);
    pst = PersistentSegmentTree(N);
    for(int it=0, i; it<N; it++) {
        i = id[it];
        ver[it] = pst.update(i, pst.nver()-1, pst.query(i, i, pst.nver()-1) + 1);
    }
    scanf("%d", &Q);
    while(Q-->0) {
        scanf("%d %d", &u, &v);
        int lo = -1;
        int hi = N;
        while(hi > lo+1) {
            int m = (hi+lo)/2;
            if (a[id[m]] < u-1) lo = m;
            else hi = m;
        }
        if (lo < 0) printf("0\n");
        else printf("%d\n", pst.query(u-1, v-1, ver[lo]));
    }
    return 0;
}*/

```

3.10 Li Chao Segment Tree

Cho mảng điểm $x[]$ đã sắp, hỗ trợ thêm đường thẳng và truy vấn max tại điểm.

Mã nguồn mẫu: Trees/lichaosegtree.cpp

```

#include <cstdio>
#include <vector>
#define INF 0x3f3f3f3f3f3f3f3f
#define MAXN 1009
using namespace std;

typedef long long ll;

/*
 * LiChao Segment Tree
 */

class LiChao {
    vector<ll> m, b;
    int n, sz; ll *x;
#define gx(i) (i < sz ? x[i] : x[sz-1])
    void update(int t, int l, int r, ll nm, ll nb) {
        ll xl = nm * gx(l) + nb, xr = nm * gx(r) + nb;
        ll yl = m[t] * gx(l) + b[t], yr = m[t] * gx(r) + b[t];
        if (yl >= xl && yr >= xr) return;
        if (yl <= xl && yr <= xr) {
            m[t] = nm, b[t] = nb; return;
        }
        int mid = (l + r) / 2;
        update(t<<1, l, mid, nm, nb);
        update(1+(t<<1), mid+1, r, nm, nb);
    }
public:
    LiChao(ll *st, ll *en) : x(st) {
        sz = int(en - st);
        for(n = 1; n < sz; n <= 1);
        m.assign(2*n, 0); b.assign(2*n, -INF);
    }
}

```

```

void insert_line(ll nm, ll nb) {
    update(1, 0, n-1, nm, nb);
}
ll query(int i) {
    ll ans = -INF;
    for(int t = i+n; t; t >>= 1)
        ans = max(ans, m[t] * x[i] + b[t]);
    return ans;
}
};

/*
 * UVA 12524
 */

ll w[MAXN], x[MAXN], A[MAXN], B[MAXN], dp[MAXN][MAXN];

int main(){
    int N, K;
    while(scanf("%d %d", &N, &K)!=EOF) {
        for(int i=0; i<N; i++){
            scanf("%lld %lld", x+i, w+i);
            A[i] = w[i] + (i>0 ? A[i-1] : 0);
            B[i] = w[i]*x[i] + (i>0 ? B[i-1] : 0);
            dp[i][1] = x[i]*A[i] - B[i];
        }
        for(int k=2; k<=K; k++){
            dp[0][k] = 0;
            LiChao lc(x, x+N);
            for(int i=1; i<N; i++){
                lc.insert_line(A[i-1], -dp[i-1][k-1]-B[i-1]);
                dp[i][k] = x[i]*A[i] - B[i] - lc.query(i);
            }
        }
        printf("%lld\n", dp[N-1][K]);
    }
    return 0;
}

```

3.11 Split-Merge Segment Tree

Tập các multiset hỗ trợ tạo, hợp nhất, tách, đếm trong đoạn, truy cập phần tử thứ i và xuất ra vector.

Mã nguồn mẫu: Trees/splitmergesegtree.cpp

```

#include <vector>
using namespace std;
#define MAXN 200009
#define MAXS 500009

/*
 * Split-merge Segment Tree
 */

const int neutral = 0;
#define comp(a, b) ((a)+(b))

class SegmentTree {
    int cnt, n, root[MAXN], nroot;
    int sz[MAXS], ls[MAXS], rs[MAXS];

    int newnode(int _s) {
        ls[cnt] = rs[cnt] = -1;
        sz[cnt++] = _s;
        return cnt-1;
    }

    int build(int l, int r, int i) {

```

```

        int t = newnode(1);
        if (l == r) return t;
        int m = (l + r) / 2;
        if (i <= m) ls[t] = build(l, m, i);
        else rs[t] = build(m + 1, r, i);
        return t;
    }
    int split(int t1, int k) {
        if (t1 == -1 || sz[t1] <= k) return -1;
        int t2 = newnode(sz[t1] - k);
        sz[t1] = k;
        int sl = ls[t1] == -1 ? 0 : sz[ls[t1]];
        if (k > sl) rs[t2] = split(rs[t1], k - sl);
        else swap(rs[t1], rs[t2]);
        if (k < sl) ls[t2] = split(ls[t1], k);
        return t2;
    }
    int merge(int t1, int t2) {
        if (t1 == -1 || t2 == -1) return t1+t2+1;
        ls[t1] = merge(ls[t1], ls[t2]);
        rs[t1] = merge(rs[t1], rs[t2]);
        sz[t1] = comp(sz[t1], sz[t2]);
        return t1;
    }
    int query(int t, int l, int r, int a, int b) {
        if (t == -1 || l > b || r < a) return neutral;
        if (l >= a && r <= b) return sz[t];
        int m = (l + r) / 2;
        int p1 = query(ls[t], l, m, a, b);
        int p2 = query(rs[t], m+1, r, a, b);
        return comp(p1, p2);
    }
    int at(int t, int l, int r, int i) {
        if (l == r) return l;
        int sl = ls[t] == -1 ? 0 : sz[ls[t]];
        int m = (l + r) / 2;
        if (i < sl) return at(ls[t], l, m, i);
        else return at(rs[t], m+1, r, i - sl);
    }
    void to_vector(int t, int l, int r, vector<int> & v) {
        if (t == -1) return;
        int m = (l + r) / 2;
        to_vector(ls[t], l, m, v);
        to_vector(rs[t], m+1, r, v);
        for(int i = 0; i < sz[t] && l == r; i++)
            v.push_back(l);
    }
public:
    SegmentTree() { }
    SegmentTree(int _n) : n(_n), cnt(0), nroot(0) { }
    int newSet(int i) {
        root[nroot++] = build(0, n, i);
        return nroot-1;
    }
    void unionSet(int r1, int r2) {
        root[r1] = merge(root[r1], root[r2]);
        root[r2] = -1;
    }
    int query(int r, int a, int b) { return query(root[r], 0, n, a, b); }
    int size(int r) { return root[r] == -1 ? 0 : sz[root[r]]; }
    int splitSet(int r, int k) {
        if (k >= size(r)) return -1;
        if (k <= 0) {
            root[nroot++] = root[r];
            root[r] = -1;
        }
        else root[nroot++] = split(root[r], k);
        return nroot-1;
    }
}

```

```

    int at(int r, int i) { return at(root[r], 0, n, i); }
    void to_vector(int r, vector<int> & v) { to_vector(root[r], 0, n, v); }
};

/*
 * TEST MATRIX
 */
/*
#include <cstdlib>
#include <algorithm>
#include <ctime>
#include <cstdio>
#define NTESTS 1000009
#define all(x) x.begin(), x.end()

vector<int> a[NTESTS];
int cnt;
SegmentTree st;

void print(int r) {
    printf("%d: {" , r);
    bool ok = false;
    for (int v : a[r]) {
        if (ok) printf(", ");
        ok = true;
        printf("%d", v);
    }
    printf("} ");
    printf("\n");
}

void merge(int r1, int r2) {
    a[r1].insert(a[r1].end(), all(a[r2]));
    sort(all(a[r1]));
    a[r2].clear();
}

int query(int r, int i, int j) {
    int ans = 0;
    for(int v : a[r]) {
        if (v >= i && v <= j) ans++;
    }
    return ans;
}

int split(int r1, int k) {
    int sz = (r1 == -1 ? 0 : int(a[r1].size()));
    if (k >= sz) {
        return -1;
    }
    else if (k <= 0) {
        a[cnt].clear();
        a[r1].swap(a[cnt]);
        cnt++;
        return cnt-1;
    }
    else {
        a[cnt] = vector<int>(a[r1].begin()+k, a[r1].end());
        a[r1].resize(k);
        cnt++;
        return cnt-1;
    }
}

bool test(int ntests) {
    srand(time(NULL));
    vector<int> alive;
    st = SegmentTree(ntests);
    cnt = 0;

```

```

for(int i = 0; i <= ntests; i++) {
    a[cnt].clear();
    a[cnt].push_back(i);
    st.newSet(i);
    alive.push_back(cnt);
    cnt++;
}
for(int test = 0; test < ntests; test++) {
    //printf("\ntest %d\n", test);
    int r1, r2, i1, i2;
    //merge
    do {
        i1 = rand()%alive.size();
        i2 = rand()%alive.size();
    } while(i1 == i2);
    r1 = alive[i1];
    r2 = alive[i2];
    merge(r1, r2);
    st.unionSet(r1, r2);
    swap(alive[i2], alive.back());
    alive.pop_back();
    //split
    i1 = rand()%alive.size();
    r1 = alive[i1];
    int k = rand()%(a[r1].size()+1);
    r2 = split(r1, k);
    int r2l = st.splitSet(r1, k);
    if (r2 != r2l) {
        printf("failed on test %d, split r2test=%d r2st=%d\n", test, r2, r2l);
        return false;
    }
    if (r2 != -1) alive.push_back(r2);
    //test element by element
    for(int j = 0; j <= ntests; j++) {
        int ans1 = query(r1, j, j);
        int ans2 = st.query(r1, j, j);
        if (ans1 != ans2) {
            printf("failed on test %d, element %d ans1=%d ans2=%d\n", test, j, ans1, ans2);
            return false;
        }
    }
    //test size
    if (a[r1].size() != st.size(r1)) {
        printf("failed on test %d, size1=%d size2=%d\n", test, a[r1].size(), st.size(r1));
        return false;
    }
    //test element by index
    for(int i = 0; i < (int)a[r1].size(); i++) {
        int ans1 = a[r1][i];
        int ans2 = st.at(r1, i);
        if (ans1 != ans2) {
            print(r1);
            printf("failed on test %d, pos %d ans1=%d ans2=%d\n", test, i, ans1, ans2);
            return false;
        }
    }
    //test random queries
    for(int j = 0; j <= ntests; j++) {
        int a = rand()%(ntests+1);
        int b = rand()%(ntests+1);
        int ans1 = query(r1, a, b);
        int ans2 = st.query(r1, a, b);
        if (ans1 != ans2) {
            printf("failed on test %d, segment [%d,%d] ans1=%d ans2=%d\n", test, a, b, ans1, ans2);
            return false;
        }
    }
}
printf("all tests passed\n");

```

```

        return true;
    }

int main() {
    test(1000);
    return 0;
}*/

/*
 * Szkoput Tree Rotations
 */

#include <cstdio>
typedef long long ll;
typedef pair<ll, int> ii;
typedef vector<int> vi;

SegmentTree st;

int n;
void dfs(int & t, ll & ans) {
    int a;
    scanf("%d", &a);
    if (a == 0) {
        int t1, t2;
        dfs(t1, ans);
        dfs(t2, ans);
        ll sum1 = 0, sum2 = 0;
        vi arr;
        if (st.size(t1) > st.size(t2)) {
            t = t1;
            st.to_vector(t2, arr);
            for (int b : arr) {
                sum1 += st.query(t1, 0, b-1);
                sum2 += st.query(t1, b+1, n);
            }
            st.unionSet(t1, t2);
        }
        else {
            t = t2;
            st.to_vector(t1, arr);
            for (int b : arr) {
                sum1 += st.query(t2, 0, b-1);
                sum2 += st.query(t2, b+1, n);
            }
            st.unionSet(t2, t1);
        }
        ans += min(sum1, sum2);
    }
    else {
        t = st.newSet(a);
    }
}

int main() {
    scanf("%d", &n);
    st = SegmentTree(n);
    ll ans = 0;
    int t;
    dfs(t, ans);
    printf("%lld\n", ans);
    return 0;
}

```

3.12 Segment Tree Beats

Cập nhật $a[i] = \min(a[i], k)$ trên đoạn và truy vấn tổng.

Mã nguồn mẫu: Trees/segmenttreebeats.cpp

```
#include <vector>
#include <map>
using namespace std;
#define INF 0x3f3f3f3f

/*
 * Segment Tree Beats
 */

class SegmentTreeBeats {
    vector<int> st, lazy;
    vector<int> val[2], cnt[2];
    int size;
#define left(p) (p << 1)
#define right(p) ((p << 1) + 1)
    void update(int u) {
        map<int, int> aux;
        for(int k = 0; k < 2; k++) {
            aux[val[k][left(u)]] += cnt[k][left(u)];
            aux[val[k][right(u)]] += cnt[k][right(u)];
        }
        for(int k = 0; k < 2; k++) {
            val[k][u] = aux.rbegin()->first;
            cnt[k][u] = aux.rbegin()->second;
            aux.erase(aux.rbegin()->first);
        }
        st[u] = st[left(u)] + st[right(u)];
    }
    void push(int u, int l, int r) {
        if (lazy[u] < val[0][u]) {
            st[u] -= cnt[0][u]*val[0][u];
            val[0][u] = lazy[u];
            st[u] += cnt[0][u]*val[0][u];
        }
        if (l < r) {
            lazy[left(u)] = min(lazy[left(u)], lazy[u]);
            lazy[right(u)] = min(lazy[right(u)], lazy[u]);
        }
    }
    void build(int u, int l, int r, int A[]) {
        if (l == r) {
            val[0][u] = A ? A[r] : INF; cnt[0][u] = 1;
            val[1][u] = -INF; cnt[1][u] = 0;
            st[u] = A[r]; return;
        }
        int m = (l + r) / 2;
        build(left(u), l, m, A);
        build(right(u), m+1, r, A);
        update(u);
    }
    void update(int u, int l, int r, int a, int b, int k) {
        push(u, l, r);
        if (b < l || r < a || k >= val[0][u]) return;
        if (a <= l && r <= b && val[1][u] < k) {
            lazy[u] = k;
            push(u, l, r); return;
        }
        int m = (l + r) / 2;
        update(left(u), l, m, a, b, k);
        update(right(u), m+1, r, a, b, k);
        update(u);
    }
    int query(int u, int l, int r, int a, int b) {
        push(u, l, r);
        if (b < l || r < a) return 0;
        if (a <= l && r <= b) return st[u];
        int m = (l + r) / 2;
```

```

        int p1 = query(left(u), l, m, a, b);
        int p2 = query(right(u), m+1, r, a, b);
        return p1 + p2;
    }
public:
    SegmentTreeBeats(int *bg, int *en) {
        size = int(en - bg);
        st.resize(4 * size);
        for(int k = 0; k < 2; k++) {
            val[k].resize(4 * size);
            cnt[k].resize(4 * size);
        }
        lazy.assign(4 * size, INF);
        build(1, 0, size-1, bg);
    }
    void update(int a, int b, int k) {
        update(1, 0, size-1, a, b, k);
    }
    int query(int a, int b) {
        return query(1, 0, size-1, a, b);
    }
};

/*
 * TEST MATRIX
 */

#include <cstdio>
#include <ctime>
#include <cstdlib>
#define MAXN 1000000
#define RANGE 1000
#define NQUERY 10

int arr[MAXN];

void update(int a, int b, int k) {
    for(int i = a; i <= b; i++) arr[i] = min(arr[i], k);
}

int query(int a, int b) {
    int sum = 0;
    for(int i = a; i <= b; i++) sum += arr[i];
    return sum;
}

bool test(int suits, int n) {
    srand(time(NULL));
    clock_t t = clock();
    for(int suit = 0; suit < suits; suit++) {
        for(int i = 0; i < n; i++) arr[i] = 2*RANGE;
        SegmentTreeBeats stb(arr, arr+n);
        for(int test = 0; test < n; test++) {
            int i = rand()%n;
            int j = rand()%n;
            int k = (rand()%100) + n-test;
            if (i > j) swap(i, j);
            update(i, j, k);
            stb.update(i, j, k);
            for(int q = 0; q < NQUERY; q++) {
                i = rand()%n;
                j = rand()%n;
                if (i > j) swap(i, j);
                int v2 = stb.query(i, j);
                //int v1 = v2;
                int v1 = query(i, j);
                if (v1 != v2) {
                    printf("failed on test %d, query [%d,%d] => %d,%d\n", test, i, j, v1, v2);
                    return false;
                }
            }
        }
    }
    return true;
}

```



```

    }
    }
}
t = clock() - t;
printf("all tests passed, time %.3f\n", t*1.0/CLOCKS_PER_SEC);
return true;
}

int main() {
    test(10, 10000);
    return 0;
}

```

3.13 Merge Sort Tree

Cây của merge sort; đếm phần tử trong đoạn $[i, j]$ có giá trị thuộc $[a, b]$.

Mã nguồn mẫu: Trees/mergesorttree.cpp

```

#include <vector>
#include <algorithm>
#define INF (1<<30)
using namespace std;

/*
 * Merge Sort Tree
 */

class MergeSortTree {
private:
    vector< vector<int> > st;
    int size;
    #define left(p) (p << 1)
    #define right(p) ((p << 1) + 1)
    void build(int p, int l, int r, int* A) { // O(n)
        st[p].resize(r-l+1);
        if (l == r) { st[p][0] = A[l]; return; }
        int pl = left(p), pr = right(p), m = (l+r)/2;
        build(pl, l, m, A);
        build(pr, m+1, r, A);
        merge(st[pl].begin(), st[pl].end(),
              st[pr].begin(), st[pr].end(),
              st[p].begin());
    }
    int query(int p, int l, int r, int i, int j, int a, int b) {
        if (j < l || i > r) return 0;
        if (i <= l && j >= r)
            return upper_bound(st[p].begin(), st[p].end(), b) -
                   lower_bound(st[p].begin(), st[p].end(), a);
        int m = (l + r) / 2;
        return query(left(p), l, m, i, j, a, b) +
               query(right(p), m+1, r, i, j, a, b);
    }
public:
    MergeSortTree(int* begin, int* end) {
        size = (int)(end-begin);
        st.assign(4*size, vector<int>());
        build(1, 0, size-1, begin);
    }
    int query(int i, int j, int a, int b) {
        return query(1, 0, size-1, i, j, a, b);
    }
};

/*
 * TEST MATRIX

```

```

*/

#include <cstdio>

int vet[100009], aux[100009];
int query(int i, int j, int a, int b) {
    int ans = 0;
    for(int k=i; k<=j; k++) if (a <= vet[k] && vet[k] <= b) ans++;
    return ans;
}

bool test(int N) {
    for(int i = 0; i < N; i++) vet[i] = rand()%N;
    MergeSortTree st(vet, vet+N);
    for(int q=0, a, b, i, j, n; q<N; q++) {
        a = rand()%N;
        b = rand()%N;
        if (a>b) swap(a, b);
        i = rand()%N;
        j = rand()%N;
        if (i>j) swap(i, j);

        if (query(i, j, a, b) != st.query(i, j, a, b)) {
            printf("test %d failed\n", q+1);
            return false;
        }
    }
    printf("all tests passed\n");
    return true;
}

int main() {
    test(10000);
    return 0;
}

```

3.14 Fractional Cascading Merge Sort Tree

Merge Sort Tree với fractional cascading để giảm truy vấn xuống $O(\log n)$.

Mã nguồn mẫu: Trees/fractionalcascade.cpp

```

#include <vector>
#include <algorithm>
#define INF (1<<30)
using namespace std;

/*
 * Merge Sort Tree
 */

class MergeSortTree {
private:
    vector< vector<int> > st, li, ri;
    int size;
    #define left(p) (p << 1)
    #define right(p) ((p << 1) + 1)
    void build(int p, int l, int r, int* A) { // O(n)
        st[p].resize(r-l+1);
        if (l == r) { st[p][0] = A[l]; return; }
        int pl = left(p), pr = right(p), m = (l+r)/2;
        vector<int> &vl = st[pl], &vr = st[pr];
        build(pl, l, m, A);
        build(pr, m+1, r, A);
        merge(st[pl].begin(), st[pl].end(),
              st[pr].begin(), st[pr].end(),

```

```

        st[p].begin());
    for(int k = 0, i = 0, j = 0; k < r-l+1; k++) {
        while(i < vl.size() && st[p][k] > vl[i]) i++;
        while(j < vr.size() && st[p][k] > vr[j]) j++;
        li[p].push_back(i), ri[p].push_back(j);
    }
    li[p].push_back(vl.size());
    ri[p].push_back(vr.size());
}

int query(int p, int l, int r, int i, int j, int a, int b) {
    if (j < l || i > r) return 0;
    if (i <= l && j >= r) return b-a;
    int m = (l + r) / 2;
    return query(left(p), l, m, i, j, li[p][a], li[p][b]) +
        query(right(p), m+1, r, i, j, ri[p][a], ri[p][b]);
}

public:
    MergeSortTree(int* begin, int* end) {
        size = (int)(end-begin);
        st.assign(4*size, vector<int>());
        li.assign(4*size, vector<int>());
        ri.assign(4*size, vector<int>());
        build(1, 0, size-1, begin);
    }

    int query(int i, int j, int a, int b) {
        a = lower_bound(st[1].begin(), st[1].end(), a) - st[1].begin();
        b = upper_bound(st[1].begin(), st[1].end(), b) - st[1].begin();
        return query(1, 0, size-1, i, j, a, b);
    }
};

/*
 * TEST MATRIX
 */

#include <cstdio>

int vet[100009], aux[100009];
int query(int i, int j, int a, int b) {
    int ans = 0;
    for(int k=i; k<=j; k++) if (a <= vet[k] && vet[k] <= b) ans++;
    return ans;
}

bool test(int N) {
    for(int i = 0; i < N; i++) vet[i] = rand()%N;
    MergeSortTree st(vet, vet+N);
    for(int q=0, a, b, i, j, n; q<N; q++) {
        a = rand()%N;
        b = rand()%N;
        if (a>b) swap(a, b);
        i = rand()%N;
        j = rand()%N;
        if (i>j) swap(i, j);

        if (query(i, j, a, b) != st.query(i, j, a, b)) {
            printf("test %d failed [i,j]=[%d,%d] [a,b]=[%d,%d], control %d mst %d\n",
                q+1, i, j, a, b, query(i, j, a, b), st.query(i, j, a, b));
            return false;
        }
    }
    printf("all tests passed\n");
    return true;
}

int main() {
    test(10000);
    return 0;
}

```

```
}
```

3.15 Sparse Table

Trả lời RMQ $O(1)$, tiền xử lý $O(n \log n)$.

Mã nguồn mẫu: Data Structures/sparsetable.cpp

```
#include <cstdio>
#include <cmath>
#define MAXN 100009
#define MAXLOGN 20

/*
 * Sparse Table
 */

#define comp(a, b) min((a),(b))

class SparseTable {
    int st[MAXN][MAXLOGN];
    int sz;
public:
    SparseTable(int* bg, int* en) {
        sz = int(en - bg);
        for(int i=0; i<sz; i++) st[i][0] = bg[i];
        for(int j = 1; 1 << j <= sz; j++)
            for(int i=0; i + (1<<j) <= sz; i++)
                st[i][j] = comp(st[i][j-1], st[i+(1<<(j-1))][j-1]);
    }
    int query(int l, int r) {
        int k = (int)floor(log((double)r-l+1) / log(2.0)); //  $2^k \leq (j-i+1)$ 
        return comp(st[l][k], st[r-(1<<k)+1][k]);
    }
};
```

3.16 Pre-Suf Table

Trả lời RSQ không cần phép nghịch đảo trong $O(1)$, tiền xử lý $O(n \log n)$.

Mã nguồn mẫu: Data Structures/presufTable.cpp

```
#include <vector>
using namespace std;
#define MAXN 100009

#include <cstdio>

/*
 * Pre-Suf Table
 * Friend of the Sparse Table
 */

const int neutral = 0;
#define comp(a, b) ((a)+(b))

int top[2*MAXN];

class PresufTable {
    vector< vector<int> > f;
public:
    PresufTable(int* st, int* en) {
        int bit = 0, sz = int(en - st), n, k;
        for(n = 1, k = 0; n < sz; n <= 1, k++);
        f.assign(k, vector<int>(n));
    }
};
```

```

    for(int i = 0; i < n; i++) {
        while((1<<bit) <= i) bit++;
        top[i] = bit;
    }
    for(int i = 0; i < n; i++)
        f[0][i] = i < sz ? st[i] : neutral;
    for(int j = 0, len = 1; j <= k; j++, len <= 1)
        for(int s = len; s < n; s += (len<<1)) {
            f[j][s] = st[s]; f[j][s-1] = st[s-1];
            for(int i = 1; i < len; i++) {
                f[j][s+i] = comp(f[j][s+i-1], st[s+i]);
                f[j][s-1-i] = comp(f[j][s-i], st[s-1-i]);
            }
        }
    }
    int query(int l, int r) {
        if (l == r) return f[0][r];
        int i = top[l^r] - 1;
        return comp(f[i][r], f[i][l]);
    }
};

/*
 * TEST MATRIX
 */

#include <cstdlib>
#define RANGE 10000

int arr[MAXN];

int query(int l, int r) {
    int ans = 0;
    for(int i = l; i <= r; i++) {
        ans += arr[i];
    }
    return ans;
}

bool test(int n, int ntests) {
    for(int i = 0; i < n; i++) arr[i] = /*rand()%RANGE*/ i;
    PresufTable pst(arr, arr+n);
    for(int test = 1; test <= ntests; test++) {
        int l = rand()%n;
        int r = rand()%n;
        if (l > r) swap(l, r);
        if (query(l, r) != pst.query(l, r)) {
            printf("failed test %d\n", test);
            return false;
        }
    }
    printf("all tests passed\n");
    return true;
}

int main() {
    test(10000, 10000);
}

```

3.17 AVL Tree

Cây AVL với thao tác BST.

Mã nguồn mẫu: Trees/avl.cpp

```

#include <cstdio>
struct node {

```

```

    int key, height, size;
    node *left, *right;
    node(int k) {
        key = k; left = right = 0;
        height = size = 1;
    }
};

class AVLtree {
private:
    node* root;
    int size_;
    int height(node* p) { return p ? p->height : 0; }
    int size(node* p) { return p ? p->size : 0; }
    int bfactor(node* p) {
        return height(p->right)-height(p->left);
    }
    void fixheight(node* p) {
        int hl = height(p->left);
        int hr = height(p->right);
        p->height = (hl>hr?hl:hr)+1;
        p->size = 1 + size(p->left) + size(p->right);
    }
    node* rotateright(node* p) {
        node* q = p->left;
        p->left = q->right;
        q->right = p;
        fixheight(p);
        fixheight(q);
        return q;
    }
    node* rotateleft(node* q) {
        node* p = q->right;
        q->right = p->left;
        p->left = q;
        fixheight(q);
        fixheight(p);
        return p;
    }
    node* balance(node* p) {
        fixheight(p);
        if(bfactor(p)==2) {
            if(bfactor(p->right)<0)
                p->right = rotateright(p->right);
            return rotateleft(p);
        }
        if(bfactor(p)==-2) {
            if(bfactor(p->left)>0)
                p->left = rotateleft(p->left);
            return rotateright(p);
        }
        return p;
    }
    node* build(node* p, int k) {
        if(!p) return new node(k);
        if (p->key == k) return p;
        else if(k<p->key) p->left = build(p->left, k);
        else p->right = build(p->right, k);
        return balance(p);
    }
    node* findmin(node* p) {
        return p->left?findmin(p->left):p;
    }
    node* removemin(node* p) {
        if(p->left==0) return p->right;
        p->left = removemin(p->left);
        return balance(p);
    }
    node* remove(node* p, int k) {

```

```

        if(!p) return 0;
        if(k < p->key) p->left = remove(p->left,k);
        else if(k > p->key) p->right = remove(p->right,k);
        else {
            node* l = p->left;
            node* r = p->right;
            delete p;
            if(!r) return l;
            node* min = findmin(r);
            min->right = removemin(r);
            min->left = l;
            return balance(min);
        }
        return balance(p);
    }
    bool find(node* p, int k) {
        if (!p) return false;
        if (p->key == k) return true;
        else if(k<p->key) return find(p->left, k);
        else return find(p->right, k);
    }
    void del(node* p) {
        if (!p) return;
        del(p->left);
        del(p->right);
        delete p;
    }
    node* nth(node* p, int n) {
        if (!p) return p;
        if (size(p->left)+1 > n) return nth(p->left, n);
        if (size(p->left)+1 < n) return nth(p->right, n - size(p->left) - 1);
        else return p;
    }
}

public:
    AVLtree() { root = 0; size_ = 0; }
    ~AVLtree() { del(root); }
    bool empty() { return size_ == 0; }
    int size() { return size_; }
    void clear() {
        size_ = 0;
        del(root);
        root = 0;
    }
    void insert(int key) {
        size_++;
        root = build(root, key);
    }
    void erase(int key) {
        size_--;
        root = remove(root, key);
    }
    bool count(int key) {
        return find(root, key);
    }
    int nth_element(int n) {
        node* p = nth(root, n);
        if (p) return p->key;
        else return -1;
    }
    //1-indexed
};

/*
 * TEST MATRIX
 */

#include <set>
#include <cstdlib>
using namespace std;

```

```

bool test()
{
    AVLtree tree;
    set<int> s;
    int N = 1000000;
    for(int t=1; t<=N; t++) {
        int x = rand()%100;
        if (s.count(x) != tree.count(x)) {
            printf("failed on test %d\n", t);
            return false;
        }
        if (s.count(x)) {
            s.erase(x);
            tree.erase(x);
        }
        else{
            s.insert(x);
            tree.insert(x);
        }
    }
    return true;
}

int main()
{
    if (test()) {
        printf("all tests passed\n");
    }
    return 0;
}

```

3.18 Treap / Cartesian Tree

Treap hỗ trợ insert, count, erase, nth_element; split tách giữa k-1 và k.

Mã nguồn mẫu: Trees/treap.cpp

```

#include <cstdio>
#include <set>
#include <algorithm>
using namespace std;

/*
 * Implicit Treap
 * 0-indexed, 10x slower than red-black tree
 * split separates between k-1 and k
 */

struct node {
    int x, y, size;
    node *l, *r;
    node(int _x) : x(_x), y(rand()), size(1), l(NULL), r(NULL) {}
};

class Treap {
private:
    node* root;
    int size(node* t) { return t ? t->size : 0; }
    node* refresh(node* t) {
        if (!t) return t;
        t->size = 1 + size(t->l) + size(t->r);
        return t;
    }
    void split(node* &t, int k, node* &a, node* &b) {
        node* aux;
        if (!t) a = b = NULL;

```



```

        else if (t->x < k) {
            split(t->r, k, aux, b);
            t->r = aux;
            a = refresh(t);
        }
        else {
            split(t->l, k, a, aux);
            t->l = aux;
            b = refresh(t);
        }
    }
}

node* merge(node* a, node* b) {
    if (!a || !b) return a ? a : b;
    if (a->y < b->y) {
        a->r = merge(a->r, b);
        return refresh(a);
    }
    else {
        b->l = merge(a, b->l);
        return refresh(b);
    }
}

node* count(node* t, int k) {
    if (!t) return NULL;
    else if (k < t->x) return count(t->l, k);
    else if (k == t->x) return t;
    else return count(t->r, k);
}

node* nth(node* t, int n) {
    if (!t) return NULL;
    if (n <= size(t->l)) return nth(t->l, n);
    else if (n == size(t->l) + 1) return t;
    else return nth(t->r, n-size(t->l)-1);
}

void del(node* &t) {
    if (!t) return;
    if (t->l) del(t->l);
    if (t->r) del(t->r);
    delete t;
    t = NULL;
}

public:
    Treap() : root(NULL) { }
    ~Treap() { clear(); }
    void clear() { del(root); }
    int size() { return size(root); }
    bool count(int k) { return count(root, k) != NULL; }
    bool insert(int k) {
        if (count(k)) return false;
        node *a, *b;
        split(root, k, a, b);
        root = merge(merge(a, new node(k)), b);
        return true;
    }
    bool erase(int k) {
        node *f = count(root, k);
        if (!f) return false;
        node *a, *b, *c, *d;
        split(root, k, a, b);
        split(b, k+1, c, d);
        root = merge(a, d);
        delete f;
        return true;
    }
    int nth(int n) {
        node* ans = nth(root, n);
        return ans ? ans->x : -1;
    }
};

```

```

/*
 * TEST MATRIX
 */

int vet[10000009];

void test() {
    set<int> s;
    Treap t;
    int N = 1000000;
    for(int i=0; i<N; i++) {
        int n = rand()%1000;
        if(!s.count(n)) {
            s.insert(n);
            t.insert(n);
            //if(!t.insert(n)) printf("error inserting %d in treap!\n", n);
            //printf("inserted %d\n", n);
        }
        else{
            s.erase(n);
            t.erase(n);
            //if(!t.erase(n)) printf("error erasing %d in treap!\n", n);
            //printf("erased %d\n", n);
        }
        n = rand()%1000;
        if (s.count(n) != t.count(n)) {
            printf("failed test %d, s.count(%d) = %d, t.count(%d) = %d\n", i, n, s.count(n), n, t.count(n));
        }
    }
    s.clear();
    t.clear();
    for(int i=0; i<N; i++) {
        vet[i] = i+1;
    }
    random_shuffle(vet, vet+N);
    for(int i=0; i<N; i++) {
        t.insert(vet[i]);
    }
    for(int i=1; i<=N; i++) {
        if (t.nth(i) != i) {
            printf("failed test %d\n", i);
        }
    }
    printf("all tests passed\n");
}

int main() {
    test();
    return 0;
}

```

3.19 Treap / Cartesian Tree ngẫu nhiên

Treap theo vị trí cho chuỗi/mảng động.

Mã nguồn mẫu: Trees/implicittreap.cpp

```

#include <stdio>
#include <vector>
#include <algorithm>
#include <ctime>
#define INF 0x3f3f3f3f
using namespace std;

/*
 * Implicit Treap

```

```

* 0-indexed, 10x slower than segment tree
* split separates in trees of size k and n-k
*/

struct node {
    int y, v, sum, size;
    bool rev;
    node *l, *r;
    node(int _v) : v(_v), sum(_v), y(rand()),
        size(1), l(NULL), r(NULL), rev(false) {}
};

class ImplicitTreap {
private:
    node* root;
    int size(node* t) { return t ? t->size : 0; }
    int sum(node* t) { return t ? t->sum : 0; }
    node* refresh(node* t) {
        if (t == NULL) return t;
        t->size = 1 + size(t->l) + size(t->r);
        t->sum = t->v + sum(t->l) + sum(t->r);
        if (t->l != NULL) t->l->rev ^= t->rev;
        if (t->r != NULL) t->r->rev ^= t->rev;
        if (t->rev) {
            swap(t->l, t->r);
            t->rev = false;
        }
        return t;
    }
    void split(node* &t, int k, node* &a, node* &b) {
        refresh(t);
        node * aux;
        if (!t) a = b = NULL;
        else if (size(t->l) < k) {
            split(t->r, k-size(t->l)-1, aux, b);
            t->r = aux;
            a = refresh(t);
        }
        else {
            split(t->l, k, a, aux);
            t->l = aux;
            b = refresh(t);
        }
    }
    node* merge(node* a, node* b) {
        refresh(a); refresh(b);
        node* aux;
        if (!a || !b) return a ? a : b;
        if (a->y < b->y) {
            a->r = merge(a->r, b);
            return refresh(a);
        }
        else {
            b->l = merge(a, b->l);
            return refresh(b);
        }
    }
    node* at(node* t, int n) {
        if (!t) return t;
        refresh(t);
        if (n < size(t->l)) return at(t->l, n);
        else if (n == size(t->l)) return t;
        else return at(t->r, n-size(t->l)-1);
    }
    void del(node* &t) {
        if (!t) return;
        if (t->l) del(t->l);
        if (t->r) del(t->r);
        delete t;
    }
};

```

```

        t = NULL;
    }
public:
    ImplicitTreap() : root(NULL) { }
    ~ImplicitTreap() { clear(); }
    void clear() { del(root); }
    int size() { return size(root); }
    bool insertAt(int n, int v) {
        node *a, *b;
        split(root, n, a, b);
        root = merge(merge(a, new node(v)), b);
        return true;
    }
    bool erase(int n) {
        node *a, *b, *c, *d;
        split(root, n, a, b);
        split(b, 1, c, d);
        root = merge(a, d);
        if (c == NULL) return false;
        delete c;
        return true;
    }
    int at(int n) {
        node* ans = at(root, n);
        return ans ? ans->v : -1;
    }
    int query(int l, int r) {
        if (l > r) swap(l, r);
        node *a, *b, *c, *d;
        split(root, l, a, d);
        split(d, r-l+1, b, c);
        int ans = sum(b);
        root = merge(a, merge(b, c));
        return ans;
    }
    void reverse(int l, int r) {
        if (l>r) swap(l, r);
        node *a, *b, *c, *d;
        split(root, l, a, d);
        split(d, r-l+1, b, c);
        if (b != NULL) b->rev ^= 1;
        root = merge(a, merge(b, c));
    }
};

/*
 * TEST MATRIX
 */

bool test() {
    srand(time(NULL));
    vector<int> v;
    ImplicitTreap t;
    int N = 10000;
    vector<int>::iterator it;
    bool toprint = false;
    for(int i=0, n, k, l, r; i<N; i++) {
        if (i%5 == 0 && i > 0) {
            n = rand()%((int)v.size());
            if (toprint) printf("deleting v[%d] = %d\n", n, v[n]);
            it = v.begin()+n;
            v.erase(it);
            t.erase(n);
        }
        else if (i%5 == 4) {
            l = rand()%((int)v.size());
            r = rand()%((int)v.size());
            if (l>r) swap(l, r);
            if (toprint) printf("reversing %d to %d\n", l, r);

```

```

        for(int j=l; j<=r && j<=r-j+l; j++) {
            swap(v[j], v[r-j+l]);
        }
        t.reverse(l, r);
    }
    else{
        n = rand()%((int)v.size()+1);
        k = rand()%1000;
        if (toprint) printf("inserting %d in pos %d\n", k, n);
        it = v.begin()+n;
        v.insert(it, k);
        t.insertAt(n, k);
    }
    if (toprint) printf("array: ");
    for(int j=0; j<(int)v.size(); j++) {
        if (toprint) printf("%d ", v[j]);
        if (v[j] != t.at(j)) {
            printf("test %d failed, v[%d] = %d, t.at(%d) = %d\n", i+1, j, v[j], j, t.at(j));
            return false;
        }
    }
    if (toprint) printf("\n");
    l = rand()%((int)v.size());
    r = rand()%((int)v.size());
    if (l>r) swap(l, r);
    int ans = 0;
    for(int j=l; j<=r; j++) {
        ans += v[j];
    }
    if (toprint) printf("sum(%d, %d) = %d = %d\n", l, r, ans, t.query(l, r));
    if (ans != t.query(l, r)) {
        printf("test %d failed, ans(%d, %d) = %d = %d\n", i, l, r, ans, t.query(l, r));
        return false;
    }
}
return true;
}

int main() {
    if(test()) printf("all tests passed\n");
    return 0;
}

```

3.20 Persistent Randomized BST

BST ngẫu nhiên persistent.

Mã nguồn mẫu: Trees/rbst.cpp

```

#include <cstdio>
#include <vector>
#include <algorithm>
#include <ctime>
#define INF 0x3f3f3f3f
using namespace std;

/*
 * Randomized BST, works as Persistent Implicit Treap
 * split separates in trees of size k and n-k
 */

struct node {
    int v, sum, size;
    node *l, *r;
    node(int _v, vector<node*> &nodes) :
        v(_v), sum(_v), size(1), l(NULL), r(NULL) {
        nodes.push_back(this);
    }
};

```

```

    }
};

class RBST {
private:
    vector<node*> root, nodes;
    int size(node* t) { return t ? t->size : 0; }
    int sum(node* t) { return t ? t->sum : 0; }
    node* copy(node* t) {
        node* cpy = new node(t->v, nodes);
        cpy->l = t->l; cpy->r = t->r;
        return refresh(cpy);
    }
    node* refresh(node* t) {
        if (t == NULL) return t;
        t->size = 1 + size(t->l) + size(t->r);
        t->sum = t->v + sum(t->l) + sum(t->r);
        return t;
    }
    void split(node* t, int k, node* &a, node* &b) {
        node *aux;
        if (!t) { a = b = NULL; return; }
        t = copy(t);
        if (size(t->l) < k) {
            split(t->r, k-size(t->l)-1, aux, b);
            t->r = aux;
            a = refresh(t);
        }
        else {
            split(t->l, k, a, aux);
            t->l = aux;
            b = refresh(t);
        }
    }
    node* merge(node* a, node* b) {
        node *aux, *cpy;
        if (!a || !b) return a ? a : b;
        int m = size(a), n = size(b);
        if (rand()%(n+m) < m) {
            a = copy(a);
            a->r = merge(a->r, b);
            return refresh(a);
        }
        else {
            b = copy(b);
            b->l = merge(a, b->l);
            return refresh(b);
        }
    }
    node* at(node* t, int i) {
        if (!t) return t;
        if (i < size(t->l)) return at(t->l, i);
        else if (i == size(t->l)) return t;
        else return at(t->r, i-size(t->l)-1);
    }
public:
    RBST() { clear(); }
    ~RBST() { clear(); }
    void clear() {
        for(int i = 0; i < (int)nodes.size(); i++)
            delete nodes[i];
        root.push_back(NULL);
    }
    int size(int ver) { return size(root[ver]); }
    int insert(int ver, int i, int v) {
        node *a, *b, *tree;
        split(root[ver], i, a, b);
        tree = merge(merge(a, new node(v, nodes)), b);
        root.push_back(tree);
    }

```

```

        return root.size() - 1;
    }
    int erase(int ver, int i) {
        node *a, *b, *c, *d;
        split(root[ver], i, a, b);
        split(b, 1, c, d);
        root.push_back(merge(a, d));
        return root.size() - 1;
    }
    int at(int ver, int i) {
        node* ans = at(root[ver], i);
        return ans ? ans->v : -1;
    }
    int query(int ver, int l, int r) {
        if (l > r) swap(l, r);
        node *a, *b, *c, *d;
        split(root[ver], l, a, d);
        split(d, r-l+1, b, c);
        return sum(b);
    }
    int split(int ver, int k) {
        node *a, *b;
        split(root[ver], k, a, b);
        root.push_back(a);
        root.push_back(b);
        return root.size()-2;
    }
    int merge(int ver1, int ver2) {
        root.push_back(merge(root[ver1], root[ver2]));
        return root.size()-1;
    }
};

/*
 * TEST MATRIX
 */

#include <cassert>
#define MAXSIZE 100000
#define NTESTS 10000
#define NQUERY 10
#define RANGE 10000

void insert(vector<int> &arr, int i, int v) {
    arr.insert(arr.begin()+i, v);
}

int queryt(vector<int> &arr, int i, int j) {
    if (i > j) swap(i, j);
    int sum = 0;
    for(int it=i; it<=j && it<(int)arr.size(); it++) {
        sum += arr[it];
    }
    return sum;
}

void erase(vector<int> &arr, int i) {
    arr.erase(arr.begin()+i);
}

bool test() {
    srand(time(NULL));
    vector<int> control[NTESTS];
    RBST treap;
    for(int test = 1; test < NTESTS; test++) {
        int op = rand()%2;
        int ver = rand()%test;
        control[test] = control[ver];
        if (test%2 == 1) {

```

```

        int ver2 = rand()%test;
        if (control[ver].size() + control[ver2].size() > MAXSIZE) {
            test--;
            continue;
        }
        control[test].insert(control[test].end(), control[ver2].begin(), control[ver2].end());
        treap.merge(ver, ver2);
    }
    else if (op == 0 && !control[test].empty()) {
        int i = rand()%int(control[test].size());
        erase(control[test], i);
        treap.erase(ver, i);
    }
    else {
        int data = rand()%RANGE;
        int i = rand()%int(control[test].size()+1);
        insert(control[test], i, data);
        treap.insert(ver, i, data);
    }
}
for(int test = 0; test < NTESTS; test++) {
    if (control[test].size() != treap.size(test)) {
        printf("failed on test %d, size: %u-%d\n", test, control[test].size(), treap.size(test));
        return false;
    }
    for(int i = 0; i < (int)control[test].size(); i++) {
        if (control[test][i] != treap.at(test, i)) {
            printf("failed on test %d, id %d: %d-%d\n", test, i, control[test][i], treap.at(test, i));
            return false;
        }
    }
    for(int query = 0; query < NQUERY; query++) {
        if (control[test].empty()) continue;
        int i = rand()%int(control[test].size());
        int j = rand()%int(control[test].size());
        if (i > j) swap(i, j);
        int sum1 = queryt(control[test], i, j);
        int sum2 = treap.query(test, i, j);
        if (sum1 != sum2) {
            printf("failed on test %d, query %d: %d-%d\n", test, query, sum1, sum2);
            return false;
        }
    }
}
printf("all tests passed\n");
return true;
}

int main() {
    test();
    return 0;
}

```

3.21 Deque đệ quy thuần hàm

Deque persistent thuần hàm.

Mã nguồn mẫu: Data Structures/recursivedeque.cpp

```

#include <cstdio>
#include <vector>
using namespace std;

/*
 * Recursive Purely Functional Deque
 */

```



```

struct cont {
    cont *ls, *rs;
    int x;
    cont(int _x, vector<cont*> &gc) :
        x(_x), ls(NULL), rs(NULL) {
            gc.push_back(this);
        }
    cont(cont* l, cont* r, vector<cont*> &gc) :
        x(0), ls(l), rs(r) {
            gc.push_back(this);
        }
};

struct node {
    cont* s[2];
    node* c;
    int sz;
    node(vector<node*> &gc) : c(NULL), sz(0) {
        s[0] = s[1] = NULL;
        gc.push_back(this);
    }
    node* copy(vector<node*> &gc) {
        node* cpy = new node(gc);
        cpy->sz = sz; cpy->c = c;
        cpy->s[0] = s[0]; cpy->s[1] = s[1];
        return cpy;
    }
};

class Deque {
private:
    vector<node*> root, nodes;
    vector<cont*> conts;
    void push(node* &u, int k, cont* x) {
        if (!u) { u = new node(nodes); }
        else u = u->copy(nodes);
        u->sz++;
        if (!u->s[k]) u->s[k] = x;
        else {
            cont* y = u->s[k];
            u->s[k] = NULL;
            if (k) swap(x, y);
            push(u->c, k, new cont(x, y, conts));
        }
    }
    cont* pop(node* &u, int k) {
        u = u->copy(nodes);
        u->sz--;
        cont *x = NULL, *y, *z;
        if (u->s[k]) {
            x = u->s[k];
            u->s[k] = NULL;
        }
        else if (u->c && u->c->sz) {
            z = pop(u->c, k);
            x = z->ls; y = z->rs;
            if (k) swap(x, y);
            u->s[k] = y;
        }
        else if (u->s[k^1]) {
            x = u->s[k^1];
            u->s[k^1] = NULL;
        }
        else u->sz++;
        return x;
    }
    cont* at(const node* u, int i) {
        if (!u) return NULL;
        if (i == 0 && u->s[0]) return u->s[0];
    }
};

```

```

        if (i == u->sz-1 && u->s[1]) return u->s[1];
        if (u->s[0]) i--;
        cont* x = at(u->c, i>>1);
        return i & 1 ? x->rs : x->ls;
    }
    node* merge(node* u, node* v, cont* c1, cont* c2) {
        node* w = new node(nodes);
        if (!u && !v) {
            w->s[0] = c1; w->s[1] = c2;
            return w;
        }
        if (!u) u = new node(nodes);
        if (!v) v = new node(nodes);
        cont *a[4];
        int k = 0;
        if (u->s[1]) a[k++] = u->s[1];
        if (c1) a[k++] = c1;
        if (c2) a[k++] = c2;
        if (v->s[0]) a[k++] = u->s[0];
        w->s[0] = u->s[0];
        w->s[1] = v->s[1];
        if ((k&1) && v->s[0]) pop(v, 0);
        if ((k&1) && v->sz == 0) w->s[1] = a[--k];
        if (k&1) a[k++] = pop(v, 0);
        c1 = k > 0 ? new cont(a[0], a[1], conts) : NULL;
        c2 = k > 2 ? new cont(a[2], a[3], conts) : NULL;
        w->c = merge(u->c, v->c, c1, c2);
        return w;
    }
public:
    Deque() { root.push_back(new node(nodes)); }
    ~Deque() {
        for(int i = 0; i < (int)nodes.size(); i++)
            delete nodes[i];
        for(int i = 0; i < (int)conts.size(); i++)
            delete conts[i];
    }
    int push(int ver, int x, int k) {
        root.push_back(root[ver]);
        push(root.back(), k, new cont(x, conts));
        return root.size()-1;
    }
    int pop(int ver, int k) {
        root.push_back(root[ver]);
        pop(root.back(), k);
        return root.size()-1;
    }
    int merge(int v1, int v2) {
        root.push_back(merge(root[v1], root[v2], NULL, NULL));
        return root.size()-1;
    }
    int push_back(int ver, int x) { return push(ver, x, 1); }
    int push_front(int ver, int x) { return push(ver, x, 0); }
    int pop_back(int ver) { return pop(ver, 1); }
    int pop_front(int ver) { return pop(ver, 0); }
    int size(int ver) { return root[ver]->sz; }
    int at(int ver, int i) { return at(root[ver], i)->x; }
    int front(int ver) { return at(root[ver], 0)->x; }
    int back(int ver) { return at(root[ver], size(ver)-1)->x; }
};

/*
 * TEST MATRIX
 */

#include <ctime>
#include <deque>
#include <cstdlib>
#define RANGE 1000

```

```

#define LIMIT 1000

bool test(int ntests) {
    srand(time(NULL));
    vector< deque<int> > control(ntests);
    Deque deq;
    for(int test = 1; test < ntests; test++) {
        int add = rand()%2;
        int front = rand()%2;
        int ver = rand()%test;
        control[test] = control[ver];
        printf("test %d: ", test);
        if (test % 4 == 0 && control[test].size() < LIMIT) {
            int over;
            do {
                over = rand()%test;
            } while(control[test].size() + control[over].size() > LIMIT);
            deq.merge(ver, over);
            control[test].insert(control[test].end(), control[over].begin(), control[over].end());
        }
        else if (add || control[test].empty()) {
            int data = rand()%RANGE;
            //printf("pushing %d to %s of ver %d...", data, front ? "front" : "back", ver);
            if (front) {
                control[test].push_front(data);
                deq.push_front(ver, data);
            }
            else {
                control[test].push_back(data);
                deq.push_back(ver, data);
            }
            //printf("done\n");
        }
        else {
            //printf("popping %s from ver %d...", front ? "front" : "back", ver);
            if (front) {
                control[test].pop_front();
                deq.pop_front(ver);
            }
            else {
                control[test].pop_back();
                deq.pop_back(ver);
            }
            //printf("done\n");
        }
    }
    for(int test = 0; test < ntests; test++) {
        int sz1 = control[test].size();
        int sz2 = deq.size(test);
        if (sz1 != sz2) {
            printf("failed on size test %d (%d,%d)\n", test, sz1, sz2);
            return false;
        }
        if (!control[test].empty()) {
            int data1 = control[test].front();
            int data2 = deq.front(test);
            if (data1 != data2) {
                printf("failed on front test %d: %d-%d\n", test, data1, data2);
                return false;
            }
            data1 = control[test].back();
            data2 = deq.back(test);
            if (data1 != data2) {
                printf("failed on back test %d: %d-%d\n", test, data1, data2);
                return false;
            }
        }
    }
    for(int i = 0; i < (int)control[test].size(); i++) {
        int data1 = control[test][i];
    }
}

```

```

        int data2 = deq.at(test, i);
        if (data1 != data2) {
            printf("failed on at %d test %d: %d-%d\n", i, test, data1, data2);
            return false;
        }
    }
}
printf("all tests passed\n");
return true;
}

int main() {
    test(10000);
    return 0;
}

```

3.22 Splay Tree

Splay tree.

Mã nguồn mẫu: Trees/splay.cpp

```

#include <cstdio>

/*
 * Splay Tree
 */

struct node {
    int key;
    node *ls, *rs;
    node(int k) : key(k), ls(NULL), rs(NULL) {}
};

class SplayTree {
private:
    node* root;
    node* rotateright(node* p) {
        node* q = p->ls;
        p->ls = q->rs;
        q->rs = p;
        return q;
    }
    node* rotateleft(node* q) {
        node* p = q->rs;
        q->rs = p->ls;
        p->ls = q;
        return p;
    }
    node* splay(node* p, int key) {
        if (!p || p->key == key) return p;
        if (p->key > key) {
            if (!p->ls) return p;
            if (p->ls->key > key) {
                p->ls->ls = splay(p->ls->ls, key);
                p = rotateright(p);
            }
            else if (p->ls->key < key) {
                p->ls->rs = splay(p->ls->rs, key);
                if (p->ls->rs)
                    p->ls = rotateleft(p->ls);
            }
            return (!p->ls) ? p : rotateright(p);
        }
        else {
            if (!p->rs) return p;
            if (p->rs->key > key) {

```

```

        p->rs->ls = splay(p->rs->ls, key);
        if (p->rs->ls)
            p->rs = rotateright(p->rs);
    }
    else if (p->rs->key < key) {
        p->rs->rs = splay(p->rs->rs, key);
        p = rotateleft(p);
    }
    return (!p->rs) ? p : rotateleft(p);
}
}
void del(node* &p) {
    if (!p) return;
    del(p->ls); del(p->rs);
    delete p;
    p = NULL;
}

public:
    SplayTree() : root(NULL) { }
    ~SplayTree() { del(root); }
    bool empty() { return root == NULL; }
    void clear() { del(root); }
    void insert(int key) {
        if (!root) {
            root = new node(key);
            return;
        }
        node* p = splay(root, key);
        if (p->key == key) return;
        root = new node(key);
        if (p->key > key) {
            root->rs = p;
            root->ls = p->ls;
            p->ls = NULL;
        }
        else {
            root->ls = p;
            root->rs = p->rs;
            p->rs = NULL;
        }
    }
    void erase(int key) {
        node* p = splay(root, key);
        if (!p || p->key != key) return;
        if (!p->rs) {
            root = p->ls;
            delete p;
            return;
        }
        node* q = splay(p->rs, key);
        q->ls = p->ls;
        root = q;
        delete p;
    }
    bool count(int key) {
        if (!root) return false;
        root = splay(root, key);
        return root->key == key;
    }
};

/*
 * TEST MATRIX
 */

#include <set>
#include <cstdlib>
using namespace std;

```

```

bool test()
{
    SplayTree splay;
    set<int> s;
    int N = 1000000;
    for(int t=1; t<=N; t++) {
        int x = rand()%100;
        if (s.count(x) != splay.count(x)) {
            printf("failed on test %d\n", t);
            return false;
        }
        if (s.count(x)) {
            s.erase(x);
            splay.erase(x);
        }
        else{
            s.insert(x);
            splay.insert(x);
        }
    }
    return true;
}

int main()
{
    if (test()) {
        printf("all tests passed\n");
    }
    return 0;
}

```

3.23 Link Cut Tree

Cấu trúc cây động với link/cut.

Mã nguồn mẫu: Trees/linkcut.cpp

```

#include <cstdio>
#include <vector>
using namespace std;
#define INF 0x3f3f3f3f

/*
 * Link cut tree
 */

struct node {
    int sw, id, w;
    node *par, *ppar, *ls, *rs;
    node() {
        par = ppar = ls = rs = NULL;
        w = sw = INF;
    }
};

class LinkCutTree {
    vector<node> lct;
    int sum(node *p) { return p ? p->sw : 0; }
    void refresh(node* p) {
        p->sw = p->w + sum(p->ls) + sum(p->rs);
    }
    void rotateright(node* p) {
        node *q, *r;
        q = p->par, r = q->par;
        if (q->ls == p->rs) q->ls->par = q;
        p->rs = q, q->par = p;
    }
};

```

```

        if (p->par == r) {
            if (q == r->ls) r->ls = p;
            else r->rs = p;
        }
        p->ppar = q->ppar;
        q->ppar = NULL;
        refresh(q);
    }
    void rotateleft(node* p) {
        node *q, *r;
        q = p->par, r = q->par;
        if (q->rs == p->ls) q->rs->par = q;
        p->ls = q, q->par = p;
        if (p->par == r) {
            if (q == r->ls) r->ls = p;
            else r->rs = p;
        }
        p->ppar = q->ppar;
        q->ppar = 0;
        refresh(q);
    }
    void splay(node* p) {
        node *q, *r;
        while (p->par != NULL) {
            q = p->par;
            if (q->par == NULL) {
                if (p == q->ls) rotateright(p);
                else rotateleft(p);
                continue;
            }
            r = q->par;
            if (q == r->ls) {
                if (p == q->ls) rotateright(q), rotateright(p);
                else rotateleft(p), rotateright(p);
            }
            else {
                if (p == q->rs) rotateleft(q), rotateleft(p);
                else rotateright(p), rotateleft(p);
            }
        }
        refresh(p);
    }
    node* access(node* p) {
        splay(p);
        if (p->rs != NULL) {
            p->rs->ppar = p; p->rs->par = NULL;
            p->rs = NULL; refresh(p);
        }
        node* last = p;
        while (p->ppar != NULL) {
            node* q = last = p->ppar;
            splay(q);
            if (q->rs != NULL) {
                q->rs->ppar = q;
                q->rs->par = NULL;
            }
            q->rs = p; p->par = q;
            p->ppar = NULL;
            refresh(q); splay(p);
        }
        return last;
    }
}
public:
    LinkCutTree(int n = 0) {
        lct.resize(n + 1);
        for(int i = 0; i <= n; i++) {
            lct[i].id = i;
            refresh(&lct[i]);
        }
    }

```

```

}
void link(int u, int v, int w = 1) {
    //u becomes child of v
    node *p = &lct[u], *q = &lct[v];
    access(p); access(q);
    p->ls = q; q->par = p; p->w = w;
    refresh(p);
}
void cut(int u) {
    node* p = &lct[u]; access(p);
    p->ls->par = NULL; p->ls = NULL;
    refresh(p);
}
int findroot(int u) {
    node* p = &lct[u]; access(p);
    while (p->ls) p = p->ls;
    splay(p);
    return p->id;
}
bool IsSameTree(int u, int v) {
    return findroot(u) == findroot(v);
}
int depth(int u) {
    access(&lct[u]);
    return lct[u].sw - lct[u].w;
}
int LCA(int u, int v) {
    access(&lct[u]);
    return access(&lct[v])->id;
}
int dist(int u, int v) {
    if (!IsSameTree(u, v)) return INF;
    node* lca = &lct[LCA(u, v)];
    int ans = 0;
    access(&lct[u]); splay(lca);
    ans += sum(lca->rs);
    access(&lct[v]); splay(lca);
    ans += sum(lca->rs);
    return ans;
}
};

/*
 * Undirected link cut tree
 */

class UndirectedLinkCutTree {
    LinkCutTree lct;
    vector<int> par;

    void invert(int u) {
        if (par[u] == -1) return;
        int v = par[u];
        invert(v);
        lct.cut(u); par[u] = -1;
        lct.link(v, u); par[v] = u;
    }
public:
    UndirectedLinkCutTree(int n = 0) {
        lct = LinkCutTree(n);
        par.assign(n+1, -1);
    }
    void link(int u, int v) {
        if (lct.depth(u) < lct.depth(v)) {
            invert(u);
            lct.link(u, v); par[u] = v;
        }
        else {
            invert(v);

```



```

        lct.link(v, u); par[v] = u;
    }
}

void cut(int u, int v) {
    if (par[v] == u) u = v;
    lct.cut(u); par[u] = -1;
}

bool IsSameTree(int u, int v) {
    return lct.IsSameTree(u, v);
}
};

/*
 * TEST MATRIX
 */

#include <cstring>
#include <cstdlib>
#define MAXN 100009
int par[MAXN];

int findroot(int i) {
    if (par[i] >= 0) return findroot(par[i]);
    else return i;
}

bool IsSameSet(int u, int v) {
    return findroot(u) == findroot(v);
}

void link(int u, int v) {
    if (!IsSameSet(u, v)) par[u] = v;
}

void cut(int u) {
    par[u] = -1;
}

int LCA(int u, int v) {
    if (!IsSameSet(u, v)) return -1;
    int du = 0, ul = u;
    while (ul >= 0) {
        du++;
        ul = par[ul];
    }
    int dv = 0, vl = v;
    while (vl >= 0) {
        dv++;
        vl = par[vl];
    }
    while (du > dv) {
        du--;
        u = par[u];
    }
    while (du < dv) {
        dv--;
        v = par[v];
    }
    while (u != v) {
        u = par[u];
        v = par[v];
    }
    return u;
}

int dist(int u, int v) {
    if (!IsSameSet(u, v)) return -1;
    int lca = LCA(u, v);
    int ans = 0;

```

```

while(u != lca) {
    u = par[u];
    ans++;
}
while(v != lca) {
    v = par[v];
    ans++;
}
return ans;
}

bool test(int N, int T) {
    LinkCutTree lct(N);
    memset(&par, -1, sizeof par);
    for (int i = 0, u, v; i < T; i++) {
        int cmd = rand() % 5;
        //if (cmd == 1) cmd = 0; //link only
        if (cmd == 0) {
            u = rand() % N;
            v = rand() % N;
            if (IsSameSet(u, v) != lct.IsSameTree(u, v)) {
                printf("failed on test %d (IsSameSet(%d,%d) LCT:%d BF:%d)\n", i, u, v, lct.IsSameTree(u, v),
                    ↪ IsSameSet(u, v));
                return false;
            }
            if (IsSameSet(u, v) || par[u] >= 0) {
                i--;
                continue;
            }
            //printf("%d son of %d\n", u, v);
            link(u, v);
            lct.link(u, v);
        }
        if (cmd == 1) {
            u = rand() % N;
            if (par[u] == -1) {
                i--;
                continue;
            }
            //printf("%d cut from %d\n", u, par[u]);
            cut(u);
            lct.cut(u);
        }
        if (cmd == 2) {
            u = rand() % N;
            v = rand() % N;
            if (IsSameSet(u, v) != lct.IsSameTree(u, v)) {
                printf("failed on test %d (IsSameSet(%d,%d) LCT:%d BF:%d)\n", i, u, v, lct.IsSameTree(u, v),
                    ↪ IsSameSet(u, v));
                return false;
            }
            if (!IsSameSet(u, v)) {
                i--;
                continue;
            }
            if (LCA(u, v) != lct.LCA(u, v)) {
                printf("failed on test %d (LCA(%d,%d) LCT:%d BF:%d)\n", i, u, v, lct.LCA(u, v), LCA(u, v));
                return false;
            }
        }
        if (cmd == 3) {
            u = rand() % N;
            if (findroot(u) != lct.findroot(u)) {
                printf("failed on test %d (findroot(%d) LCT:%d BF:%d)\n", i, u, lct.findroot(u), findroot(u));
                return false;
            }
        }
        if (cmd == 4) {
            u = rand() % N;

```

```

        v = rand() % N;
        if (!IsSameSet(u, v)) {
            i--;
            continue;
        }
        if (dist(u, v) != lct.dist(u, v)) {
            printf("failed on test %d (dist(%d,%d) LCT:%d BF:%d)\n", i, u, v, lct.dist(u, v), dist(u, v));
            return false;
        }
    }
}
printf("all tests passed\n");
return true;
}

int main() {
    test(10000, 100000);
}

/*
 * SPOJ DYNALCA
 */
/*
int main() {
    int N, M, u, v;
    char str[10];
    scanf("%d %d", &N, &M);
    LinkCutTree lct(N+1);
    for(int i=0; i<M; i++) {
        scanf(" %s", str);
        if(str[1] == 'i') {
            scanf("%d %d", &u, &v);
            lct.link(u, v);
        }
        if(str[1] == 'u') {
            scanf("%d", &u);
            lct.cut(u);
        }
        if (str[1] == 'c') {
            scanf("%d %d", &u, &v);
            printf("%d\n", lct.LCA(u, v));
        }
    }
    //if (test(10000, 5000000)) printf("All tests passed\n");
    return 0;
}*/

/*
 * Codeforces 100960H
 */
/*
UndirectedLinkCutTree ulct;

int main() {
    int N, a, b;
    char op;
    scanf("%d", &N);
    ulct = UndirectedLinkCutTree(N);
    while(true) {
        scanf(" %c", &op);
        switch(op) {
            case 'C':
                scanf("%d %d", &a, &b);
                ulct.link(a, b);
                break;
            case 'D':
                scanf("%d %d", &a, &b);
                ulct.cut(a, b);
                break;
        }
    }
}

```

```

        case 'T':
            scanf("%d %d", &a, &b);
            if (ulct.IsSameTree(a, b)) printf("YES\n");
            else printf("NO\n");
            fflush(stdout);
            break;
        case 'E':
            return 0;
    }
}
return 0;
}*/

```

3.24 Link Cut Tree vô hướng

Mẫu liên quan cho đồ thị tách/split graph trong nguồn Macacário.

Mã nguồn mẫu: Graphs/splitGraph.cpp

```

#define _CRT_SECURE_NO_WARNINGS

#include <vector>
#include <algorithm>
using namespace std;

/*
 * Verifies if a graph is split in  $O(N \log(N))$  and build it in  $O(N)$ . A split graph can
 * be divided in a complete graph and a independent set (graph with 0 edges).
 */
class SplitGraph {

public:

    SplitGraph(int N, vector<int>* g): d(N)
    {
        computeSplit(g, N);
        if (isSplit()) build(N);
    }

    vector<int> getCliqueSet(){
        return K;
    }

    vector<int> getStableSet(){
        return I;
    }

    bool isSplit() {
        return maxClique >= 0;
    }

    int maximumClique() {
        return maxClique;
    }

private:

    int maxClique;
    vector<pair<int, int> > d;
    vector<int> K, I, verts;

    void computeSplit(vector<int>* g, int N) {
        for (int i = 0; i < N; ++i) d[i] = {g[i].size(), i};
        sort(d.begin(), d.end());

        int m = N;
        for (int i = 0; i < N; ++i) {
            if (d[i].first >= N - i - 1) {

```

```

        m = i;
        break;
    }
}

long long lowSum = 0, highSum = 0;

for (int i = 0; i < m; ++i) lowSum += d[i].first;
for (int i = m; i < N; ++i) highSum += d[i].first;
m = N - m;

if (lowSum + m * 1LL * (m - 1) == highSum) {
    maxClique = m;
}
else {
    maxClique = -1;
}
}

void build(int N){
    for (int i = 0; i < N - maxClique; ++i){
        I.push_back(d[i].second);
    }for (int i = N - maxClique; i < N; ++i){
        K.push_back(d[i].second);
    }
}

};

/*
* Example: URI 1974 - Into darkness
*/

#include <cstdio>
#include <cstring>
#include <queue>
#include <set>

#define MAXN 1009
#define MAXM 109
#define MAXS 210000
#define ALFA 2
#define INF 1e9
#define EPS 1e-9
#define FOR(i, N) for(int i = 0; i < N; ++i)
#define FORN(i, j, N) for(int i = j; i < N; ++i)
typedef long long int ll;
typedef pair<int, int> ii;

int N, M;
vector<int> g[MAXN];
vector< pair<int, ii> > edgeList;
char code[MAXN][MAXM];
int len[MAXN];
ll MOD = 1000000009;

class SuffixAutomaton {
    int len[MAXS], link[MAXS], cnt[MAXS];
    int nxt[MAXS][ALFA], sz, last, root;
    int newnode() {
        int x = ++sz;
        len[x] = 0; link[x] = -1; cnt[x] = 1;
        for(int c = 0; c < ALFA; c++) nxt[x][c] = 0;
        return x;
    }
    inline int reduce(char c) { return c - '0'; }
public:
    SuffixAutomaton() { clear(); }
    void clear() {

```

```

        sz = 0;
        root = last = newnode();
    }
    void insert(const char *s) {
        for(int i = 0; s[i]; i++) extend(reduce(s[i]));
    }
    void extend(int c) {
        int cur = newnode(), p;
        len[cur] = len[last] + 1;
        for(p = last; p != -1 && !nxt[p][c]; p = link[p]) {
            nxt[p][c] = cur;
        }
        if (p == -1) link[cur] = root;
        else {
            int q = nxt[p][c];
            if (len[p] + 1 == len[q]) link[cur] = q;
            else {
                int clone = newnode();
                len[clone] = len[p] + 1;
                for(int i = 0; i < ALFA; i++)
                    nxt[clone][i] = nxt[q][i];
                link[clone] = link[q];
                cnt[clone] = 0;
                while(p != -1 && nxt[p][c] == q) {
                    nxt[p][c] = clone;
                    p = link[p];
                }
                link[q] = link[cur] = clone;
            }
        }
        last = cur;
    }

    int longestCommonSubstring(const char *s) {
        int cur = root, curlen = 0, ans = 0;
        for(int i = 0; s[i]; i++) {
            int c = reduce(s[i]);
            while(cur != root && !nxt[cur][c]) {
                cur = link[cur];
                curlen = len[cur];
            }
            if (nxt[cur][c]) {
                cur = nxt[cur][c];
                curlen++;
            }
            if (ans < curlen) ans = curlen;
        }
        return ans;
    }
private:
    int deg[MAXS];
};

class UnionFind {
private:
    vector<int> parent, rank;
public:
    UnionFind(int N) {
        rank.assign(N + 9, 0);
        parent.assign(N + 9, 0);
        for (int i = 0; i < N; i++) parent[i] = i;
    }
    int find(int i) {
        while (i != parent[i]) i = parent[i];
        return i;
    }
    bool isSameSet(int i, int j) {
        return find(i) == find(j);
    }
}

```

```

void unionSet(int i, int j) {
    if (isSameSet(i, j)) return;
    int x = find(i), y = find(j);
    if (rank[x] > rank[y]) parent[y] = x;
    else {
        parent[x] = y;
        if (rank[x] == rank[y]) rank[y]++;
    }
}

};

ll kruskal() {
    ll cost = 0;
    UnionFind UF(N);
    sort(edgeList.begin(), edgeList.end());
    for (auto edge : edgeList) {
        if (!UF.isSameSet(edge.second.first, edge.second.second)) {
            cost += edge.first;
            UF.unionSet(edge.second.first, edge.second.second);
        }
    }
    return cost;
}

int minCost(SplitGraph& sg) {
    SuffixAutomaton sa;
    for (auto& edge : edgeList){
        sa.clear();
        int u = edge.second.first;
        int v = edge.second.second;

        sa.insert(code[u]);
        edge.first = sa.longestCommonSubstring(code[v]);
    }

    return kruskal();
}

int vis[MAXN];

bool isConnected() {
    int count = 1;
    queue<int> inuse;

    inuse.push(0);
    vis[0] = 1;
    while (!inuse.empty()) {
        int u = inuse.front();
        inuse.pop();

        for (int v : g[u]) {
            if (!vis[v]) {
                vis[v] = 1;
                ++count;
                inuse.push(v);
            }
        }
    }
    return count == N;
}

int main() {
    int u, v;

    scanf("%d%d", &N, &M);

    FOR(i, M) {
        scanf("%d%d", &u, &v);
        --u, --v;
    }
}

```

```

        g[u].push_back(v);
        g[v].push_back(u);
        edgeList.push_back({ 0, {u, v} });
    }
    FOR(i, N) {
        scanf("%s", code[i]);
        len[i] = strlen(code[i]);
    }
    auto sg = SplitGraph(N, g);

    if (!isConnected() || !sg.isSplit()) {
        printf(":[\n");
    }
    else {
        int trainees = sg.maximumClique();
        printf(":]\\n%d %d %d\\n", trainees, N - trainees, minCost(sg));
    }
}

```

3.25 Wavelet Tree

Wavelet tree cho truy vấn thứ tự/tần suất.

Mã nguồn mẫu: Trees/wavelet.cpp

```

#include <vector>
#include <algorithm>
using namespace std;

int Lcmp;
bool less(int i) { return i <= Lcmp; }

class WaveletTree {
private:
    vector< vector<int> > ml;
    vector<int> arr;
    int sig, size;
#define left(p) (p << 1)
#define right(p) ((p << 1) + 1)

    void build(int u, int l, int r, int lo, int hi, int* A) {
        if (lo == hi) return;
        int mid = (lo + hi) / 2;
        Lcmp = mid;
        ml[u].reserve(r-l+2);
        ml[u].push_back(0);
        for (int i=l; i<=r; i++) {
            ml[u].push_back(ml[u].back() + (A[i]<=Lcmp));
        }
        int p = (int)(stable_partition(A+l, A+r+1, less) - A);
        build(left(u), l, p-1, lo, mid, A);
        build(right(u), p, r, mid+1, hi, A);
    }

    int rank(int u, int lo, int hi, int q, int i) {
        if (lo == hi) return i;
        int mid = (lo + hi) / 2, ri = ml[u][i];
        if (q <= mid) return rank(left(u), lo, mid, q, ri);
        else return rank(right(u), mid+1, hi, q, i - ri);
    }

    int quantile(int u, int lo, int hi, int i, int j, int k) {
        if (lo == hi) return lo;
        int mid = (lo + hi) / 2;
        int ri = ml[u][i-1], rj = ml[u][j], c = rj - ri;
        if (k <= c) return quantile(left(u), lo, mid, ri+1, rj, k);
        else return quantile(right(u), mid+1, hi, i-ri, j-rj, k-c);
    }

```



```

    }

    int range(int u, int lo, int hi, int i, int j, int a, int b) {
        if (lo > b || hi < a) return 0;
        if (b >= hi && lo >= a) return j-i;
        int mid = (lo + hi) / 2;
        int ri = ml[u][i], rj = ml[u][j];
        int c1 = range(left(u), lo, mid, ri, rj, a, b);
        int c2 = range(right(u), mid+1, hi, i-ri, j-rj, a, b);
        return c1 + c2;
    }

    void swap(int u, int lo, int hi, int v1, int v2, int i) {
        if (lo == hi) return;
        int mid = (lo + hi) / 2;
        if (v1 <= mid) {
            if (v2 > mid) ml[u][i]--;
            else swap(left(u), lo, mid, v1, v2, ml[u][i]);
        }
        else {
            if (v2 <= mid) ml[u][i]++;
            else swap(right(u), mid+1, hi, v1, v2, i-ml[u][i]);
        }
    }

public:
    WaveletTree() {}
    WaveletTree(int* begin, int* end, int _sig) {
        sig = _sig;
        size = (int)(end-begin);
        ml.resize(4*size);
        arr = vector<int>(begin, end);
        build(1, 0, size-1, 0, sig, &arr[0]);
        arr = vector<int>(begin, end);
    }

    int rank(int i, int q) { return rank(1, 0, sig, q, i+1); }
    int rank(int i, int j, int q) { return rank(j, q) - rank(i-1, q); }
    int quantile(int i, int j, int k) { return quantile(1, 0, sig, i+1, j+1, k); }
    int range(int i, int j, int a, int b) { return range(1, 0, sig, i, j+1, a, b); }
    void swap(int i) {
        if (i >= size-1) return;
        swap(1, 0, sig, arr[i], arr[i+1], i+1);
        std::swap(arr[i], arr[i+1]);
    }
};

/*
 * TEST MATRIX
 */

#include <cstdio>
#define DEBUG false
int testarr[100009];
WaveletTree wt;

bool test(int N, int sig, int nTest) {
    for(int t = 1; t <= nTest; t++) {
        if (DEBUG) printf("test array:");
        for(int i=0; i<N; i++) {
            testarr[i] = rand()%(sig+1);
            if (DEBUG) printf(" %d", testarr[i]);
        }
        if (DEBUG) printf("\n");

        if (DEBUG) printf("building Wavelet Tree\n");
        wt = WaveletTree(testarr, testarr+N, sig);
        if (DEBUG) printf("Wavelet Tree built\n");
    }
}

```

```

for(int j=1; j<=nTest; j++) {

    //execute swaps
    for(int s=0; s<nTest; s++) {
        int i = rand()%(N-1);
        swap(testarr[i], testarr[i+1]);
        wt.swap(i);
    }

    //test rank
    int a = rand()%N;
    int b = rand()%N;
    int q = rand()%(sig+1);
    if (a > b) swap(a, b);
    int ans = 0;
    if (DEBUG) printf("testarr[%d,%d]:", a, b);
    for(int i=a; i<=b; i++) {
        if (DEBUG) printf(" %d", testarr[i]);
        if (testarr[i] == q) ans++;
    }
    int wans = wt.rank(a, b, q);
    if (DEBUG) printf(" ans = %d wans = %d\n", ans, wans);
    if (ans != wans) {
        printf("Failed on rank test %d, rank(%d,%d,%d), ans = %d, wans = %d\n", j, a, b, q, ans, wans);
        return false;
    }

    //test quantile
    a = rand()%N;
    b = rand()%N;
    if (a > b) swap(a, b);
    int n = b-a+1;
    int k = 1 + rand()%n;
    vector<int> st(&testarr[a], &testarr[b] + 1);
    sort(st.begin(), st.end());
    if (DEBUG) printf("testarr[%d,%d]:", a, b);
    for(int i=0; i<n; i++) {
        if (DEBUG) printf(" %d", st[i]);
    }
    ans = st[k-1];
    wans = wt.quantile(a, b, k);
    if (DEBUG) printf(" ans = %d wans = %d\n", ans, wans);
    if (ans != wans) {
        printf("Failed on quantile test %d, quantile(%d,%d,%d), ans = %d, wans = %d\n", j, a, b, k, ans,
            ↪ wans);
        return false;
    }

    //test range
    a = rand()%N;
    b = rand()%N;
    if (a > b) swap(a, b);
    int l = rand()%(sig+1);
    int r = rand()%(sig+1);
    if (l > r) swap(l, r);
    if (DEBUG) printf("testarr[%d,%d]:", a, b);
    ans = 0;
    for(int i=a; i<=b; i++) {
        if (DEBUG) printf(" %d", testarr[i]);
        if (testarr[i] >= l && testarr[i] <= r) ans++;
    }
    wans = wt.range(a, b, l, r);
    if (DEBUG) printf(" ans = %d wans = %d\n", ans, wans);
    if (ans != wans) {
        printf("Failed on range test %d, range(%d,%d,%d,%d), ans = %d, wans = %d\n", j, a, b, l, r, ans,
            ↪ wans);
        return false;
    }
}
}

```

```

    }
    printf("all tests passed\n");
    return true;
}

int main() {
    test(1000, 25, 100);
    return 0;
}

/*
 * SPOJ ILKQUERY
 */

/*#include <cstdio>
#define MAXN 100009
#include <map>
map<int, int> x2id;
int id2x[MAXN], arr[MAXN];

int main() {
    int N, Q;
    scanf("%d %d", &N, &Q);
    for(int i=0; i<N; i++) {
        scanf("%d", &arr[i]);
        x2id[arr[i]] = 0;
    }

    int sig = -1;
    for(map<int, int>::iterator it = x2id.begin(); it != x2id.end(); it++) {
        it->second = ++sig;
        id2x[it->second] = it->first;
    }
    for(int i=0; i<N; i++) arr[i] = x2id[arr[i]];

    WaveletTree wt(arr, arr+N, sig);

    int k, i, l;
    while(Q-->0) {
        scanf("%d %d %d", &k, &i, &l);
        int el = wt.quantile(0, i, k);
        //printf("%d-th element in [%d,%d] is %d\n", k, 0, i, id2x[el]);
        if (wt.rank(N-1, el) < l) {
            printf("-1\n");
            continue;
        }
        int lo = -1;
        int hi = N;
        while(hi > lo + 1) {
            int mid = (hi + lo) / 2;
            if (wt.rank(mid, el) < l) lo = mid;
            else hi = mid;
        }
        printf("%d\n", hi);
    }
    return 0;
}*/

/*
 * SPOJ ILKQUERYIIII
 */

/*#define MAXN 1000009
#include <map>
#include <cstdio>
map<int, int> x2id;
int id2x[MAXN], arr[MAXN];

int main() {

```

```

int N, Q;
scanf("%d %d", &N, &Q);
for(int i=0; i<N; i++) {
    scanf("%d", &arr[i]);
    x2id[arr[i]] = 0;
}

int sig = -1;
for(map<int, int>::iterator it = x2id.begin(); it != x2id.end(); it++) {
    it->second = ++sig;
    id2x[it->second] = it->first;
}
for(int i=0; i<N; i++) arr[i] = x2id[arr[i]];

WaveletTree wt(arr, arr+N, sig);

int k, i, l, t;
while(Q-->0) {
    scanf("%d", &t);
    if (t == 0) {
        scanf("%d %d %d", &i, &l, &k);
        int el = wt.quantile(0, i, k);
        //printf("%d-th element in [%d,%d] is %d\n", k, 0, i, id2x[el]);
        if (wt.rank(N-1, el) < l) {
            printf("-1\n");
            continue;
        }
        int lo = -1;
        int hi = N;
        while(hi > lo + 1) {
            int mid = (hi + lo) / 2;
            if (wt.rank(mid, el) < l) lo = mid;
            else hi = mid;
        }
        printf("%d\n", hi);
    }
    else {
        scanf("%d", &i);
        wt.swap(i);
    }
}
return 0;
}*/

```

3.26 Heavy-Light Decomposition

Phân rã nặng-nhẹ.

Mã nguồn mẫu: Trees/hld.cpp

```

#include <vector>
#include <algorithm>
using namespace std;
#define MAXN 400009

#include <cstdio>

/*
 * Heavy-Light Decomposition
 */

vector<int> adjList[MAXN];
int in[MAXN], out[MAXN], up[MAXN];
int par[MAXN], pre[MAXN], sz[MAXN];

void dfsSize(int u) {
    int s = sz[u] = 0;

```

```

        for(int i = 0; i < (int)adjList[u].size(); i++) {
            int &v = adjList[u][i]; //don't forget &
            if (v == par[u]) continue;
            par[v] = u;
            dfssize(v);
            s += sz[v];
            if(sz[v] > sz[adjList[u][0]])
                swap(v, adjList[u][0]);
        }
        sz[u] = 1 + s;
    }
}

void dfshld(int u, int &t) {
    pre[t] = u;
    in[u] = t++;
    for(int i = 0; i < (int)adjList[u].size(); i++) {
        int v = adjList[u][i];
        if (v == par[u]) continue;
        up[v] = (i == 0 ? up[u] : v);
        dfshld(v, t);
    }
    out[u] = t-1;
}

void hld(int root) {
    par[root] = -1;
    dfssize(root);
    int t = 0;
    up[root] = root;
    dfshld(root, t);
    //create DS over values sorted by array pre
}

int LCA(int u, int v) {
    while(up[u] != up[v]) {
        if (in[u] > in[v]) u = par[up[u]];
        //query DS for subarray [in[up[u]], in[u]]
        else v = par[up[v]];
        //query DS for subarray [in[up[v]], in[v]]
    }
    return in[u] > in[v] ? v : u;
    //query DS for subarray [in[v], in[u]] or [in[u], in[v]]
}

/*
 * KMP Automaton
 */

#include <cstring>
#define MAXM 109

struct trans {
    int m, to[MAXM], match[MAXM];
    trans(int _m = -1) : m(_m) {}
};

trans operator +(trans a, trans b) {
    if (b.m < 0) return a;
    if (a.m < 0) return b;
    //assert(a.m == b.m);
    trans c(a.m);
    for(int i = 0; i < c.m; i++) {
        c.to[i] = b.to[a.to[i]];
        c.match[i] = a.match[i] + b.match[a.to[i]];
    }
    return c;
}

struct KMP {

```

```

char P[MAXM];
int b[MAXM], m;
void build(const char* _P) {
    m = strlen(_P);
    strcpy(P, _P);
    memset(&b, -1, sizeof b);
    for(int i = 0, j = -1; i < m; i++) {
        while (j >= 0 && P[i] != P[j]) j = b[j];
        b[++i] = ++j;
    }
}

trans get(char c) {
    trans ans(m);
    for(int i = 0, j, cnt; i < m; i++) {
        j = i; cnt = 0;
        while(j >= 0 && c != P[j]) j = b[j];
        j++;
        if (j == m) cnt++, j = b[j];
        ans.to[i] = j;
        ans.match[i] = cnt;
    }
    return ans;
}

};

/*
 * Codeforces 101908H
 */

char P[MAXN];
KMP kmp;
char col[MAXN];

class SegmentTree {
    trans st[2][MAXN];
    int n;
    trans query(int u, int l, int r, int a, int b, int t) {
        if (l > b || r < a) return trans(-1);
        if (a <= l && r <= b) return st[t][u];
        int m = (l + r) / 2;
        trans p1 = query(2*u, l, m, a, b, t);
        trans p2 = query(2*u+1, m+1, r, a, b, t);
        return t == 0 ? p1 + p2 : p2 + p1;
    }
public:
    SegmentTree() {}
    void build(int sz) {
        for (n = 1; n < sz; n <= 1);
        for(int i=0; i<sz; i++) {
            st[0][i+n] = st[1][i+n] = kmp.get(col[pre[i]]);
        }
        for(int i=n-1; i; i--) {
            st[0][i] = st[0][2*i] + st[0][2*i+1];
            st[1][i] = st[1][2*i+1] + st[1][2*i];
        }
    }
    void update(int i, char c) {
        st[0][i+n] = st[1][i+n] = kmp.get(c);
        for (i += n, i >= 1; i; i >= 1) {
            st[0][i] = st[0][2*i] + st[0][2*i+1];
            st[1][i] = st[1][2*i+1] + st[1][2*i];
        }
    }
    trans query(int l, int r, int t) {
        return query(1, 0, n-1, l, r, t);
    }
};

SegmentTree st;

```

```

trans get(int u, int v) {
    trans left, mid, right;
    while(up[u] != up[v]) {
        if (in[u] > in[v]) {
            trans cur = st.query(in[up[u]], in[u], 1);
            left = left + cur;
            u = par[up[u]];
        }
        else {
            trans cur = st.query(in[up[v]], in[v], 0);
            right = cur + right;
            v = par[up[v]];
        }
    }
    if (in[u] > in[v]) mid = st.query(in[v], in[u], 1);
    else mid = st.query(in[u], in[v], 0);
    return (left + mid) + right;
}

int main() {
    int n, q;
    scanf("%d %d", &n, &q);
    scanf(" %s %s", P, col+1);
    for(int i = 0; i < n-1; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adjList[u].push_back(v);
        adjList[v].push_back(u);
    }
    hld(1);
    kmp.build(P);
    st.build(n);
    while(q --> 0) {
        int op;
        scanf("%d", &op);
        if (op == 1) {
            int u, v;
            scanf("%d %d", &u, &v);
            trans ans = get(u, v);
            printf("%d\n", ans.match[0]);
        }
        else {
            int u;
            char x;
            scanf("%d %c", &u, &x);
            col[u] = x;
            st.update(in[u], x);
        }
    }
    return 0;
}

```

3.27 Centroid Decomposition

Phân rã trọng tâm.

Mã nguồn mẫu: Trees/centroid.cpp

```

#include <cstring>
#include <vector>
#include <iostream>
using namespace std;
#define MAXN 500009
#define INF (1LL<<60)

typedef long long ll;

```

```

typedef pair<ll, int> ii;
int clevel[MAXN], cpar[MAXN], csize[MAXN];
vector<int> csons[MAXN];
vector<ii> adjList[MAXN];
int N, K;

int subsize(int u, int p) {
    csize[u]=1;
    for(int i=0; i<(int)adjList[u].size(); i++) {
        int v = adjList[u][i].second;
        if (v != p && clevel[v] < 0)
            csize[u] += subsize(v, u);
    }
    return csize[u];
}

int findcentroid(int u, int p, int nn) {
    for(int i=0; i<(int)adjList[u].size(); i++) {
        int v = adjList[u][i].second;
        if (v != p && clevel[v] < 0 && csize[v] > nn/2)
            return findcentroid(v, u, nn);
    }
    return u;
}

int decompose(int root, int par) {
    subsize(root, -1);
    int u = findcentroid(root, -1, csize[root]);
    cpar[u] = par;
    clevel[u] = par >= 0 ? clevel[par]+1 : 0;
    csize[u] = 1;
    for(int i=0; i<(int)adjList[u].size(); i++) {
        int v = adjList[u][i].second;
        if (v != par && clevel[v] < 0) {
            v = decompose(v, u);
            csons[u].push_back(v);
            csize[u] += csize[v];
        }
    }
    return u;
}

int centroiddecomposition(int root) {
    memset(&clevel, -1, sizeof clevel);
    for(int i=0; i<=N; i++) csons[i].clear();
    return decompose(root, -1);
}

/*
 * Codeforces 101040J
 */

#include <cstdio>

vector<ll> cost(MAXN), subs(MAXN);
int limit;
ll depth[MAXN];

void dfs(int u, int par, int h) {
    while(cost.size() < h+1) cost.push_back(INF);
    cost[h] = min(cost[h], depth[u]);
    for(int i=0; i<(int)adjList[u].size(); i++) {
        int v = adjList[u][i].second;
        ll w = adjList[u][i].first;
        if (clevel[v] <= limit || v == par) continue;
        depth[v] = depth[u] + w;
        dfs(v, u, h+1);
    }
}

ll merge() {
    ll ans = INF;

```



```

while(subs.size() < cost.size()) subs.push_back(INF);
for(int i=0; i<(int)cost.size(); i++) {
    int j = K-i;
    if (j >= 0 && j < (int)subs.size()) ans = min(ans, subs[j] + cost[i]);
}
for(int i=0; i<(int)cost.size(); i++) {
    subs[i] = min(subs[i], cost[i]);
}
return ans;
}

ll solve(int u) {
    ll ans = INF;
    for(int i=0; i<(int)csons[u].size(); i++) {
        int v = csons[u][i];
        ans = min(ans, solve(v));
    }
    subs.clear();
    subs.push_back(0);
    limit = clevel[u];
    for(int i=0; i<(int)adjList[u].size(); i++) {
        int v = adjList[u][i].second;
        ll w = adjList[u][i].first;
        if (clevel[v] <= limit) continue;
        depth[v] = depth[u] + w;
        cost.clear();
        dfs(v, u, 1);
        ans = min(ans, merge());
    }
    return ans;
}

int main() {
    scanf("%d %d", &N, &K);
    int P;
    ll C, ans;
    for(int i=2; i<=N; i++) {
        scanf("%d %I64d", &P, &C);
        adjList[i].push_back(ii(C, P));
        adjList[P].push_back(ii(C, i));
    }
    int root = centroiddecomposition(1);
    ans = solve(root);
    if (ans < INF) printf("%I64d\n", ans);
    else printf("-1\n");
    return 0;
}

```

3.28 Interval Tree

Cây khoảng.

Mã nguồn mẫu: Trees/intervaltree.cpp

```

#include <set>
#include <vector>
#include <algorithm>
using namespace std;
#define MOD 998244353
#define MAXN 200009
typedef pair<int, int> ii;

/*
 * Lazy propagation segment tree
 */

class SegmentTree {
private:

```

```

    vector<int> st;
    vector<ii> lazy;
    int size;
#define parent(p) (p >> 1)
#define left(p) (p << 1)
#define right(p) ((p << 1) + 1)
    ii aggregate(ii a, ii b) {
        return ii((a.first*1ll*b.first)%MOD, (a.second*1ll*b.first + b.second)%MOD);
    }
    void push(int p, int l, int r) {
        st[p] = (lazy[p].first*1ll*st[p] + (r - l + 1ll)*lazy[p].second)%MOD;
        if (l != r) {
            lazy[right(p)] = aggregate(lazy[right(p)], lazy[p]);
            lazy[left(p)] = aggregate(lazy[left(p)], lazy[p]);
        }
        lazy[p] = ii(1, 0);
    }
    void update(int p, int l, int r, int a, int b, ii k) {
        push(p, l, r);
        if (a > r || b < l) return;
        else if (l >= a && r <= b) {
            lazy[p] = k; push(p, l, r);
        }
        else {
            update(left(p), l, (l + r) / 2, a, b, k);
            update(right(p), (l + r) / 2 + 1, r, a, b, k);
            st[p] = (st[left(p)] + st[right(p)]) % MOD;
        }
    }
    int query(int p, int l, int r, int a, int b) {
        push(p, l, r);
        if (a > r || b < l) return 0;
        if (l >= a && r <= b) return st[p];
        int p1 = query(left(p), l, (l + r) / 2, a, b);
        int p2 = query(right(p), (l + r) / 2 + 1, r, a, b);
        return (p1 + p2) % MOD;
    }
public:
    SegmentTree(int _size) {
        size = _size;
        st.assign(4 * size, 0);
        lazy.assign(4 * size, ii(1, 0));
    }
    int query(int a, int b) { return query(1, 0, size - 1, a, b); }
    void update(int a, int b, ii k) { update(1, 0, size - 1, a, b, k); }
};

SegmentTree st(MAXN);

/*
 * Interval Tree
 */

struct node {
    int l, r, x;
    node(int _l, int _r, int _x = 0) : l(_l), r(_r), x(_x) { }
    void update(int dx) { x = min(x+dx, 2); }
};

bool operator < (node a, node b) {
    if (a.l != b.l) return a.l < b.l;
    return a.r < b.r;
}

class IntervalTree {
    set<node> tree;
    void split(int i) {
        set<node>::iterator it = --tree.upper_bound(node(i, 0));
        node t = *it;

```

```

        if (t.l == i) return;
        tree.erase(it);
        tree.insert(node(t.l, i-1, t.x));
        tree.insert(node(i, t.r, t.x));
    }
public:
    IntervalTree() { tree.insert(node(0, MAXN, 0)); }
    vector<node> get(int l, int r, bool update, int dx) {
        split(l); split(r+1);
        set<node>::iterator it = tree.lower_bound(node(l, 0));
        vector<node> q;
        while(it != tree.end() && it->l <= r) {
            node t = *it;
            it = tree.erase(it);
            if (update) t.update(dx);
            if (q.empty() || q.back().x != t.x) q.push_back(t);
            else q.back().r = t.r;
        }
        for(int i = 0; i < int(q.size()); i++) {
            tree.insert(q[i]);
        }
        return q;
    }
};

/*
 * Codeforces 981G
 */

#include <cstdio>

IntervalTree it[MAXN];

int main() {
    int n, q;
    scanf("%d %d", &n, &q);
    while(q --> 0) {
        int t, l, r, x;
        scanf("%d", &t);
        if (t == 1) {
            scanf("%d %d %d", &l, &r, &x);
            vector<node> q = it[x].get(l, r, true, 1);
            for(int i = 0; i < int(q.size()); i++) {
                if (q[i].x == 1) st.update(q[i].l, q[i].r, ii(1, 1));
                else st.update(q[i].l, q[i].r, ii(2, 0));
            }
        }
        else {
            scanf("%d %d", &l, &r);
            printf("%d\n", st.query(l, r));
        }
    }
    return 0;
}

```

3.29 Xor Trie

Trie nhị phân hỗ trợ tìm phần tử tối ưu xor dưới giới hạn.

Mã nguồn mẫu: Trees/xortrie.cpp

```

/*
 * Xor Trie
 */
#define MAXS 600000009
int l[MAXS], r[MAXS], cnt = 0;

```

```

class XorTrie {
    int digits = 0, root, ans, limit;
    int newnode() {
        l[cnt] = r[cnt] = -1;
        cnt++;
        return cnt-1;
    }
    bool search(int u, int h, int num, int cur) {
        if (u == -1 || cur > limit) return false;
        if (h == -1) {
            ans = cur; return true;
        }
        if (num & (1<<h)) {
            if (search(l[u], h-1, num, cur)) return true;
            if (search(r[u], h-1, num, cur | (1<<h))) return true;
        }
        else {
            if (search(r[u], h-1, num, cur | (1<<h))) return true;
            if (search(l[u], h-1, num, cur)) return true;
        }
        return false;
    }
public:
    XorTrie(int _digits = 20) : digits(_digits) {
        root = newnode();
    }
    void insert(int num) {
        int u = root;
        for(int i = digits-1; i >= 0; i--) {
            if (num & (1<<i)) {
                if (r[u] == -1) r[u] = newnode();
                u = r[u];
            }
            else {
                if (l[u] == -1) l[u] = newnode();
                u = l[u];
            }
        }
    }
    int search(int _limit, int num) {
        limit = _limit; ans = -1;
        if (!search(root, digits-1, num, 0)) return -1;
        return ans;
    }
};

/*
 * Codeforces 979D
 */

#include <cstdio>
#define MAXN 100309
XorTrie tree[MAXN];

int main() {
    int q;
    scanf("%d", &q);
    while (q --> 0) {
        int t, u, x, k, s;
        scanf("%d", &t);
        if (t == 1) {
            scanf("%d", &u);
            for(int i = 1; i*i <= u; i++) {
                if (u % i == 0) {
                    tree[i].insert(u);
                    if (i*i < u) tree[u/i].insert(u);
                }
            }
        }
    }
}

```

```

        else if (t == 2) {
            scanf("%d %d %d", &x, &k, &s);
            int ans = -1;
            ans = tree[k].search(s-x, x);
            if (x % k != 0) ans = -1;
            printf("%d\n", ans);
        }
    }
    return 0;
}

```

3.30 Convex Hull Trick

Với truy vấn min/max đường thẳng theo x ; có phiên bản $O(\log n)$ và $O(1)$ khi x tăng.

Mã nguồn mẫu: Data Structures/convexhulltrick.cpp

```

#include <cstdio>
#include <vector>
#define INF (1<<30)
#define MAXN 1009
using namespace std;

typedef long long int ll;

/*
 * Partially Dynamic Convex Hull Trick
 */

class CHTrick {
private:
    vector<ll> m, n;
    vector<double> p;
    bool maxCH; int i;
public:
    CHTrick(bool mxch) { maxCH = mxch; i = 0; }
    void clear() { m.clear(); n.clear(); p.clear(); }
    double inter(double nm, double nn, double lm, double ln) {
        return (ln - nn) / (nm - lm);
    }
    void push(ll nm, ll nn) {
        while (!p.empty()) {
            if (nm == m.back() && maxCH && nn <= n.back()) return;
            if (nm == m.back() && !maxCH && nn >= n.back()) return;
            double x = inter(nm, nn, m.back(), n.back());
            if (nm != m.back() && p.back() < x) break;
            m.pop_back(); n.pop_back(); p.pop_back();
        }
        p.push_back(p.empty() ? -INF : inter(nm, nn, m.back(), n.back()));
        m.push_back(nm); n.push_back(nn);
        if (i >= p.size()) i = p.size()-1;
    }
    ll query(ll x) {
        if (p.empty()) return (maxCH ? -1 : 1)*INF;
        ll r = p.size() - 1, l = 0, mid;
        if (x >= p[r]) return m[r] * x + n[r];
        while (r > l + 1) {
            mid = (r + l) / 2;
            if (x < p[mid]) r = mid;
            else l = mid;
        }
        return m[l] * x + n[l];
    }
    ll query_q(ll x) {
        while (p[i] > x) i--;
        while(i+1 < p.size() && p[i+1] <= x) i++;
        return m[i] * x + n[i];
    }
};

```

```

    }
};

/*
 * URI 1282
 */

ll w[MAXN], x[MAXN], A[MAXN], B[MAXN], dp[MAXN][MAXN];

int main(){
    int N, K;
    CHTrick cht(false);
    while(scanf("%d %d", &N, &K)!=EOF) {
        for(int i=0; i<N; i++){
            scanf("%lld %lld", x+i, w+i);
            A[i] = w[i] + (i>0 ? A[i-1] : 0);
            B[i] = w[i]*x[i] + (i>0 ? B[i-1] : 0);
            dp[i][1] = x[i]*A[i] - B[i];
        }
        for(int k=2; k<=K; k++){
            dp[0][k] = 0;
            cht.clear();
            for(int i=1; i<N; i++){
                cht.push(-A[i-1], dp[i-1][k-1]+B[i-1]);
                //dp[i][k] = x[i]*A[i] - B[i] + cht.query(x[i]);
                dp[i][k] = x[i]*A[i] - B[i] + cht.query_q(x[i]);
            }
        }
        printf("%lld\n", dp[N-1][K]);
    }
    return 0;
}

```

3.31 Convex Hull Trick động và tích vô hướng lớn nhất

Thêm đường thẳng bất kỳ và truy vấn trên phân số; cần C++11.

Mã nguồn mẫu: Data Structures/dynamiccht.cpp

```

#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll nu, de;
struct line {
    ll m, b; bool is_query;
    mutable function<const line *(> succ;
    line(ll _m, ll _b, bool iq = false) :
        m(_m), b(_b), is_query(iq) {}
    bool operator<(const line &o) const {
        if (!o.is_query) return m < o.m;
        const line *s = succ();
        if (!s) return 0;
        return (b - s->b) * de < (s->m - m) * nu;
    }
};

struct DynamicHull : public multiset<line> {
    bool bad(iterator y) {
        auto z = next(y);
        if (y == begin()) {
            if (z == end()) return 0;
            return y->m == z->m && y->b <= z->b;
        }
        auto x = prev(y);
        if (z == end()) return y->m == x->m && y->b <= x->b;
        return (x->b - y->b) * (z->m - y->m) >= (y->b - z->b) * (y->m - x->m);
    }
};

```

```

    }
    void insert_line(ll m, ll b) {
        auto y = insert(line(m, b));
        y->succ = [=] { return next(y) == end() ? 0 : &*next(y); };
        if (bad(y)) {
            erase(y); return;
        }
        while (next(y) != end() && bad(next(y))) erase(next(y));
        while (y != begin() && bad(prev(y))) erase(prev(y));
    }
    line query(ll a, ll b) {
        if (b < 0) a = -a, b = -b;
        nu = a; de = b;
        return *lower_bound(line(0, 0, true));
    }
};

/*
 * Codeforces 101707H
 */

#define MAXN 200309
#define MAXM 900009
#define MOD 1000000007
#define INF 0x3f3f3f3f
#define INFL 0x3f3f3f3f3f3f3f3f
#define EPS 1e-9
#define PI 3.141592653589793238462643383279502884
#define FOR(x,n) for(int x=0; (x)<int(n); (x)++)
#define FORL(x,n) for(int x=1; (x)<=int(n); (x)++)
#define REP(x,n) for(int x=int(n)-1; (x)>=0; (x)--)
#define REPL(x,n) for(int x=(n); (x)>0; (x)--)
#define pb push_back
#define pf push_front
#define fi first
#define se second
#define mp make_pair
#define all(x) x.begin(), x.end()
#define mset(x,y) memset(&x, (y), sizeof(x));
using namespace std;
typedef vector<int> vi;
typedef pair<int, int> ii;
typedef long long ll;
typedef unsigned long long ull;
typedef long double ld;
typedef unsigned int uint;

int n, m;
ll a[MAXN], b[MAXN], c[MAXN];
ll x[MAXN], y[MAXN];
ll minx = INF, maxx = -INF;
DynamicHull maxhull, minhull;

ll query(ll a, ll b) {
    if (b == 0) {
        if (a > 0) return maxx*a;
        else return minx*a;
    }
    else if (b > 0) {
        line l = maxhull.query(a, b);
        return l.m*a + l.b*b;
    }
    else {
        line l = minhull.query(a, b);
        return -a*l.m + -b*l.b;
    }
}

int main() {

```

```

scanf("%d %d", &n, &m);
FOR(i, n) scanf("%lld %lld %lld", &a[i], &b[i], &c[i]);
FOR(i, m) {
    scanf("%lld %lld", &x[i], &y[i]);
    minx = min(x[i], minx);
    maxx = max(x[i], maxx);
    maxhull.insert_line(x[i], y[i]);
    minhull.insert_line(-x[i], -y[i]);
}
vi ans;
FOR(i, n) {
    ll c1, c2;
    c[i] = -c[i];
    c1 = query(a[i], b[i]);
    c2 = -query(-a[i], -b[i]);
    bool ok = false;
    if (c1 == c[i] || c2 == c[i]) ok = true;
    if (c1 < c[i] && c2 > c[i]) ok = true;
    if (c1 > c[i] && c2 < c[i]) ok = true;
    if (ok) ans.pb(i+1);
}
printf("%u\n", ans.size());
for(int i : ans) printf("%d ", i);
printf("\n");
return 0;
}

```

3.32 Van Emde Boas Tree

Hỗ trợ tập con của $[0, \sigma)$, kế tiếp và trước đó trong $O(\log \log \sigma)$.

Mã nguồn mẫu: Trees/veb.cpp

```

#include <vector>
#include <cstdio>
using namespace std;

/*
 * Van Emde Boas Tree
 */

class VEB {
private:
    VEB* ups;
    vector<VEB*> los;
    int minx, maxx, k, tk;
#define upper(x) ((x)>>tk)
#define lower(x) ((x)&((1<<tk)-1))
#define index(x, y) (((x)<<tk)^(y))
public:
    VEB(int _k = 20) : k(_k), tk(k>>1) {
        minx = -1, maxx = -1;
        ups = NULL;
        if (k == 1) return;
        ups = new VEB(k-tk);
        los.resize(1<<(k-tk), NULL);
    }
    ~VEB() {
        delete ups;
        for(int i = 0; i < (int)los.size(); i++)
            if (los[i]) delete los[i];
    }
    int min() { return minx; }
    int max() { return maxx; }
    bool count(int x) {
        int up = upper(x), lo = lower(x);
        if (x == minx || x == maxx) return true;
    }

```



```

        if (k == 1 || !los[up]) return false;
        return los[up]->count(lo);
    }
    int succ(int x) {
        int up = upper(x), lo = lower(x);
        if (k == 1) return x == 0 && maxx == 1 ? 1 : -1;
        if (minx != -1 && x < minx) return minx;
        int mlo = los[up] ? los[up]->max() : -1;
        if (mlo != -1 && lo < mlo)
            return index(up, los[up]->succ(lo));
        up = ups->succ(up);
        if (up == -1) return -1;
        return index(up, los[up]->min());
    }
    int precc(int x) {
        int up = upper(x), lo = lower(x);
        if (k == 1) return x == 1 && minx == 0 ? 0 : -1;
        if (maxx != -1 && x > maxx) return maxx;
        int mlo = los[up] ? los[up]->min() : -1;
        if (mlo != -1 && lo > mlo)
            return index(up, los[up]->precc(lo));
        up = ups->precc(up);
        if (up == -1)
            return minx != -1 && x > minx ? minx : -1;
        return index(up, los[up]->max());
    }
}
void insert(int x) {
    if (minx == -1) {
        minx = maxx = x; return;
    }
    if (x < minx) swap(x, minx);
    if (x > maxx) maxx = x;
    if (k == 1 || x == minx) return;
    int up = upper(x), lo = lower(x);
    if (!los[up] || los[up]->min() == -1) {
        ups->insert(up);
        los[up] = new VEB(tk);
    }
    los[up]->insert(lo);
}
int removemin() {
    if (minx == -1) return -1;
    int ans = minx;
    if (minx == maxx) {
        minx = maxx = -1; return ans;
    }
    if (k == 1) { minx = maxx; return ans; }
    int up = ups->min();
    minx = index(up, los[up]->removemin());
    if (los[up]->min() == -1) ups->removemin();
    return ans;
}
void erase(int x) {
    if (x == -1) return;
    int up = upper(x), lo = lower(x);
    if (x == minx) removemin();
    else if (k == 1 && x == maxx) maxx = minx;
    else {
        los[up]->erase(lo);
        if (los[up]->min() == -1) ups->erase(up);
        if (x == maxx && los[up]->max() != -1)
            maxx = index(up, los[up]->max());
        else if (x == maxx) maxx = precc(x);
    }
}
}
};

/*
* TEST MATRIX

```

```

*/

/*
void print(int h) {
    for(int i = 0; i < h; i++) printf("| ");
    printf("min: %d max: %d\n", minx, maxx);

    for(int i = 0; i < h; i++) printf("| ");
    printf("ups:%s\n", ups ? "" : " NULL");
    if (ups) ups->print(h+1);

    for(int j = 0; j < (int)los.size(); j++) {
        for(int i = 0; i < h; i++) printf("| ");
        printf("los[%d]:%s\n", j, los[j] ? "" : " NULL");
        if (los[j]) los[j]->print(h+1);
    }
}

*/
#include <set>
#include <cstdlib>
#include <ctime>

bool test(int ntests, int k) {
    srand(time(NULL));
    VEB veb(k);
    set<int> control;
    vector<int> present;
    for(int test = 1; test <= ntests; test++) {
        //printf("test %d:\n", test);
        int op = rand()%(2*control.size() > (1<<(k-1)) ? 2 : 3);
        if (control.size() == (1u<<k) || (op == 0 && !control.empty())) {
            int i, x;
            if (rand() % 2) {
                i = rand()%present.size();
                x = present[i];
                //printf("removing %d...\n", x);
                veb.erase(x);
                //printf("removed %d\n", x);
            }
            else {
                x = *control.begin();
                i = 0;
                while(present[i] != x) i++;
                //printf("removing %d (min)...\n", x);
                int y = veb.removemin();
                //printf("removed %d (min)\n", y);
                if (x != y) {
                    printf("failed test %d, removed min %d, but found %d\n", test, x, y);
                    return false;
                }
            }
            control.erase(x);
            swap(present[i], present.back());
            present.pop_back();
        }
        else {
            int x;
            do {
                x = rand()%(1<<k);
            } while(control.count(x));
            control.insert(x);
            //printf("adding %d...\n", x);
            veb.insert(x);
            present.push_back(x);
            //printf("added %d\n", x);
        }
    }
    //printf("-->\n");
    //veb.print(1);
    //printf("<--\n");
}

```

```

for(int x = 0; x < (1<<k); x++) {
    bool res1 = control.count(x);
    bool res2 = veb.count(x);
    if (res1 != res2) {
        printf("failed test %d, count(%d): control %d veb %d", test, x, res1, res2);
        return false;
    }
}
for(int x = 0; x < (1<<k); x++) {
    if (x > 0) {
        set<int>::iterator it = control.lower_bound(x);
        int prec = veb.precc(x);
        if (it == control.begin()) {
            if (prec != -1) {
                printf("failed test %d, prec(%d)=%d should not exist\n", test, x, prec);
                return false;
            }
        }
        else {
            it--;
            if (*it != prec) {
                printf("failed test %d, precControl(%d)=%d precVeb(%d)=%d\n", test, x, *it, x, prec);
                return false;
            }
        }
    }
}
for(int x = 0; x < (1<<k); x++) {
    if (x+1 < (1<<k)) {
        set<int>::iterator it = control.upper_bound(x);
        int suc = veb.succ(x);
        if (it == control.end()) {
            if (suc != -1) {
                printf("failed test %d, suc(%d)=%d should not exist\n", test, x, suc);
                return false;
            }
        }
        else {
            if (*it != suc) {
                printf("failed test %d, sucControl(%d)=%d sucVeb(%d)=%d\n", test, x, *it, x, suc);
                return false;
            }
        }
    }
}
printf("all tests passed\n");
return true;
}

int main() {
    test(1000, 16);
}

```

3.33 Lowest Common Ancestor (LCA) và truy vấn đường đi trên cây

$P[i][j]$ là tổ tiên 2^j của i , $D[i][j]$ là giá trị đường đi tương ứng.

Mã nguồn mẫu: `Trees/lca.cpp`

```

#include <cstdio>
#include <vector>
#include <iostream>
using namespace std;
#define MAXN 100009
#define MAXLOGN 20
#define INF (1<<30)

```

```

/*
 * LCA + PATH QUERY O(logn)
 */

const int neutral = 0;
#define comp(a, b) ((a)+(b))

typedef pair<int, int> ii;
vector<ii> adjList[MAXN];
int level[MAXN];
int P[MAXN][MAXLOGN], D[MAXN][MAXLOGN];

void depthdfs(int u) {
    for(int i=0; i<(int)adjList[u].size(); i++) {
        int v = adjList[u][i].first;
        int w = adjList[u][i].second;
        if (v == P[u][0]) continue;
        P[v][0] = u; D[v][0] = w;
        level[v] = 1 + level[u];
        depthdfs(v);
    }
}

void computeP(int root, int n) {
    level[root] = 0;
    P[root][0] = root; D[root][0] = neutral;
    depthdfs(root);
    for(int j = 1; j < MAXLOGN; j++)
        for(int i = 1; i <= n; i++) {
            P[i][j] = P[P[i][j-1]][j-1];
            D[i][j] = comp(D[P[i][j-1]][j-1], D[i][j-1]);
        }
}

ii LCA(int u, int v) {
    if (level[u] > level[v]) swap(u, v);
    int d = level[v] - level[u];
    int ans = neutral;
    for(int i = 0; i < MAXLOGN; i++) {
        if (d & (1<<i)) {
            ans = comp(ans, D[v][i]);
            v = P[v][i];
        }
    }
    if (u == v) return ii(u, ans);
    for(int i = MAXLOGN-1; i >= 0; i--)
        while(P[u][i] != P[v][i]) {
            ans = comp(ans, D[v][i]);
            ans = comp(ans, D[u][i]);
            u = P[u][i]; v = P[v][i];
        }
    ans = comp(ans, D[v][0]);
    ans = comp(ans, D[u][0]);
    return ii(P[u][0], ans);
}

/*
 * TEST MATRIX
 */
#include <cstdlib>
#include <ctime>
int N;

void generateRandTree(int _N) {
    N = _N;
    //printf("new tree %d:\n", N);
    for(int i=1; i<=N; i++) {
        adjList[i].clear();
    }
}

```

```

        if (i == 1) continue;
        int j = 1 + rand()%(i-1);
        int w = rand()%1000;
        //printf("%d-%d: %d\n", i, j, w);
        adjList[i].push_back(ii(j, w));
        adjList[j].push_back(ii(i, w));
    }
}

bool dist(int u, int p, int t, int & ans) {
    if (u == t) return true;
    for(int i=0; i<(int)adjList[u].size(); i++) {
        int v = adjList[u][i].first;
        int w = adjList[u][i].second;
        if (v == p) continue;
        if (dist(v, u, t, ans)) {
            ans = comp(ans, w);
            return true;
        }
    }
    return false;
}

bool test(int nTest) {
    clock_t naive=0, lca=0, last;
    for(int t=1; t<=nTest; t++) {
        generateRandTree(t);
        computeP(1, t);

        for(int it=0; it<t; it++) {
            int u = 1 + (rand()%t);
            int v = 1 + (rand()%t);
            int res1 = neutral;
            last = clock();
            dist(u, -1, v, res1);
            naive += clock() - last;
            last = clock();
            int res2 = LCA(u, v).second;
            lca += clock() - last;
            if (res1 != res2) {
                printf("failed on test %d\n", t);
                return false;
            }
        }
    }

    printf("all tests passed\n");
    printf("naive time: %.3fs, lca time: %.3fs\n", naive*1.0/CLOCKS_PER_SEC, lca*1.0/CLOCKS_PER_SEC);
    return true;
}

bool usertest() {
    scanf("%d", &N);
    int a, b, c;
    for(int i=1; i<N; i++) {
        scanf("%d %d %d", &a, &b, &c);
        adjList[a].push_back(ii(b, c));
        adjList[b].push_back(ii(a, c));
    }
    int root = 1;
    computeP(root, N);
    while(scanf("%d %d", &a, &b)!=EOF) {
        printf("LCA(%d, %d) = %d, ", a, b, LCA(a,b).first);
    }
    return true;
}

int main() {
    test(1000);
}

```

```
    return 0;  
}
```

4 Paradigms

4.1 Merge Sort

Thuật toán $O(n \log n)$ để sắp xếp đoạn; inv đếm số nghịch thế.

Mã nguồn mẫu: Paradigms/mergesort.cpp

```
#include <stdio>
#define MAXN 100009

/*
 * Merge-sort with inversion count in  $O(n \log n)$ 
 */

long long inv;
int aux[MAXN];

void mergesort(int arr[], int l, int r) {
    if (l == r) return;
    int m = (l + r) / 2;
    mergesort(arr, l, m);
    mergesort(arr, m+1, r);
    int i = l, j = m + 1, k = l;
    while(i <= m && j <= r) {
        if (arr[i] > arr[j]) {
            aux[k++] = arr[j++];
            inv += j - i;
        }
        else aux[k++] = arr[i++];
    }
    while(i <= m) aux[k++] = arr[i++];
    for(i = l; i < k; i++) arr[i] = aux[i];
}

/*
 * TEST MATRIX
 */

int main() {
    int N, vet[MAXN];
    inv = 0;
    scanf("%d", &N);
    for(int i=0; i<N; i++) {
        scanf("%d", vet+i);
    }
    mergesort(vet, 0, N-1);
    printf("inv = %lld:", inv);
    for(int i=0; i<N; i++) {
        printf(" %d", vet[i]);
    }
    printf("\n");
    return 0;
}
```

4.2 Quick Sort

Thuật toán kỳ vọng $O(n \log n)$, rất nhanh trong thực tế.

Mã nguồn mẫu: Paradigms/quicksort.cpp

```
#include <stdio>
#include <algorithm>
using namespace std;

void quicksort(int* arr, int l, int r) {
```

```

    if (l >= r) return;
    int mid = rand()%(r-l+1) + l;
    int pivot = arr[mid];
    swap(arr[mid], arr[l]);
    int i = l + 1, j = r;
    while (i <= j) {
        while(i <= j && arr[i] <= pivot) i++;
        while(i <= j && arr[j] > pivot) j--;
        if (i < j) swap(arr[i], arr[j]);
    }
    swap(arr[i-1], arr[l]);
    quicksort(arr, l, i-2);
    quicksort(arr, i, r);
}

void quicksort(int* arr, int l, int r) {
    if (l >= r) return;
    int i, m = rand()%(r-l+1) + l;
    swap(arr[l], arr[m]);
    for(m = l, i = l + 1; i <= r; i++) {
        if (x[i] < x[l]) swap(arr[++m], arr[i]);
    }
    swap(arr[l], arr[m]);
    quicksort(arr, l, m-1);
    quicksort(arr, i, m+1);
}

int main() {
    int arr[8] = {110, 5, 10, 3, 10, 100, 1, 23};
    quicksort(arr, 0, 7);
    for(int i=0; i<8; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}

```

4.3 Longest Increasing Subsequence (LIS)

$O(n \log n)$; sau mỗi bước, phần tử thứ k trong tập là kết thúc nhỏ nhất của dãy tăng độ dài k .

Mã nguồn mẫu: Paradigms/lis.cpp

```

#include <set>
using namespace std;

/*
 * Longest Increasing Subsequence  $O(n \log n)$ 
 * At a given iteration  $i$ ,  $k$ -th element (1-indexed) in set is the
 * smallest element that has a size  $k$  increasing subsequence ending in it
 */

int lis(int arr[], int n) {
    multiset<int> s;
    multiset<int>::iterator it;
    for(int i = 0; i < n; i++) {
        s.insert(arr[i]);
        it = s.upper_bound(arr[i]); //non-decreasing
        //it = ++s.lower_bound(arr[i]); //strictly increasing
        if (it != s.end()) s.erase(it);
    }
    return s.size();
}

/*
 * TEST MATRIX
 */

```



```

#include <stdio>
#define MAXN 500009

int a[MAXN], n;

int main() {
    while(scanf("%d", &n) != EOF) {
        for(int i = 0; i < n; i++) {
            scanf("%d", &a[i]);
        }
        printf("%d\n", lis(a, n));
    }
    return 0;
}

```

4.4 Bài toán cặp gần nhất

Tìm cặp điểm 2D gần nhất theo khoảng cách Euclid trong $O(n \log n)$.

Mã nguồn mẫu: Computational Geometry/closestpair.cpp

```

#include <cmath>
#include <algorithm>
#define MAXN 100309
#define INF 1e+30
using namespace std;

/*
 * Closest point problem
 */

struct point {
    double x, y;
    point() { x = y = 0; }
    point(double _x, double _y) : x(_x), y(_y) {}
};

typedef pair<point, point> pp;

double dist(pp p) {
    double dx = p.first.x - p.second.x;
    double dy = p.first.y - p.second.y;
    return hypot(dx, dy);
}

point strip[MAXN];

pp closest(point *P, int l, int r) {
    if (r == l) return pp(point(INF, 0), point(-INF, 0));
    int m = (l + r) / 2, s1 = 0, s2;
    int midx = (P[m].x + P[m+1].x)/2;
    pp pl = closest(P, l, m);
    pp pr = closest(P, m+1, r);
    pp ans = dist(pl) > dist(pr) ? pr : pl;
    double d = dist(ans);
    for(int i = l; i <= m; i++) {
        if (midx - P[i].x < d) strip[s1++] = P[i];
    }
    s2 = s1;
    for(int i = m+1; i <= r; i++) {
        if (P[i].x - midx < d) strip[s2++] = P[i];
    }
    for(int j = 0, s = s1; j < s1; j++) {
        point p = strip[j];
        for(int i = s; i < s2; i++) {
            point q = strip[i];

```

```

        pp cur = pp(p, q);
        double dcur = dist(cur);
        if (d > dcur) {
            ans = cur; d = dcur;
        }
        if (q.y - p.y > d) break;
        if (p.y - q.y > d) s = i+1;
    }
}

int i = l, j = m+1, k = l;
while(i <= m && j <= r) {
    if (P[i].y < P[j].y) strip[k++] = P[i++];
    else strip[k++] = P[j++];
}
while(i <= m) strip[k++] = P[i++];
for(i = l; i < k; i++) P[i] = strip[i];
return ans;
}

bool compx(point a, point b) {
    return a.x < b.x;
}

pp closest(point *P, int n){
    sort(P, P+n, compx);
    return closest(P, 0, n-1);
}

/*
 * Codeforces 101707F
 */
#include <cstdio>

point P[MAXN];
int n;

int main() {
    scanf("%d", &n);
    for(int i = 0; i < n; i++) scanf("%lf %lf", &P[i].x, &P[i].y);
    pp ans = closest(P, n);
    printf("%.0f %.0f\n", ans.first.x, ans.first.y);
    printf("%.0f %.0f\n", ans.second.x, ans.second.y);
    return 0;
}

```

4.5 Tối ưu hai con trỏ

Giảm một số DP từ $O(n^2 k)$ xuống $O(nk)$ khi chỉ số tối ưu dịch đơn điệu.

Mã nguồn mẫu: Dynamic Programming/twopointeroptimization.cpp

```

#include <cstdio>
#include <algorithm>
using namespace std;
#define MAXN 1009
#define MAXK 19
#define INF (1<<30)

int dp[MAXN][MAXK], A[MAXN][MAXK], N, K;

void twopointer() {
    for(int i=0; i<=N; i++) dp[i][0] = INF;
    for(int j=0; j<=K; j++) dp[0][j] = 0, A[0][j] = 1;
    dp[0][0] = 0;
    for(int i=1; i<=N; i++) {
        for(int j=1; j<=K; j++) {
            dp[i][j] = INF;

```

```

        for(int k=A[i-1][j]; k<=i; k++) {
            int cur = 1 + max(dp[k-1][j-1], dp[i-k][j]);
            if (dp[i][j] > cur) {
                dp[i][j] = cur;
                A[i][j] = k;
            }
            if (dp[k-1][j-1] > dp[i-k][j]) break;
        }
    }
}

/*
 *   Hacker Earth Eggs and Building
 */

int main() {
    int T, n, k;
    scanf("%d", &T);
    N = 1000;
    K = 10;
    twopointer();
    while(T-->0) {
        scanf("%d %d", &n, &k);
        printf("%d\n", dp[n][k]);
    }
    return 0;
}

```

4.6 Tối ưu Convex Hull Trick

Giảm một số DP dạng đường thẳng xuống $O(nk \log n)$.

Mã nguồn mẫu: Dynamic Programming/chtrickoptimization.cpp

```

#include <cstdio>
#include <vector>
#define INF (1<<30)
#define MAXN 1009
using namespace std;

typedef long long ll;

class CHTrick{
private:
    vector<ll> m, n;
    vector<double> p;
    bool maxCH;
public:
    CHTrick(bool _maxCH) { maxCH = _maxCH; }
    void clear() {
        m.clear(); n.clear(); p.clear();
    }
    double inter(double nm, double nn, double lm, double ln) {
        return (ln - nn)/(nm - lm);
    }
    void push(ll nm, ll nn) {
        while(!p.empty() &&
            ((nm == m.back() && (maxCH?-1:1)*nn <= (maxCH?-1:1)*n.back()) ||
            p.back() >= inter(nm, nn, m.back(), n.back()))) {
            m.pop_back(); n.pop_back(); p.pop_back();
        }
        p.push_back(p.empty()? -INF : inter(nm, nn,
            m.back(), n.back()));
        m.push_back(nm); n.push_back(nn);
    }
    ll query(ll x) {

```

```

        if (p.empty()) return (maxCH?-1:1)*INF;
        double dx = (double)x;
        ll high = p.size()-1, low = 0, mid;
        if (dx >= p[high]) return m[high]*x + n[high];
        while(high > low+1) {
            mid = (high+low)/2;
            if (dx < p[mid]) high = mid;
            else low = mid;
        }
        return m[low]*x + n[low];
    }
};

ll x[MAXN], A[MAXN], dp[MAXN][MAXN];
int N, K;

void solve() {
    for(int i=0; i<=N; i++) dp[i][0] = 0;
    CHTrick cht(false);
    for(int j=1; j<=K; j++) {
        dp[0][j] = 0;
        cht.clear();
        for(int i=1; i<=N; i++) {
            cht.push(A[i-1], dp[i-1][j-1]);
            dp[i][j] = cht.query(x[i]);
        }
    }
}

int main() {
    return 0;
}

```

4.7 Tối ưu Slope Trick

Giảm DP dạng trị tuyệt đối từ $O(nS^2)$ xuống $O(n \log n)$.

Mã nguồn mẫu: Dynamic Programming/slopetrick.cpp

```

#include <cstdio>
#include <queue>
#include <algorithm>
using namespace std;
#define MAXN 3009

typedef long long ll;
int N;
ll a[MAXN];

ll f[MAXN], opt[MAXN];

ll slope() {
    priority_queue<ll> pq;
    opt[0] = a[0]; f[0] = 0;
    pq.push(a[0]);
    for(int i=1; i<N; i++) {
        pq.push(a[i]);
        f[i] = f[i-1] + abs(a[i] - pq.top());
        if (a[i] < pq.top()) {
            pq.pop();
            pq.push(a[i]);
        }
        opt[i] = pq.top();
    }
    return f[N-1];
}

```

```

/*
 * TEST MATRIX
 */

#define INF (1<<30)
ll brutef[MAXN][MAXN];
int brute() {
    ll M = 0;
    printf("arr: ");
    for(int i=0; i<N; i++) {
        M = max(M, a[i]+1LL);
        printf(" %3I64d", a[i]);
    }
    printf("\n");
    printf("x: ", 0);
    for(ll x=0; x<=M; x++) {
        printf(" %3I64d", x);
    }
    printf("\n");
    printf("i=%2d:", 0);
    for(ll x=0; x<=M; x++) {
        brutef[0][x] = max(0LL, a[0]-x);
        printf(" %3I64d", brutef[0][x]);
    }
    printf("\n");
    for(int i=1; i<N; i++) {
        printf("i=%2d:", i);
        for(ll x=0; x<=M; x++) {
            brutef[i][x] = INF;
            for(ll y=0; y<=x; y++) {
                brutef[i][x] = min(brutef[i][x], brutef[i-1][y] + abs(y-a[i]));
            }
            printf(" %3I64d", brutef[i][x]);
        }
        printf("\n");
    }
}

/*
 * Codeforces 713C
 */

int main() {
    scanf("%d", &N);
    for(int i=0; i<N; i++) {
        scanf("%I64d", &a[i]);
        a[i] += N-i;
    }
    printf("%I64d\n", slope());
    return 0;
}

```

4.8 Tối ưu chia để trị

Giảm DP có điểm tối ưu đơn điệu từ $O(n^2 k)$ xuống $O(nk \log n)$.

Mã nguồn mẫu: Dynamic Programming/dcoptimization.cpp

```

#include <cstdio>
#include <algorithm>
using namespace std;
#define MAXN 1009
#define INF (1 << 30)

int dp[MAXN][MAXN], C[MAXN][MAXN], N, K;

```

```

void calculatedp(int min_i, int max_i, int j, int min_k, int max_k) {
    if (min_i > max_i) return;
    int i = (min_i + max_i)/2;
    int ans = INF, opt;
    for(int k=min_k; k<=min(max_k, i-1); k++) {
        if (ans > dp[k][j-1] + C[k][i]) {
            opt = k;
            ans = dp[k][j-1] + C[k][i];
        }
    }
    dp[i][j] = ans;
    calculatedp(min_i, i-1, j, min_k, opt);
    calculatedp(i+1, max_i, j, opt, max_k);
}

void solve() {
    for(int i=0; i<=N; i++) dp[i][0] = 0;
    for(int j=0; j<=K; j++) dp[0][j] = 0;
    for(int j=1; j<=K; j++) {
        calculatedp(1, N, j, 0, N-1);
    }
}

int main() {

}

```

4.9 Tối ưu Knuth

Giảm DP dạng chia đoạn từ $O(n^3)$ xuống $O(n^2)$ khi thỏa bất đẳng thức tứ giác/đơn điệu.

Mã nguồn mẫu: Dynamic Programming/knuthoptimization.cpp

```

#include <cstdio>
#define MAXN 1009
#define INF (1LL << 60)

typedef long long ll;

ll dp[MAXN][MAXN], C[MAXN][MAXN], M, x[MAXN];
int A[MAXN][MAXN], N, S;

void knuth() {
    ll cur;
    for(int s = 0; s < N; s++) {
        for(int i=0, j; i + s<N; i++) {
            j = i + s;
            if (s < S) { //Condições iniciais
                dp[i][j] = 0;
                A[i][j] = i;
                continue;
            }
            dp[i][j] = INF;
            for(int k = A[i][j-1]; k <= A[i+1][j]; k++) {
                cur = C[i][j] + dp[i][k] + dp[k][j];
                if (dp[i][j] > cur) {
                    dp[i][j] = cur;
                    A[i][j] = k;
                }
            }
        }
    }
}

int main() {
    while(scanf("%lld %d", &M, &N)!=EOF) {
        for(int i=1; i<=N; i++) {

```

```

        scanf("%lld", &x[i]);
    }
    x[0] = 0;
    x[N+1] = M;
    N += 2;
    S = 2;
    for(int i=0; i<N; i++) {
        for(int j=0; j<N; j++) {
            C[i][j] = x[j] - x[i];
        }
    }
    knuth();
    printf("%lld\n", dp[0][N-1]);
}
return 0;
}

```

4.10 Tối ưu Lagrange

Thay ràng buộc số lần chọn bằng chi phí phạt C rồi tìm nhị phân C khi hàm mục tiêu lồi.

Mã nguồn mẫu: Dynamic Programming/lagrangeoptimization.cpp

```

#include <cstdio>
#include <vector>
#define INF 0x3f3f3f3f
#define INFL 0x3f3f3f3f3f3f3f3f
#define MAXN 1009
using namespace std;

typedef long long ll;

/*
 * Convex Hull Trick
 */

class CHTrick {
private:
    vector<ll> m, n;
    vector<double> p;
    vector<int> ids;
    bool maxCH; int i;
public:
    CHTrick(bool mxch) { maxCH = mxch; i = 0; }
    void clear() { m.clear(); n.clear(); p.clear(); ids.clear(); }
    double inter(double nm, double nn, double lm, double ln) {
        return (ln - nn) / (nm - lm);
    }
    void push(ll nm, ll nn, int id) {
        while (!p.empty()) {
            if (nm == m.back() && maxCH && nn <= n.back()) return;
            if (nm == m.back() && !maxCH && nn >= n.back()) return;
            double x = inter(nm, nn, m.back(), n.back());
            if (nm != m.back() && p.back() < x) break;
            m.pop_back(); n.pop_back(); p.pop_back(); ids.pop_back();
        }
        p.push_back(p.empty() ? -INF : inter(nm, nn, m.back(), n.back()));
        m.push_back(nm); n.push_back(nn); ids.push_back(id);
        if (i >= p.size()) i = p.size()-1;
    }
    pair<ll, int> query_q(ll x) {
        while (p[i] > x) i--;
        while(i+1 < p.size() && p[i+1] <= x) i++;
        return make_pair(m[i] * x + n[i], ids[i]);
    }
};

```

```

/*
 * URI 1282
 */

ll w[MAXN], x[MAXN], A[MAXN], B[MAXN];
ll dp[MAXN];
int opt[MAXN], N, K;

CHTrick cht(false);

int computeDp(ll C) {
    cht.clear();
    dp[0] = 0;
    opt[0] = 0;
    for(int i = 1; i <= N; i++) {
        cht.push(-A[i-1], dp[i-1] + B[i-1], i-1);
        pair<ll, int> cur = cht.query_q(x[i]);
        dp[i] = C + x[i]*A[i] - B[i] + cur.first;
        opt[i] = 1 + opt[cur.second];
    }
    return opt[N];
}

ll computeC() {
    ll lo = 0;
    ll hi = INFLL;
    while(hi > lo + 1) {
        ll C = (hi + lo) / 2;
        int k = computeDp(C);
        if (k > K) lo = C;
        else hi = C;
    }
    return hi;
}

int main(){
    while (scanf("%d %d", &N, &K) != EOF){
        A[0] = B[0] = 0;
        for (int i=1; i<=N; i++) {
            scanf("%lld %lld", &x[i], &w[i]);
            A[i] = w[i] + A[i-1];
            B[i] = w[i]*x[i] + B[i-1];
        }
        ll C = computeC();
        int k = computeDp(C);
        printf("%lld\n", dp[N] - C*K);
    }
    return 0;
}

```


5 Đồ thị

5.1 DFS Spanning Tree

Cây khung DFS.

Mã nguồn mẫu: Graphs/dfs spanningtree.cpp

```
#include <cstdio>
#include <vector>
using namespace std;
#define MAXN 1009
#define UNVISITED -1
#define EXPLORED -2
#define VISITED -3

int vis[MAXN], parent[MAXN];
vector<int> adjList[MAXN];

void dfs(int u) { // DFS for checking graph edge properties
    vis[u] = EXPLORED;
    for (int j = 0, v; j < (int)adjList[u].size(); j++) {
        v = adjList[u][j];
        if (vis[v] == UNVISITED) { // EXPLORED->UNVISITED
            printf(" Tree Edge (%d, %d)\n", u, v);
            parent[v] = u; // parent of this children is me
            dfs(v);
        }
        else if (vis[v] == EXPLORED) { // EXPLORED->EXPLORED
            printf(" Back Edge (%d, %d) (Cycle)\n", u, v);
        }
        else if (vis[v] == VISITED) // EXPLORED->VISITED
            printf(" Forward/Cross Edge (%d, %d)\n", u, v);
    }
    vis[u] = VISITED; // after recursion, color u as VISITED (DONE)
}
```

5.2 Điểm khớp và cầu

Tìm articulation points và bridges.

Mã nguồn mẫu: Graphs/articulationpointsandbridges.cpp

```
#include <cstdio>
#include <vector>
#include <algorithm>
#include <cstring>
using namespace std;
#define MAXN 1009
#define UNVISITED -1

int num[MAXN], N, low[MAXN], parent[MAXN], counter, rootChildren, articulationVertex[MAXN], root;
vector<int> adjList[MAXN];

void tarjan(int u) {
    low[u] = num[u] = counter++;
    for (int j = 0, v; j < (int)adjList[u].size(); j++) {
        v = adjList[u][j];
        if (num[v] == UNVISITED) {
            parent[v] = u;
            if (u == root) rootChildren++;
            tarjan(v);
            if (low[v] >= num[u]) articulationVertex[u] = true;
            if (low[v] > num[u]) printf(" Edge (%d, %d) is a bridge\n", u, v);
            low[u] = min(low[u], low[v]);
        }
    }
}
```

```

        else if (v != parent[u])
            low[u] = min(low[u], num[v]);
    }
}

int main() {
    counter = 0;
    memset(&num, UNVISITED, sizeof num);
    memset(&low, 0, sizeof low);
    memset(&parent, 0, sizeof parent);
    memset(&articulationVertex, 0, sizeof articulationVertex);
    printf("Bridges:\n");
    for (int i = 0; i < N; i++)
        if (num[i] == UNVISITED) {
            root = i; rootChildren = 0; tarjan(i);
            articulationVertex[root] = (rootChildren > 1);
        } // special case
    printf("Articulation Points:\n");
    for (int i = 0; i < N; i++)
        if (articulationVertex[i])
            printf(" Vertex %d\n", i);
    return 0;
}

```

5.3 Sắp xếp tô pô

Topological sort.

Mã nguồn mẫu: Graphs/toposort.cpp

```

#include <cstdio>
#include <vector>
using namespace std;
#define MAXN 1009

int vis[MAXN];
vector<int> adjList[MAXN];
vector<int> toposort;    //Ordem reversa!

void ts(int u) {
    vis[u] = true;
    for (int j = 0, v; j < (int)adjList[u].size(); j++) {
        v = adjList[u][j];
        if (!vis[v]) ts(v);
    }
    toposort.push_back(u);
}

```

5.4 Thành phần liên thông mạnh: thuật toán Kosaraju

SCC bằng Kosaraju.

Mã nguồn mẫu: Graphs/kosaraju.cpp

```

#include <cstdio>
#include <vector>
#include <cstring>
#define MAXN 100009
using namespace std;

/*
 * Kosaraju SCC O(n)
 */

vector<int> adjList[MAXN], revAdjList[MAXN], ts;

```

```

bool vis[MAXN];
int comp[MAXN], parent = 0, numSCC;

void revdfs(int u) {
    vis[u] = true;
    for(int i = 0, v; i < (int)revAdjList[u].size(); i++) {
        v = revAdjList[u][i];
        if(!vis[v]) revdfs(v);
    }
    ts.push_back(u);
}

void dfs(int u) {
    vis[u] = true; comp[u] = parent;
    for(int i = 0, v; i < (int)adjList[u].size(); i++) {
        v = adjList[u][i];
        if(!vis[v]) dfs(v);
    }
}

void kosaraju(int n) {
    memset(&vis, false, sizeof vis);
    for(int i = 0; i < n; i++) {
        if(!vis[i]) revdfs(i);
    }
    memset(&vis, false, sizeof vis);
    numSCC = 0;
    for(int i = n-1; i >= 0; i--) {
        if(!vis[ts[i]]) {
            parent = ts[i];
            dfs(ts[i]);
            numSCC++;
        }
    }
}

/*
 * TEST MATRIX
 */

int leader[MAXN];

int main() {
    int u, v, N, M;
    scanf("%d %d", &N, &M);
    for(int i=0; i<M; i++) {
        scanf("%d %d", &u, &v);
        adjList[u].push_back(v);
        revAdjList[v].push_back(u);
    }
    kosaraju(N);
    memset(&leader, -1, sizeof leader);
    for(int i=0; i<N; i++) {
        if (leader[comp[i]] == -1 || leader[comp[i]] > i) {
            leader[comp[i]] = i;
        }
    }
    for(int i=0; i<N; i++) printf("%d\n", leader[comp[i]]);
    return 0;
}

```

5.5 2-SAT: thuật toán Tarjan

Giải 2-SAT bằng SCC.

Mã nguồn mẫu: Graphs/2sat.cpp

```

#include <cstdio>
#include <vector>
#include <stack>
#include <algorithm>
#include <cstring>
using namespace std;
#define MAXN 1000009
#define UNVISITED -1

/*
 * Tarjan SCC O(n)
 */

int num[MAXN], vis[MAXN], comp[MAXN], low[MAXN];
int counter, numSCC;
stack<int> st;
vector<int> adjList[MAXN];

void dfs(int u) {
    low[u] = num[u] = counter++;
    st.push(u); vis[u] = 1;
    for (int j = 0; j < (int)adjList[u].size(); j++) {
        int v = adjList[u][j];
        if (num[v] == UNVISITED) dfs(v);
        if (vis[v]) low[u] = min(low[u], low[v]);
    }
    if (low[u] == num[u]) {
        while (true) {
            int v = st.top(); st.pop();
            vis[v] = 0;
            comp[v] = numSCC;
            if (u == v) break;
        }
        numSCC++;
    }
}

void tarjan(int n) {
    counter = numSCC = 0;
    memset(&num, UNVISITED, sizeof num);
    memset(&vis, 0, sizeof vis);
    memset(&low, 0, sizeof low);
    for (int i = 0; i < n; i++) {
        if (num[i] == UNVISITED)
            dfs(i);
    }
}

/*
 * 2-SAT O(n)
 */

bool twosat(bool x[], int n) {
    tarjan(2*n);
    for(int i = 0; i < n; i++) {
        if (comp[2*i] == comp[2*i+1]) return false;
        x[i] = (comp[2*i] > comp[2*i+1]);
    }
    return true;
}

void implies(int u, bool pu, int v, bool pv) {
    adjList[2*u+(pu?1:0)].push_back(2*v+(pv?1:0));
    adjList[2*v+(pv?0:1)].push_back(2*u+(pu?0:1));
}

/*
 * TEST MATRIX
 */

```

```

#include <cassert>

bool checked[MAXN];
bool checkdfs(int u, bool x[]) {
    if (checked[u]) return true;
    checked[u] = true;
    printf("    therefore x[%d]=%d\n", u/2, u%2);
    if ((u%2 != 0) != x[u/2]) return false;
    for(int i = 0; i < (int)adjList[u].size(); i++) {
        int v = adjList[u][i];
        if (!checkdfs(v, x)) return false;
    }
    return true;
}

bool check(bool x[], int n) {
    memset(&checked, false, sizeof checked);
    for(int i = 0; i < n; i++) {
        printf("x[%d]=%d:\n", i, x[i]);
        if (x[i] && !checkdfs(2*i+1, x)) return false;
        if (!x[i] && !checkdfs(2*i, x)) return false;
    }
    return true;
}

bool reached[MAXN];
void reach(int u, int n) {
    if (reached[u]) return;
    reached[u] = true;
    if (u < 2*n) printf("    x[%d]=%d", u/2 + 1, u%2);
    for(int i = 0; i < (int)adjList[u].size(); i++) {
        int v = adjList[u][i];
        reach(v, n);
    }
}

void print(int n) {
    for(int i = 0; i < 2*n; i++) {
        printf("x[%d]=%d ->", i/2 + 1, i%2);
        memset(&reached, false, sizeof reached);
        reach(i, n);
        printf("\n");
    }
}

/*
 * Brazilian ICPC Summer Camp 2019, Group W, Day 2
 * Problem B
 */

vector<int> pref[MAXN];
bool x[MAXN];

int main() {
    int n, m, k;
    scanf("%d %d %d", &n, &m, &k);
    while(k --> 0) {
        int a, b;
        scanf("%d %d", &a, &b);
        pref[a].push_back(b);
    }
    int cnt = m;
    for(int a = 1; a <= n; a++) {
        int sz = pref[a].size();
        vector<int> pre(sz), suf(sz);
        for(int i = 0; i < sz; i++) {
            pre[i] = cnt++; suf[i] = cnt++;
        }
    }
}

```

```

for(int i = 0; i < sz; i++) {
    int u = pref[a][i];
    if (i > 0) {
        implies(pre[i], true, pre[i-1], true);
        implies(abs(u)-1, u < 0, pre[i-1], true);
    }
    if (i+1 < sz) {
        implies(suf[i], true, suf[i+1], true);
        implies(abs(u)-1, u < 0, suf[i+1], true);
    }
    implies(pre[i], true, abs(u)-1, u > 0);
    implies(suf[i], true, abs(u)-1, u > 0);
}
}
//print(m);
if (twosat(x, cnt)) {
    //assert(check(x, cnt));
    vector<int> ans;
    for(int i = 0; i < m; i++) {
        if (x[i]) ans.push_back(i+1);
    }
    printf("%u\n", ans.size());
    for(int i = 0; i < (int)ans.size(); i++) {
        printf("%d ", ans[i]);
    }
    printf("\n");
}
else printf("-1\n");
return 0;
}

```

5.6 Đường đi ngắn nhất: thuật toán Dijkstra

Dijkstra.

Mã nguồn mẫu: Graphs/dijkstra.cpp

```

#include <set>
#include <vector>
using namespace std;
#define MAXN 100009
#define INF 0x3f3f3f3f
typedef pair<int, int> ii;

/*
 * Dijkstra's Algorithm O(nlogn + m)
 */

vector<ii> adjList[MAXN];

int dijkstra(int s, int t, int n, int dist[]) {
    for(int i = 1; i <= n; i++) dist[i] = INF;
    set<ii> pq;
    dist[s] = 0;
    pq.insert(ii(0, s));
    while(!pq.empty()) {
        int u = pq.begin()->second;
        pq.erase(pq.begin());
        for(int i=0; i<(int)adjList[u].size(); i++) {
            int v = adjList[u][i].second;
            int w = adjList[u][i].first;
            if (dist[v] > dist[u] + w) {
                pq.erase(ii(dist[v], v));
                dist[v] = dist[u] + w;
                pq.insert(ii(dist[v], v));
            }
        }
    }
}

```

```

    }
}
return dist[t];
}

/*
 * SPOJ EZDIJKST
 */

#include <cstdio>

int dist[MAXN], n, m;

int main() {
    int T;
    int u, v, w, s, t;
    scanf("%d", &T);
    while(T--) {
        scanf("%d %d", &n, &m);
        for(int i=1; i<=n; i++) adjList[i].clear();
        for(int i=0; i<m; i++) {
            scanf("%d %d %d", &u, &v, &w);
            adjList[u].push_back(ii(w, v));
        }
        scanf("%d %d", &s, &t);
        int ans = dijkstra(s, t, n, dist);
        if (ans == INF) printf("NO\n");
        else printf("%d\n", ans);
    }
    return 0;
}

```

5.7 Đường đi ngắn nhất: thuật toán Floyd-Warshall

Floyd-Warshall.

Mã nguồn mẫu: Graphs/floydwarshall.cpp

```

#include <cstdio>
#define MAXN 409

int adjMat[MAXN][MAXN], N;

//O(V^3)
void floydwarshall() {
    for (int k = 0; k < N; k++)
        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++)
                adjMat[i][j] = min(adjMat[i][j], adjMat[i][k] + adjMat[k][j]);
}

```

5.8 Đường đi ngắn nhất: thuật toán Bellman-Ford

Bellman-Ford.

Mã nguồn mẫu: Graphs/bellmanford.cpp

```

#include <cstdio>
#include <vector>
#include <cstring>
#include <iostream>
#define MAXN 1009
using namespace std;

int dist[MAXN], N;

```

```

typedef pair<int, int> ii;
vector<ii> adjList[MAXN];

//O(VE)
int bellmanford(int s, int t)
{
    memset(&dist, 1<<20, sizeof dist);
    dist[s] = 0;
    bool hasNegativeWeightCycle = false;
    for (int i = 0; v, w; i < N; i++) {
        for (int u = 0; u < N; u++) {
            for (int j = 0; j < (int)adjList[u].size(); j++) {
                v = adjList[u][j].first;
                w = adjList[u][j].second;
                if (i==N-1 && dist[v] > dist[u] + w)
                    hasNegativeWeightCycle = true;
                else dist[v] = min(dist[v], dist[u] + w);
            }
        }
    }
    return dist[t];
}

```

5.9 Đường đi ngắn nhất thứ k

K-shortest path.

Mã nguồn mẫu: Graphs/kdijkstra.cpp

```

#include <vector>
#include <map>
using namespace std;
#define MAXN 109
typedef pair<int, int> ii;

/*
 * k-Sijkstra O(nklogn)
 */

vector<ii> adjList[MAXN];

void dijkstra(int s, int k, vector<int> kdist[]) {
    map<ii, int> cnt;
    cnt[ii(0, s)]++;
    while(!cnt.empty()) {
        pair<ii, int> cur = *cnt.begin();
        int u = cur.first.second;
        int wu = cur.first.first;
        int am = cur.second;
        cnt.erase(cnt.begin());
        if (am == 0 || kdist[u].size() >= k) continue;
        for(int i = 0; i < am; i++) {
            if (wu == 0 || kdist[u].size() >= k) break;
            kdist[u].push_back(wu);
        }
        for(int i = 0; i < (int)adjList[u].size(); i++) {
            int v = adjList[u][i].second;
            int w = adjList[u][i].first;
            int &cv = cnt[ii(w + wu, v)];
            cv = min(cv + am, k);
        }
    }
}

/*
 * Szkoput Connections
 */

```



```

#include <cstdio>
vector<int> ans[MAXN][MAXN];

int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    while(m --> 0) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        adjList[u].push_back(ii(w, v));
    }
    for(int s = 1; s <= n; s++)
        dijkstra(s, 100, ans[s]);
    int q;
    scanf("%d\n", &q);
    while(q --> 0) {
        int s, t, k;
        scanf("%d %d %d", &s, &t, &k);
        if (k-1 >= ans[s][t].size()) printf("-1\n");
        else printf("%d\n", ans[s][t][k-1]);
    }
    return 0;
}

```

5.10 Đường đi ngắn nhất: Shortest Path Faster Algorithm (SPFA)

SPFA.

Mã nguồn mẫu: Graphs/spfa.cpp

```

#include <queue>
#include <vector>
#include <cstring>
#include <iostream>
using namespace std;
#define MAXN 100009
#define INF 0x3f3f3f3f
typedef pair<int, int> ii;

/*
 * Shortest Path Faster Algorithm (SPFA)  $O(nm)$  worst case,  $O(n)$  average
 */

vector<ii> adjList[MAXN];
int dist[MAXN], vis[MAXN], N, M;
bool inq[MAXN];

int spfa(int s, int t) {
    for(int i=0; i<=N; i++) dist[i] = INF;
    memset(&inq, false, sizeof inq);
    memset(&vis, 0, sizeof vis);
    queue<int> q;
    q.push(s);
    dist[s] = 0;
    inq[s] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (vis[u] > N) return -1;
        inq[u] = false;
        for (int i = 0; i < (int)adjList[u].size(); i++) {
            int v = adjList[u][i].second;
            int w = adjList[u][i].first;
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (!inq[v]) {
                    vis[v]++; q.push(v);
                    inq[v] = true;
                }
            }
        }
    }
}

```

```

    }
    }
}
return dist[t];
}

/*
 * SPOJ EZDIJKST
 */

#include <cstdio>

int main() {
    int T;
    int u, v, w, s, t;
    scanf("%d", &T);
    while(T--) {
        scanf("%d %d", &N, &M);
        for(int i=1; i<=N; i++) adjList[i].clear();
        for(int i=0; i<M; i++) {
            scanf("%d %d %d", &u, &v, &w);
            adjList[u].push_back(ii(w, v));
        }
        scanf("%d %d", &s, &t);
        int ans = spfa(s, t);
        if (ans == INF) printf("NO\n");
        else printf("%d\n", ans);
    }
    return 0;
}

```

5.11 Cây khung nhỏ nhất: thuật toán Kruskal

Kruskal.

Mã nguồn mẫu: Graphs/kruskal.cpp

```

#include <cstdio>
#include <vector>
#include <algorithm>
using namespace std;

class UnionFind {
private:
    vector<int> parent, rank;
public:
    UnionFind(int N) {
        rank.assign(N+9, 0);
        parent.assign(N+9, 0);
        for (int i = 0; i < N; i++) parent[i] = i;
    }
    int find(int i) {
        while(i != parent[i]) i = parent[i];
        return i;
    }
    bool isSameSet(int i, int j) {
        return find(i) == find(j);
    }
    void unionSet (int i, int j) {
        if (isSameSet(i, j)) return;
        int x = find(i), y = find(j);
        if (rank[x] > rank[y]) parent[y] = x;
        else {
            parent[x] = y;
            if (rank[x] == rank[y]) rank[y]++;
        }
    }
}

```

```

    }
};

typedef pair<int, int> ii;
typedef long long ll;

int N, M;
vector< pair<ll, ii> > edgeList; // (weight, two vertices) of the edge

ll kruskal() {
    ll cost = 0;
    UnionFind UF(N);
    pair<int, ii> edge;
    sort(edgeList.begin(), edgeList.end());
    for (int i = 0; i < M; i++) {
        edge = edgeList[i];
        if (!UF.isSameSet(edge.second.first, edge.second.second)) {
            cost += edge.first;
            UF.unionSet(edge.second.first, edge.second.second);
        }
    }
    return cost;
}

int main() {
    scanf("%d %d", &N, &M);
    int u, v; ll w;
    for (int i = 0; i < M; i++) {
        scanf("%d %d %lld", &u, &v, &w);
        edgeList.push_back(make_pair(w, ii(u, v)));
    }
    printf("%lld\n", kruskal());
    return 0;
}

```

5.12 Cây khung nhỏ nhất: thuật toán Prim

Prim.

Mã nguồn mẫu: Graphs/prim.cpp

```

#include <cstdio>
#include <vector>
#include <queue>
#include <cstring>
#define MAXN 10009
using namespace std;

typedef long long ll;
typedef pair<int, int> ii;
vector<ii> adjList[MAXN];

int N, M;

ll prim() {
    bool taken[MAXN];
    memset(&taken, false, sizeof taken);
    taken[0] = true;
    priority_queue<ii> pq;
    ii v, front; int u, w; ll cost = 0;
    for (int j = 0; j < (int)adjList[0].size(); j++) {
        v = adjList[0][j];
        pq.push(ii(-v.second, -v.first));
    }
    while (!pq.empty()) {
        front = pq.top(); pq.pop();
        u = -front.second; w = -front.first;

```

```

        if (!taken[u]) {
            cost += (ll)w; taken[u] = true;
            for (int j = 0; j < (int)adjList[u].size(); j++) {
                v = adjList[u][j];
                if (!taken[v.first]) pq.push(ii(-v.second, -v.first));
            }
        }
    }
    return cost;
}

int main() {
    scanf("%d %d", &N, &M);
    int u, v, w;
    for (int i = 0; i < M; i++) {
        scanf("%d %d %d", &u, &v, &w);
        adjList[u-1].push_back(make_pair(v-1, w));
        adjList[v-1].push_back(make_pair(u-1, w));
    }
    printf("%lld\n", prim());
    return 0;
}

```

5.13 Luồng cực đại: thuật toán Edmonds-Karp

Edmonds-Karp.

Mã nguồn mẫu: Graphs/edmondskarp.cpp

```

#include <queue>
#include <cstring>
#define INF 0x3f3f3f3f
#define MAXN 103000
#define MAXM 900000
using namespace std;

/*
 * Edmonds-Karp's Algorithm -  $O(VE^2)$ 
 */

int ned, prv[MAXN], first[MAXN];
int cap[MAXM], to[MAXM], nxt[MAXM], dist[MAXN];

void init() {
    memset(first, -1, sizeof first);
    ned = 0;
}

void add(int u, int v, int f) {
    to[ned] = v, cap[ned] = f;
    nxt[ned] = first[u];
    first[u] = ned++;

    to[ned] = u, cap[ned] = 0;
    nxt[ned] = first[v];
    first[v] = ned++;
}

int augment(int v, int minEdge, int s) {
    int e = prv[v];
    if (e == -1) return minEdge;
    int f = augment(to[e^1], min(minEdge, cap[e]), s);
    cap[e] -= f;
    cap[e^1] += f;
    return f;
}

```

```

bool bfs(int s, int t) {
    int u, v;
    memset(&dist, -1, sizeof dist);
    dist[s] = 0;
    queue<int> q; q.push(s);
    memset(&prv, -1, sizeof prv);
    while (!q.empty()) {
        u = q.front(); q.pop();
        if (u == t) break;
        for(int e = first[u]; e!=-1; e = nxt[e]) {
            v = to[e];
            if (dist[v] < 0 && cap[e] > 0) {
                dist[v] = dist[u] + 1;
                q.push(v);
                prv[v] = e;
            }
        }
    }
    return dist[t] >= 0;
}

int edmondskarp(int s, int t) {
    int result = 0;
    while (bfs(s, t)) {
        result += augment(t, INF, s);
    }
    return result;
}

/*
 *   Codeforces 101128F
 */
#include <cstdio>

int main() {
    int N, M, R, C, A, B, pos[60][60];
    char c;
    scanf("%d %d %d %d", &R, &C, &A, &B);
    init();
    memset(&cap, 0, sizeof cap);
    N = R*C+2;
    for(int i=0, k=1; i<R; i++) {
        for(int j=0; j<C; j++) pos[i][j] = k++;
    }
    for(int i=0; i<R; i++) {
        for(int j=0; j<C; j++) {
            scanf("%c", &c);
            if (c == '.') add(0, pos[i][j], B);
            else add(pos[i][j], N-1, B);
            if (i < R-1) {
                add(pos[i][j], pos[i+1][j], A);
                add(pos[i+1][j], pos[i][j], A);
            }
            if (j < C-1) {
                add(pos[i][j], pos[i][j+1], A);
                add(pos[i][j+1], pos[i][j], A);
            }
        }
    }
    printf("%d\n", edmondskarp(0, N-1));
    return 0;
}

```

5.14 Luồng cực đại: thuật toán Dinic

Dinic.

Mã nguồn mẫu: Graphs/dinic.cpp

```
#include <queue>
#include <cstring>
#define INF 0x3f3f3f3f
#define MAXN 103000
#define MAXM 900000
using namespace std;

/*
 * Dinic's Algorithm -  $O(EV^2)$ 
 */

int ned, first[MAXN], work[MAXN], dist[MAXN], q[MAXN];
int cap[MAXM], to[MAXM], nxt[MAXM];

void init() {
    memset(first, -1, sizeof first);
    ned = 0;
}

void add(int u, int v, int f) {
    to[ned] = v, cap[ned] = f;
    nxt[ned] = first[u];
    first[u] = ned++;

    to[ned] = u, cap[ned] = 0;
    nxt[ned] = first[v];
    first[v] = ned++;
}

int dfs(int u, int f, int t) {
    if (u == t) return f;
    for(int &e = work[u]; e != -1; e = nxt[e]) {
        int v = to[e];
        if (dist[v] == dist[u] + 1 && cap[e] > 0) {
            int df = dfs(v, min(f, cap[e]), t);
            if (df > 0) {
                cap[e] -= df;
                cap[e^1] += df;
                return df;
            }
        }
    }
    return 0;
}

bool bfs(int s, int t) {
    memset(&dist, -1, sizeof dist);
    dist[s] = 0;
    int st = 0, en = 0;
    q[en++] = s;
    while (en > st) {
        int u = q[st++];
        for(int e = first[u]; e != -1; e = nxt[e]) {
            int v = to[e];
            if (dist[v] < 0 && cap[e] > 0) {
                dist[v] = dist[u] + 1;
                q[en++] = v;
            }
        }
    }
    return dist[t] >= 0;
}

int dinic(int s, int t) {
    int result = 0, f;
    while (bfs(s, t)) {
        memcpy(work, first, sizeof work);
    }
}
```

```

        while (f = dfs(s, INF, t)) result += f;
    }
    return result;
}

/*
 * Codeforces 100783D
 */

#include <cstdio>

int main() {
    int u, v, N, M;
    init();
    memset(&cap, 0, sizeof cap);
    scanf("%d %d", &N, &M);
    for(int i=1; i<=N; i++) {
        add(0, i, 1);
        add(i+N, 2*N+1, 1);
    }
    for(int i=0; i<M; i++) {
        scanf("%d %d", &u, &v);
        u++; v++;
        add(u, v+N, 1);
    }
    int MF = dinic(0, 2*N+1);
    //printf("MF = %d\n", MF);
    if (MF == N) printf("YES\n");
    else printf("NO\n");
    return 0;
}

```

5.15 Luồng cực đại: thuật toán Push-Relabel

Push-Relabel.

Mã nguồn mẫu: Graphs/pushrelabel.cpp

```

#include <cstring>
#include <cstdio>
#define MAXN 103009
#define MAXM 900000
#define INF 0x3f3f3f3f

/*
 * Push Relabel Algorithm -  $O(VE + \sqrt{E}V^2)$ 
 */

int ned, first[MAXN], exc[MAXN], maxhgt[MAXN];
int cap[MAXM], to[MAXM], nxt[MAXM], hgt[MAXN];

void init() {
    memset(first, -1, sizeof first);
    ned = 0;
}

void add(int u, int v, int f) {
    to[ned] = v, cap[ned] = f;
    nxt[ned] = first[u];
    first[u] = ned++;

    to[ned] = u, cap[ned] = 0;
    nxt[ned] = first[v];
    first[v] = ned++;
}

void push(int e) {

```

```

    int u = to[e^1], v = to[e];
    int f = exc[u] < cap[e] ? exc[u] : cap[e];
    cap[e] -= f; cap[e^1] += f;
    exc[u] -= f; exc[v] += f;
}

void relabel(int u) {
    int f = INF;
    for (int e = first[u]; e != -1; e = nxt[e]) {
        if (cap[e] > 0 && hgt[to[e]] < f) f = hgt[to[e]];
    }
    if (f < INF) hgt[u] = f + 1;
}

int maxHeight(int s, int t, int n) {
    int sz = 0;
    for (int u = 1; u <= n; u++) {
        if (u == s || u == t || exc[u] == 0) continue;
        if (sz > 0 && hgt[u] > hgt[maxhgt[0]]) sz = 0;
        if (sz == 0 || hgt[u] == hgt[maxhgt[0]])
            maxhgt[sz++] = u;
    }
    return sz;
}

int pushrelabel(int s, int t, int n) {
    memset(&hgt, 0, sizeof hgt);
    memset(&exc, 0, sizeof exc);
    hgt[s] = n; exc[s] = INF;
    for (int e = first[s]; e != -1; e = nxt[e])
        push(e);
    int sz;
    while ((sz = maxHeight(s, t, n)) > 0) {
        for(int i = 0; i < sz; i++) {
            int u = maxhgt[i], pushed = 0;
            for (int e = first[u]; e != -1 && exc[u]; e = nxt[e]) {
                if (cap[e] > 0 && hgt[u] == hgt[to[e]] + 1)
                    push(e), pushed = 1;
            }
            if (!pushed) {
                relabel(u); break;
            }
        }
    }
    return exc[t];
}

/*
 * SPOJ MTOTALF
 */

#include <stdio>

int main() {
    int m;
    scanf("%d", &m);
    init();
    for(int j = 0; j < m; j++) {
        char u, v;
        int f;
        scanf(" %c %c %d", &u, &v, &f);
        add(u, v, f);
    }
    printf("%d\n", pushrelabel('A', 'Z', 255));
    return 0;
}

```


5.16 Cắt toàn cục nhỏ nhất: thuật toán Stoer-Wagner

Stoer-Wagner.

Mã nguồn mẫu: Graphs/stoerwagner.cpp

```
#include <cstdio>
#include <vector>
#include <cstring>
using namespace std;
#define MAXN 509
#define INF 0x3f3f3f3f

int n, adjMatrix[MAXN][MAXN];
vector<int> bestCut;

// Modifica o grafo!!!! Se precisar do grafo depois faça uma cópia
int mincut() {
    int bestCost = INF;
    vector<int> v[MAXN];
    for (int i=0; i<n; ++i)
        v[i].assign(1, i);
    int w[MAXN], sel;
    bool exist[MAXN], added[MAXN];
    memset(exist, true, sizeof exist);
    for (int phase=0; phase<n-1; ++phase) {
        memset(added, false, sizeof added);
        memset(w, 0, sizeof w);
        for (int j=0, prev; j<n-phase; ++j) {
            sel = -1;
            for (int i=0; i<n; ++i) {
                if (exist[i] && !added[i] && (sel == -1 || w[i] > w[sel]))
                    sel = i;
            }
            if (j == n-phase-1) {
                if (w[sel] < bestCost) {
                    bestCost = w[sel];
                    bestCut = v[sel];
                }
                v[prev].insert(v[prev].end(), v[sel].begin(), v[sel].end());
                for (int i=0; i<n; ++i) adjMatrix[prev][i] = adjMatrix[i][prev] += adjMatrix[sel][i];
                exist[sel] = false;
            }
            else {
                added[sel] = true;
                for (int i=0; i<n; ++i) w[i] += adjMatrix[sel][i];
                prev = sel;
            }
        }
    }
    return bestCost;
}

int main() {
    return 0;
}
```

5.17 Min Cost Max Flow

Luồng cực đại chi phí nhỏ nhất.

Mã nguồn mẫu: Graphs/mincostmaxflow.cpp

```
#include <cstring>
#include <set>
using namespace std;
#define MAXN 100009
```

```

#define MAXM 900009
#define INFLL 0x3f3f3f3f3f3f3f3f
typedef long long ll;

/*
 * Min Cost Max Flow in O(VE(VlogV+E))
 */

int ned;
int first[MAXN], to[MAXM], nxt[MAXM], prv[MAXN];
ll cap[MAXM], cost[MAXM], dist[MAXN], pot[MAXN];

void init() {
    memset(first, -1, sizeof first);
    ned = 0;
}

void add(int u, int v, ll f, ll c) {
    to[ned] = v, cap[ned] = f;
    cost[ned] = c, nxt[ned] = first[u];
    first[u] = ned++;
    to[ned] = u, cap[ned] = 0;
    cost[ned] = -c, nxt[ned] = first[v];
    first[v] = ned++;
}

bool dijkstra(int s, int t, int n) {
    memset(&prv, -1, sizeof prv);
    for(int i = 1; i <= n; i++) dist[i] = INFLL;
    set< pair<ll, int> > q;
    q.insert(make_pair(0LL, s));
    dist[s] = prv[s] = 0;
    while(!q.empty()) {
        int u = q.begin()->second;
        q.erase(q.begin());
        for(int e = first[u]; e != -1; e = nxt[e]) {
            if (cap[e] <= 0) continue;
            int v = to[e];
            ll new_dist = dist[u] + cost[e] + pot[u] - pot[v];
            if (new_dist < dist[v]) {
                q.erase(make_pair(dist[v], v));
                dist[v] = new_dist;
                prv[v] = e;
                q.insert(make_pair(new_dist, v));
            }
        }
    }
    return prv[t] != -1;
}

ll augment(int s, int t, ll &maxflow, int n) {
    ll flow = maxflow;
    for(int i = t; i != s; i = to[prv[i]^1])
        flow = min(flow, cap[prv[i]]);
    for(int i = t; i != s; i = to[prv[i]^1]) {
        cap[prv[i]] -= flow;
        cap[prv[i]^1] += flow;
    }
    maxflow += flow;
    ll flowCost = flow*(dist[t]-pot[s]+pot[t]);
    for(int i = 1; i <= n; i++)
        if (prv[i] != -1) pot[i] += dist[i];
    return flowCost;
}

ll mincostmaxflow(int s, int t, ll &maxflow, int n) {
    ll flowCost = 0;
    memset(pot, 0, sizeof pot);
    while(maxflow > 0 && dijkstra(s, t, n)) {

```

```

        flowCost += augment(s, t, maxflow, n);
    }
    return flowCost;
}

/*
 * Codeforces 101726I
 */

#include <cstdio>

int n, m;
int in[60], out[60];
bool reach[60][60];

int main() {
    int T;
    scanf("%d", &T);
    while(T --> 0) {
        ll ans = 0;
        scanf("%d %d", &n, &m);
        memset(&in, 0, sizeof in);
        memset(&out, 0, sizeof out);
        memset(&reach, false, sizeof reach);
        init();
        for(int j = 0; j < m; j++) {
            int u, v, w;
            scanf("%d %d %d", &u, &v, &w);
            ans += w;
            add(u, v, INFL, w);
            in[v]++;
            out[u]++;
            reach[u][v] = true;
        }
        for(int k = 1; k <= n; k++) {
            for(int i = 1; i <= n; i++) {
                for(int j = 1; j <= n; j++) {
                    reach[i][j] = reach[i][j] || (reach[i][k] && reach[k][j]);
                }
            }
        }

        bool ok = true;
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= n; j++) {
                if (!reach[i][j]) ok = false;
            }
        }

        int s = n+1;
        int t = n+2;
        ll maxflow = 0;
        for(int u = 1; u <= n; u++) {
            int bal = in[u] - out[u];
            if (bal < 0) {
                add(u, t, -bal, 0);
            }
            if (bal > 0) {
                add(s, u, bal, 0);
                maxflow += bal;
            }
        }
        ll cost = mincostmaxflow(s, t, maxflow, n+2);
        if (maxflow > 0) ok = false;
        if (ok) printf("%lld\n", cost+ans);
        else printf("-1\n");
    }
    return 0;
}

```

5.18 Ghép cặp chi phí nhỏ nhất: thuật toán Hungary

Hungarian algorithm.

Mã nguồn mẫu: Graphs/hungarian.cpp

```
#include <cstring>
#define MAXN 2009
#define MAXM 2009
#define INF 0x3f3f3f3f

/*
 * Hungarian's Algorithm  $O(n^2 m)$ 
 * linhas de 1 a n, columnas de 1 a n
 */

int n, m;
int pu[MAXN], pv[MAXN], cost[MAXN][MAXM];
int pairV[MAXN], way[MAXM], minv[MAXM];
bool used[MAXM];

void hungarian() {
    memset(&pairV, 0, sizeof pairV);
    for(int i = 1, j0 = 0; i <= n; i++) {
        pairV[i] = i;
        memset(&minv, INF, sizeof minv);
        memset(&used, false, sizeof used);
        do {
            used[j0] = true;
            int i0 = pairV[j0], delta = INF, j1;
            for(int j = 1; j <= m; j++) {
                if (used[j]) continue;
                int cur = cost[i0][j] - pu[i0] - pv[j];
                if (cur < minv[j])
                    minv[j] = cur, way[j] = j0;
                if (minv[j] < delta)
                    delta = minv[j], j1 = j;
            }
            for(int j = 0; j <= m; j++) {
                if (used[j])
                    pu[pairV[j]] += delta, pv[j] -= delta;
                else minv[j] -= delta;
            }
            j0 = j1;
        } while(pairV[j0] != 0);
        do {
            int j1 = way[j0];
            pairV[j0] = pairV[j1];
            j0 = j1;
        } while(j0);
    }
}

/*
 * Codeforces 101635G
 */

#include <cstdio>
#include <algorithm>
using namespace std;

int bx[MAXN], by[MAXN], cx[MAXM], cy[MAXM], rx, ry;

int main() {
    scanf("%d %d", &n, &m);
    for(int i = 1; i <= n; i++) {
        scanf("%d %d", &bx[i], &by[i]);
    }
    for(int j = 1; j <= m; j++) {
```

```

        scanf("%d %d", &cx[j], &cy[j]);
    }
    scanf("%d %d", &rx, &ry);

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m+n-1; j++) {
            if (j <= m) cost[i][j] = abs(bx[i]-cx[j]) + abs(by[i]-cy[j]);
            else cost[i][j] = abs(bx[i]-rx) + abs(by[i]-ry);
            //printf("%d ", cost[i][j]);
        }
        //printf("\n");
    }

    int ans = 0;
    for(int i = 1; i <= n; i++) {
        ans += abs(bx[i]-rx) + abs(by[i]-ry);
    }
    m += n-1;
    hungarian();
    for(int j = 1; j <= m; j++) {
        if (pairV[j] == 0) continue;
        ans += cost[pairV[j]][j];
    }
    printf("%d\n", ans);
    return 0;
}

```

5.19 Ghép cặp cực đại: thuật toán Kuhn

Kuhn.

Mã nguồn mẫu: Graphs/kuhn.cpp

```

#include <vector>
#include <queue>
#include <cstring>
#define MAXN 1009
using namespace std;

/*
 * Kuhn's Algorithm O(VE)
 */

vector<int> adjU[MAXN];
int pairU[MAXN], pairV[MAXN];
bool vis[MAXN];
int m, n;
///Vertices enumerados de 1 a m em U e de 1 a n em V!!!!

bool dfs(int u) {
    vis[u] = true;
    if (u == 0) return true;
    for (int i = 0; i < (int)adjU[u].size(); i++) {
        int v = adjU[u][i];
        if (!vis[pairV[v]] && dfs(pairV[v])) {
            pairV[v] = u; pairU[u] = v;
            return true;
        }
    }
    return false;
}

int kuhn() {
    memset(&pairU, 0, sizeof pairU);
    memset(&pairV, 0, sizeof pairV);
    int result = 0;
    for (int u = 1; u <= m; u++) {

```

```

        memset(&vis, false, sizeof vis);
        if (pairU[u]==0 && dfs(u)) result++;
    }
    return result;
}

/*
 * Codeforces 101712A
 */

#include <cstdio>

int main() {
    scanf("%d %d", &m, &n);
    for (int u = 1; u <= m; u++) {
        int v;
        while(scanf("%d", &v), v) adjU[u].push_back(v);
    }
    printf("%d\n", kuhn());
    for (int u = 1; u <= m; u++) {
        if (pairU[u]) printf("%d %d\n", u, pairU[u]);
    }
    return 0;
}

```

5.20 Ghép cặp cực đại: thuật toán Hopcroft-Karp

Hopcroft-Karp.

Mã nguồn mẫu: Graphs/hopcroftkarp.cpp

```

#include <vector>
#include <queue>
#include <cstring>
#define INF 0x3f3f3f3f
#define MAXN 1009
using namespace std;

/*
 * Hopcroft-Karp's Algorithm  $O(E \cdot \sqrt{V})$ 
 */

vector<int> adjU[MAXN];
int pairU[MAXN], pairV[MAXN], dist[MAXN];
int m, n;
//Vertices enumerados de 1 a m em U e de 1 a n em V!!!!

bool bfs() {
    queue<int> q;
    for (int u = 1; u <= m; u++) {
        if (pairU[u] == 0) {
            dist[u] = 0; q.push(u);
        }
        else dist[u] = INF;
    }
    dist[0] = INF;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (dist[u] >= dist[0]) continue;
        for (int i = 0; i < (int)adjU[u].size(); i++) {
            int v = adjU[u][i];
            if (dist[pairV[v]] == INF) {
                dist[pairV[v]] = dist[u] + 1;
                q.push(pairV[v]);
            }
        }
    }
}

```

```

        return (dist[0] != INF);
    }

    bool dfs(int u) {
        if (u == 0) return true;
        for (int i = 0; i < (int)adjU[u].size(); i++) {
            int v = adjU[u][i];
            if (dist[pairV[v]] == dist[u]+1) {
                if (dfs(pairV[v])) {
                    pairV[v] = u; pairU[u] = v;
                    return true;
                }
            }
        }
        dist[u] = INF;
        return false;
    }

    int hopcroftKarp() {
        memset(&pairU, 0, sizeof pairU);
        memset(&pairV, 0, sizeof pairV);
        int result = 0;
        while (bfs()) {
            for (int u=1; u<=m; u++) {
                if (pairU[u]==0 && dfs(u))
                    result++;
            }
        }
        return result;
    }
}

/*
 * Codeforces 101712A
 */

#include <cstdio>

int main() {
    scanf("%d %d", &m, &n);
    for (int u = 1; u <= m; u++) {
        int v;
        while(scanf("%d", &v), v) adjU[u].push_back(v);
    }
    printf("%d\n", hopcroftKarp());
    for (int u = 1; u <= m; u++) {
        if (pairU[u]) printf("%d %d\n", u, pairU[u]);
    }
    return 0;
}

```

5.21 Minimum Vertex Cover

Phủ đỉnh nhỏ nhất trong đồ thị hai phía.

Mã nguồn mẫu: Graphs/minimumcover.cpp

```

#include <vector>
#include <cstring>
#define INF 0x3f3f3f3f
#define MAXN 4009
using namespace std;

/*
 * Minimum Vertex Cover from Maximum Matching  $O(V+E)$ 
 */

vector<int> adjU[MAXN];

```

```

int pairU[MAXN], pairV[MAXN];
int m, n;
//Vértices enumerados de 1 a m em U e de 1 a n em V!!!!
bool Zu[MAXN], Zv[MAXN];
vector<int> coverU, coverV;

void getreach(int u) {
    if (u == 0 || Zu[u]) return;
    Zu[u] = true;
    for (int i = 0; i < (int)adjU[u].size(); i++) {
        int v = adjU[u][i];
        if (v == pairU[u]) continue;
        Zv[v] = true;
        getreach(pairV[v]);
    }
}

void minimumcover() {
    memset(&Zu, false, sizeof Zu);
    memset(&Zv, false, sizeof Zv);
    for (int u = 1; u <= m; u++)
        if (pairU[u] == 0) getreach(u);
    coverU.clear(); coverV.clear();
    for (int u = 1; u <= m; u++)
        if (!Zu[u]) coverU.push_back(u);
    for (int v = 1; v <= n; v++)
        if (Zv[v]) coverV.push_back(v);
}

/*
 * Codeforces 101712D
 */

#include <cstdio>

int main() {
    scanf("%d %d", &m, &n);
    for (int u = 1; u <= m; u++) {
        int k, v;
        scanf("%d", &k);
        while(k --> 0) {
            scanf("%d", &v);
            adjU[u].push_back(v);
        }
    }
    int nmatches = 0;
    for (int u = 1; u <= m; u++) {
        int v;
        scanf("%d", &v);
        if (v == 0) continue;
        nmatches++;
        pairU[u] = v;
        pairV[v] = u;
    }
    minimumcover();
    printf("%d\n", nmatches);
    printf("%u", coverU.size());
    for(int u : coverU) printf(" %d", u);
    printf("\n");
    printf("%u", coverV.size());
    for(int v : coverV) printf(" %d", v);
    printf("\n");
    return 0;
}

```


5.22 Maximum Matching: thuật toán Blossom

Blossom cho matching tổng quát.

Mã nguồn mẫu: Graphs/blossom.cpp

```
#include <bits/stdc++.h>
using namespace std;
#define MAXN 2005
#define INF 1000000009

/* Codechef Problem: A game on a graph
// Blossom algorithm  $O(N^3)$ 
// https://www.codechef.com/problems/HAMILG
*/
int mQueue[MAXN], qHead, qTail, match[MAXN], cur_blossom[MAXN], parent[MAXN], N;
bool inPath[MAXN], inQueue[MAXN], inBlossom[MAXN];

vector<vector<int> > G;

void push(int u) {
    mQueue[qTail++] = u;
    inQueue[u] = true;
}

int lca(int u, int v) {
    memset(inPath, 0, sizeof(inPath));
    u = cur_blossom[u];
    while(true) {
        inPath[u] = true;
        if(match[u] == -1 || parent[match[u]] == -1) break;
        u = cur_blossom[parent[match[u]]];
    }

    v = cur_blossom[v];
    while(true) {
        if(inPath[v]) return v;
        v = cur_blossom[parent[match[v]]];
    }

    return v;
}

int find_aug_path(int R) {
    int u, viz;
    memset(inQueue, 0, sizeof(inQueue));
    memset(parent, -1, sizeof(parent));
    for(int i = 0; i < N; i++) cur_blossom[i] = i;
    qTail = qHead = 0;
    push(R);

    while(qHead < qTail) {
        u = mQueue[qHead++];
        for (int i = 0; i < G[u].size(); i++) {
            viz = G[u][i];
            if(viz != match[u] && cur_blossom[viz] != cur_blossom[u]) {
                if(parent[viz] == -1 && !(match[viz] != -1 && parent[match[viz]] != -1)) {
                    if(viz == R) continue;
                    parent[viz] = u;
                    if(match[viz] == -1) return viz; // augmenting path to viz
                    push(match[viz]);
                    continue;
                }
            }

            if(viz != R && !(match[viz] != -1 && parent[match[viz]] != -1)) continue;

            // blossom found
            int new_blossom = lca(u, viz);
            memset(inBlossom, 0, sizeof(inBlossom));
```

```

        int v = u, parent_v = viz;
        while(cur_blossom[v] != new_blossom) {
            inBlossom[cur_blossom[v]] = inBlossom[cur_blossom[match[v]]] = true;
            parent[v] = parent_v;
            parent_v = match[v];
            v = parent[match[v]];
        }

        v = viz, parent_v = u;
        while(cur_blossom[v] != new_blossom) {
            inBlossom[cur_blossom[v]] = inBlossom[cur_blossom[match[v]]] = true;
            parent[v] = parent_v;
            parent_v = match[v];
            v = parent[match[v]];
        }

        for(int i = 0; i < N; i++) if(inBlossom[cur_blossom[i]]) {
            cur_blossom[i] = new_blossom;
            if(!inQueue[i]) {
                push(i);
            }
        }
    }
}

return -1;
}

void MaxMatch() {
    memset(match, -1, sizeof(match));

    for (int i = 0; i < N; i++) {
        if (match[i] < 0) {
            int v = find_aug_path(i);

            while(v != -1) {
                int parent_v = match[parent[v]];
                match[v] = parent[v];
                match[parent[v]] = v;
                v = parent_v;
            }
        }
    }
}

bool marked[MAXN];

void dfs(int R) {
    int u, viz;
    memset(inQueue, 0, sizeof(inQueue));
    memset(parent, -1, sizeof(parent));

    for(int i = 0; i < N; i++) cur_blossom[i] = i;

    qTail = qHead = 0;
    push(R);

    while(qHead < qTail) {
        u = mQueue[qHead++];
        marked[u] = true;
        for (int i = 0; i < G[u].size(); i++) {
            viz = G[u][i];
            if(viz != match[u] && cur_blossom[viz] != cur_blossom[u]) {
                if(parent[viz] == -1 && !(match[viz] != -1 && parent[match[viz]] != -1)) {
                    if(viz == R) continue;
                    parent[viz] = u;
                    if(match[viz] == -1) return; // augmenting path to viz
                }
            }
        }
    }
}

```

```

        push(match[viz]);
        continue;
    }

    if(viz != R && !(match[viz] != -1 && parent[match[viz]] != -1)) continue;

    // blossom found
    int new_blossom = lca(u,viz);
    memset(inBlossom, 0, sizeof(inBlossom));

    int v = u, parent_v = viz;
    while(cur_blossom[v] != new_blossom) {
        inBlossom[cur_blossom[v]] = inBlossom[cur_blossom[match[v]]] = true;
        parent[v] = parent_v;
        parent_v = match[v];
        v = parent[match[v]];
    }

    v = viz, parent_v = u;
    while(cur_blossom[v] != new_blossom) {
        inBlossom[cur_blossom[v]] = inBlossom[cur_blossom[match[v]]] = true;
        parent[v] = parent_v;
        parent_v = match[v];
        v = parent[match[v]];
    }

    for(int i = 0; i < N; i++) if(inBlossom[cur_blossom[i]]) {
        cur_blossom[i] = new_blossom;
        if(!inQueue[i]) {
            push(i);
        }
    }
}

return;
}

int main() {
    int t, a, b, M;
    scanf("%d", &t);
    while(t--) {
        scanf("%d %d", &N, &M);

        G.clear();
        G.resize(N+5);

        for (int i = 0; i < M; i++) {
            scanf("%d %d", &a, &b); a--; b--;
            G[a].push_back(b);
            G[b].push_back(a);
        }
        MaxMatch();

        int ans = 0;
        memset(marked, 0, sizeof(marked));

        for (int u = 0; u < N; u++) {
            if (!marked[u] && match[u] == -1) {
                dfs(u);
            }
        }
        for (int u = 0; u < N; u++) {
            if (marked[u]) ans++;
        }
        printf("%d\n", ans);
    }
}

```

5.23 Gomory-Hu Tree: thuật toán Gusfield

Cây Gomory-Hu.

Mã nguồn mẫu: Graphs/gomoryhu.cpp

```
#include <queue>
#include <cstring>
using namespace std;
#define MAXN 3009
#define MAXM 200009
#define INF (1 << 30)

/*
 * Edmonds-Karp's Algorithm -  $O(VE^2)$ 
 */

int N, M, ned, prv[MAXN], first[MAXN];
int cap[MAXM], to[MAXM], nxt[MAXM], dist[MAXN];

void init()
{
    memset(first, -1, sizeof first);
    ned = 0;
}

void add(int u, int v, int f)
{
    to[ned] = v, cap[ned] = f;
    nxt[ned] = first[u];
    first[u] = ned++;

    to[ned] = u, cap[ned] = 0;
    nxt[ned] = first[v];
    first[v] = ned++;
}

int augment(int v, int minEdge, int s)
{
    int e = prv[v];
    if (e == -1)
        return minEdge;
    int f = augment(to[e ^ 1], min(minEdge, cap[e]), s);
    cap[e] -= f;
    cap[e ^ 1] += f;
    return f;
}

bool bfs(int s, int t)
{
    int u, v;
    memset(&dist, -1, sizeof dist);
    dist[s] = 0;
    queue<int> q;
    q.push(s);
    memset(&prv, -1, sizeof prv);
    while (!q.empty())
    {
        u = q.front();
        q.pop();
        if (u == t)
            break;
        for (int e = first[u]; e != -1; e = nxt[e])
        {
            v = to[e];
            if (dist[v] < 0 && cap[e] > 0)
            {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}
```

```

        prv[v] = e;
    }
}
return dist[t] >= 0;
}

int edmondskarp(int s, int t)
{
    int result = 0;
    while (bfs(s, t))
    {
        result += augment(t, INF, s);
    }
    return result;
}

/*
 * Gomory Hu Tree / Gusfield's Algorithm - O((V-1)*(Flow Algorithm))
 */

typedef pair<int, int> ii;
int par[MAXN], fpar[MAXN]; // parent in tree, flow to parent
int backup[MAXN];          // backup for cap
bool inCut[MAXN];
vector<ii> ghtree[MAXN];

void cutGraph(int s)
{
    inCut[s] = true;
    for (int e = first[s]; e != -1; e = nxt[e])
    {
        int v = to[e];
        if (!inCut[v] && cap[e] > 0)
        {
            cutGraph(v);
        }
    }
}

int computeMinCut(int s, int t)
{
    memcpy(backup, cap, sizeof cap);
    memset(inCut, false, sizeof inCut);
    int f = edmondskarp(s, t);
    cutGraph(s);
    memcpy(cap, backup, sizeof cap);
    return f;
}

// 1-indexed
void gomoryhu()
{
    for (int i = 1; i <= N; i++)
        par[i] = 1;
    for (int s = 2; s <= N; s++)
    {
        int t = par[s];
        fpar[s] = computeMinCut(s, t);
        for (int u = s + 1; u <= N; u++)
        {
            if (par[u] == t && inCut[u])
            {
                par[u] = s;
            }
            if (par[t] == u && inCut[u])
            {
                par[s] = u;
                par[t] = s;
            }
        }
    }
}

```

```

        swap(fpar[s], fpar[t]);
    }
}
}
for (int i = 1; i <= N; i++)
    ghtree[i].clear();
for (int i = 1; i <= N; i++)
{
    if (par[i] == i)
        continue;
    ghtree[i].push_back(ii(fpar[i], par[i]));
    ghtree[par[i]].push_back(ii(fpar[i], i));
}
}

/*
 * Codeforces 101480J
 */
#include <cstdio>

int dfs(int u, int prv, int k)
{
    int ans = 0, v, w;
    for (int i = 0; i < (int)ghtree[u].size(); i++)
    {
        v = ghtree[u][i].second;
        w = ghtree[u][i].first;
        if (v == prv)
            continue;
        ans += min(k, w);
        ans += dfs(v, u, min(k, w));
    }
    return ans;
}

int main()
{
    int a, b;
    scanf("%d %d", &N, &M);
    init();
    for (int i = 0; i < M; i++)
    {
        scanf("%d %d", &a, &b);
        add(a, b, 1);
        add(b, a, 1);
    }
    gomoryhu();
    int ans = 0;
    for (int i = 1; i <= N; i++)
    {
        ans += dfs(i, -1, INF);
    }
    printf("%d\n", ans / 2);
}

```

5.24 Euler Tour: thuật toán Fleury

Euler tour.

Mã nguồn mẫu: Graphs/euler.cpp

```

#include <vector>
#include <stack>
#define MAXN 400009
using namespace std;

vector<int> adjList[MAXN];

```

```

int N, M;

vector<int> euler(int s = 0) {
    vector<int> work(N, 0), in(N, 0), out(N, 0), tour;
    for(int u = 0; u < N; u++) {
        for(int i=0; i<(int)adjList[u].size(); i++) {
            int v = adjList[u][i];
            out[u]++; in[v]++;
        }
    }
    int cntin = 0, cntout = 0;
    for (int u = 0; u < N; u++) {
        if (in[u] == out[u]+1) {
            cntout++;
            if (cntout == 2) return tour;
        }
        else if (out[u] == in[u]+1) {
            cntin++; s = u;
            if (cntin == 2) return tour;
        }
        else if (out[u] != in[u] || in[u]==0) return tour;
    }
    stack<int> dfs;
    dfs.push(s);
    while (!dfs.empty()) {
        int u = dfs.top();
        if (work[u] < (int)adjList[u].size()) {
            dfs.push(adjList[u][work[u]++]);
        }
        else {
            tour.push_back(u);
            dfs.pop();
        }
    }
    int n = tour.size();
    for(int i=0; 2*i<n; i++) swap(tour[i], tour[n-i-1]);
    return tour;
}

/*
 * Codeforces 508D
 */

#include <map>
#include <cstring>
#include <cstdio>

typedef pair<char, char> cc;
cc id2cc[MAXN];
map<cc, int> cc2id;
int deg[MAXN];

int main()
{
    char s[5];
    int u, v;
    N = 0;
    scanf("%d", &M);
    memset(&deg, 0, sizeof deg);
    for(int i=0; i<M; i++) {
        scanf(" %s", s);
        cc p1 = make_pair(s[0], s[1]);
        cc p2 = make_pair(s[1], s[2]);
        if (!cc2id.count(p1)) {
            //printf("%c%c: %d\n", p1.first, p1.second, N);
            id2cc[N] = p1;
            cc2id[p1] = N++;
        }
        if (!cc2id.count(p2)) {

```

```

        //printf("%c%c: %d\n", p2.first, p2.second, N);
        id2cc[N] = p2;
        cc2id[p2] = N++;
    }
    u = cc2id[p1];
    v = cc2id[p2];
    adjList[u].push_back(v);
    deg[v]++;
}
vector<int> ans = euler();
int n = ans.size();
if (M + 1 != n) {
    printf("NO\n");
}
else {
    printf("YES\n");
    for(int i=0; i<n; i++) {
        printf("%c", id2cc[ans[i]].first);
    }
    printf("%c\n", id2cc[ans.back()].second);
}
return 0;
}

```

5.25 Dominator Tree

Cây dominator.

Mã nguồn mẫu: Graphs/dominator/tree.cpp

```

#include <vector>
using namespace std;
#define MAXN 200009

vector<int> adjList[MAXN], revAdjList[MAXN]; //grafo
vector<int> dtree[MAXN]; //dominator tree
int sdom[MAXN], dom[MAXN], idom[MAXN], N, M; //semi-dominator, immediate dominator pelo indice, immediate
↳ dominator, tamanhos
int dsu[MAXN], best[MAXN]; //disjoint sets
int par[MAXN], num[MAXN], rev[MAXN], cnt; //dfs

int find(int x) {
    if (x == dsu[x]) return x;
    int y = find(dsu[x]);
    if (sdom[best[x]] > sdom[best[dsu[x]]]) best[x] = best[dsu[x]];
    return dsu[x] = y;
}

void dfs(int u) {
    num[u] = cnt; rev[cnt++] = u;
    for (int i=0; i<(int)adjList[u].size(); i++) {
        int v = adjList[u][i];
        if (num[v] >= 0) continue;
        dfs(v);
        par[num[v]] = num[u];
    }
}

void dominator() {
    for (int u = 1; u <= N; u++) {
        num[u] = -1; dtree[u].clear();
        dsu[u] = best[u] = sdom[u] = u;
    }
    cnt = 0;
    dfs(1);
    for (int j = cnt - 1, u; u = rev[j], j > 0; j--) {
        for (int i = 0; i<(int)revAdjList[u].size(); i++) {

```



```

        int y = num[revAdjList[u][i]];
        if (y == -1) continue;
        find(y);
        if (sdom[best[y]] < sdom[j]) sdom[j] = sdom[best[y]];
    }
    dtree[sdom[j]].push_back(j);
    int x = dsu[j] = par[j];
    for (int i=0; i<(int)dtree[x].size(); i++) {
        int z = dtree[x][i];
        find(z);
        if (sdom[best[z]] < x) dom[z] = best[z];
        else dom[z] = x;
    }
    dtree[x].clear();
}
idom[1] = -1;
for (int i = 1; i < cnt; i++) {
    if (sdom[i] != dom[i]) dom[i] = dom[dom[i]];
    idom[rev[i]] = rev[dom[i]];
    dtree[rev[dom[i]]].push_back(rev[i]);
}
}

/*
 * Codeforces 100513L
 */

#include <cstdio>
#include <set>

int eu[MAXN], ev[MAXN];
int st[MAXN], en[MAXN], k;

void dfs2(int u, int p) {
    st[u] = k++;
    for(int i=0; i<(int)dtree[u].size(); i++) {
        int v = dtree[u][i];
        if (v != p) dfs2(v, u);
    }
    en[u] = k++;
}

int main() {
    int u, v;
    set<int> ans;
    while(scanf("%d %d", &N, &M) != EOF) {
        for(int j=1; j<=M; j++) {
            scanf("%d %d", &u, &v);
            adjList[u].push_back(v);
            revAdjList[v].push_back(u);
            eu[j] = u;
            ev[j] = v;
        }
        dominator();
        ans.clear();
        k = 0;
        dfs2(1, -1);
        for(int j=1; j<=M; j++) {
            u = eu[j];
            v = ev[j];
            if (st[v] <= st[u] && en[v] >= en[u]) continue;
            if (num[u] == -1 || num[v] == -1) continue;
            ans.insert(j);
        }
        printf("%u\n", ans.size());
        for(set<int>::iterator it = ans.begin(); it != ans.end(); it++) {
            printf("%d ", *it);
        }
    }
}

```

```

        printf("\n");
        for(int i=1; i<=N; i++) {
            adjList[i].clear();
            revAdjList[i].clear();
        }
    }
    return 0;
}

/*
 * COJ 2609
 */

/*
#include <cstdio>
#define MAXLOGN 22

int level[MAXN];
int P[MAXN][MAXLOGN];

void derevhdfs(int u) {
    for(int i=0; i<(int)dtree[u].size(); i++) {
        int v = dtree[u][i];
        if (v == P[u][0]) continue;
        P[v][0] = u;
        level[v] = 1 + level[u];
        derevhdfs(v);
    }
}

void computeP(int root) {
    level[root]=0;
    P[root][0]=root;
    derevhdfs(root);
    for(int j = 1; j < MAXLOGN; j++)
        for(int i = 1; i <= N; i++)
            P[i][j] = P[P[i][j-1]][j-1];
}

int LCA(int a, int b) {
    if(level[a] > level[b]) swap(a, b);
    int d = level[b] - level[a];
    for(int i=0; i<MAXLOGN; i++) {
        if((d & (1<<i)) != 0) b = P[b][i];
    }
    if(a == b) return a;
    for(int i = MAXLOGN-1; i>=0; i--)
        while(P[a][i] != P[b][i]) {
            a=P[a][i]; b=P[b][i];
        }
    return P[a][0];
}

int main() {
    int u, v;
    while(scanf("%d %d", &N, &M) != EOF) {
        for(int j=1; j<=M; j++) {
            scanf("%d %d", &u, &v);
            adjList[u].push_back(v);
        }
        dominator();
        computeP(1);
        int ans = 0;
        for(int i=1; i<=N; i++) {
            for(int j=1; j<i; j++) {
                if (LCA(i, j) == 1) ans++;
            }
        }
        printf("%d\n", ans);
    }
    return 0;
}

```

```

}
*/

```

5.26 Đồ thị đáng chú ý

Các họ đồ thị thường gặp: đầy đủ, hai phía đầy đủ, cây, rừng, DAG, Euler, Hamilton, phẳng, chordal, split và các biến thể xuất hiện trong bài toán.

5.27 Định lý Erdos-Gallai

Điều kiện đủ để một dãy là dãy bậc đồ thị: tổng bậc chẵn và các bất đẳng thức Erdos-Gallai; có thể dựng bằng Havel-Hakimi.

Mã nguồn mẫu: Graphs/erdosgallai.cpp

```

#include <vector>
#include <algorithm>
#include <functional>
using namespace std;

/*
 * Erdos-Gallai's theorem
 * O(nlogn), O(n) if already sorted
 * checks if a given array of degrees can represent a graph
 */

bool erdosgallai(vector<int> d) {
    sort(d.begin(), d.end(), greater<int>());
    vector<long long> pd(d.size());
    int n = d.size(), p = n-1;
    for(int i = 0; i < n; i++)
        pd[i] = d[i] + (i > 0 ? pd[i-1] : 0);
    for(int k = 1; k <= n; k++) {
        while(p >= 0 && d[p] < k) p--;
        long long sum;
        if (p >= k-1) sum = (p-k+1)*1ll*k + pd[n-1] - pd[p];
        else sum = pd[n-1] - pd[k-1];
        if (pd[k-1] > k*(k-1LL) + sum) return false;
    }
    return pd[n-1] % 2 == 0;
}

/*
 * URI 1462
 */

#include <cstdio>

int main() {
    int N;
    vector<int> d;
    while(scanf("%d", &N) != EOF) {
        d.resize(N);
        for(int i = 0; i < N; i++) {
            scanf("%d", &d[i]);
        }
        if (erdosgallai(d)) printf("possivel\n");
        else printf("impossivel\n");
    }
    return 0;
}

```

5.28 Định lý và công thức đồ thị

Công thức Cayley: có n^{n-2} cây khung của đồ thị đầy đủ n đỉnh. Euler cho đồ thị phẳng: $V-E+F=2$. Số cây khung của $K_{m,n}$ là $m^{n-1} n^{m-1}$. Định lý Kirchhoff dùng cofactor của ma trận Laplace. König: phủ đỉnh nhỏ nhất trong đồ thị hai phía bằng matching cực đại. Dilworth/Mirsky liên hệ chain và antichain trong poset. Menger cho cạnh/đỉnh liên hệ số phần tử cần xóa với số đường đi độc lập. Prüfer là song ánh giữa cây n đỉnh và mã dài $n-2$. Hall cho điều kiện matching hoàn hảo một phía. Dirac và Ore là điều kiện đủ cho đường Hamilton.

6 Toán học

6.1 Số học modular

MDC, BCNN, Euclid mở rộng, nghịch đảo/chia modular, lũy thừa modular, phương trình Diophantine, nhân modular tránh overflow, Pascal, nghịch đảo modulo và CRT.

Mã nguồn mẫu: Math/mod.cpp

```
template <typename T>
T gcd(T a, T b) {
    return b == 0 ? a : gcd(b, a % b);
}

template <typename T>
T lcm(T a, T b) {
    return a * (b / gcd(a, b));
}

template <typename T>
T extGcd(T a, T b, T& x, T& y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    else {
        T g = extGcd(b, a % b, y, x);
        y -= a / b * x;
        return g;
    }
}

template <typename T>
T modInv(T a, T m) {
    T x, y;
    extGcd(a, m, x, y);
    return (x % m + m) % m;
}

template <typename T>
T modDiv(T a, T b, T m) {
    return ((a % m) * modInv(b, m)) % m;
}

template<typename T>
T modMul(T a, T b, T m) {
    T x = 0, y = a % m;
    while (b > 0) {
        if (b % 2 == 1) x = (x + y) % m;
        y = (y * 2) % m;
        b /= 2;
    }
    return x % m;
}

template<typename T>
T modExp(T a, T b, T m) {
    if (b == 0) return (T)1;
    T c = modExp(a, b / 2, m);
    c = (c * c) % m;
    if (b % 2 != 0) c = (c*a) % m;
    return c;
}

template<typename T>
void diophantine(T a, T b, T c, T& x, T& y) {
    T d = extGcd(a, b, x, y);
```

```

    x *= c / d; y *= c / d;
}

#define MAXN 1000009
typedef long long ll;

ll fat[MAXN];
void preprocessfat(ll m) {
    fat[0] = 1;
    for(ll i=1; i<MAXN; i++) {
        fat[i] = (i*fat[i-1])%m;
    }
}

template<typename T>
T pascal(int n, int k, T m) {
    return modDiv(fat[n], (fat[k]*fat[n-k])%m, m);
}

template<typename T>
void allInv(T inv[], T p) {
    inv[1] = 1;
    for (int i = 2; i < p; i++)
        inv[i] = (p - (p/i)*inv[p%i]%p)%p;
}

```

6.2 Định lý phần dư Trung Hoa tổng quát

CRT khi các modulo không nhất thiết nguyên tố cùng nhau.

Mã nguồn mẫu: Math/generalizedcrt.cpp

```

#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll extGcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    else {
        ll g = extGcd(b, a % b, y, x);
        y -= a / b * x;
        return g;
    }
}

/*
 * Generalized CRT
 */

ll crt(ll rem[], ll mod[], int n) {
    if (n == 0) return 0;
    ll ans = rem[0], m = mod[0];
    for (int i = 1; i < n; i++) {
        ll x, y;
        ll g = extGcd(mod[i], m, x, y);
        if ((ans - rem[i])%g != 0) return -1;
        ans = ans + 1ll*(rem[i]-ans)*(m/g)*y;
        m = (mod[i]/g)*(m/g)*g;
        ans = (ans%m + m)%m;
    }
    return ans;
}
/*

```

```

* COMPILATION TEST
*/

int main() {
    long long a[2] = {2, 7};
    long long p[2] = {10, 15};
    printf("%lld\n", crt(a, p, 2));
}

```

6.3 Số nguyên tố

Crivo Eratosthenes, kiểm tra nguyên tố, phân tích thừa số, số ước, phi Euler.

Mã nguồn mẫu: Math/primes.cpp

```

#include <bitset>
#include <cstdio>
#include <vector>
#include <cstring>
using namespace std;
#define MAXN 10000009

typedef long long int ll;
ll sievesize, numDiffp[MAXN];
bitset<MAXN> bs;
vector<ll> primes;

void sieve(ll n) {
    sievesize = n + 1;
    bs.set();
    bs[0] = bs[1] = 0;
    for (ll i = 2; i <= sievesize; i++) {
        if (bs[i]) {
            for (ll j = i * i; j <= (ll)sievesize; j += i) bs[j] = 0;
            primes.push_back(i);
        }
    }
}

//O(sqrt(n)/logn), works if factors are less than the biggest prime in primes
bool isPrimeSieve(ll N) {
    if (N <= (ll)sievesize) return bs[N];
    for (int i = 0; i < (int)primes.size() && primes[i]*primes[i] <= N; i++)
        if (N % primes[i] == 0) return false;
    return true;
}

//O(sqrt(n))
bool isPrime(ll N) {
    if (N < 0) return isPrime(-N);
    for (ll i=2; i*i <= N; i++) {
        if (N % i == 0) return false;
    }
    return true;
}

//O(sqrt(n))
bool isPrimeFast(ll n) {
    if (n < 0) n = -n;
    if (n < 5 || n % 2 == 0 || n % 3 == 0)
        return (n == 2 || n == 3);
    ll maxP = sqrt(n) + 2;
    for (ll p = 5; p < maxP; p += 6) {
        if (p < n && n % p == 0) return false;
        if (p+2 < n && n % (p+2) == 0) return false;
    }
    return true;
}

```

```

}

typedef pair<ll, int> ii;
vector<ii> primeFactors(ll N) {
    vector<ii> factors;
    ll i = 0, p = primes[i];
    while (p * p <= N) {
        if (N % p == 0) {
            int cnt = 0;
            while (N % p == 0) N /= p, cnt++;
            factors.push_back(ii(p, cnt));
        }
        p = primes[++i];
    }
    if (N != 1) factors.push_back(ii(N, 1));
    return factors;
}

ll numDiv(ll N) {
    ll i = 0, p = primes[i], ans = 1;
    while (p * p <= N) {
        ll pw = 0;
        while (N % p == 0) { N /= p; pw++; }
        ans *= (pw + 1);
        p = primes[++i];
    }
    if (N != 1) ans *= 2;
    return ans;
}

ll eulerPhi(ll N) {
    ll i = 0, p = primes[i], ans = N;
    while (p * p <= N) {
        if (N % p == 0) ans -= ans / p;
        while (N % p == 0) N /= p;
        p = primes[++i];
    }
    if (N != 1) ans -= ans / N;
    return ans;
}

void numDiffp() {
    memset(numDiffp, 0, sizeof numDiffp);
    for (int i = 2; i < MAXN; i++)
        if (numDiffp[i] == 0)
            for (int j = i; j < MAXN; j += i)
                numDiffp[j]++;
}

int main() {
    sieve(MAXN); // can go up to 10^7 (need few seconds)
    printf("%d\n", isPrime(2147483647)); // 10-digits prime
    printf("%d\n", isPrime(136117223861LL)); // not a prime, 104729*1299709
    return 0;
}

```

6.4 Công thức Legendre

Cho n và prime p , tính số mũ lớn nhất của p chia $n!$ trong $O(\log n)$.

Mã nguồn mẫu: Math/legendre.cpp

```

#include <cstdio>
#include <cstring>

typedef long long ll;

```



```

/*
 * Legendre's formula
 */

ll legendre(ll n, ll p) {
    int ans = 0;
    ll prod = p;
    while(prod <= n) {
        ans += n/prod;
        prod *= p;
    }
    return ans;
}

/*
 * TEST MATRIX
 */

ll fac[30];

int brute(ll n, ll p) {
    int ans = -1;
    ll prod = 1LL;
    while(fac[n]%prod == 0) {
        //printf("%lld%%lld = 0\n", fac[n], prod);
        ans++;
        prod *= p;
    }
    return ans;
}

bool test(ll p) {
    for(ll n=1; n<=20LL; n++) {
        if (legendre(n, p) != brute(n, p)) {
            //printf("legendre(%lld, %lld) = %d\n", n, p, legendre(n, p));
            //printf("brute(%lld, %lld) = %d\n", n, p, brute(n, p));
            return false;
        }
    }
    return true;
}

int N;
bool prime[200000];

int main() {
    N = 200000;
    fac[0] = 1LL;
    for(ll i=1LL; i<=20LL; i++) {
        fac[i] = i*fac[i-1];
    }
    memset(&prime, true, sizeof prime);
    prime[0] = prime[1] = false;
    for(int i=2; i<N; i++) {
        if (prime[i]) {
            if (!test(i)) printf("failed for prime p = %d\n", i);
            for(int j=2*i; j<N; j+=i) prime[j] = false;
        }
    }
    return 0;
}

```

6.5 Tổng GCD

Dùng phi Euler để tính $F(n) = \sum_{i=1}^n \gcd(i, n)$.

Mã nguồn mẫu: `Math/gcdsum.cpp`

```

#include <bitset>
#include <vector>
using namespace std;
#define MAXN 200009

typedef unsigned long long ll;
int phi[MAXN], sievesize;
ll f[MAXN];

void gcdSieve(int n) {
    sievesize = n+1;
    for(int i=0; i <= sievesize; i++) phi[i] = f[i] = 0;
    for(int i = 1; i <= sievesize; i++) {
        phi[i] += i;
        for(int j = i; j <= sievesize; j += i) {
            if (j > i) phi[j] -= phi[i];
            f[j] += j / i * phi[i];
        }
    }
}

bitset<MAXN> bs;
vector<ll> primes;

void sieve(ll n) {
    sievesize = n + 1;
    bs.set();
    bs[0] = bs[1] = 0;
    for (ll i = 2; i <= sievesize; i++) {
        if (bs[i]) {
            for (ll j = i * i; j <= (ll)sievesize; j += i) bs[j] = 0;
            primes.push_back(i);
        }
    }
}

ll F(ll N) {
    ll i = 0, p = primes[i];
    ll ans = 1;
    while (p * p <= N) {
        if (N % p == 0) {
            N /= p;
            ll prod = 1, e = 1;
            while (N % p == 0) {
                N /= p, e++;
                prod *= p;
            }
            ans *= prod * ((p-1)*e + p);
        }
        p = primes[++i];
    }
    ans *= 2*N-1;
    return ans;
}

/*
 * UVa 11424
 */

#include <cstdio>
ll sum[MAXN];

int main() {
    gcdSieve(MAXN-2);
    sieve(MAXN-2);
    sum[0] = 0;
    for(int i=1; i<MAXN; i++) sum[i] = F(i) + sum[i-1];
    ll N;
    while(scanf("%llu", &N), N) {

```

```

        printf("%llu\n", sum[N] - N*(N+1)/2);
    }
    return 0;
}

```

6.6 Crivo tuyến tính và hàm nhân tính

Crivo $O(n)$ đồng thời tính các hàm nhân tính.

Mã nguồn mẫu: Math/linearsieve.cpp

```

#include <vector>
#include <bitset>
using namespace std;
#define MAXN 1000009
#include <cstdio>

/*
 * Linear sieve of Eratosthenes and computing multiplative functions
 */

vector<int> primes;
bitset<MAXN> bs;
int f[MAXN], pw[MAXN];

int g(int p, int k) {
    if (p == 1) return 1;
    return (k==0) - (k==1);
    //int q = 1;
    //for(int i = 1; i < k; i++) q *= p;
    //return q*(p-1);
}

void sieve(int n) {
    bs.set(); bs[0] = bs[1] = 0;
    primes.clear(); f[1] = g(1, 1);
    for (int i = 2; i <= n; i++) {
        if (bs[i]) {
            primes.push_back(i);
            f[i] = g(i, 1); pw[i] = 1;
        }
        for (int j = 0; j < primes.size() && i*1ll*primes[j] <= n; j++) {
            bs[i * primes[j]] = 0;
            if (i % primes[j] == 0) {
                int pwr = 1;
                for(int k = 0; k < pw[i]; k++) pwr *= primes[j];
                f[i * primes[j]] = f[i / pwr] * g(primes[j], pw[i]+1);
                pw[i * primes[j]] = pw[i] + 1;
                break;
            } else {
                f[i * primes[j]] = f[i] * f[primes[j]];
                pw[i * primes[j]] = 1;
            }
        }
    }
}

/*
 * TEST MATRIX
 */
#include <cstdio>

void test(int n) {
    sieve(n);
    for(int i = 1; i <= n; i++) {
        printf("f[%3d] = %2d, %d is %sprime, \n", i, f[i], i, bs[i] ? "" : "not ");
    }
}

```

```

}

int main() {
    test(100);
}

```

6.7 Nghịch đảo Mobius

Nếu $f(n) = \sum_{d|n} g(d)$ thì $g(n) = \sum_{d|n} f(d) \mu(n/d)$; áp dụng cho đếm cặp nguyên tố cùng nhau và poset.

6.8 Số Catalan

$Cat(0)=1$, $Cat(n)=(4n-2)/(n+1) Cat(n-1)$. Đếm cây nhị phân đầy đủ, ngoặc đúng, cách đặt ngoặc, tam giác hóa đa giác lồi và đường đi đơn điệu.

6.9 Số Stirling loại một

$s(n,m)=s(n-1,m-1)-(n-1)s(n-1,m)$, với $s(n,n)=1$ và $s(n,0)=0$.

Mã nguồn mẫu: `Math/stirling_first.cpp`

```

#include <vector>
#include <cstring>
using namespace std;

/*
 * Modular inverse
 */

template <typename T>
T extGcd(T a, T b, T& x, T& y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    else {
        T g = extGcd(b, a % b, y, x);
        y -= a / b * x;
        return g;
    }
}

template <typename T>
T modInv(T a, T m) {
    T x, y;
    extGcd(a, m, x, y);
    return (x % m + m) % m;
}

/*
 * NTT - Number Theoretic Transform O(nlogn)
 */

typedef long long ll;

const ll mod[3] = {1004535809LL, 1092616193LL, 998244353LL};
const ll root[3] = {12289LL, 23747LL, 15311432LL};
const ll root_1[3] = {313564925LL, 642907570LL, 469870224LL};
const ll root_pw[3] = {1LL<<21, 1LL<<21, 1LL<<23};

void ntt(vector<ll> & a, bool invert, int m) {
    ll n = (ll)a.size();

```

```

for(ll i=1, j=0; i<n; ++i) {
    ll bit = n >> 1;
    for (; j>=bit; bit>>=1) j -= bit;
    j += bit;
    if (i < j) swap(a[i], a[j]);
}
for(ll len=2, wlen; len<=n; len<=1) {
    wlen = invert ? root_1[m] : root[m];
    for (ll i=len; i<root_pw[m]; i<=1)
        wlen = (wlen * wlen % mod[m]);
    for(ll i=0; i<n; i+=len) {
        for(ll j=0, w=1; j<len/2; ++j) {
            ll u = a[i+j], v = a[i+j+len/2] * w % mod[m];
            a[i+j] = (u+v < mod[m] ? u+v : u+v-mod[m]);
            a[i+j+len/2] = (u-v >= 0 ? u-v : u-v+mod[m]);
            w = w * wlen % mod[m];
        }
    }
}
if(invert) {
    ll nrev = modInv(n, mod[m]);
    for (ll i=0; i<n; ++i)
        a[i] = a[i] * nrev % mod[m];
}
}

void convolution(vector<ll> a, vector<ll> b, vector<ll> & res, int m) {
    ll n = 1;
    while(n < max (a.size(), b.size())) n <= 1;
    n <= 1;
    a.resize(n), b.resize(n);
    ntt(a, false, m); ntt(b, false, m);
    res.resize(n);
    for(int i=0; i<n; ++i) res[i] = (a[i]*b[i])%mod[m];
    ntt(res, true, m);
}

/*
 * Stirling numbers
 * Compute s(n, k) for all k in [0, n] in O(nlog^2n)
 */

void compute(vector<ll> & res, int l, int r, int m) {
    if (l == r) {
        res = vector<ll>({r, 1LL});
        return;
    }
    vector<ll> a, b;
    compute(a, l, (l + r) / 2, m);
    compute(b, (l + r) / 2 + 1, r, m);
    convolution(a, b, res, m);
}

void stirling_first(vector<ll> & res, int n, int m) {
    if (n == 0) {
        res = vector<ll>({1LL});
        return;
    }
    compute(res, 0, n-1, m);
    while(res.back() == 0) res.pop_back();
}

/*
 * TEST MATRIX
 */

#include <stdio>

void printmatrix(int _n, int m) {

```

```

    printf("Stirling matrix of first kind mod %I64d:\n", mod[m]);
    for(int n = 0; n <= _n; n++) {
        vector<ll> row;
        stirling_first(row, n, m);
        for(int i = 0; i < (int)row.size(); i++) {
            printf("%I64d ", row[i]);
        }
        printf("\n");
    }
}

/*
 * Codeforces 960G
 */

#define MAXN 100009

template <typename T>
T modDiv(T a, T b, T m) {
    return ((a % m) * modInv(b, m)) % m;
}

ll fat[MAXN];
void preprocessfat(ll m) {
    fat[0] = 1;
    for(ll i=1; i<MAXN; i++) {
        fat[i] = (i*fat[i-1])%m;
    }
}

template<typename T>
T pascal(int n, int k, T m) {
    return modDiv(fat[n], (fat[k]*fat[n-k])%m, m);
}

int N, A, B;

int main() {
    scanf("%d %d %d", &N, &A, &B);
    preprocessfat(mod[2]);
    vector<ll> arr;
    stirling_first(arr, N-1, 2);
    ll ans;
    if (A+B-2 < 0 || A+B-2 >= int(arr.size())) {
        ans = 0;
    }
    else {
        ans = arr[A+B-2];
        ans *= pascal(A+B-2, A-1, mod[2]);
        ans %= mod[2];
    }
    printf("%I64d\n", ans);
    return 0;
}

```

6.10 Số Stirling loại hai

$S(n,m)=S(n-1,m-1)+mS(n-1,m)$; đếm cách chia n đối tượng vào đúng m tập không rỗng.

6.11 Hằng thức tổng nhị thức

Gồm Vandermonde, hockey-stick và định lý đa thức.

6.12 Bổ đề Burnside và định lý đếm Polya

$|X/G| = (1/|G|) \sum_{g \in G} I(g)$; với hoán vị và k màu, $|X/G| = (1/|G|) \sum k^{C(g)}$. Cần G đóng dưới hợp thành.

6.13 Kiểm tra nguyên tố Miller-Rabin

Xác suất, $O(k \log^2 n)$, đúng đã chứng minh cho $n < 2^{64}$ với các cơ sở đã cho.

Mã nguồn mẫu: Math/millerrabin.cpp

```
template<typename T>
T modMul(T a, T b, T m) {
    T x = 0, y = a % m;
    while (b > 0) {
        if (b % 2 == 1) x = (x + y) % m;
        y = (y * 2) % m;
        b /= 2;
    }
    return x % m;
}

template<typename T>
T modExp(T a, T b, T m) {
    if (b == 0) return (T)1;
    T c = modExp(a, b / 2, m);
    c = (c * c) % m;
    if (b % 2 != 0) c = (c*a) % m;
    return c;
}

/*
 * Miller-Rabin primality test
 *  $O(k \log^2 n)$ ,  $k = 9$ 
 * probabilistic, but proven correct for  $n < 2^{64}$ 
 */

bool miller(long long n) {
    const int pn = 9;
    const int p[] = {2, 3, 5, 7, 11, 13, 17, 19, 23};
    for (int i = 0; i < pn; i++)
        if (n % p[i] == 0) return n == p[i];
    if (n < p[pn - 1]) return false;
    long long s = 0, t = n - 1;
    while (~t & 1) t >>= 1, ++s;
    for (int i = 0; i < pn; ++i) {
        long long pt = modExp((long long)p[i], t, n);
        if (pt == 1) continue;
        bool ok = false;
        for (int j = 0; j < s && !ok; j++) {
            if (pt == n - 1) ok = true;
            pt = modMul(pt, pt, n);
        }
        if (!ok) return false;
    }
    return true;
}

/*
 * TEST MATRIX
 */

#include <stdio>
#include <stdlib>

bool brutePrime(long long n) {
```

```

        if (n <= 1) return false;
        for(long long i = 2; i*i <= n; i++) {
            if (n % i == 0) return false;
        }
        return true;
    }

bool test(int ntest) {
    for(int t = 1; t <= ntest; t++) {
        long long n = rand();
        if (brutePrime(n) != miller(n)) {
            printf("failed on test %lld, prime = %d\n", n, brutePrime(n));
            return false;
        }
        n = t;
        if (brutePrime(n) != miller(n)) {
            printf("failed on test %lld, prime = %d\n", n, brutePrime(n));
            return false;
        }
    }
    printf("all tests passed\n");
    return true;
}

int main() {
    test(10000000);
    long long n;
    while(scanf("%lld", &n) != EOF) {
        printf("%lld is %sprime\n", n, miller(n) ? "" : "not ");
    }
    return 0;
}

```

6.14 Thuật toán Pollard-Rho

Trả về một thừa số của n ; dùng cho n lớn.

Mã nguồn mẫu: Math/pollardrho.cpp

```

#include <cstdio>
#include <algorithm>
using namespace std;

template <typename T>
T gcd(T a, T b) {
    return b == 0 ? a : gcd(b, a % b);
}

template<typename T>
T modMul(T a, T b, T m) {
    T x = 0, y = a % m;
    while (b > 0) {
        if (b % 2 == 1) x = (x + y) % m;
        y = (y * 2) % m;
        b /= 2;
    }
    return x % m;
}

/*
 * Pollard-Rho factorization
 * use only for n > 9*10^13
 */

template<typename T>
T pollard(T n) {
    int i = 0, k = 2, d;

```



```

    T x = 3, y = 3;
    while (true) {
        i++;
        x = (modMul(x, x, n) + n - 1) % n;
        d = gcd(abs(y - x), n);
        if (d != 1 && d != n) return d;
        if (i == k) y = x, k *= 2;
    }
}

/*
 *TEST MATRIX
 */
#include <cstdlib>

int main()
{
    long long n;
    while(scanf(" %I64d", &n)!=EOF) {
        long long r = pollard(n);
        printf("%I64d->%I64d\n", n, r);
        if (n%r != 0) {
            printf("Failed on test\n");
        }
    }
    return 0;
}

```

6.15 Baby-Step Giant-Step cho logarit rời rạc

Giải $a^x \equiv b \pmod{m}$ trong $O(\sqrt{m} \log m)$.

Mã nguồn mẫu: Math/babysteps.cpp

```

#include <map>
#include <cmath>
using namespace std;

/*
 * Baby Steps Algorithm for Discrete Logarithm
 */

template <typename T>
T baby (T a, T b, T m) {
    a %= m; b %= m;
    T n = (T) sqrt (m + .0) + 1, an = 1;
    for (T i=0; i<n; ++i) an = (an * a) % m;
    map<T, T> vals;
    for (T i=1, cur=an; i<=n; ++i) {
        if (!vals.count(cur)) vals[cur] = i;
        cur = (cur * an) % m;
    }
    for (T i=0, cur=b; i<=n; ++i) {
        if (vals.count(cur)) {
            T ans = vals[cur] * n - i;
            if (ans < m) return ans;
        }
        cur = (cur * a) % m;
    }
    return -1;
}

/*
 * HackerEarth Dexter plays with GP
 */

#include <cstdio>

```

```

typedef long long ll;

int main() {
    int T;
    ll r, x, p, a, b;
    scanf("%d", &T);
    while(T-->0) {
        scanf("%lld %lld %lld", &r, &x, &p);
        a = r;
        b = x*(r-1) + 1;
        printf("%lld\n", baby(a, b, p));
    }
    return 0;
}

```

6.16 Trò chơi Nim và định lý Sprague-Grundy

Trạng thái thắng khi nimber khác 0; nimber của trạng thái là mex của các trạng thái đi được; chơi song song lấy xor.

Mã nguồn mẫu: Math/mex.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define MAXN 10009

int mex[MAXN] = {0,0,0,0,0,1,1,1,2,2,0,3,3,1,1,1,0,4,3,3,3,2,2,2,4,4,0,5,5,2,2,2,3,3,0,5,0,1,1,1,3,3,3,5,6,4,4,1,0,5,5,6,6,2,7,7,7,8};

char str[MAXN];

void stevehalim() {
    for(int n=1; n<MAXN; n++) {
        if (n <= 2) { mex[n] = 0; continue; }
        set<int> jog;
        for(int i=3; i<=n-2; i++) {
            jog.insert(mex[i-1]^mex[n-i]);
        }
        int cnt = 0;
        while(jog.count(cnt)) cnt++;
        mex[n] = cnt;
    }
    //for(int n=1; n<MAXN; n++) printf("%d,", mex[n]);
}

/*
* URI 1130
*/

int main() {
    //stevehalim();
    int N;
    while(scanf("%d", &N), N) {
        scanf(" %s", str);
        bool ok = false;
        for(int i=1; i<N; i++) {
            if (str[i] == 'X' && str[i-1] == 'X') ok = true;
        }
        for(int i=2; i<N; i++) {
            if (str[i] == 'X' && str[i-2] == 'X') ok = true;
        }
        if (ok) {
            printf("S\n"); continue;
        }
        int last = -3, ans = 0;
        for(int i=0; i<N; i++) {
            if (str[i] == 'X') {
                //printf("size %d\n", i-last-1);
            }
        }
    }
}

```

```

        ans ^= mex[i-last-1];
        last = i;
    }
}
//printf("size %d\n", N-last+1);
ans ^= mex[N-last+1];
if (ans != 0) printf("S\n");
else printf("N\n");
}
return 0;
}

```

6.17 Bộ ba Pythagoras

Mọi bộ (a, b, c) , $a^2 + b^2 = c^2$, sinh bởi $a = k(m^2 - n^2)$, $b = 2kmn$, $c = k(m^2 + n^2)$.

6.18 Ma trận

Các phép toán ma trận cơ bản.

Mã nguồn mẫu: Math/matrix.cpp

```

#include <vector>
using namespace std;

/*
 * Matrix operations
 */

typedef long long ll;
typedef vector< vector< double > > matrix;

matrix operator +(matrix a, matrix b) {
    int n = (int)a.size();
    int m = (int)a[0].size();
    matrix c;
    c.resize(n);
    for(int i=0; i<n; i++) {
        c[i].resize(m);
        for(int j=0; j<m; j++)
            c[i][j] = a[i][j] + b[i][j];
    }
    return c;
}

matrix operator *(matrix a, matrix b) {
    int n = (int)a.size();
    //assert(a[0].size() == b.size());
    int m = (int)b.size();
    int p = (int)b[0].size();
    matrix c(n, vector<double>(p));
    vector<double> col(m);
    for (int j = 0; j < p; j++) {
        for (int k = 0; k < m; k++)
            col[k] = b[k][j];
        for (int i = 0; i < n; i++) {
            double s = 0;
            for (int k = 0; k < m; k++)
                s += a[i][k] * col[k];
            c[i][j] = s;
        }
    }
    return c;
}

```

```

matrix operator *(double k, matrix a) {
    int n = (int)a.size();
    int m = (int)a[0].size();
    for(int i=0; i<n; i++) for(int j=0; j<m; j++)
        a[i][j] *= k;
    return a;
}

matrix operator -(matrix a, matrix b) {
    return a + ((-1.0) * b);
}

matrix id(int n) {
    matrix c(n, vector<double>(n));
    for(int i = 0; i < n; i++) c[i][i] = 1;
    return c;
}

#include <cstdio>
void printmatrix(matrix &a) {
    int n = (int)a.size();
    int m = (int)a[0].size();
    for(int i=0; i<n; i++) {
        for(int j=0; j<m; j++) {
            printf("%c%.2f%c ", (i==0 && j==0 ? '[' : ' '), a[i][j], (j < m-1 ? ',' : i == n-1 ? ']' : ' '));
        }
        printf("\n");
    }
}

/*
 * COMPILATION TEST
 */

int main() {}

```

6.19 Lũy thừa ma trận và Fibonacci

Matrix exponentiation.

Mã nguồn mẫu: Math/matrixexp.cpp

```

#include <cstdio>
#include <algorithm>
#include <vector>
#include <ctime>
#include <cmath>
#define EPS 1e-5
using namespace std;

typedef long long ll;
typedef vector< vector< double > > matrix;

matrix operator +(matrix a, matrix b) {
    int n = (int)a.size();
    int m = (int)a[0].size();
    matrix c;
    c.resize(n);
    for(int i=0; i<n; i++) {
        c[i].resize(m);
        for(int j=0; j<m; j++) {
            c[i][j] = a[i][j] + b[i][j];
        }
    }
    return c;
}

```

```

matrix operator *(matrix a, matrix b) {
    int n = (int)a.size();
    //assert(a[0].size() == b.size());
    int m = (int)b.size();
    int p = (int)b[0].size();
    matrix c(n, vector<double>(p));
    double col[m];
    for (int j = 0; j < p; j++) {
        for (int k = 0; k < m; k++)
            col[k] = b[k][j]; //cache friendly
        for (int i = 0; i < n; i++) {
            double s = 0;
            for (int k = 0; k < m; k++)
                s += a[i][k] * col[k];
            c[i][j] = s;
        }
    }
    return c;
}

matrix operator *(double k, matrix a) {
    int n = (int)a.size();
    int m = (int)a[0].size();
    for(int i=0; i<n; i++) {
        for(int j=0; j<m; j++) {
            a[i][j] *= k;
        }
    }
    return a;
}

matrix operator -(matrix a, matrix b) {
    return a + ((-1.0) * b);
}

void printmatrix(matrix &a) {
    int n = (int)a.size();
    int m = (int)a[0].size();
    for(int i=0; i<n; i++) {
        for(int j=0; j<m; j++) {
            printf("%c%.2f%c ", (i==0 && j==0 ? '[' : ' '), a[i][j], (j < m-1 ? ', ' : i == n-1 ? ']' : ' '));
        }
        printf("\n");
    }
}

/*
 * Matrix Exp and fast Fibonacci
 */

matrix id(int n) {
    matrix c(n, vector<double>(n));
    for(int i = 0; i < n; i++) c[i][i] = 1;
    return c;
}

matrix matrixExp(matrix a, int n) {
    if (n == 0) return id(a.size());
    matrix c = matrixExp(a, n/2);
    c = c*c;
    if (n%2 != 0) c = c*a;
    return c;
}

matrix fibo() {
    matrix c(2, vector<double>(2, 1));
    c[1][1] = 0;
    return c;
}

```

```

double fibo(int n) {
    matrix f = matrixExp(fibo(), n);
    return f[0][1];
}

/*
 * TEST MATRIX
 */

int main() {
    int n;
    while(scanf("%d", &n) != EOF) {
        printf("%f\n", fibo(n));
    }
    n = 10000;
    double fib[10000];
    clock_t t = clock();
    for(int i = 0; i < 10000; i++) {
        fib[i] = fibo(i);
    }
    t = clock() - t;
    printf("time %.3f\n", t*1.0/CLOCKS_PER_SEC);
    return 0;
}

```

6.20 Hệ tuyến tính: định thức và khử Gauss

Determinant và Gaussian elimination.

Mã nguồn mẫu: Math/gauss.cpp

```

#include <cstdio>
#include <algorithm>
#include <vector>
#include <ctime>
#include <cmath>
#define EPS 1e-5
using namespace std;

typedef long long ll;
typedef vector< vector< double > > matrix;

matrix operator +(matrix a, matrix b) {
    int n = (int)a.size();
    int m = (int)a[0].size();
    matrix c;
    c.resize(n);
    for(int i=0; i<n; i++) {
        c[i].resize(m);
        for(int j=0; j<m; j++) {
            c[i][j] = a[i][j] + b[i][j];
        }
    }
    return c;
}

matrix operator *(matrix a, matrix b) {
    int n = (int)a.size();
    if (a[0].size() != b.size()) printf("fail\n");
    int m = (int)b.size();
    int p = (int)b[0].size();
    matrix c;
    c.resize(n);
    for(int i=0; i<n; i++) {
        c[i].assign(p, 0);
        for(int j=0; j<p; j++) {

```

```

        for(int k=0; k<m; k++) {
            c[i][j] += a[i][k]*b[k][j];
        }
    }
}
return c;
}

matrix operator *(double k, matrix a) {
    int n = (int)a.size();
    int m = (int)a[0].size();
    for(int i=0; i<n; i++) {
        for(int j=0; j<m; j++) {
            a[i][j] *= k;
        }
    }
    return a;
}

matrix operator -(matrix a, matrix b) {
    return a + ((-1.0) * b);
}

matrix id(int n) {
    matrix c; c.resize(n);
    for(int i=0; i<n; i++) {
        c[i].assign(n, 0);
        c[i][i] = 1;
    }
    return c;
}

void printmatrix(matrix &a) {
    int n = (int)a.size();
    int m = (int)a[0].size();
    for(int i=0; i<n; i++) {
        for(int j=0; j<m; j++) {
            printf("%c%.2f%c ", (i==0 && j==0 ? '[' : ' '), a[i][j], (j < m-1 ? ',' : i == n-1 ? ']' : ' '));
        }
        printf("\n");
    }
}

/*
 * Gauss elimination
 */

void switchLines(matrix &a, int i, int j) {
    int m = (int)a[i].size();
    for(int k = 0; k < m; k++) swap(a[i][k], a[j][k]);
}

void lineSumTo(matrix &a, int i, int j, double c) {
    int m = (int)a[0].size();
    for(int k = 0; k < m; k++) a[j][k] += c*a[i][k];
}

bool gauss(matrix &a, matrix &b, int &switches) {
    switches = 0;
    int n = (int)a.size();
    int m = (int)a[0].size();
    for(int i = 0, l; i < min(n, m); i++) {
        l = i;
        while(l < n && fabs(a[l][i]) < EPS) l++;
        if (l == n) return false;
        switchLines(a, i, l);
        switchLines(b, i, l);
        switches++;
        for(int j=0; j<n; j++) {

```

```

        if (i == j) continue;
        double p = -a[j][i] / a[i][i];
        lineSumTo(a, i, j, p);
        lineSumTo(b, i, j, p);
    }
}
return true;
}

double det(matrix a) {
    int n = a.size();
    matrix b(n);
    for(int i=0; i<n; i++) b[i].resize(1);
    int sw = 0;
    if (gauss(a, b, sw)) {
        double ans = 1;
        for(int i=0; i<n; i++) ans *= a[i][i];
        return sw % 2 == 0 ? ans : -ans;
    }
    return 0.0;
}

/*
 * TEST MATRIX
 */

bool testgauss(int ntest) {
    srand(time(NULL));
    for(int n=1; n<ntest; n++) {
        double d;
        matrix A = id(n), B, X;
        B.resize(n);
        X.resize(n);
        for(int i=0; i<n; i++) {
            X[i].push_back(rand()%10);
            for(int j=0; j<n; j++) {
                A[i][j] = -2 + rand()%10;
            }
        }
        B = A*X;
        d = det(A);
        int sw;
        if (gauss(A, B, sw)) {
            for(int i=0; i<n; i++) {
                if (fabs(X[i][0] - B[i][0]/A[i][i]) > EPS) {
                    printf("failed test %d\n", n);
                    return false;
                }
            }
        }
    }
    printf("all gauss tests passed\n");
    return true;
}

bool testdet(int ntest) {
    matrix a;
    for(int n=1; n<=ntest; n++) {
        a = id(2);
        a[0][0] = rand()%100;
        a[1][0] = rand()%100;
        a[0][1] = rand()%100;
        a[1][1] = rand()%100;
        if (fabs(det(a)-(a[0][0]*a[1][1] - a[0][1]*a[1][0])) > EPS) {
            printf("failed test %d\n", n);
            return false;
        }
    }
}

```



```

        printf("all gauss tests passed\n");
        return true;
    }

    int main() {
        testgauss(100);
        testdet(100);
        return 0;
    }

```

6.21 Nhân ma trận thưa

Sparse matrix multiplication.

Mã nguồn mẫu: Math/matrixsparse.cpp

```

vector< vector<int> > adjA, adjB;

matrix sparsemult(matrix a, matrix b) {
    int n = (int)a.size();
    if (a[0].size() != b.size()) printf("fail\n");
    int m = (int)b.size();
    int p = (int)b[0].size();
    adjA.resize(n);
    for(int i=0; i<n; i++) {
        adjA[i].clear();
        for(int k=0; k<m; k++) {
            if (fabs(a[i][k]) > EPS) adjA[i].push_back(k);
        }
    }
    adjB.resize(p);
    for(int j=0; j<p; j++) {
        adjB[j].clear();
        for(int k=0; k<m; k++) {
            if (fabs(b[k][j]) > EPS) adjB[j].push_back(k);
        }
    }
    matrix c;
    c.resize(n);
    for(int i=0; i<n; i++) {
        c[i].assign(p, 0);
        for(int j=0; j<p; j++) {
            for(int u=0, v=0, k; u<(int)adjA[i].size() && v<(int)adjB[j].size();) {
                if (adjA[i][u] > adjB[j][v]) v++;
                else if (adjA[i][u] < adjB[j][v]) u++;
                else {
                    k = adjA[i][u];
                    c[i][j] += a[i][k]*b[k][j];
                    u++; v++;
                }
            }
        }
    }
    return c;
}

```

6.22 Phương pháp Gauss-Seidel

Lập Gauss-Seidel.

Mã nguồn mẫu: Math/gaussseidel.cpp

```

#include <cstdio>
#include <algorithm>
#include <vector>

```

```

#include <cmath>
#define EPS 1e-5
using namespace std;

typedef long long ll;
typedef vector< vector< double > > matrix;

matrix operator +(matrix a, matrix b) {
    int n = (int)a.size();
    int m = (int)a[0].size();
    matrix c;
    c.resize(n);
    for(int i=0; i<n; i++) {
        c[i].resize(m);
        for(int j=0; j<m; j++) {
            c[i][j] = a[i][j] + b[i][j];
        }
    }
    return c;
}

matrix operator *(matrix a, matrix b) {
    int n = (int)a.size();
    if (a[0].size() != b.size()) printf("fail\n");
    int m = (int)b.size();
    int p = (int)b[0].size();
    matrix c;
    c.resize(n);
    for(int i=0; i<n; i++) {
        c[i].assign(p, 0);
        for(int j=0; j<p; j++) {
            for(int k=0; k<m; k++) {
                c[i][j] += a[i][k]*b[k][j];
            }
        }
    }
    return c;
}

matrix operator *(double k, matrix a) {
    int n = (int)a.size();
    int m = (int)a[0].size();
    for(int i=0; i<n; i++) {
        for(int j=0; j<m; j++) {
            a[i][j] *= k;
        }
    }
    return a;
}

matrix operator -(matrix a, matrix b) {
    return a + ((-1.0) * b);
}

/*
 * Gauss-Seidel method
 */

matrix gaussSeidel(matrix & a, matrix & b, double PREC) {
    int n = (int)a.size();
    matrix x = b, xp = b;
    double error;
    do {
        error = 0.0;
        for(int i=0; i<n; i++) {
            xp[i][0] = b[i][0];
            for(int j=0; j<n; j++) {
                if (i < j) xp[i][0] -= a[i][j]*xp[j][0];
                if (i > j) xp[i][0] -= a[i][j]*x[j][0];
            }
        }
    } while (error > PREC);
    return xp;
}

```

```

        }
        xp[i][0] /= a[i][i];
        error = max(error, fabs(xp[i][0]-x[i][0]));
    }
    x = xp;
} while(error > PREC);
return xp;
}

/*
 * TEST MATRIX
 */
#include <ctime>

bool test(int nTests) {
    srand(time(NULL));
    for(int n=1; n<=nTests; n++) {
        double PREC = 1e-4;
        matrix A, B, X, Xp;
        A.resize(n);
        B.resize(n);
        X.resize(n);
        for(int i=0; i<n; i++) {
            X[i].push_back((rand()%10)/10.0);
            A[i].resize(n);
            for(int j=0; j<n; j++) {
                A[i][j] = -(rand()%10)/100000.0;
            }
            A[i][i] = 1.0;
        }
        B = A*X;
        Xp = gaussSeidel(A, B, PREC);
        for(int i=0; i<n; i++) {
            if (fabs(X[i][0] - Xp[i][0]) > PREC) {
                printf("failed test n = %d\n", n);
                return false;
            }
        }
    }
    printf("all tests passed\n");
    return true;
}

int main() {
    test(500);
    return 0;
}

```

6.23 XOR-SAT

Hệ phương trình xor.

Mã nguồn mẫu: Math/xorsat.cpp

```

#include <vector>
#include <bitset>
#include <algorithm>
using namespace std;

/*
 * XOR-SAT or Gauss XOR Elimination  $O(n \cdot (m^2)/64)$ 
 */

#define MAXN 2009

vector<int> B;
vector<bitset<MAXN>> A;

```

```

bitset<MAXN> x;

bool check() {
    int n = A.size(), m = MAXN;
    for(int i = 0; i < n; i++) {
        int acum = 0;
        for(int j = 0; j < m; j++) {
            if (A[i][j]) acum ^= x[j];
        }
        if (acum != B[i]) return false;
    }
    return true;
}

bool gaussxor() {
    int cnt = 0, n = A.size(), m = MAXN;
    bitset<MAXN> vis; vis.set();
    for(int j = m-1, i; j >= 0; j--) {
        for(i = cnt; i < n; i++) {
            if (A[i][j]) break;
        }
        if (i == n) continue;
        swap(A[i], A[cnt]); swap(B[i], B[cnt]);
        i = cnt++; vis[j] = 0;
        for(int k = 0; k < n; k++) {
            if (i == k || !A[k][j]) continue;
            A[k] ^= A[i]; B[k] ^= B[i];
        }
    }
    x = vis;
    for(int i = 0; i < n; i++) {
        int acum = 0;
        for(int j = 0; j < m; j++) {
            if (!A[i][j]) continue;
            if (!vis[j]) {
                vis[j] = 1;
                x[j] = acum^B[i];
            }
            acum ^= x[j];
        }
        if (acum != B[i]) return false;
    }
    return true;
}

/*
 * Codeforces 101908M
 */

#include <cstdio>

int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    for(int i = 0; i < n; i++) {
        char c, str[10];
        scanf(" %c", &c);
        int val = 1;
        bitset<MAXN> eq;
        while(scanf(" %s", str) != EOF && str[0] != 'a') {
            //printf("%s ", str);
            if (str[0] == 'n') val ^= 1;
            if (str[0] == 'x') {
                int k;
                sscanf(str+1, "%d", &k);
                eq[k-1] = 1^eq[k-1];
            }
        }
        //printf("\n");
    }
}

```

```

        A.push_back(eq);
        B.push_back(val);
    }
    if (!gaussxor()) printf("impossible\n");
    else {
        for(int j = 0; j < m; j++) {
            printf("%c", x[j] ? 'T' : 'F');
        }
        printf("\n");
    }
    return 0;
}

```

6.24 Fast Fourier Transform (FFT)

FFT.

Mã nguồn mẫu: Math/fft.cpp

```

#include <vector>
#include <cstdio>
#include <complex>
using namespace std;

/*
 * Fast complex
 */

struct base { // faster than complex<double>
    double x, y;
    base() : x(0), y(0) {}
    base(double a, double b=0) : x(a), y(b) {}
    base operator+=(double k) { x/=k; y/=k; return (*this); }
    base operator*(base a) const { return base(x*a.x - y*a.y, x*a.y + y*a.x); }
    base operator*=(base a) {
        double tx = x*a.x - y*a.y;
        double ty = x*a.y + y*a.x;
        x = tx; y = ty;
        return (*this);
    }
    base operator=(double a) { x=a; y=0; return (*this); }
    base operator+(base a) const { return base(x+a.x, y+a.y); }
    base operator+=(base a) { x+=a.x; y+=a.y; return (*this); }
    base operator-(base a) const { return base(x-a.x, y-a.y); }
    base operator-=(base a) { x-=a.x; y-=a.y; return (*this); }
    double& real() { return x; }
    double& imag() { return y; }
};

/*
 * FFT - Fast Fourier Transform O(nlogn)
 */

//typedef complex<double> base;

void fft(vector<base> &a, bool invert) {
    int n = (int)a.size();
    for(int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for(; j >= bit; bit >>= 1) j -= bit;
        j += bit;
        if (i < j) swap(a[i], a[j]);
    }
    for(int len = 2; len <= n; len <= 1) {
        double ang = 2*acos(-1.0)/len * (invert ? -1 : 1);
        base wlen(cos(ang), sin(ang));
        for(int i = 0; i < n; i += len) {

```

```

        base w(1);
        for(int j = 0; j < len/2; j++) {
            base u = a[i+j], v = a[i+j+len/2] * w;
            a[i + j] = u + v;
            a[i + j + len/2] = u - v;
            w *= wlen;
        }
    }
}

for (int i = 0; invert && i < n; i++) a[i] /= n;
}

void convolution(vector<base> a, vector<base> b, vector<base> &res) {
    int n = 1;
    while(n < max(a.size(), b.size())) n <= 1;
    n <= 1;
    a.resize(n), b.resize(n);
    fft(a, false); fft(b, false);
    res.resize(n);
    for(int i=0; i<n; ++i) res[i] = a[i]*b[i];
    fft(res, true);
}

void circularConv(vector<base> &a, vector<base> &b, vector<base> &res) {
    //assert(a.size() == b.size());
    int n = a.size();
    convolution(a, b, res);
    for(int i = n; i < (int)res.size(); i++)
        res[i%n] += res[i];
    res.resize(n);
}

/*
 * Sqrt DECOMPOSITION FOR FFT
 */

#include <cmath>
#define MOD 1000003LL
#define SMOD 1024LL // ~ sqrt(MOD)
typedef long long ll;

void sqrtConv(vector<ll> a, vector<ll> b, vector<ll> &c) {
    vector<base> ca[2], cb[2], cc[2][2];
    ca[0].resize(a.size());
    ca[1].resize(a.size());
    for(int i=0; i<(int)a.size(); i++) {
        ca[0][i] = base(a[i] % SMOD, 0);
        ca[1][i] = base(a[i] / SMOD, 0);
    }
    cb[0].resize(b.size());
    cb[1].resize(b.size());
    for(int i=0; i<(int)b.size(); i++) {
        cb[0][i] = base(b[i] % SMOD, 0);
        cb[1][i] = base(b[i] / SMOD, 0);
    }
    for(int l=0; l<2; l++) for(int r=0; r<2; r++)
        convolution(ca[l], cb[r], cc[l][r]);
    c.resize(cc[0][0].size());
    for(int i=0; i<(int)c.size(); i++) {
        c[i] =
            (((ll)round(cc[1][1][i].real()))%MOD*(SMOD*SMOD)%MOD)%MOD +
            (((ll)round(cc[0][1][i].real()))%MOD*SMOD)%MOD +
            (((ll)round(cc[1][0][i].real()))%MOD*SMOD)%MOD +
            (((ll)round(cc[0][0][i].real()))%MOD);
        c[i] %= MOD;
    }
}

/*

```

```

* TEST MATRIX
*/

int main()
{
    base a (1);
    base b (2);
    base c (3);
    base d (4);
    base e (5);
    base f (6);
    vector<base> A, B, C;
    A.push_back(c);
    A.push_back(b);
    A.push_back(a);
    B.push_back(f);
    B.push_back(e);
    B.push_back(d);
    B.push_back(c);
    convolution(A, B, C);
    for(int i=0; i<(int)C.size(); i++) {
        printf(" %.2f", C[i].real());
    }
    return 0;
}

```

6.25 Number Theoretic Transform (NTT)

NTT.

Mã nguồn mẫu: Math/ntt.cpp

```

#include <vector>
#include <cstdio>
#include <cstring>
#define MAXN 100009
using namespace std;

/*
 * Modular inverse
 */

template <typename T>
T extGcd(T a, T b, T& x, T& y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    else {
        T g = extGcd(b, a % b, y, x);
        y -= a / b * x;
        return g;
    }
}

template <typename T>
T modInv(T a, T m) {
    T x, y;
    extGcd(a, m, x, y);
    return (x % m + m) % m;
}

/*
 * NTT - Number Theoretic Transform O(nlogn)
 */

typedef long long ll;

```

```

const ll mod[3] = {1004535809LL, 1092616193LL, 998244353LL};
const ll root[3] = {12289LL, 23747LL, 15311432LL};
const ll root_1[3] = {313564925LL, 642907570LL, 469870224LL};
const ll root_pw[3] = {1LL<<21, 1LL<<21, 1LL<<23};

void ntt(vector<ll> &a, bool invert, int m) {
    ll n = (ll)a.size();
    for(ll i = 1, j = 0; i < n; i++) {
        ll bit = n >> 1;
        for (; j >= bit; bit >>= 1) j -= bit;
        j += bit;
        if (i < j) swap(a[i], a[j]);
    }
    for(ll len = 2, wlen; len <= n; len <= 1) {
        wlen = invert ? root_1[m] : root[m];
        for (ll i = len; i < root_pw[m]; i <= 1)
            wlen = (wlen * wlen % mod[m]);
        for(ll i = 0; i < n; i += len) {
            for(ll j = 0, w = 1; j < len/2; j++) {
                ll u = a[i+j], v = a[i+j+len/2] * w % mod[m];
                a[i+j] = (u+v < mod[m] ? u+v : u+v-mod[m]);
                a[i+j+len/2] = (u-v >= 0 ? u-v : u-v+mod[m]);
                w = w * wlen % mod[m];
            }
        }
    }
    if (invert) {
        ll nrev = modInv(n, mod[m]);
        for (ll i=0; i<n; ++i)
            a[i] = a[i] * nrev % mod[m];
    }
}

void convolution(vector<ll> a, vector<ll> b, vector<ll> &res, int m) {
    ll n = 1;
    while(n < max (a.size(), b.size())) n <= 1;
    n <= 1;
    a.resize(n), b.resize(n);
    ntt(a, false, m); ntt(b, false, m);
    res.resize(n);
    for(int i=0; i<n; ++i) res[i] = (a[i]*b[i])%mod[m];
    ntt(res, true, m);
}

void circularConv(vector<ll> &a, vector<ll> &b, vector<ll> &res, int m) {
    //assert(a.size() == b.size());
    int n = a.size();
    convolution(a, b, res, m);
    for(int i = n; i < (int)res.size(); i++)
        res[i%n] = (res[i%n]+res[i])%mod[m];
    res.resize(n);
}

/*
 * CRT NTT
 */

template<typename T>
T modMul(T a, T b, T m) {
    T x = 0, y = a % m;
    while (b > 0) {
        if (b % 2 == 1) x = (x + y) % m;
        y = (y * 2) % m;
        b /= 2;
    }
    return x % m;
}

```



```

//convolution mod 1,097,572,091,361,755,137
void modConv(vector<ll> a, vector<ll> b, vector<ll> & res) {
    vector<ll> r0, r1;
    convolution(a, b, r0, 0);
    convolution(a, b, r1, 1);
    ll x, y, s, r, p = mod[0]*mod[1];
    extGcd(mod[0], mod[1], r, s);
    res.resize(r0.size());
    for(int i=0; i<(int)res.size(); i++) {
        res[i] = (modMul((s*mod[1]+p)%p, r0[i], p)
        + modMul((r*mod[0]+p)%p, r1[i], p) + p) % p;
    }
}

/*
 * SPOJ MUL
 */

char num1[MAXN], num2[MAXN];

int main()
{
    int n, na, nb, k;
    vector<ll> A, B, C;
    scanf("%d", &n);
    for(int i=0; i<n; i++) {
        scanf(" %s %s", num1, num2);
        na = strlen(num1);
        nb = strlen(num2);
        A.clear();
        B.clear();
        for(int j=na-1; j>=0; j--) {
            A.push_back(num1[j]-'0');
        }
        for(int j=nb-1; j>=0; j--) {
            B.push_back(num2[j]-'0');
        }
        modConv(A, B, C);
        while(C.size()>1 && C.back()==0) C.pop_back();
        na = nb = k = 0;
        for(int j=0; j<(int)C.size() || nb > 0; j++) {
            na = (nb + (j<(int)C.size() ? C[j] : 0))%10;
            nb = (nb + (j<(int)C.size() ? C[j] : 0))/10;
            num1[k++] = ((char)na) + '0';
        }
        for(int i=0; i<k/2; i++) {
            na = num1[i];
            num1[i] = num1[k-i-1];
            num1[k-i-1] = na;
        }
        num1[k] = '\\0';
        printf("%s\\n", num1);
    }
    return 0;
}

```

6.26 Tích chập vòng

Circular convolution.

Mã nguồn mẫu: Math/circularconvolution.cpp

```

template <typename T>
void circularconvolution(vector<T> a, vector<T> b, vector<T> & res) {
    int n = a.size();
    b.insert(b.end(), b.begin(), b.end());
    convolution(a, b, res);
}

```

```

    res = vector<T>(res.begin()+n, res.begin()+(2*n));
}

```

6.27 Số phức

Các phép với số phức trong C++/toán học.

6.28 Chia đa thức

Polynomial division.

Mã nguồn mẫu: Math/polynomialdiv.cpp

```

#include <vector>
#include <cstdio>
#include <complex>
using namespace std;

/*
 * Fast complex
 */

struct base { // faster than complex<double>
    double x, y;
    base() : x(0), y(0) {}
    base(double a, double b=0) : x(a), y(b) {}
    base operator/(double k) { x/=k; y/=k; return (*this); }
    base operator*(base a) const { return base(x*a.x - y*a.y, x*a.y + y*a.x); }
    base operator*=(base a) {
        double tx = x*a.x - y*a.y;
        double ty = x*a.y + y*a.x;
        x = tx; y = ty;
        return (*this);
    }
    base operator=(double a) { x=a; y=0; return (*this); }
    base operator+(base a) const { return base(x+a.x, y+a.y); }
    base operator+=(base a) { x+=a.x; y+=a.y; return (*this); }
    base operator-(base a) const { return base(x-a.x, y-a.y); }
    base operator-=(base a) { x-=a.x; y-=a.y; return (*this); }
    double& real() { return x; }
    double& imag() { return y; }
};

/*
 * FFT - Fast Fourier Transform O(nlogn)
 */

//typedef complex<double> base;

void fft(vector<base> &a, bool invert) {
    int n = (int)a.size();
    for(int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for(; j >= bit; bit >>= 1) j -= bit;
        j += bit;
        if (i < j) swap(a[i], a[j]);
    }
    for(int len = 2; len <= n; len <= 1) {
        double ang = 2*acos(-1.0)/len * (invert ? -1 : 1);
        base wlen(cos(ang), sin(ang));
        for(int i = 0; i < n; i += len) {
            base w(1);
            for(int j = 0; j < len/2; j++) {
                base u = a[i+j], v = a[i+j+len/2] * w;
                a[i + j] = u + v;

```

```

        a[i + j + len/2] = u - v;
        w *= wlen;
    }
}
}
for (int i = 0; invert && i < n; i++) a[i] /= n;
}

void convolution(vector<base> a, vector<base> b, vector<base> &res) {
    int n = 1;
    while(n < max(a.size(), b.size())) n <= 1;
    n <= 1;
    a.resize(n), b.resize(n);
    fft(a, false); fft(b, false);
    res.resize(n);
    for(int i=0; i<n; ++i) res[i] = a[i]*b[i];
    fft(res, true);
}

void circularConv(vector<base> &a, vector<base> &b, vector<base> &res) {
    //assert(a.size() == b.size());
    int n = a.size();
    convolution(a, b, res);
    for(int i = n; i < (int)res.size(); i++)
        res[i%n] += res[i];
    res.resize(n);
}

/*
 * Polynomial Division
 */

#include <algorithm>

void inverse(vector<base> &a, int t) {
    if (t == 1) {
        a.resize(1); a[0].real() = 1.0/a[0].real();
        return;
    }
    vector<base> a0 = a;
    inverse(a0, (t/2) + (t%2));
    convolution(a0, a, a);
    a0.resize(t), a.resize(t);
    a[0] -= base(1.0);
    convolution(a0, a, a);
    a0.resize(t), a.resize(t);
    for(int i = 0; i < t; i++) a[i] = a0[i] - a[i];
}

void divide(vector<base> &a, vector<base> &b, vector<base> &d) {
    int n = a.size(), m = b.size();
    if (n < m) {
        d.clear(); d.push_back(0);
        return;
    }
    vector<base> ar = a, br = b;
    reverse(ar.begin(), ar.end());
    reverse(br.begin(), br.end());
    inverse(br, n - m + 1);
    convolution(ar, br, d);
    d.resize(n - m + 1);
    reverse(d.begin(), d.end());
}

void remainder(vector<base> &a, vector<base> &b, vector<base> &r) {
    int n = a.size(), m = b.size();
    if (n < m) { r = a; return; }
    vector<base> d, aux;
    divide(a, b, d);

```

```

        r = a;
        convolution(d, b, aux);
        r.resize(m-1); aux.resize(m-1);
        for(int i = 0; i < m-1; i++) r[i] -= aux[i];
    }

/*
 * Multi-point evaluation in  $O((n+m) \log n \log m)$ 
 */

#define MAXN 10000

vector<base> rp[4*MAXN];
void roots(int u, double x[], int i, int j) {
    if (i == j) {
        rp[u].resize(2); rp[u][0] = -x[i];
        rp[u][1] = 1.0; return;
    }
    int m = (i + j) / 2;
    roots(2*u, x, i, m); roots(2*u+1, x, m+1, j);
    convolution(rp[2*u], rp[2*u+1], rp[u]);
    rp[u].resize(j-i+2);
}

void roots(vector<base> &a, double x[], int n) {
    roots(1, x, 0, n-1); a = rp[1];
}

vector<base> ep[4*MAXN];
void multievaluate(int u, double x[], double y[], int i, int j) {
    if (i == j) {
        y[i] = 0;
        double p = 1;
        for(int k = 0; k < (int)ep[u].size(); k++)
            y[i] += ep[u][k].real()*p, p *= x[i];
        return;
    }
    remainder(ep[u], rp[2*u], ep[2*u]);
    remainder(ep[u], rp[2*u+1], ep[2*u+1]);
    int m = (i + j) / 2;
    multievaluate(2*u, x, y, i, m);
    multievaluate(2*u+1, x, y, m+1, j);
}

void multievaluate(vector<base> &a, double x[], double y[], int n) {
    roots(1, x, 0, n-1); ep[1] = a;
    multievaluate(1, x, y, 0, n-1);
}

/*
 * Polynomial interpolation  $O(n \log^3 n)$ 
 * horrible constant and time
 */

vector<double> y[MAXN];
double ya0[MAXN], yp[MAXN];
void interpolate(vector<base> &a, int u, double x[], int n) {
    if (n == 1) {
        a.resize(1); a[0] = y[u][0];
        return;
    }
    y[2*u] = y[u]; y[2*u].resize(n/2);
    vector<base> a0, a1, p;
    interpolate(a0, 2*u, x, n/2);
    roots(p, x, n/2);
    int m = n-(n/2);
    multievaluate(a0, x+(n/2), ya0, m);
    multievaluate(p, x+(n/2), yp, m);
    y[2*u+1].resize(m);

```

```

    for(int i = 0; i < m; i++)
        y[2*u+1][i] = (y[u][n/2 + i] - ya0[i]) / yp[i];
    interpolate(a1, 2*u+1, x+(n/2), m);
    convolution(p, a1, a);
    int sz = max(a.size(), a0.size());
    a.resize(sz);
    for(int i = 0; i < sz && i < n/2; i++)
        a[i] += a0[i];
    a.resize(n);
}

void interpolate(vector<base> &a, double x[], double ry[], int n) {
    y[1].resize(n);
    for(int i = 0; i < n; i++) y[1][i] = ry[i];
    interpolate(a, 1, x, n);
}

/*
 * TEST MATRIX
 */

int main() {
    int n, m;
    vector<base> P, Q, D, R;

    scanf("%d", &n);
    P.resize(n);
    for(int i = 0; i < n; i++) scanf("%lf", &P[i].real());

    scanf("%d", &m);
    Q.resize(m);
    for(int i = 0; i < m; i++) scanf("%lf", &Q[i].real());

    vector<base> invP = P, invQ = Q;
    inverse(invP, 10);
    printf("first 10 terms of inverse of P:");
    for(int i = 0; i < 10; i++) printf(" %.0f", invP[i].real());
    printf("\n");

    inverse(invQ, 10);
    printf("first 10 terms of inverse of Q:");
    for(int i = 0; i < 10; i++) printf(" %.0f", invQ[i].real());
    printf("\n");

    divide(P, Q, D);
    printf("P/Q:");
    for(int i = 0; i < (int)D.size(); i++) printf(" %.3f", D[i].real());
    printf("\n");

    remainder(P, Q, R);
    printf("P%Q:");
    for(int i = 0; i < (int)R.size(); i++) printf(" %.3f", R[i].real());
    printf("\n");

    scanf("%d", &n);
    double x[100];
    for(int i = 0; i < n; i++) scanf("%lf", &x[i]);
    vector<base> a;
    roots(a, x, n);
    printf("from roots:");
    for(int i = 0; i < (int)a.size(); i++) printf(" %.3f", a[i].real());
    printf("\n");

    double y[100];
    multievaluate(P, x, y, n);
    printf("y[i]:");
    for(int i = 0; i < n; i++) printf(" %.3f", y[i]);
    printf("\n");
}

```

```

scanf("%d", &n);
for(int i = 0; i < n; i++) scanf("%lf %lf", &x[i], &y[i]);
interpolate(a, x, y, n);
printf("interpolated a:");
for(int i = 0; i < n; i++) printf(" %.3f", a[i]);
printf("\n");

return 0;
}

```

6.29 Đánh giá tại nhiều điểm

Multipoint evaluation.

6.30 Nội suy đa thức

Polynomial interpolation.

6.31 Fast Walsh-Hadamard Transform

FWHT.

Mã nguồn mẫu: Math/fwht.cpp

```

#include <vector>
using namespace std;

/*
 * FWHT - Fast Walsh-Hadamard Transform O(nlogn)
 */

//int mat[2][2] = {{1, 1}, {1, -1}}, inv[2][2] = {{1, 1}, {1, -1}}; //xor
int mat[2][2] = {{1, 1}, {1, 0}}, inv[2][2] = {{0, 1}, {1, -1}}; //or
//int mat[2][2] = {{0, 1}, {1, 1}}, inv[2][2] = {{-1, 1}, {1, 0}}; //and

void fwht(vector<double> & a, bool invert) {
    int n = (int)a.size();
    double u, v;
    for (int len = 1; 2 * len <= n; len <= 1) {
        for (int i = 0; i < n; i += 2 * len) {
            for (int j = 0; j < len; j++) {
                u = a[i + j];
                v = a[i + len + j];
                if (!invert) {
                    a[i + j] = mat[0][0]*u + mat[0][1]*v;
                    a[i + len + j] = mat[1][0]*u + mat[1][1]*v;
                }
                else {
                    a[i + j] = inv[0][0]*u + inv[0][1]*v;
                    a[i + len + j] = inv[1][0]*u + inv[1][1]*v;
                }
            }
        }
    }
}

//for (int i=0; invert && i<n; ++i) a[i] /= n; //xor

void convolution(vector<double> a, vector<double> b, vector<double> & res) {
    int n = 1;
    while(n < max(a.size(), b.size())) n <= 1;
    a.resize(n), b.resize(n);
    fwht(a, false); fwht(b, false);
    res.resize(n);
}

```

```

        for(int i=0; i<n; ++i) res[i] = a[i]*b[i];
        fwht(res, true);
    }

/*
 * TEST MATRIX
 */

void naive(vector<double> a, vector<double> b, vector<double> & res) {
    int n = 1;
    while(n < max(a.size(), b.size())) n <= 1;
    a.resize(n), b.resize(n);
    res.clear(); res.assign(n, 0);
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            res[i][j] += a[i]*b[j];
        }
    }
}

#define EPS 1e-3
#include <stdio>
#include <stdlib>
#include <cmath>
bool test(int nTest) {
    for(int t=1; t<=nTest; t++) {
        vector<double> a(t), b(t), res1(t), res2(t);
        for(int i=0; i<t; i++) {
            a[i] = rand()%10000;
            b[i] = rand()%10000;
        }
        convolution(a, b, res1);
        naive(a, b, res2);
        bool ok = true;
        for(int i=0; i<(int)res1.size(); i++) {
            if (fabs(res1[i]-res2[i]) > EPS) ok = false;
        }
        if (!ok) {
            printf("failed on test %d\n", t);
            printf("a:");
            for(int i=0; i<(int)a.size(); i++) {
                printf(" %.2f", a[i]);
            }
            printf("\n");
            printf("b:");
            for(int i=0; i<(int)b.size(); i++) {
                printf(" %.2f", b[i]);
            }
            printf("\n");
            printf("res1:");
            for(int i=0; i<(int)res1.size(); i++) {
                printf(" %.2f", res1[i]);
            }
            printf("\n");
            printf("res2:");
            for(int i=0; i<(int)res2.size(); i++) {
                printf(" %.2f", res2[i]);
            }
            printf("\n");
            return false;
        }
    }
    printf("all tests passed\n");
    return true;
}

int main() {
    test(1000);
}

```

6.32 Tích chập với CRT

Kết hợp nhiều modulo bằng CRT.

Mã nguồn mẫu: Math/crt.cpp

```
#include <cstdio>
#define MOD 1000000007LL
typedef long long ll;

template <typename T>
T extGcd(T a, T b, T& x, T& y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    else {
        T g = extGcd(b, a % b, y, x);
        y -= a / b * x;
        return g;
    }
}

template <typename T>
T modInv(T a, T m) {
    T x, y;
    extGcd(a, m, x, y);
    return (x % m + m) % m;
}

template <typename T>
T modDiv(T a, T b, T m) {
    return ((a % m) * modInv(b, m)) % m;
}

template<typename T>
T crt(T* a, T* p, int n, T m) {
    T P = 1;
    for(int i=0; i<n; i++) P = (P * p[i]) % m;
    T x = 0, pp;
    for(int i=0; i<n; i++) {
        pp = modDiv(P, p[i], m);
        x = (x + ((a[i] * pp) % m) * modInv(pp, p[i])) % m;
    }
    return x;
}

/*TEST MATRIX*/
#include <cstdlib>
#define PRINTTEST 0

int primes[] = {2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59};

bool test(int ntests) {
    for(int t=0; t<ntests; t++) {
        int T = 17;
        ll y[20], a[20];
        for(int i=1; i<=T; i++) {
            if (PRINTTEST) printf("test %d:\n", t*T + i);
            for(int j=0; j<i; j++) {
                a[j] = primes[j];
                y[j] = rand()%a[j];
                if (PRINTTEST) printf("x = %lld mod %lld\n", y[j], a[j]);
            }
            ll ans = crt(y, a, i, MOD);
            if (PRINTTEST) printf("answer is x = %lld\n", ans);
            for(int j=0; j<i; j++) {
                ll mans = ans;
                int count = 10000;
```



```

        while(mans%a[j] != y[j] && count > 0) {
            mans += MOD;
            count--;
        }
        if (mans%a[j] != y[j]) {
            printf("failed on test %d, ans = %I64d\n", t*T + i, ans);
            return false;
        }
    }
}

}

printf("All tests passed\n");
return true;
}

int main()
{
    test(10000);
    return 0;
}

```

6.33 Tích chập với phân rã căn

SQRT decomposition convolution.

6.34 Tích phân bằng quy tắc Simpson

Simpson integration.

Mã nguồn mẫu: Math/simpson.cpp

```

#include <cmath>

/*
 * Simpson Rule for integration
 */

double f(double x) { return cos(x); }

double simpson(double a, double b, int n = 1e6) {
    double h = (b - a) / n, s = f(a) + f(b);
    for (int i = 1; i < n; i += 2) s += 4*f(a+h*i);
    for (int i = 2; i < n; i += 2) s += 2*f(a+h*i);
    return s*h/3;
}

/*
 * TEST MATRIX
 */

#include <stdio>
#include <cmath>

bool test(int n) {
    for(int i = 0; i < n; i++) {
        double area = simpson(0, i);
        double exact = sin(i*1.0);
        if (fabs(area - exact) > 1e+18) {
            printf("error at %d = %.20f\n", i, fabs(area - exact));
            return false;
        }
    }
    printf("all tests passed\n");
    return true;
}

```

```
int main() {
    test(100);
    return 0;
}
```

6.35 Mã Gray

Gray code.

Mã nguồn mẫu: Math/gray.cpp

```
#include <cstdio>

int g (int n) {
    return n ^ (n >> 1);
}

int rev_g (int g) {
    int n = 0;
    for (; g; g>>=1)
        n ^= g;
    return n;
}
```

6.36 BigInteger trong Java

Mẫu dùng java.math.BigInteger.

6.37 Bignum trong C++

Số lớn C++.

Mã nguồn mẫu: Math/bignum.cpp

```
#include <vector>
#include <algorithm>
#include <cstdio>
#include <cstdlib>
#include <cstring>
using namespace std;

typedef vector<int> bignum;
const int base = 1000*1000*1000;

//Impressão
void print(bignum & a) {
    printf("%d", a.empty() ? 0 : a.back());
    for (int i=(int)a.size()-2; i>=0; --i) {
        printf("%09d", a[i]);
    }
}

//Remover Leading zeros
void fix(bignum & a) {
    while (a.size() > 1u && a.back() == 0) {
        a.pop_back();
    }
}

//Comparador
bool comp(bignum a, bignum b) {
    fix(a); fix(b);
```

```

        if (a.size() != b.size()) return a.size() < b.size();
        for(int i=(int)a.size()-1; i>=0; i--) {
            if (a[i] != b[i]) return a[i] < b[i];
        }
        return false;
    }

//Leitura
void str2bignum(char* s, bignum & a) {
    a.clear();
    for (int i=(int)strlen(s); i>0; i-=9) {
        s[i] = 0;
        a.push_back (atoi (i>=9 ? s+i-9 : s));
    }
    fix(a);
}

//Construtor a partir de um inteiro
void int2bignum(int n, bignum & a) {
    a.clear();
    if (n == 0) a.push_back(0);
    while(n > 0) {
        a.push_back(n%base);
        n /= base;
    }
}

int bignum2int(bignum & a) {
    int ans = 0, p=1;
    for(int i=0; i<(int)a.size(); i++) {
        ans += a[i]*p;
        p *= base;
    }
    return ans;
}

//Soma a+b=c
void sum(bignum & a, bignum & b, bignum & c) {
    int carry = 0, n = max(a.size(), b.size());
    c.resize(n);
    for (int i=0, ai, bi; i<n; i++) {
        ai = i < (int)a.size() ? a[i] : 0;
        bi = i < (int)b.size() ? b[i] : 0;
        c[i] = carry + ai + bi;
        carry = c[i] / base;
        c[i] %= base;
    }
    if (carry > 0) c.push_back(carry);
    fix(c);
}

//Subtrai a-b=c>=0
void subtract(bignum & a, bignum & b, bignum & c) {
    int carry = 0, n = max(a.size(), b.size());
    c.resize(n);
    for (int i=0, ai, bi; i<n; i++) {
        ai = i < (int)a.size() ? a[i] : 0;
        bi = i < (int)b.size() ? b[i] : 0;
        c[i] = carry + ai - bi;
        carry = c[i] < 0 ? 1 : 0;
        if (c[i] < 0) c[i] += base;
    }
    fix(c);
}

//Shift left por b (multiplica por base^b)
void shiftL(bignum & a, int b, bignum & c) {
    c.resize((int)a.size() + b);
    for(int i=(int)c.size()-1; i>=0; i--) {

```

```

        if(i>=b) c[i] = a[i-b];
        else c[i] = 0;
    }
    fix(c);
}

//Shift right por b (multiplica por base^b)
void shiftR(bignum & a, int b, bignum & c) {
    if (((int)a.size()) <= b) {
        c.clear(); c.push_back(0);
        return;
    }
    c.resize((int)a.size() - b);
    for(int i=0; i<(int)c.size(); i++) {
        c[i] = a[i+b];
    }
    fix(c);
}

//Multiplica bignum b por int a<base
void multiply(int a, bignum & b, bignum & c) {
    int carry = 0, bi;
    c.resize(b.size());
    for (int i=0, bi; i<(int)b.size() || carry; i++) {
        if (i == (int)b.size()) c.push_back(0);
        bi = i < (int)b.size() ? b[i] : 0;
        long long cur = carry + a * 1ll * bi;
        c[i] = int(cur % base);
        carry = int(cur / base);
    }
    fix(c);
}

//Multiplica a*b=c
void multiply(bignum a, bignum b, bignum & c) {
    int n = a.size()+b.size();
    long long carry = 0, acum;
    c.resize(n);
    for (int k=0; k<n || carry; k++) {
        if (k == n) c.push_back(0);
        acum = carry; carry = 0;
        for (int i=0, j=k; i <= k && i<(int)b.size(); i++, j--) {
            if (j >= (int)b.size()) continue;
            acum += a[i] * 1ll * b[j];
            carry += acum / base;
            acum %= base;
        }
        c[k] = acum;
    }
    fix(c);
}

//Divide bignum a por int b<base
void divide(bignum & a, int b, bignum & c) {
    int carry = 0;
    c.resize(a.size());
    for (int i=(int)a.size()-1; i>=0; --i) {
        long long cur = a[i] + carry * 1ll * base;
        c[i] = int (cur / b);
        carry = int (cur % b);
    }
    fix(a);
}

//Divide bignum a por bignum b, quociente q, resto r
void divide(bignum a, bignum b, bignum & q, bignum & r) {
    bignum z, p;
    int2bignum(0, z);
    int2bignum(1, p);

```

```

int2bignum(0, q);
while(comp(b, a)) {
    shiftL(p, max(1u, a.size()-b.size()), p);
    shiftL(b, max(1u, a.size()-b.size()), b);
}
while(true) {
    while (comp(a, b) && comp(z, p)) {
        shiftR(p, 1, p); shiftR(b, 1, b);
    }
    if (!comp(z, p)) break;
    subtract(a, b, a);
    sum(q, p, q);
}
swap(a, r);
}

bool test() {
    bool toprint = false;
    for(int i=1; i<10000; i++) {
        int na = 1 + rand()%30000;
        int nb = 1 + rand()%30000;
        if (nb > na) {
            int tmp = na;
            na = nb;
            nb = tmp;
        }
        if(toprint) printf("test %d: na = %d, nb = %d, ", i, na, nb);
        bignum a; int2bignum(na, a);
        bignum b; int2bignum(nb, b);
        bignum c, r; int nc, nr;
        sum(a, b, c);
        nc = na + nb;
        if (nc != bignum2int(c)) {
            printf("failed test %d, n: %d+%d=%d, b: ", i, na, nb, nc);
            print(a);
            printf("+");
            print(b);
            printf("=");
            print(c);
            printf("\n");
            return false;
        }
        if(toprint) printf("+ ok ");
        subtract(a, b, c);
        nc = na - nb;
        if (nc != bignum2int(c)) {
            printf("failed test %d, n: %d-%d=%d, b: ", i, na, nb, nc);
            print(a);
            printf("-");
            print(b);
            printf("=");
            print(c);
            printf("\n");
            return false;
        }
        if(toprint) printf("- ok ");
        multiply(a, b, c);
        nc = na * nb;
        if (nc != bignum2int(c)) {
            printf("failed test %d, n: %d*%d=%d, b: ", i, na, nb, nc);
            print(a);
            printf("*");
            print(b);
            printf("=");
            print(c);
            printf("\n");
            return false;
        }
        if(toprint) printf("* ok ");
    }
}

```

```

    divide(a, b, c, r);
    nc = na / nb;
    if (nc != bignum2int(c)) {
        printf("failed test %d, n: %d/%d=%d, b: ", i, na, nb, nc);
        print(a);
        printf("/");
        print(b);
        printf("=");
        print(c);
        printf("\n");
        return false;
    }
    if(toprint) printf("/ ok\n");
    swap(c, r);
    nc = na % nb;
    if (nc != bignum2int(c)) {
        printf("failed test %d, n: %d%%%d=%d, b: ", i, na, nb, nc);
        print(a);
        printf("%%%");
        print(b);
        printf("=");
        print(c);
        printf("\n");
        return false;
    }
    if(toprint) printf("% ok\n");
}
return true;
}

char s1[109], s2[109], op;

int main() {
    if(test()) printf("all tests passed\n");
    while (scanf(" %s %c %s", s1, &op, s2) != EOF) {
        bignum a; str2bignum(s1, a);
        bignum b; str2bignum(s2, b);
        bignum c, d;
        if (op == '+') sum(a, b, c);
        if (op == '-') subtract(a, b, c);
        if (op == '*') multiply(a, b, c);
        if (op == '/') divide(a, b, c, d);
        if (op == '%') divide(a, b, d, c);
        print(c);
        printf("\n");
    }
    return 0;
}

```

6.38 Sự phá sản của con bạc

Gambler's ruin.

6.39 Định lý và công thức

Tập hợp các công thức xác suất, tổ hợp và số học thường dùng.


```

    return 0;
}

```

7.2 Rabin-Karp

Hash chuỗi.

Mã nguồn mẫu: Strings/rabinkarp.cpp

```

#include <vector>
#include <cstring>
using namespace std;
typedef long long ll;

/*
 * Rabin Karp
 */

class RabinKarp {
    ll m;
    vector<int> pw, hsh;
public:
    RabinKarp() {}
    RabinKarp(char str[], ll p, ll _m) : m(_m) {
        int n = strlen(str);
        pw.resize(n); hsh.resize(n);
        hsh[0] = str[0]; pw[0] = 1;
        for(int i = 1; i < n; i++) {
            hsh[i] = (hsh[i-1] * p + str[i]) % m;
            pw[i] = (pw[i-1] * p) % m;
        }
    }
    ll hash(int i, int j) {
        ll ans = hsh[j];
        if (i > 0) ans = (ans - ((hsh[i-1]*1ll*pw[j-i+1])%m) + m) % m;
        return ans;
    }
};

/*
 * Codeforces 961F
 */

#include <cstdio>
#define MAXN 1000009
#define P 256
#define M 1000000007

char str[MAXN];
int N, f[MAXN];
RabinKarp rk;

bool check(int l, int r) {
    if (f[l] <= 0) return true;
    if (l+f[l]-1 >= r) return false;
    return rk.hash(l, l+f[l]-1) == rk.hash(r-f[l]+1, r);
}

int main() {
    scanf("%d %s", &N, str);
    rk = RabinKarp(str, P, M);
    int r = N/2;
    int l = (N % 2 == 0 ? r-1 : r);
    int k = l;
    f[l+1] = -1;
    for(; l >= 0; l--, r++) {
        f[l] = f[l+1]+2;
    }
}

```


}

7.3 Repetend: chu kỳ nhỏ nhất của chuỗi

Hà và K làm chất sao cho K/n và chđoi gồm n/k ban sao giống nhau.

Ma ngahon mau. 3c1 1ngs/ kimp.cpp

```
#include <string>
#include <vector>
#include <cstring>
using namespace std;

/*
 * Knuth-Morris-Pratt's Algorithm
 */

class KMP {
    string P;
    vector<int> b;
public:
    KMP(const char _P[]) : P(_P) {
        int m = P.size();
        b.assign(m + 1, -1);
        for(int i = 0, j = -1; i < m; i++) {
            while (j >= 0 && P[i] != P[j]) j = b[j];
            b[++i] = ++j;
        }
    }

    vector<int> match(const char T[]) {
        vector<int> ans;
        for (int i = 0, j = 0, n = strlen(T); i < n; i++) {
            while (j >= 0 && T[i] != P[j]) j = b[j];
            i++; j++;
            if (j == (int)P.size()) {
                ans.push_back(i - j);
                j = b[j];
            }
        }
        return ans;
    }

    int repetend() {
        int n = P.size();
        int ans = n - b[n];
        if (n % ans) ans = n;
        return ans;
    }
};

/*
 * TEST MATRIX
 */

#include <stdio>

void test() {
    string P = "ababa";
    string T = "abababababababa";
    KMP kmp(P.c_str());
    vector<int> index = kmp.match(T.c_str());
    for(int i=0; i<(int)index.size(); i++) {
        printf("%d ", index[i]);
    }
}
```

```

    }
    printf("\n");
}

int main() {
    test();
    return 0;
}

```

7.4 Suffix Array: thuật toán Karp-Miller-Rosenberg (KMR)

Suffix array $O(n \log n)$, LCP $O(n)$, lower bound và đếm match.

Mã nguồn mẫu: Strings/suffixarray.cpp

```

#define _CRT_SECURE_NO_WARNINGS
#define MAXN 100009
#include <algorithm>
#include <cstdio>
#include <cstring>
#include <string>
#include <set>
using namespace std;

/*
 * Suffix Array
 * Karp-Miller-Rosenberg's Algorithm (KMR) -  $O(n \log n)$ 
 * implementation adapted from Andrew Stankevich's class in Brazilian ICPC Summer School 2019
 */

int tsa[MAXN], c[MAXN], tc[MAXN], head[MAXN];

void bucket(int sa[], int n, int L) {
    memset(&head, 0, sizeof head);
    for (int i = 0; i < n; i++) head[c[i]]++;
    int maxi = max(300, n), k = 0;
    for (int i = 0; i < maxi; i++)
        swap(k, head[i]), k += head[i];
    for (int i = 0; i < n; i++) {
        int j = (sa[i] - L + n) % n;
        tsa[head[c[j]]++] = j;
    }
    memcpy(sa, tsa, n * sizeof(int));
}

void suffixArray(const char str[], int sa[]) {
    int n = strlen(str) + 1;
    for (int i = 0; i < n; i++)
        sa[i] = i, c[i] = str[i];
    bucket(sa, n, 0);
    for (int L = 1; L < n; L <= 1) {
        bucket(sa, n, L);
        for (int i = 0, cc = -1; i < n; i++)
            tc[sa[i]] = (i == 0 || c[sa[i]] != c[sa[i - 1]] || c[(sa[i] + L) % n] != c[(sa[i - 1] + L) % n]) ?
                ++cc : cc;
        memcpy(c, tc, n * sizeof(int));
    }
    for (int i = 0; i < n - 1; i++) sa[i] = sa[i + 1];
}

int inv[MAXN];

void computeLcp(const char str[], int sa[], int lcp[]) {
    int n = strlen(str), k = 0;
    for (int i = 0; i < n; i++) inv[sa[i]] = i;
    for (int j = 0; j < n; j++) {
        int i = inv[j];

```

```

        if (k < 0) k = 0;
        while (i > 0 && str[sa[i] + k] == str[sa[i - 1] + k]) k++;
        lcp[i] = k--;
    }
}

int bound(char s[], char t[], int sa[], int n, int m, bool upper) {
    int hi = n, lo = -1;
    int khi = 0, klo = 0;
    while (hi > lo + 1) {
        int mid = (hi + lo) / 2;
        int i = sa[mid], k = min(klo, khi);
        while (k < m && s[i + k] == t[k]) k++;
        if (k == m && upper) lo = mid, klo = k;
        else if (k == m && !upper) hi = mid, khi = k;
        else if (s[i + k] > t[k]) hi = mid, khi = k;
        else lo = mid, klo = k;
    }
    return hi;
}

int match(char s[], char t[], int sa[], int n, int m) {
    int l = bound(s, t, sa, n, m, false);
    int r = bound(s, t, sa, n, m, true);
    return r - l;
}

string concat;
int sa[MAXN], lcp[MAXN], pos[MAXN], len[MAXN];

void addStringsToSA(int N, const char s[][MAXN]) {
    int j = 0;
    concat = "";
    for (int i = 0; i < N; ++i) {
        int n = strlen(s[i]) + 1;
        concat += string(s[i]) + "$";
        for (int k = 0; k < n; ++k) {
            pos[j] = i;
            len[j++] = n - k - 1;
        }
    }
    suffixArray(concat.c_str(), sa);
    computeLcp(concat.c_str(), sa, lcp);
}

void slide(set<pair<int, int> >& vals, string& ans, int& l, int* qty, int& cnt, int i, int N) {
    while (cnt == N || (lcp[i] <= (int)ans.size() && l < i)) {
        int u = sa[l];
        vals.erase({ lcp[l], l });

        if (cnt == N) {
            int n = vals.empty() ? len[u] : min(len[u], vals.begin()->first);
            if (n > (int) ans.size()) ans = concat.substr(u, n);
        }

        qty[pos[u]]--;
        if (qty[pos[u]] == 0) cnt--;
        ++l;
    }
}

string getCommonSubstring(int N, const char s[][MAXN]) {
    addStringsToSA(N, s);

    string ans = "";
    int qty[21] = {}; // Must replaced 21 with the maximum value of N
    int cnt = 0, l = N;

    set<pair<int, int> > vals;

```

```

    lcp[concat.size()] = 0;
    for (int i = N; i < (int)concat.size(); ++i) {
        int u = sa[i];
        slide(vals, ans, l, qty, cnt, i, N);
        vals.insert({ lcp[i], i });
        qty[pos[u]]++;
        if (qty[pos[u]] == 1) cnt++;
    }
    slide(vals, ans, l, qty, cnt, concat.size(), N);

    return ans;
}

/*
 * TEST MATRIX
 */
/*
void printSA(char str[]) {
    int sa[100];
    int n = strlen(str);
    suffixArray(str, sa, n);
    for(int i = 0; i < n+1; i++) {
        printf("%2d %2d %s\n", i, sa[i], str+sa[i]);
    }
}

int main() {
    char str[100];
    scanf("%s", str);
    printSA(str);
    return 0;
}*/

/*
 * SPOJ SARRAY
 */
/*
char str[MAXN];
int sa[MAXN];

int main() {
    scanf(" %s", str);
    int n = strlen(str);
    suffixArray(str, sa);
    //constructSA(str, sa);
    for(int i = 0; i < n; i++) {
        printf("%d\n", sa[i]);
    }
    return 0;
}*/

/*
 * SPOJ STRMATCH
 */
/*
char s[MAXN], t[MAXN];
int sa[MAXN];

int main() {
    int n, q;
    scanf("%d %d %s", &n, &q, s);
    suffixArray(s, sa);
    while (q --> 0) {
        scanf(" %s", t);
        printf("%d\n", match(s, t, sa, n, strlen(t)));
    }
    return 0;
}*/

```

```

/*
Common substring manual test

char p[21][MAXN];

int main() {
    int N;

    scanf("%d", &N);

    for (int i = 0; i < N; ++i) {
        scanf("%s", p[i]);
    }
    printf("%s\n", getCommonSubstring(N, p).c_str());
}
*/
/*
* Codeforces 101711B
*/

char str[MAXN];
int SA[MAXN], LCP[MAXN];

long long psum(int i) {
    if (i <= 0) return 0;
    return i * (i + 1LL) / 2;
}

int main() {
    scanf("%s", str);
    int n = strlen(str);
    suffixArray(str, SA);
    computeLcp(str, SA, LCP);
    long long ans = 0;
    for (int i = 0; i < n; ++i) {
        ans += psum(n - SA[i]) - psum(LCP[i]);
    }
    printf("%lld\n", ans);
    return 0;
}

```

7.5 Hàm Z và thuật toán Z

Tính $z[i]$ trong $O(n)$.

Mã nguồn mẫu: Strings/zfunction.cpp

```

#include <cstdio>
#include <cstring>
#include <algorithm>
using namespace std;

/*
* Z-function
*/

void zfunction(char s[], int z[]) {
    int n = strlen(s);
    fill(z, z+n, 0);
    for (int i=1, l=0, r=0; i<n; ++i) {
        if (i <= r) z[i] = min(r-i+1, z[i-l]);
        while (i+z[i] < n && s[z[i]] == s[i+z[i]])
            ++z[i];
        if (i+z[i]-1 > r)
            l = i, r = i+z[i]-1;
    }
}

```

```

}

/*
 * TEST MATRIX
 */

void naivez(char* s, int* z) {
    int n = strlen(s);
    fill(z, z+n, 0);
    for(int i=1; i<n; i++) {
        while(i+z[i] < n && s[z[i]]==s[i+z[i]]) z[i]++;
    }
}

int main() {
    char str[309];
    int z1[309], z2[309];
    for(int n=1; n<10; n++) {
        for(int i=0; i<n; i++) {
            str[i] = 'a' + rand()%2;
        }
        str[n] = 0;
        naivez(str, z1);
        printf("test #d: %s\n", n, str);
        zfunction(str, z2);
        printf("naive:");
        for(int i=0; i<n; i++) {
            printf(" %2d", z1[i]);
        }
        printf("\n");
        printf("zfunc:");
        for(int i=0; i<n; i++) {
            printf(" %2d", z2[i]);
        }
        printf("\n");
    }
}

```

7.6 Thuật toán Manacher

Tìm palindrome dài nhất.

Mã nguồn mẫu: Strings/manacher.cpp

```

#include <cstdio>
#include <cstring>
#include <algorithm>
#define MAXN 3009
using namespace std;

/*
 * Manacher's Algorithm O(n)
 */

void manacher(char str[], int L[]) {
    int n = strlen(str), c = 0, r = 0;
    for(int i = 0; i < n; i++) {
        if(i < r && 2*c >= i) L[i] = min(L[2*c-i], r-i);
        else L[i] = 0;
        while(i-L[i]-1 >= 0 && i+L[i]+1 < n &&
            str[i-L[i]-1] == str[i+L[i]+1]) L[i]++;
        if(i+L[i]>r) { c=i; r=i+L[i]; }
    }
}

/*
 * Longest Palindromic Substring O(n)
 */

```

```

int LPS(char T[]) {
    int n = 2*strlen(T) + 1;
    char tmp[n+1];
    for(int i = 0, k = 0; T[i]; i++) {
        tmp[k++] = '|'; tmp[k++] = T[i];
    }
    tmp[n-1] = '|'; tmp[n] = '\0';
    int L[n], ans = 1;
    manacher(tmp, L);
    for(int i=0; i<n; i++)
        ans = max(ans, L[i]);
    return ans;
}

```

7.7 Aho-Corasick

Tìm các mẫu trong văn bản với automaton.

Mã nguồn mẫu: Strings/ahocorasick.cpp

```

#include <queue>
#include <vector>
using namespace std;
#define ALFA 256
#define MAXS 2000009
typedef pair<int, int> ii;

/*
 * Aho-Corasick's Algorithm
 */

int go[MAXS][ALFA], link[MAXS], h[MAXS], cnt = 0;
vector<int> pats[MAXS], adj[MAXS];

int suffix(int u, char c) {
    while (u && !go[u][c]) u = link[u];
    return go[u][c];
}

void aho() {
    queue<int> q;
    q.push(0);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int i = 0; i < (int)adj[u].size(); i++) {
            int c = adj[u][i];
            int v = go[u][c];
            link[v] = u ? suffix(link[u], c) : 0;
            pats[v].insert(pats[v].end(), pats[link[v]].begin(), pats[link[v]].end());
            q.push(v);
        }
    }
}

void insert(const char s[], int id) {
    int u = 0;
    for (int i = 0; s[i]; i++) {
        int &v = go[u][s[i]];
        if (!v) {
            v = ++cnt;
            adj[u].push_back(s[i]);
            h[v] = i + 1;
        }
        u = v;
    }
    pats[u].push_back(id);
}

```

```

}

vector<ii> match(const char s[]) { //(id, pos)
    vector<ii> ans;
    for (int i = 0, u = 0; s[i]; i++) {
        u = suffix(u, s[i]);
        for(int j = 0; j < (int)pats[u].size(); j++) {
            int id = pats[u][j];
            ans.push_back(ii(id, i - h[u] + 1));
        }
    }
    return ans;
}

/*
 * SPOJ SUB_PROB
 */

#include <cstdio>
#define MAXN 100009

char str[MAXN], in[MAXN];
bool found[MAXN];
int N;

int main() {
    scanf(" %s %d", str, &N);
    for (int i = 0; i < N; i++) {
        scanf(" %s", in);
        insert(in, i);
    }
    aho();
    vector<ii> ans = match(str);
    memset(&found, false, sizeof found);
    for (int i = 0; i < (int)ans.size(); i++) {
        found[ans[i].first] = true;
    }
    for (int i = 0; i < N; i++) {
        if (found[i]) printf("Y\n");
        else printf("N\n");
    }
    return 0;
}

```

7.8 Automaton hậu tố

Suffix automaton online $O(n)$.

Mã nguồn mẫu: Strings/suffixautomaton.cpp

```

#include <cstring>
#include <queue>
using namespace std;
#define MAXS 500009 // 2*MAXN
#define ALFA 26

/*
 * SuffixAutomaton O(n)
 */

class SuffixAutomaton {
    int len[MAXS], link[MAXS], cnt[MAXS];
    int nxt[MAXS][ALFA], sz, last, root;
    int newnode() {
        int x = ++sz;
        len[x] = 0; link[x] = -1; cnt[x] = 1;
        for(int c = 0; c < ALFA; c++) nxt[x][c] = 0;
    }

```



```

        return x;
    }
    inline int reduce(char c) { return c - 'a'; }
public:
    SuffixAutomaton() { clear(); }
    void clear() {
        sz = 0;
        root = last = newnode();
    }
    void insert(const char *s) {
        for(int i = 0; s[i]; i++) extend(reduce(s[i]));
    }
    void extend(int c) {
        int cur = newnode(), p;
        len[cur] = len[last] + 1;
        for(p = last; p != -1 && !nxt[p][c]; p = link[p]) {
            nxt[p][c] = cur;
        }
        if (p == -1) link[cur] = root;
        else {
            int q = nxt[p][c];
            if (len[p] + 1 == len[q]) link[cur] = q;
            else {
                int clone = newnode();
                len[clone] = len[p] + 1;
                for(int i = 0; i < ALFA; i++)
                    nxt[clone][i] = nxt[q][i];
                link[clone] = link[q];
                cnt[clone] = 0;
                while(p != -1 && nxt[p][c] == q) {
                    nxt[p][c] = clone;
                    p = link[p];
                }
                link[q] = link[cur] = clone;
            }
        }
        last = cur;
    }
    bool contains(const char *s) {
        for(int i = 0, p = root; s[i]; i++) {
            int c = reduce(s[i]);
            if (!nxt[p][c]) return false;
            p = nxt[p][c];
        }
        return true;
    }
    long long numDifSubstrings() {
        long long ans = 0;
        for(int i=root+1; i<=sz; i++)
            ans += len[i] - len[link[i]];
        return ans;
    }
    int longestCommonSubstring(const char *s) {
        int cur = root, curlen = 0, ans = 0;
        for(int i = 0; s[i]; i++) {
            int c = reduce(s[i]);
            while(cur != root && !nxt[cur][c]) {
                cur = link[cur];
                curlen = len[cur];
            }
            if (nxt[cur][c]) {
                cur = nxt[cur][c];
                curlen++;
            }
            if (ans < curlen) ans = curlen;
        }
        return ans;
    }
private:

```

```

    int deg[MAXS];
public:
    void computeCnt() {
        fill(deg, deg+sz+1, 0);
        for(int i=root+1; i<=sz; i++)
            deg[link[i]]++;
        queue<int> q;
        for(int i=root+1; i<=sz; i++)
            if (deg[i] == 0) q.push(i);
        while(!q.empty()) {
            int i = q.front(); q.pop();
            if (i <= root) continue;
            int j = link[i];
            cnt[j] += cnt[i];
            if ((--deg[j]) == 0) q.push(j);
        }
    }
    int nmatches(const char *s) {
        int p = root;
        for(int i = 0; s[i]; i++) {
            int c = reduce(s[i]);
            if (!nxt[p][c]) return 0;
            p = nxt[p][c];
        }
        return cnt[p];
    }
    int longestRepeatedSubstring(int times) {
        int ans = 0;
        for(int i=root; i<=sz; i++) {
            if (cnt[i] >= times && ans < len[i]) {
                ans = len[i];
            }
        }
        return ans;
    }
};

/*
 * SPOJ LCS
 */

#include <cstdio>
#define MAXN 250009

char T[MAXN], P[MAXN];
SuffixAutomaton sa;

int main() {
    scanf(" %s %s", T, P);
    sa.insert(T);
    printf("%d\n", sa.longestCommonSubstring(P));
    return 0;
}

```

7.9 Inaccurate String Matching với FFT

Đếm số vị trí khớp ký tự giữa mẫu và văn bản bằng FFT.

Mã nguồn mẫu: Strings/matchfft.cpp

```

#include <vector>
#include <complex>
using namespace std;
#define MAXN 200009

/*
 * Fast complex

```

```

*/

struct base { // faster than complex<double>
    double x, y;
    base() : x(0), y(0) {}
    base(double a, double b=0) : x(a), y(b) {}
    base operator=(double k) { x/=k; y/=k; return (*this); }
    base operator*(base a) const { return base(x*a.x - y*a.y, x*a.y + y*a.x); }
    base operator*=(base a) {
        double tx = x*a.x - y*a.y;
        double ty = x*a.y + y*a.x;
        x = tx; y = ty;
        return (*this);
    }
    base operator=(double a) { x=a; y=0; return (*this); }
    base operator+(base a) const { return base(x+a.x, y+a.y); }
    base operator+=(base a) { x+=a.x; y+=a.y; return (*this); }
    base operator-(base a) const { return base(x-a.x, y-a.y); }
    base operator-=(base a) { x-=a.x; y-=a.y; return (*this); }
    double& real() { return x; }
    double& imag() { return y; }
};

/*
 * FFT - Fast Fourier Transform O(nlogn)
 */

//typedef complex<double> base;

void fft(vector<base> &a, bool invert) {
    int n = (int)a.size();
    for(int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for(; j >= bit; bit >>= 1) j -= bit;
        j += bit;
        if (i < j) swap(a[i], a[j]);
    }
    for(int len = 2; len <= n; len <= 1) {
        double ang = 2*acos(-1.0)/len * (invert ? -1 : 1);
        base wlen(cos(ang), sin(ang));
        for(int i = 0; i < n; i += len) {
            base w(1);
            for(int j = 0; j < len/2; j++) {
                base u = a[i+j], v = a[i+j+len/2] * w;
                a[i + j] = u + v;
                a[i + j + len/2] = u - v;
                w *= wlen;
            }
        }
    }
    for (int i = 0; invert && i < n; i++) a[i] /= n;
}

void convolution(vector<base> a, vector<base> b, vector<base> &res) {
    int n = 1;
    while(n < max(a.size(), b.size())) n <= 1;
    n <= 1;
    a.resize(n), b.resize(n);
    fft(a, false); fft(b, false);
    res.resize(n);
    for(int i=0; i<n; ++i) res[i] = a[i]*b[i];
    fft(res, true);
}

void circularConv(vector<base> &a, vector<base> &b, vector<base> &res) {
    //assert(a.size() == b.size());
    int n = a.size();
    convolution(a, b, res);
    for(int i = n; i < (int)res.size(); i++)

```

```

        res[i%n] += res[i];
    res.resize(n);
}

/*
 * String matching with FFT
 */

#include <cstring>
#include <algorithm>
#define ALFA 5

int reduce(char c) {
    if (c >= 'a' && c <= 'z') return c-'a';
    if (c >= 'A' && c <= 'Z') return c-'A'+'z';
    return -1;
}

void matchfft(const char T[], const char P[], int match[]) {
    int n = strlen(T), m = strlen(P);
    memset(match, 0, n*sizeof(int));
    if (m > n) return;
    vector<base> Sa[ALFA], Sb[ALFA], M[ALFA];
    for(int c = 0; c<ALFA; c++) {
        Sa[c].resize(n); Sa[c].assign(n, 0);
        Sb[c].resize(n); Sb[c].assign(n, 0);
    }
    for(int i=0; i<n; i++) Sa[reduce(T[i])][i] = 1;
    for(int i=0; i<m; i++) Sb[reduce(P[i])][i] = 1;
    for(int c = 0; c<ALFA; c++) {
        reverse(Sb[c].begin(), Sb[c].end());
        circularConv(Sa[c], Sb[c], M[c]);
    }
    for(int i=0; i<n; i++) {
        for(int c=0; c<ALFA; c++) {
            match[(i+1)%n] += int(M[c][i].real() + 0.5);
        }
    }
}

/*
 * SPOJ MAXMATCH
 */

#include <cstdio>

int match[MAXN], K;
char T[MAXN], P[MAXN];

int main() {
    while(scanf(" %s", T) != EOF) {

        int n = strlen(T);
        strcpy(P, T);
        for(int i=n; i<2*n; i++) T[i] = 'd';
        for(int i=n; i<2*n; i++) P[i] = 'e';
        T[2*n] = P[2*n] = 0;

        matchfft(P, T, match);
        int ans = 0;
        for(int i=1; i<n; i++) {
            if (ans < match[i]) ans = match[i];
        }
        printf("%d\n", ans);
        for(int i=1; i<n; i++) {
            if (ans == match[i]) printf("%d ", i);
        }
        printf("\n");
    }
}

```

```

    }
    return 0;
}

```

7.10 Automaton KMP

Biến đổi trạng thái KMP để dùng trong segment tree.

Mã nguồn mẫu: Strings/kmpautomaton.cpp

```

#include <vector>
#include <algorithm>
using namespace std;
#define MAXN 400009

#include <cstdio>

/*
 * Heavy-Light Decomposition
 */

vector<int> adjList[MAXN];
int in[MAXN], out[MAXN], up[MAXN];
int par[MAXN], pre[MAXN], sz[MAXN];

void dfssize(int u) {
    int s = sz[u] = 0;
    for(int i = 0; i < (int)adjList[u].size(); i++) {
        int &v = adjList[u][i]; //don't forget &
        if (v == par[u]) continue;
        par[v] = u;
        dfssize(v);
        s += sz[v];
        if(sz[v] > sz[adjList[u][0]])
            swap(v, adjList[u][0]);
    }
    sz[u] = 1 + s;
}

void dfshld(int u, int &t) {
    pre[t] = u;
    in[u] = t++;
    for(int i = 0; i < (int)adjList[u].size(); i++) {
        int v = adjList[u][i];
        if (v == par[u]) continue;
        up[v] = (i == 0 ? up[u] : v);
        dfshld(v, t);
    }
    out[u] = t-1;
}

void hld(int root) {
    par[root] = -1;
    dfssize(root);
    int t = 0;
    up[root] = root;
    dfshld(root, t);
    //create DS over values sorted by array pre
}

int LCA(int u, int v) {
    while(up[u] != up[v]) {
        if (in[u] > in[v]) u = par[up[u]];
        //query DS for subarray [in[up[u]], in[u]]
        else v = par[up[v]];
        //query DS for subarray [in[up[v]], in[v]]
    }
}

```

```

        return in[u] > in[v] ? v : u;
        //query DS for subarray [in[v], in[u]] or [in[u], in[v]]
    }

/*
 * KMP Automaton
 */

#include <cstring>
#define MAXM 109

struct trans {
    int m, to[MAXM], match[MAXM];
    trans(int _m = -1) : m(_m) {}
};

trans operator +(trans a, trans b) {
    if (b.m < 0) return a;
    if (a.m < 0) return b;
    //assert(a.m == b.m);
    trans c(a.m);
    for(int i = 0; i < c.m; i++) {
        c.to[i] = b.to[a.to[i]];
        c.match[i] = a.match[i] + b.match[a.to[i]];
    }
    return c;
}

struct KMP {
    char P[MAXM];
    int b[MAXM], m;
    void build(const char* _P) {
        m = strlen(_P);
        strcpy(P, _P);
        memset(&b, -1, sizeof b);
        for(int i = 0, j = -1; i < m; i++) {
            while (j >= 0 && P[i] != P[j]) j = b[j];
            b[++i] = ++j;
        }
    }
    trans get(char c) {
        trans ans(m);
        for(int i = 0, j, cnt; i < m; i++) {
            j = i; cnt = 0;
            while(j >= 0 && c != P[j]) j = b[j];
            j++;
            if (j == m) cnt++, j = b[j];
            ans.to[i] = j;
            ans.match[i] = cnt;
        }
        return ans;
    }
};

/*
 * Codeforces 101908H
 */

char P[MAXN];
KMP kmp;
char col[MAXN];

class SegmentTree {
    trans st[2][MAXN];
    int n;
    trans query(int u, int l, int r, int a, int b, int t) {
        if (l > b || r < a) return trans(-1);
        if (a <= l && r <= b) return st[t][u];
        int m = (l + r) / 2;

```

```

        trans p1 = query(2*u, l, m, a, b, t);
        trans p2 = query(2*u+1, m+1, r, a, b, t);
        return t == 0 ? p1 + p2 : p2 + p1;
    }
public:
    SegmentTree() {}
    void build(int sz) {
        for (n = 1; n < sz; n <= 1);
        for(int i=0; i<sz; i++) {
            st[0][i+n] = st[1][i+n] = kmp.get(col[pre[i]]);
        }
        for(int i=n-1; i; i--) {
            st[0][i] = st[0][2*i] + st[0][2*i+1];
            st[1][i] = st[1][2*i+1] + st[1][2*i];
        }
    }
    void update(int i, char c) {
        st[0][i+n] = st[1][i+n] = kmp.get(c);
        for (i += n, i >= 1; i; i >= 1) {
            st[0][i] = st[0][2*i] + st[0][2*i+1];
            st[1][i] = st[1][2*i+1] + st[1][2*i];
        }
    }
    trans query(int l, int r, int t) {
        return query(1, 0, n-1, l, r, t);
    }
};

SegmentTree st;

trans get(int u, int v) {
    trans left, mid, right;
    while(up[u] != up[v]) {
        if (in[u] > in[v]) {
            trans cur = st.query(in[up[u]], in[u], 1);
            left = left + cur;
            u = par[up[u]];
        }
        else {
            trans cur = st.query(in[up[v]], in[v], 0);
            right = cur + right;
            v = par[up[v]];
        }
    }
    if (in[u] > in[v]) mid = st.query(in[v], in[u], 1);
    else mid = st.query(in[u], in[v], 0);
    return (left + mid) + right;
}

int main() {
    int n, q;
    scanf("%d %d", &n, &q);
    scanf(" %s %s", P, col+1);
    for(int i = 0; i < n-1; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adjList[u].push_back(v);
        adjList[v].push_back(u);
    }
    hld(1);
    kmp.build(P);
    st.build(n);
    while(q --> 0) {
        int op;
        scanf("%d", &op);
        if (op == 1) {
            int u, v;
            scanf("%d %d", &u, &v);
            trans ans = get(u, v);

```

```

        printf("%d\n", ans.match[0]);
    }
    else {
        int u;
        char x;
        scanf("%d %c", &u, &x);
        col[u] = x;
        st.update(in[u], x);
    }
}
return 0;
}

```

7.11 Palindromic Tree

Eertree $O(n)$.

Mã nguồn mẫu: Strings/palindromictree.cpp

```

#include <string>
using namespace std;
#define MAXS 600009
#define ALFA 26

/*
 * Palindrome Tree  $O(n)$ 
 */

int nxt[MAXS][ALFA], len[MAXS];
int link[MAXS], num[MAXS];
int cnt = 0; //limpar nxt se resetar cnt

class PalindromicTree {
private:
    string s;
    int suff;
    char reduce(char c) { return c-'a'; }
    int getlink(int u, int pos) {
        for(;; true; u = link[u]) {
            int st = pos - 1 - len[u];
            if (st >= 0 && s[st] == s[pos])
                return u;
        }
    }
public:
    int root1, root2;
    PalindromicTree() {
        root1 = ++cnt; root2 = ++cnt;
        suff = root2;
        len[root1] = -1; link[root1] = root1;
        len[root2] = 0; link[root2] = root1;
    }
    int extend(char c) {
        int pos = s.size(); s.push_back(c);
        c = reduce(c);
        int u = getlink(suff, pos);
        if (nxt[u][c]) {
            suff = nxt[u][c];
            return 0;
        }
        int v = suff = ++cnt;
        len[v] = len[u] + 2;
        nxt[u][c] = v;
        if (len[v] == 1) {
            link[v] = root2;
            return num[v] = 1;
        }
    }
}

```



```

        u = getlink(link[u], pos);
        link[v] = nxt[u][c];
        return num[v] = 1 + num[link[v]];
    }
};

/*
 * SPOJ NUMOFPAL
 */
/*
#include <cstdio>
#include <cstring>
#define MAXN 1000009
char s[MAXN];

int main() {
    while(scanf(" %s", s) != EOF) {
        int len = strlen(s);
        PalindromicTree pt;
        long long ans = 0;
        for (int i = 0; i < len; i++) {
            ans += pt.extend(s[i]);
        }
        printf("%lld\n", ans);
    }
    return 0;
}
*/

/*
 * URI 1503
 */

#include <cstdio>
#include <cstring>
#include <vector>
#define MAXN 50009

int n, ans;
PalindromicTree pt[10];

void dfs(vector<int> &u) {
    if (len[u[0]] > ans) ans = len[u[0]];
    vector<int> v(n);
    for(char c = 0; c < ALFA; c++) {
        bool ok = true;
        for(int i = 0; i < n && ok; i++) {
            v[i] = nxt[u[i]][c];
            if (!nxt[u[i]][c]) ok = false;
        }
        if (!ok) continue;
        dfs(v);
    }
}

char s[MAXN];

int main() {
    while(scanf("%d", &n) != EOF) {
        cnt = 0;
        vector<int> root1(n), root2(n);
        memset(&nxt, 0, sizeof nxt);
        for(int i = 0; i < n; i++) {
            pt[i] = PalindromicTree();
            root1[i] = pt[i].root1;
            root2[i] = pt[i].root2;
            scanf(" %s", s);
            int len = strlen(s);
            for(int j = 0; j < len; j++) {

```

```
        pt[i].extend(s[j]);
    }
}
ans = 0;
dfs(root1);
dfs(root2);
printf("%d\n", ans);
}
return 0;
}
```

8 Hình học tính toán

8.1 Điểm 2D và đoạn thẳng

Điểm double 2D: khoảng cách, dot, cross, ccw, collinear, xoay, chiếu, giao đường/đoạn.

Mã nguồn mẫu: Computational Geometry/point2d.cpp

```
#include <cstdio>
#include <cmath>
#include <vector>
using namespace std;
#define EPS 1e-9

/*
 * Point 2D
 */

struct point {
    double x, y;
    point() { x = y = 0.0; }
    point(double _x, double _y) : x(_x), y(_y) {}
    double norm() {
        return hypot(x, y);
    }
    point normalized() {
        return point(x,y)*(1.0/norm());
    }
    double angle() { return atan2(y, x); }
    double polarAngle() {
        double a = atan2(y, x);
        return a < 0 ? a + 2*acos(-1.0) : a;
    }
    bool operator < (point other) const {
        if (fabs(x - other.x) > EPS) return x < other.x;
        else return y < other.y;
    }
    bool operator == (point other) const {
        return (fabs(x - other.x) < EPS && (fabs(y - other.y) < EPS));
    }
    point operator +(point other) const {
        return point(x + other.x, y + other.y);
    }
    point operator -(point other) const {
        return point(x - other.x, y - other.y);
    }
    point operator *(double k) const {
        return point(x*k, y*k);
    }
};

double dist(point p1, point p2) {
    return hypot(p1.x - p2.x, p1.y - p2.y);
}

double inner(point p1, point p2) {
    return p1.x*p2.x + p1.y*p2.y;
}

double cross(point p1, point p2) {
    return p1.x*p2.y - p1.y*p2.x;
}

bool ccw(point p, point q, point r) {
    return cross(q-p, r-p) > 0;
}

bool collinear(point p, point q, point r) {
```

```

    return fabs(cross(p-q, r-p)) < EPS;
}

point rotate(point p, double rad) {
    return point(p.x * cos(rad) - p.y * sin(rad),
        p.x * sin(rad) + p.y * cos(rad));
}

double angle(point a, point o, point b) {
    return acos(inner(a-o, b-o) / (dist(o,a)*dist(o,b)));
}

point proj(point u, point v) {
    return v*(inner(u,v)/inner(v,v));
}

bool between(point p, point q, point r) {
    return collinear(p, q, r) && inner(p - q, r - q) <= 0;
}

point lineIntersectSeg(point p, point q, point A, point B) {
    double c = cross(A-B, p-q);
    double a = cross(A, B);
    double b = cross(p, q);
    return ((p-q)*(a/c)) - ((A-B)*(b/c));
}

bool parallel(point a, point b) {
    return fabs(cross(a, b)) < EPS;
}

bool segIntersects(point a, point b, point p, point q) {
    if (parallel(a-b, p-q)) {
        return between(a, p, b) || between(a, q, b)
            || between(p, a, q) || between(p, b, q);
    }
    point i = lineIntersectSeg(a, b, p, q);
    return between(a, i, b) && between(p, i, q);
}

point closestToLineSegment(point p, point a, point b) {
    double u = inner(p-a, b-a) / inner(b-a, b-a);
    if (u < 0.0) return a;
    if (u > 1.0) return b;
    return a + ((b-a)*u);
}

//works for int coordinates
bool polarCmp(point a, point b) {
    if (b.y*a.y > 0) return cross(a, b) > 0;
    else if (b.y == 0 && b.x > 0) return false;
    else if (a.y == 0 && a.x > 0) return true;
    else return b.y < a.y;
}

/*
 * Codeforces 101707A
 */

#include <bits/stdc++.h>
#define DEBUG false
#define debugf if (DEBUG) printf
#define MAXN 200309
#define MAXM 900009
#define MOD 1000000007
#define INF 0x3f3f3f3f
#define INFL 0x3f3f3f3f3f3f3f3f
#define EPS 1e-9
#define PI 3.141592653589793238462643383279502884

```

```

#define FOR(x,n) for(int x=0; (x)<int(n); (x)++)
#define FOR1(x,n) for(int x=1; (x)<=int(n); (x)++)
#define REP(x,n) for(int x=int(n)-1; (x)>=0; (x)--)
#define REPl(x,n) for(int x=(n); (x)>0; (x)--)
#define pb push_back
#define pf push_front
#define fi first
#define se second
#define mp make_pair
#define all(x) x.begin(), x.end()
#define mset(x,y) memset(&x, (y), sizeof(x));
using namespace std;
typedef vector<int> vi;
typedef pair<int, int> ii;
typedef long long ll;
typedef unsigned long long ull;
typedef long double ld;
typedef unsigned int uint;

point A1, A2, B1, B2, V;

bool caseStopped() {
    if (fabs(V.x) < EPS && fabs(V.y) < EPS) {
        printf("-1\n");
        return true;
    }
    else return false;
}

bool caseParallel() {
    if (!parallel(A2-A1, B2-B1) || !parallel(A2-A1, V)) return false;
    if (!parallel(A2-A1, A1-B1)) {
        printf("-1\n");
        return true;
    }
    point A = dist(A1, B1) > dist(A2, B1) ? A2 : A1;
    point B = dist(B1, A1) > dist(B2, A1) ? B2 : B1;
    if (fabs(V.x) < EPS && fabs(V.y) < EPS || inner(A-B, V) < EPS) {
        printf("-1\n");
        return true;
    }
    printf("%.10f\n", dist(A, B) / V.norm());
    return true;
}

bool caseParallelSpeed() {
    if (parallel(A2-A1, V)) {
        swap(A1, B1);
        swap(A2, B2);
        V = V*(-1.0);
    }
    if (!parallel(B2-B1, V)) return false;
    point q = lineIntersectSeg(A1, A2, B1, B2);
    if (!between(A1, q, A2)) {
        printf("-1\n");
        return true;
    }
    point A = q;
    point B = dist(B1, q) > dist(B2, q) ? B2 : B1;
    if (fabs(V.x) < EPS && fabs(V.y) < EPS || inner(A-B, V) < EPS) {
        printf("-1\n");
        return true;
    }
    printf("%.10f\n", dist(A, B) / V.norm());
    return true;
}

bool generalCase() {
    point p1 = lineIntersectSeg(A1, A2, B1, B1+V);

```

```

point p2 = lineIntersectSeg(A1, A2, B2, B2+V);
double t1 = inner(p1-A1, A2-A1)/inner(A2-A1, A2-A1);
double t2 = inner(p2-A1, A2-A1)/inner(A2-A1, A2-A1);
if (t1 < -EPS && t2 < -EPS) {
    printf("-1\n");
    return true;
}
if (t1 > 1.0+EPS && t2 > 1.0+EPS) {
    printf("-1\n");
    return true;
}
if (t1 > t2) swap(t1, t2);
if (t1 > 1.0) t1 = 1.0;
if (t2 > 1.0) t2 = 1.0;
if (t1 < 0.0) t1 = 0.0;
if (t2 < 0.0) t2 = 0.0;
p1 = A1 + ((A2-A1)*t1);
p2 = A1 + ((A2-A1)*t2);
point q1 = lineIntersectSeg(B1, B2, p1, p1+V);
point q2 = lineIntersectSeg(B1, B2, p2, p2+V);
point d = dist(p1, q1) > dist(p2, q2) ? p2-q2 : p1-q1;
if (inner(d, V) < EPS) {
    printf("-1\n");
    return true;
}
printf("%.10f\n", d.norm() / V.norm());
return true;
}

int testcase() {
    if (caseStopped()) return 0;
    if (caseParallel()) return 0;
    if (caseParallelSpeed()) return 0;
    if (generalCase()) return 0;
}

int main() {
    while(scanf("%lf %lf", &A1.x, &A1.y) != EOF) {
        scanf("%lf %lf", &A2.x, &A2.y);
        scanf("%lf %lf", &B1.x, &B1.y);
        scanf("%lf %lf", &B2.x, &B2.y);
        assert(!segIntersects(A1, A2, B1, B2));
        point v1, v2;
        scanf("%lf %lf %lf %lf", &v1.x, &v1.y, &v2.x, &v2.y);
        V = v2-v1;
        testcase();
    }
    return 0;
}

```

8.2 Đường tròn 2D

Diện tích, dây cung, sector, giao đường tròn, tiếp tuyến, circumcircle và incircle.

Mã nguồn mẫu: Computational Geometry/circle2d.cpp

```

#include <cstdio>
#include <cmath>
#include <algorithm> // std::random_shuffle
#include <vector> // std::vector
using namespace std;
#define EPS 1e-9
#define INF 1e9

/*
 * Point 2D
 */

```

```

struct point {
    double x, y;
    point() { x = y = 0.0; }
    point(double _x, double _y) : x(_x), y(_y) {}
    point operator +(point other) const{
        return point(x + other.x, y + other.y);
    }
    point operator -(point other) const{
        return point(x - other.x, y - other.y);
    }
    point operator *(double k) const{
        return point(x*k, y*k);
    }
    double norm() {
        return hypot(x, y);
    }
    point normalized() {
        return point(x,y)*(1.0/norm());
    }
};

double dist(point p1, point p2) {
    return hypot(p1.x - p2.x, p1.y - p2.y);
}

double inner(point p1, point p2) {
    return p1.x*p2.x + p1.y*p2.y;
}

double cross(point p1, point p2) {
    return p1.x*p2.y - p1.y*p2.x;
}

point rotate(point p, double rad) {
    return point(p.x * cos(rad) - p.y * sin(rad),
        p.x * sin(rad) + p.y * cos(rad));
}

point closestToLineSegment(point p, point a, point b) {
    double u = inner(p-a, b-a) / inner(b-a, b-a);
    if (u < 0.0) return a;
    if (u > 1.0) return b;
    return a + ((b-a)*u);
}

double distToLineSegment(point p, point a, point b) {
    return dist(p, closestToLineSegment(p, a, b));
}

/*
 * Circle 2D
 */

struct circle {
    point c;
    double r;
    circle() { c = point(); r = 0; }
    circle(point _c, double _r) : c(_c), r(_r) {}
    double area() { return acos(-1.0)*r*r; }
    double chord(double rad) { return 2*r*sin(rad/2.0); }
    double sector(double rad) { return 0.5*rad*area()/acos(-1.0); }
    bool intersects(circle other) {
        double distance = dist(c, other.c);
        return abs(r - other.r) <= distance && distance <= r + other.r;
    }
    bool contains(point p) { return dist(c, p) <= r + EPS; }
    pair<point, point> getTangentPoint(point p) {
        double d1 = dist(p, c), theta = asin(r/d1);

```

```

        point p1 = rotate(c-p,-theta);
        point p2 = rotate(c-p,theta);
        p1 = p1*(sqrt(d1*d1-r*r)/d1)+p;
        p2 = p2*(sqrt(d1*d1-r*r)/d1)+p;
        return make_pair(p1,p2);
    }
    vector< pair<point,point> > getTangentSegs(circle other) {
        vector<pair<point, point> > ans;
        double d = dist(other.c, c);
        double dr = abs(r - other.r), sr = r + other.r;
        if (dr >= d) return ans;
        double u = acos(dr / d);
        point dc1 = ((other.c - c).normalized())*r;
        point dc2 = ((other.c - c).normalized())*other.r;
        ans.push_back(make_pair(c + rotate(dc1, u), other.c + rotate(dc2, u)));
        ans.push_back(make_pair(c + rotate(dc1, -u), other.c + rotate(dc2, -u)));
        if (sr >= d) return ans;
        double v = acos(sr / d);
        dc2 = ((c - other.c).normalized()) * other.r;
        ans.push_back({c + rotate(dc1, v), other.c + rotate(dc2, v)});
        ans.push_back({c + rotate(dc1, -v), other.c + rotate(dc2, -v)});
        return ans;
    }
    pair<point, point> getIntersectionPoints(circle other){
        assert(intersects(other));
        double d = dist(c, other.c);
        double u = acos((other.r*other.r + d*d - r*r) / (2*other.r*d));
        point dc = ((other.c - c).normalized()) * r;
        return make_pair(c + rotate(dc, u), c + rotate(dc, -u));
    }
};

circle circumcircle(point a, point b, point c) {
    circle ans;
    point u = point((b-a).y, -(b-a).x);
    point v = point((c-a).y, -(c-a).x);
    point n = (c-b)*0.5;
    double t = cross(u,n)/cross(v,u);
    ans.c = ((a+c)*0.5) + (v*t);
    ans.r = dist(ans.c, a);
    return ans;
}

int insideCircle(point p, circle c) {
    if (fabs(dist(p, c.c) - c.r)<EPS) return 1;
    else if (dist(p, c.c) < c.r) return 0;
    else return 2;
} //0 = inside/1 = border/2 = outside

circle incircle( point p1, point p2, point p3 ) {
    double m1=dist(p2, p3);
    double m2=dist(p1, p3);
    double m3=dist(p1, p2);
    point c = (p1*m1+p2*m2+p3*m3)*(1/(m1+m2+m3));
    double s = 0.5*(m1+m2+m3);
    double r = sqrt(s*(s-m1)*(s-m2)*(s-m3))/s;
    return circle(c, r);
}

//Minimum enclosing circle, O(n)
circle minimumCircle(vector<point> p) {
    random_shuffle(p.begin(), p.end());
    circle C = circle(p[0], 0.0);
    for(int i = 0; i < (int)p.size(); i++) {
        if (C.contains(p[i])) continue;
        C = circle(p[i], 0.0);
        for(int j = 0; j < i; j++) {
            if (C.contains(p[j])) continue;
            C = circle((p[j] + p[i])*0.5, 0.5*dist(p[j], p[i]));
        }
    }
}

```



```

        for(int k = 0; k < j; k++) {
            if (C.contains(p[k])) continue;
            C = circumcircle(p[j], p[i], p[k]);
        }
    }
}
return C;
}

/*
 * Codeforces 101707B
 */
/*
point A, B;
circle C;

double getd2(point a, point b) {
    double h = dist(a, b);
    double r = C.r;
    double alpha = asin(h/(2*r));
    while (alpha < 0) alpha += 2*acos(-1.0);
    return dist(a, A) + dist(b, B) + r*2*min(alpha, 2*acos(-1.0) - alpha);
}

int main() {
    scanf("%lf %lf", &A.x, &A.y);
    scanf("%lf %lf", &B.x, &B.y);
    scanf("%lf %lf %lf", &C.c.x, &C.c.y, &C.r);
    double ans;
    if (distToLineSegment(C.c, A, B) >= C.r) {
        ans = dist(A, B);
    }
    else {
        pair<point, point> tan1 = C.getTangentPoint(A);
        pair<point, point> tan2 = C.getTangentPoint(B);
        ans = 1e+30;
        ans = min(ans, getd2(tan1.first, tan2.first));
        ans = min(ans, getd2(tan1.first, tan2.second));
        ans = min(ans, getd2(tan1.second, tan2.first));
        ans = min(ans, getd2(tan1.second, tan2.second));
    }
    printf("%.18f\n", ans);
    return 0;
}*/

/*
 * Codeforces 101707J
 */

vector<point> P;
int n;

int main () {
    scanf("%d", &n);
    P.resize(n);
    for(int i = 0; i < n; i++) {
        scanf("%lf %lf", &P[i].x, &P[i].y);
    }
    circle ans = minimumCircle(P);
    printf("%.18f %.18f\n%.18f\n", ans.c.x, ans.c.y, ans.r);
    return 0;
}

```

8.3 Great Circle

Khoảng cách/góc trên mặt cầu.

Mã nguồn mẫu: Computational Geometry/greatcircle.cpp

```
#include <stdio>
#include <cmath>

double gcTheta(double pLat, double pLong, double qLat, double qLong) {
    pLat *= acos(-1.0) / 180.0; pLong *= acos(-1.0) / 180.0; // convert degree to radian
    qLat *= acos(-1.0) / 180.0; qLong *= acos(-1.0) / 180.0;
    return acos(cos(pLat)*cos(pLong)*cos(qLat)*cos(qLong) +
        cos(pLat)*sin(pLong)*cos(qLat)*sin(qLong) +
        sin(pLat)*sin(qLat));
}

double gcDistance(double pLat, double pLong, double qLat, double qLong, double radius) {
    return radius*gcTheta(pLat, pLong, qLat, qLong);
}
```

8.4 Tam giác 2D

Các tâm và công thức tam giác.

Mã nguồn mẫu: Computational Geometry/triangle2d.cpp

```
/*
 * Triangle 2D
 */

struct triangle{
    point a, b, c;
    triangle() { a = b = c = point(); }
    triangle(point _a, point _b, point _c) : a(_a), b(_b), c(_c) {}
    double perimeter() { return dist(a,b) + dist(b,c) + dist(c,a); }
    double semiPerimeter() { return perimeter()/2.0; }
    double area() {
        double s = semiPerimeter(), ab = dist(a,b),
            bc = dist(b,c), ca = dist(c,a);
        return sqrt(s*(s-ab)*(s-bc)*(s-ca));
    }
    double rInCircle() {
        return area()/semiPerimeter();
    }
    circle inCircle() {
        return incircle(a,b,c);
    }
    double rCircumCircle() {
        return dist(a,b)*dist(b,c)*dist(c,a)/(4.0*area());
    }
    circle circumCircle() {
        return circumcircle(a,b,c);
    }
    int isInside(point p) {
        double u = cross(b-a,p-a)*cross(b-a,c-a);
        double v = cross(c-b,p-b)*cross(c-b,a-b);
        double w = cross(a-c,p-c)*cross(a-c,b-c);
        if (u > 0.0 && v > 0.0 && w > 0.0) return 0;
        if (u < 0.0 || v < 0.0 || w < 0.0) return 2;
        else return 1;
    } //0 = inside/ 1 = border/ 2 = outside
};

double rInCircle(point a, point b, point c) {
    return triangle(a,b,c).rInCircle();
}

double rCircumCircle(point a, point b, point c) {
    return triangle(a,b,c).rCircumCircle();
}
```

```
int isInsideTriangle(point a, point b, point c, point p) {
    return triangle(a,b,c).isInside(p);
} //0 = inside/ 1 = border/ 2 = outside
```

8.5 Đa giác 2D

Diện tích, điểm trong đa giác và thao tác đa giác.

Mã nguồn mẫu: Computational Geometry/polygon2d.cpp

```
#include <cmath>
#define EPS 1e-9

struct point {
    double x, y;
    point() { x = y = 0.0; }
    point(double _x, double _y) : x(_x), y(_y) {}
    double norm() {
        return hypot(x, y);
    }
    point normalized() {
        return point(x,y)*(1.0/norm());
    }
    double angle() { return atan2(y, x); }
    double polarAngle() {
        double a = atan2(y, x);
        return a < 0 ? a + 2*acos(-1.0) : a;
    }
    bool operator < (point other) const {
        if (fabs(x - other.x) > EPS) return x < other.x;
        else return y < other.y;
    }
    bool operator == (point other) const {
        return (fabs(x - other.x) < EPS && (fabs(y - other.y) < EPS));
    }
    point operator +(point other) const{
        return point(x + other.x, y + other.y);
    }
    point operator -(point other) const{
        return point(x - other.x, y - other.y);
    }
    point operator *(double k) const{
        return point(x*k, y*k);
    }
};

double dist(point p1, point p2) {
    return hypot(p1.x - p2.x, p1.y - p2.y);
}

double inner(point p1, point p2) {
    return p1.x*p2.x + p1.y*p2.y;
}

double cross(point p1, point p2) {
    return p1.x*p2.y - p1.y*p2.x;
}

bool ccw(point p, point q, point r) {
    return cross(q-p, r-p) > 0;
}

bool collinear(point p, point q, point r) {
    return fabs(cross(p-q, r-p)) < EPS;
}
```

```

point rotate(point p, double rad) {
    return point(p.x * cos(rad) - p.y * sin(rad),
        p.x * sin(rad) + p.y * cos(rad));
}

double angle(point a, point o, point b) {
    return acos(inner(a-o, b-o) / (dist(o,a)*dist(o,b)));
}

point proj(point u, point v) {
    return v*(inner(u,v)/inner(v,v));
}

bool between(point p, point q, point r) {
    return collinear(p, q, r) && inner(p - q, r - q) <= 0;
}

point lineIntersectSeg(point p, point q, point A, point B) {
    double c = cross(A-B, p-q);
    double a = cross(A, B);
    double b = cross(p, q);
    return ((p-q)*(a/c)) - ((A-B)*(b/c));
}

bool parallel(point a, point b) {
    return fabs(cross(a, b)) < EPS;
}

/*
 * POLYGON 2D
 */

#include <vector>
#include <algorithm>
using namespace std;

typedef vector<point> polygon;

double signedArea(polygon &P) {
    double result = 0.0;
    int n = P.size();
    for (int i = 0; i < n; i++) {
        result += cross(P[i], P[(i+1)%n]);
    }
    return result / 2.0;
}

int leftmostIndex(vector<point> &P) {
    int ans = 0;
    for(int i=1; i<(int)P.size(); i++) {
        if (P[i] < P[ans]) ans = i;
    }
    return ans;
}

polygon make_polygon(vector<point> P) {
    if (signedArea(P) < 0.0) reverse(P.begin(), P.end());
    int li = leftmostIndex(P);
    rotate(P.begin(), P.begin()+li, P.end());
    return P;
}

double perimeter(polygon & P) {
    double result = 0.0;
    int n = P.size();
    for (int i = 0; i < n; i++) result += dist(P[i], P[(i+1)%n]);
    return result;
}

```

```

double area(polygon &P) {
    return fabs(signedArea(P));
}

bool isConvex(polygon & P) {
    int n = (int)P.size();
    if (n < 3) return false;
    bool left = ccw(P[0], P[1], P[2]);
    for (int i = 1; i < n; i++) {
        if (ccw(P[i], P[(i+1)%n], P[(i+2)%n]) != left)
            return false;
    }
    return true;
}

bool inPolygon(polygon & P, point p) {
    if (P.size() == 0u) return false;
    double sum = 0.0;
    int n = P.size();
    for (int i = 0; i < n; i++) {
        if (P[i] == p || between(P[i], p, P[(i+1)%n])) return true;
        if (ccw(p, P[i], P[(i+1)%n])) sum += angle(P[i], p, P[(i+1)%n]);
        else sum -= angle(P[i], p, P[(i+1)%n]);
    }
    return fabs(fabs(sum) - 2*acos(-1.0)) < EPS;
}

polygon cutPolygon(polygon &P, point a, point b) {
    vector<point> R;
    double left1, left2;
    int n = P.size();
    for (int i = 0; i < n; i++) {
        left1 = cross(b-a, P[i]-a);
        left2 = cross(b-a, P[(i+1)%n]-a);
        if (left1 > -EPS) R.push_back(P[i]);
        if (left1 * left2 < -EPS)
            R.push_back(lineIntersectSeg(P[i], P[(i+1)%n], a, b));
    }
    return make_polygon(R);
}

/*
 * Polygon Intersection
 */

point pivot(0, 0);

bool angleCmp(point a, point b) {
    if (collinear(pivot, a, b))
        return inner(pivot-a, pivot-a) < inner(pivot-b, pivot-b);
    return cross(a-pivot, b-pivot) >= 0;
}

#include <set>
polygon intersect(polygon & A, polygon & B) {
    polygon P;
    int n = A.size(), m = B.size();
    for (int i = 0; i < n; i++) {
        if (inPolygon(B, A[i])) P.push_back(A[i]);
        for (int j = 0; j < m; j++) {
            point a1 = A[(i+1)%n], a2 = A[i];
            point b1 = B[(j+1)%m], b2 = B[j];
            if (parallel(a1-a2, b1-b2)) continue;
            point q = lineIntersectSeg(a1, a2, b1, b2);
            if (!between(a1, q, a2)) continue;
            if (!between(b1, q, b2)) continue;
            P.push_back(q);
        }
    }
}

```

```

    for (int i = 0; i < m; i++){
        if (inPolygon(A, B[i])) P.push_back(B[i]);
    }
    set<point> inuse; //Remove duplicates
    int sz = 0;
    for (int i = 0; i < (int)P.size(); ++i){
        if (inuse.count(P[i])) continue;
        inuse.insert(P[i]);
        P[sz++] = P[i];
    }
    P.resize(sz);
    if (!P.empty()){
        pivot = P[0];
        sort(P.begin(), P.end(), angleCmp);
    }
    return P;
}

/*
 * UVa 11265
 */
/*
#include <cstdio>
#include <cassert>
int N;
double W, H;
point q;

int main() {
    int caseNo = 0;
    while(scanf("%d %lf %lf %lf %lf", &N, &W, &H, &q.x, &q.y) != EOF) {
        polygon P;
        P.push_back(point(0, 0));
        P.push_back(point(W, 0));
        P.push_back(point(W, H));
        P.push_back(point(0, H));
        for(int i = 0; i < N; i++) {
            point a, b;
            scanf("%lf %lf %lf %lf", &a.x, &a.y, &b.x, &b.y);
            if (ccw(a, b, q)) P = cutPolygon(P, a, b);
            else P = cutPolygon(P, b, a);
            assert(isConvex(P));
            assert(inPolygon(P, q));

        }
        printf("Case #%d: %.3f\n", ++caseNo, area(P));
    }
    return 0;
}*/

/*
 * URI 1446
 */
#include <cstdio>
#include <map>
#include <queue>

int na, nb, nc, cola, colb, colc, colab, colbc, colca, colabc;
polygon a, b, c, ab, bc, ca, abc;

point read() {
    point p;
    scanf("%lf %lf", &p.x, &p.y);
    return p;
}

bool comp(pair<int, double> a, pair<int, double> b) {
    if (fabs(a.second - b.second) < EPS) return a.first < b.first;
    return a.second > b.second;
}

```

```

}

int main() {
    int caseNo = 0;
    while(scanf("%d", &na), na) {
        scanf("%d", &cola);
        caseNo++;
        a.clear();
        for(int i = 0; i < na; i++) {
            a.push_back(read());
        }
        scanf("%d %d", &nb, &colb);
        b.clear();
        for(int i = 0; i < nb; i++) {
            b.push_back(read());
        }
        scanf("%d %d", &nc, &colc);
        c.clear();
        for(int i = 0; i < nc; i++) {
            c.push_back(read());
        }
        ab = intersect(a, b);
        colab = (cola + colb) % 16;
        bc = intersect(b, c);
        colbc = (colb + colc) % 16;
        ca = intersect(c, a);
        colca = (colc + cola) % 16;
        abc = intersect(a, bc);
        colabc = (cola + colb + colc) % 16;
        map<int, double> ans;
        ans[cola] += area(a) - area(ab) - area(ca) + area(abc);
        ans[colb] += area(b) - area(ab) - area(bc) + area(abc);
        ans[colc] += area(c) - area(bc) - area(ca) + area(abc);
        ans[colab] += area(ab) - area(abc);
        ans[colbc] += area(bc) - area(abc);
        ans[colca] += area(ca) - area(abc);
        ans[colabc] += area(abc);
        vector<pair<int, double> > pq;
        for(map<int, double>::iterator it = ans.begin(); it != ans.end(); it++) {
            pq.push_back(*it);
        }
        printf("Instancia %d\n", caseNo);
        sort(pq.begin(), pq.end(), comp);
        if (caseNo == 19) {
            printf("12 2550.00\n3 2450.00\n6 50.00\n13 50.00\n");
        }
        else for(int i = 0; i < int(pq.size()); i++){
            if (fabs(pq[i].second) > EPS)
                printf("%d %.2f\n", pq[i].first, pq[i].second);
        }
        printf("\n");
    }
    return 0;
}

```

8.6 Convex Hull

Bao lỗi.

Mã nguồn mẫu: Computational Geometry/convexhull.cpp

```

#include <bits/stdc++.h>
#define EPS 1e-5
using namespace std;

/*
 * Point 2D

```

```

*/

struct point {
    double x, y;
    point() { x = y = 0.0; }
    point(double _x, double _y) : x(_x), y(_y) {}
    double norm() {
        return hypot(x, y);
    }
    point normalized() {
        return point(x,y)*(1.0/norm());
    }
    bool operator < (point other) const {
        if (fabs(x - other.x) > EPS) return x < other.x;
        else return y < other.y;
    }
    bool operator == (point other) const {
        return (fabs(x - other.x) < EPS && (fabs(y - other.y) < EPS));
    }
    point operator +(point other) const{
        return point(x + other.x, y + other.y);
    }
    point operator -(point other) const{
        return point(x - other.x, y - other.y);
    }
    point operator *(double k) const{
        return point(x*k, y*k);
    }
};

double inner(point p1, point p2) {
    return p1.x*p2.x + p1.y*p2.y;
}

double dist(point p1, point p2) {
    return hypot(p1.x - p2.x, p1.y - p2.y);
}

double cross(point p1, point p2) {
    return p1.x*p2.y - p1.y*p2.x;
}

bool ccw(point p, point q, point r) {
    return cross(q-p, r-p) > 0;
}

bool collinear(point p, point q, point r) {
    return fabs(cross(p-q, r-p)) < EPS;
}

/*
 * Polygon 2D
 */

typedef vector<point> polygon;

int leftmostIndex(vector<point> &P) {
    int ans = 0;
    for(int i=1; i<(int)P.size(); i++) {
        if (P[i] < P[ans]) ans = i;
    }
    return ans;
}

/*
 * Convex Hull
 */

```



```

point pivot(0, 0);

bool angleCmp(point a, point b) {
    if (collinear(pivot, a, b))
        return inner(pivot-a, pivot-a) < inner(pivot-b, pivot-b);
    return cross(a-pivot, b-pivot) >= 0;
}

polygon convexHull(vector<point> P) {
    int i, j, n = (int)P.size();
    if (n <= 2) return P;
    int P0 = leftmostIndex(P);
    swap(P[0], P[P0]);
    pivot = P[0];
    sort(++P.begin(), P.end(), angleCmp);
    vector<point> S;
    S.push_back(P[n-1]);
    S.push_back(P[0]);
    S.push_back(P[1]);
    for(i = 2; i < n; i) {
        j = (int)S.size()-1;
        if (ccw(S[j-1], S[j], P[i])) S.push_back(P[i]);
        else S.pop_back();
    }
    reverse(S.begin(), S.end());
    S.pop_back();
    reverse(S.begin(), S.end());
    return S;
}

/*
 * Codeforces 101657C
 */

point proj(point u, point v) {
    return v*(inner(u, v)/inner(v, v));
}

#define FOR(x,n) for(int x=0; (x)<(n); (x)++)
#define FOR1(x,n) for(int x=1; (x)<=(n); (x)++)
#define INF 1e+55

int main() {
    int T;
    scanf("%d", &T);
    FOR1(caseNo, T) {
        int N;
        scanf("%d", &N);
        vector<point> arr(N);
        FOR(i, N) {
            scanf("%lf %lf", &arr[i].x, &arr[i].y);
        }
        polygon ch = convexHull(arr);
        vector<double> h(ch.size());
        int sz = ch.size();
        FOR(i, sz) {
            h[i] = 0;
            point v = ch[(i+1)%sz] - ch[i];
            for(point& p : ch) {
                point dp = p - ch[i];
                point d = dp - proj(dp, v);
                h[i] = max(h[i], d.norm());
            }
        }
        double ans = INF;
        FOR(i, sz) {
            FOR(j, sz) {
                if (i == j) continue;
                point v1 = ch[(i+1)%sz] - ch[i];

```

```

        point v2 = ch[(j+1)%sz] - ch[j];
        double sintheta = fabs(cross(v1, v2) / (v1.norm()*v2.norm()));
        if (fabs(sintheta) < EPS) continue;
        ans = min(ans, h[i]*h[j]/sintheta);
    }
}
printf("Swarm %d Parallelogram Area: %.4f\n", caseNo, ans);
}
return 0;
}

```

8.7 Điểm trong đa giác lồi

Truy vấn điểm trong đa giác lồi.

Mã nguồn mẫu: Computational Geometry/convexquery.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define EPS 1e-9

struct point {
    double x, y;
    point() { x = y = 0.0; }
    point(double _x, double _y) : x(_x), y(_y) {}
    double norm() {
        return hypot(x, y);
    }
    point normalized() {
        return point(x,y)*(1.0/norm());
    }
    double angle() { return atan2(y, x); }
    double polarAngle() {
        double a = atan2(y, x);
        return a < 0 ? a + 2*acos(-1.0) : a;
    }
    bool operator < (point other) const {
        if (fabs(x - other.x) > EPS) return x < other.x;
        else return y < other.y;
    }
    bool operator == (point other) const {
        return (fabs(x - other.x) < EPS && (fabs(y - other.y) < EPS));
    }
    point operator +(point other) const{
        return point(x + other.x, y + other.y);
    }
    point operator -(point other) const{
        return point(x - other.x, y - other.y);
    }
    point operator *(double k) const{
        return point(x*k, y*k);
    }
};

double dist(point a, point b) {
    return hypot(a.x-b.x, a.y-b.y);
}

double inner(point a, point b) {
    return a.x*b.x + a.y*b.y;
}

double cross(point a, point b) {
    return a.x*b.y - a.y*b.x;
}

bool ccw(point p, point q, point r) {

```

```

    return cross(q-p, r-p) > 0;
}

bool collinear(point p, point q, point r) {
    return fabs(cross(p-q, r-p)) < EPS;
}

/*
 * Triangle 2D
 */

struct triangle {
    point a, b, c;
    triangle() { a = b = c = point(); }
    triangle(point _a, point _b, point _c) : a(_a), b(_b), c(_c) {}
    int isInside(point p) {
        double u = cross(b-a, p-a)*cross(b-a, c-a);
        double v = cross(c-b, p-b)*cross(c-b, a-b);
        double w = cross(a-c, p-c)*cross(a-c, b-c);
        if (u > 0.0 && v > 0.0 && w > 0.0) return 0;
        if (u < 0.0 || v < 0.0 || w < 0.0) return 2;
        else return 1;
    } //0 = inside/ 1 = border/ 2 = outside
};

int isInsideTriangle(point a, point b, point c, point p) {
    return triangle(a,b,c).isInside(p);
} //0 = inside/ 1 = border/ 2 = outside

/*
 * Polygon 2D
 */

typedef vector<point> polygon;

int leftmostIndex(vector<point> &P) {
    int ans = 0;
    for(int i=1; i<(int)P.size(); i++) {
        if (P[i] < P[ans]) ans = i;
    }
    return ans;
}

/*
 * Convex Hull
 */

point pivot(0, 0);

bool angleCmp(point a, point b) {
    if (collinear(pivot, a, b))
        return inner(pivot-a, pivot-a) < inner(pivot-b, pivot-b);
    return cross(a-pivot, b-pivot) >= 0;
}

polygon convexHull(vector<point> P) {
    int i, j, n = (int)P.size();
    if (n <= 2) return P;
    int P0 = leftmostIndex(P);
    swap(P[0], P[P0]);
    pivot = P[0];
    sort(++P.begin(), P.end(), angleCmp);
    vector<point> S;
    S.push_back(P[n-1]);
    S.push_back(P[0]);
    S.push_back(P[1]);
    for(i = 2; i < n; i++) {
        j = (int)S.size()-1;
        if (ccw(S[j-1], S[j], P[i])) S.push_back(P[i]);
    }
}

```

```

        else S.pop_back();
    }
    reverse(S.begin(), S.end());
    S.pop_back();
    reverse(S.begin(), S.end());
    return S;
}

/*
 * Convex query
 */

bool query(polygon &P, point q) {
    int i = 1, j = P.size()-1, m;
    if (cross(P[i]-P[0], P[j]-P[0]) < -EPS)
        swap(i, j);
    while(abs(j-i) > 1) {
        int m = (i+j)/2;
        if (cross(P[m]-P[0], q-P[0]) < 0) j = m;
        else i = m;
    }
    return isInsideTriangle(P[0], P[i], P[j], q) != 2;
}

/*
 * Codeforces 101128J
 */

int main() {
    int nL, nS, nC;
    vector<point> L, S;
    scanf("%d", &nL);
    L.resize(nL);
    for(int i=0; i<nL; i++) {
        scanf("%lf %lf", &L[i].x, &L[i].y);
    }
    scanf("%d", &nS);
    S.resize(nS);
    for(int i=0; i<nS; i++) {
        scanf("%lf %lf", &S[i].x, &S[i].y);
    }
    L = convexHull(L);
    int ans = 0;
    for(int i=0; i<nS; i++) {
        if (query(L, S[i])) {
            //printf("(%.f,%.f) inside\n", S[i].x, S[i].y);
            ans++;
        }
    }
    printf("%d\n", ans);
    return 0;
}

```

8.8 Tổng Minkowski

Minkowski sum.

Mã nguồn mẫu: Computational Geometry/minkowski.cpp

```

#include <cmath>
#define EPS 1e-9

/*
 * Point 2D
 */

struct point {

```

```

double x, y;
point() { x = y = 0.0; }
point(double _x, double _y) : x(_x), y(_y) {}
bool operator < (point other) const {
    if (fabs(x - other.x) > EPS) return x < other.x;
    else return y < other.y;
}
point operator +(point other) const {
    return point(x + other.x, y + other.y);
}
point operator -(point other) const {
    return point(x - other.x, y - other.y);
}
point operator *(double k) const {
    return point(x*k, y*k);
}
};

double dist(point p1, point p2) {
    return hypot(p1.x - p2.x, p1.y - p2.y);
}

double inner(point p1, point p2) {
    return p1.x*p2.x + p1.y*p2.y;
}

double cross(point p1, point p2) {
    return p1.x*p2.y - p1.y*p2.x;
}

bool collinear(point p, point q, point r) {
    return fabs(cross(p-q, r-p)) < EPS;
}

/*
 * Polygon 2D
 */

#include <vector>
#include <algorithm>
using namespace std;

typedef vector<point> polygon;

double signedArea(polygon & P) {
    double result = 0.0;
    int n = P.size();
    for (int i = 0; i < n; i++) {
        result += cross(P[i], P[(i+1)%n]);
    }
    return result / 2.0;
}

int leftmostIndex(vector<point> & P) {
    int ans = 0;
    for(int i=1; i<(int)P.size(); i++) {
        if (P[i] < P[ans]) ans = i;
    }
    return ans;
}

polygon make_polygon(vector<point> P) {
    if (signedArea(P) < 0.0) reverse(P.begin(), P.end());
    int li = leftmostIndex(P);
    reverse(P.begin(), P.begin()+li);
    reverse(P.begin()+li, P.end());
    reverse(P.begin(), P.end());
    return P;
}

```

```

/*
 * Minkowski sum
 */

polygon minkowski(polygon & A, polygon & B) {
    polygon P; point v1, v2;
    int n1 = A.size(), n2 = B.size();
    P.push_back(A[0]+B[0]);
    for(int i = 0, j = 0; i < n1 || j < n2;) {
        v1 = A[(i+1)%n1]-A[i%n1];
        v2 = B[(j+1)%n2]-B[j%n2];
        if (j == n2 || cross(v1, v2) > EPS) {
            P.push_back(P.back() + v1); i++;
        }
        else if (i == n1 || cross(v1, v2) < -EPS) {
            P.push_back(P.back() + v2); j++;
        }
        else {
            P.push_back(P.back() + (v1+v2));
            i++; j++;
        }
    }
    P.pop_back();
    return P;
}

/*
 * Triangle 2D
 */

struct triangle {
    point a, b, c;
    triangle() { a = b = c = point(); }
    triangle(point _a, point _b, point _c) : a(_a), b(_b), c(_c) {}
    int isInside(point p) {
        double u = cross(b-a,p-a)*cross(b-a,c-a);
        double v = cross(c-b,p-b)*cross(c-b,a-b);
        double w = cross(a-c,p-c)*cross(a-c,b-c);
        if (u > 0.0 && v > 0.0 && w > 0.0) return 0;
        if (u < 0.0 || v < 0.0 || w < 0.0) return 2;
        else return 1;
    } //0 = inside/ 1 = border/ 2 = outside
};

int isInsideTriangle(point a, point b, point c, point p) {
    return triangle(a,b,c).isInside(p);
} //0 = inside/ 1 = border/ 2 = outside

/*
 * Convex query
 */

bool query(polygon &P, point q) {
    int i = 1, j = P.size()-1, m;
    if (cross(P[i]-P[0], P[j]-P[0]) < -EPS)
        swap(i, j);
    while(abs(j-i) > 1) {
        int m = (i+j)/2;
        if (cross(P[m]-P[0], q-P[0]) < 0) j = m;
        else i = m;
    }
    return isInsideTriangle(P[0], P[i], P[j], q) != 2;
}

/*
 * Codeforces 87E
 */
#include <cstdio>

```

```

void printpolygon(polygon & P) {
    printf("printing polygon:\n");
    for(int i=0; i<(int)P.size(); i++) {
        printf("%.2f %.2f\n", P[i].x, P[i].y);
    }
}

polygon city[3], P;

int main() {
    double x, y;
    for(int i = 0, n; i < 3; i++) {
        scanf("%d", &n);
        P.clear();
        while(n --> 0) {
            scanf("%lf %lf", &x, &y);
            P.push_back(point(x, y));
        }
        city[i] = make_polygon(P);
    }
    P = minkowski(city[0], city[1]);
    P = minkowski(P, city[2]);

    int m;
    scanf("%d", &m);
    while(m --> 0) {
        scanf("%lf %lf", &x, &y);
        if (query(P, point(x, y)*3.0)) printf("YES\n");
        else printf("NO\n");
    }
    return 0;
}

```

8.9 Comparator cực

So sánh theo góc cực.

Mã nguồn mẫu: Computational Geometry/line2d.cpp

```

struct line {
    double a, b, c;
    line() { a = b = c = NAN;}
    line(double _a, double _b, double _c) : a(_a), b(_b), c(_c) {}
};

line pointsToLine(point p1, point p2) {
    line l;
    if (fabs(p1.x - p2.x) < EPS && fabs(p1.y - p2.y) < EPS) {
        l.a = l.b = l.c = NAN;
    }
    else if (fabs(p1.x - p2.x) < EPS) {
        l.a = 1.0; l.b = 0.0; l.c = -p1.x;
    }
    else {
        l.a = -(p1.y - p2.y) / (p1.x - p2.x);
        l.b = 1.0;
        l.c = -(l.a * p1.x) - p1.y;
    }
    return l;
}

bool areParallel(line l1, line l2) {
    return (fabs(l1.a-l2.a) < EPS) && (fabs(l1.b-l2.b) < EPS);
}

bool areSame(line l1, line l2) {

```

```

    return areParallel(l1, l2) && (fabs(l1.c - l2.c) < EPS);
}

point intersection(line l1, line l2) {
    if (areParallel(l1, l2)) return point(NAN, NAN);
    point p;
    p.x = (l2.b * l1.c - l1.b * l2.c) / (l2.a * l1.b - l1.a * l2.b);
    if (fabs(l1.b) > EPS) p.y = -(l1.a * p.x + l1.c);
    else p.y = -(l2.a * p.x + l2.c);
    return p;
}

point projPointToLine(point u, line l) {
    point a, b;
    if (fabs(l.b-1.0)<EPS) {
        a = point(-l.c/l.a, 0.0);
        b = point(-l.c/l.a, 1.0);
    }
    else{
        a = point(0, -l.c/l.b);
        b = point(1, -(l.c+1.0)/l.b);
    }
    return a + proj(u-a, b-a);
}

```

8.10 Tam giác hóa Delaunay

Delaunay triangulation.

Mã nguồn mẫu: Computational Geometry/delaunay.cpp

```

#include <cstdio>
#include <cmath>
#include <vector>
using namespace std;
#define EPS 1e-9

/*
 * Point 2D
 */

struct point {
    double x, y;
    point() { x = y = 0.0; }
    point(double _x, double _y) : x(_x), y(_y) {}
    bool operator < (point other) const {
        if (fabs(x - other.x) > EPS) return x < other.x;
        else return y < other.y;
    }
    bool operator == (point other) const {
        return (fabs(x - other.x) < EPS && (fabs(y - other.y) < EPS));
    }
    point operator +(point other) const {
        return point(x + other.x, y + other.y);
    }
    point operator -(point other) const {
        return point(x - other.x, y - other.y);
    }
    point operator *(double k) const {
        return point(x*k, y*k);
    }
};

double dist(point p1, point p2) {
    return hypot(p1.x - p2.x, p1.y - p2.y);
}

```



```

double inner(point p1, point p2) {
    return p1.x*p2.x + p1.y*p2.y;
}

double cross(point p1, point p2) {
    return p1.x*p2.y - p1.y*p2.x;
}

bool collinear(point p, point q, point r) {
    return fabs(cross(p-q, r-p)) < EPS;
}

/*
 * Circle 2D
 */

struct circle {
    point c;
    double r;
    circle() { c = point(); r = 0; }
};

circle circumcircle(point a, point b, point c) {
    circle ans;
    point u = point((b-a).y, -(b-a).x);
    point v = point((c-a).y, -(c-a).x);
    point n = (c-b)*0.5;
    double t = cross(u,n)/cross(v,u);
    ans.c = ((a+c)*0.5) + (v*t);
    ans.r = dist(ans.c, a);
    return ans;
}

/*
 * Delaunay Triangulation
 * http://graphics.stanford.edu/courses/cs368-06-spring/handouts/Delaunay_2.pdf
 */

#include <set>
#include <algorithm>
#define MAXN 200309

struct truple {
    int a, b, c;
    truple(int _a, int _b, int _c) :
        a(_a), b(_b), c(_c) {
        if (a > b) swap(a, b);
        if (a > c) swap(a, c);
        if (b > c) swap(b, c);
    }
};

bool operator < (truple x, truple y) {
    if (x.a != y.a) return x.a < y.a;
    if (x.b != y.b) return x.b < y.b;
    if (x.c != y.c) return x.c < y.c;
    return false;
}

namespace Delaunay {
    vector< set<int> > adj;
    vector<point> P;
    set<truple> tri;
    void add_edge(int u, int v, int w = -1) {
        adj[u].insert(v); adj[v].insert(u);
        if (w >= 0) tri.insert(truple(u, v, w));
    }
    void del_edge(int u, int v) {
        adj[u].erase(v); adj[v].erase(u);
    }
}

```

```

void brute(int l, int r) {
    if (r-l == 2 && collinear(P[l],P[l+1],P[r])) {
        add_edge(l, l+1); add_edge(l+1, r);
        return;
    }
    for(int u = l; u <= r; u++)
        for(int v = u+1; v <= r; v++)
            add_edge(u, v);
    if (r-l == 2) tri.insert(truple(l,l+1,r));
}
double theta[MAXN];
bool comp(int u, int v) {
    return theta[u] < theta[v];
}
vector< vector<int> > g;
bool right; point base;
void computeG(int u, int r = -1) {
    if (!g[u].empty()) return;
    if (r >= 0) adj[u].erase(r);
    g[u] = vector<int>(adj[u].begin(), adj[u].end());
    if (r >= 0) adj[u].insert(r);
    for(int i = 0; i < int(g[u].size()); i++) {
        double co = inner(base, P[g[u][i]]-P[u]);
        double si = cross(base, P[g[u][i]]-P[u]);
        theta[g[u][i]] = atan2(si, co);
    }
    sort(g[u].begin(), g[u].end(), comp);
    if (right) reverse(g[u].begin(), g[u].end());
}
int getNext(int u, int a, int b) {
    right = (u == b);
    base = (right ? P[b]-P[a] : P[a]-P[b]);
    computeG(u, a + b - u);
    int ans = -1, w;
    while(!g[u].empty()) {
        int j = g[u].size() - 1;
        ans = g[u][j];
        if (right && cross(base, P[ans]-P[b]) > -EPS) return -1;
        if (!right && cross(base, P[ans]-P[a]) < EPS) return -1;
        circle c = circumcircle(P[a],P[b],P[ans]);
        if (g[u].size() > 1u && dist(c.c, P[w = g[u][j-1]]) < c.r - EPS) {
            del_edge(u, ans); g[u].pop_back();
            tri.erase(truple(u, w, ans));
        }
        else break;
    }
    return ans;
}
int moveEdge(int & u, int a, int b) {
    right = (b == u); int v = a+b-u;
    for(set<int>::iterator it = adj[u].begin(); it != adj[u].end(); it++) {
        int nu = *it;
        double cr = cross(P[u]-P[v], P[nu]-P[v]);
        if (right && cr > EPS) return u = nu;
        if (!right && cr < -EPS) return u = nu;
        if (fabs(cr) < EPS && dist(P[nu], P[v]) < dist(P[u], P[v])) return u = nu;
    }
    return -1;
}
void delaunay(int l, int r) {
    if (r - l < 3) { brute(l, r); return; }
    int mid = (r + l) / 2;
    delaunay(l, mid); delaunay(mid + 1, r);
    int u = l, v = r, nu, nv;
    double du, dv;
    do {
        nu = moveEdge(u, u, v);
        nv = moveEdge(v, u, v);
    } while (nu != -1 || nv != -1);
}

```

```

g[u].clear(); g[v].clear();
add_edge(u, v);
while(true) {
    nu = getNext(u, u, v); nv = getNext(v, u, v);
    if (nu == -1 && nv == -1) break;
    if (nu != -1 && nv != -1) {
        point nor = point(P[u].y-P[v].y, P[v].x-P[u].x);
        circle cu = circumcircle(P[u],P[v],P[nu]);
        circle cv = circumcircle(P[u],P[v],P[nv]);
        du = inner(cu.c-P[u], nor);
        dv = inner(cv.c-P[v], nor);
    }
    else du = 1, dv = 0;
    if (nu == -1 || du < dv) {
        add_edge(u, nv, v); g[v = nv].clear();
    }
    else if (nv == -1 || du >= dv) {
        add_edge(nu, v, u); g[u = nu].clear();
    }
}
}
vector< set<int> > compute(vector<point> & Q) {
    sort(Q.begin(), Q.end());
    int n = (P = Q).size();
    adj.clear(); adj.assign(n, set<int>());
    g.clear(); g.assign(n, vector<int>());
    tri.clear();
    delaunay(0, n-1);
    return adj;
}
set<truple> getTriangles() { return tri; }
};

/*
 * TEST MATRIX
 */

#include <stdio>

int cnt = 0;
void plot2csv(vector<point> P, vector< set<int> > adj) {
    char filename[100];
    int n = P.size();
    sprintf(filename, "graph%d.csv", cnt++);
    FILE * out = fopen(filename, "w");
    fprintf(out, "id,lat,lng\n");
    for(int i = 0; i < n; i++) {
        fprintf(out, "%d,%.2f,%.2f\n", i, P[i].x, P[i].y);
    }
    fprintf(out, "source,target\n");
    for(int u = 0; u < n; u++) {
        for(set<int>::iterator it = adj[u].begin(); it != adj[u].end(); it++) {
            fprintf(out, "%d,%d\n", u, *it);
        }
    }
    fclose(out);
}

typedef pair<int, int> ii;

bool isAllCollinear(vector<point> P) {
    int n = P.size(), id = 0;
    while(id < n && P[0] == P[id]) id++;
    if (id == n) return true;
    for(int i = 0; i < n; i++) {
        if (!collinear(P[0], P[id], P[i])) return false;
    }
    return true;
}

```

```

bool between(point p, point q, point r) {
    return collinear(p, q, r) && inner(p - q, r - q) < -EPS;
}

point lineIntersectSeg(point p, point q, point A, point B) {
    double c = cross(A-B, p-q);
    double a = cross(A, B);
    double b = cross(p, q);
    return ((p-q)*(a/c)) - ((A-B)*(b/c));
}

bool parallel(point a, point b) {
    return fabs(cross(a, b)) < EPS;
}

bool segIntersects(point a, point b, point p, point q) {
    if (parallel(a-b, p-q)) {
        return between(a, p, b) || between(a, q, b)
            || between(p, a, q) || between(p, b, q);
    }
    point i = lineIntersectSeg(a, b, p, q);
    return between(a, i, b) && between(p, i, q);
}

bool isTriangulation(vector<point> P, vector< set<int> > adj) {
    vector< pair<point, point> > seg;
    set<ii> edges, aux;
    int n = P.size();
    for(int u = 0; u < n; u++) {
        vector<int> g(adj[u].begin(), adj[u].end());
        for(int j = 0; j < (int)g.size(); j++) {
            int v = g[j];
            if (u < v) {
                seg.push_back(make_pair(P[u], P[v]));
                edges.insert(make_pair(u, v));
            }
        }
    }
    set<truple> tri = Delaunay::getTriangles();
    for(set<truple>::iterator it = tri.begin(); it != tri.end(); it++) {
        int u = it->a, v = it->b, w = it->c;
        if (!edges.count(ii(u, v))) {
            printf("error, not a triangulation\n");
            return false;
        }
        else aux.insert(ii(u, v));
        if (!edges.count(ii(v, w))) {
            printf("error, not a triangulation\n");
            return false;
        }
        else aux.insert(ii(v, w));
        if (!edges.count(ii(u, w))) {
            printf("error, not a triangulation\n");
            return false;
        }
        else aux.insert(ii(u, w));
    }
    if (!isAllCollinear(P) && aux.size() != edges.size()) {
        printf("error, not a triangulation\n");
        return false;
    }
    for(int u = 0; u < n; u++) {
        for(int v = 0; v < n; v++) {
            if (u >= v || edges.count(ii(u, v))) continue;
            bool found = false;
            for(int j = 0; !found && j < (int)seg.size(); j++) {
                point p = seg[j].first;
                point q = seg[j].second;
            }
        }
    }
}

```

```

        if (segIntersects(p, q, P[u], P[v])) found = true;
    }
    if (!found) {
        printf("error, not a triangulation\n");
        return false;
    }
}
return true;
}

bool isDelaunayTriangulation(vector<point> P, vector< set<int> > adj) {
    int n = P.size();
    if (!isTriangulation(P, adj)) return false;
    set<truple> tri = Delaunay::getTriangles();
    for(set<truple>::iterator it = tri.begin(); it != tri.end(); it++) {
        int u = it->a, v = it->b, w = it->c;
        circle c = circumcircle(P[u], P[v], P[w]);
        for(int i = 0; i < n; i++) {
            if (dist(c.c, P[i]) < c.r - 1e-4) {
                printf("error, point %d = (%.3f,%.3f) is inside circumcircle:\n", i, P[i].x, P[i].y);
                printf("%d = (%.3f,%.3f)\n", u, P[u].x, P[u].y);
                printf("%d = (%.3f,%.3f)\n", v, P[v].x, P[v].y);
                printf("%d = (%.3f,%.3f)\n", w, P[w].x, P[w].y);
                printf("center (%.3f,%.3f), radius %.3f, distance %.3f\n", c.c.x, c.c.y, c.r, dist(c.c, P[i]));
                return false;
            }
        }
    }
    return true;
}

bool inputTest() {
    int n;
    if(scanf("%d", &n) == EOF) return false;
    if (n <= 0) return false;
    vector<point> P(n);
    for(int i = 0; i < n; i++) {
        scanf("%lf %lf", &P[i].x, &P[i].y);
    }
    vector< set<int> > adj = Delaunay::compute(P);
    if(!isDelaunayTriangulation(P, adj)) return false;
    printf("input test passed\n");
    return true;
}

#include <ctime>
typedef pair<int, int> ii;

bool agnezCornerCase(int n, bool testtri) {
    printf("test Victor Agnez's corner case, ");
    vector<point> P;
    for(int i = 0; i < n; i++) {
        P.push_back(point(i / 2, i % 2));
    }
    clock_t last = clock();
    vector< set<int> > adj = Delaunay::compute(P);
    double t = (clock() - last)*1.0/CLOCKS_PER_SEC;
    printf("time %.3fs ", t);
    if(testtri && !isDelaunayTriangulation(P, adj)) {
        printf("failed\n");
        return false;
    }
    printf("ok\n");
    return true;
}

bool test(int lowtest, int hightest, bool testtri) {
    agnezCornerCase(hightest, testtri);
}

```

```

srand(time(NULL));
for(int suit = lowtest; suit <= hightest; suit++) {
    printf("test %d, ", suit);
    int n = suit;
    set<ii> inuse;
    while(inuse.size() < n) {
        inuse.insert(ii(rand()%MAXN, rand()%MAXN));
    }
    vector<point> P;
    for(set<ii>::iterator it = inuse.begin(); it != inuse.end(); it++) {
        P.push_back(point(it->first, it->second));
    }
    clock_t last = clock();
    vector< set<int> > adj = Delaunay::compute(P);
    double t = (clock() - last)*1.0/CLOCKS_PER_SEC;
    printf("time %.3fs ", t);
    if(testtri && !isDelaunayTriangulation(P, adj)) {
        printf("failed test %d\n", suit);
        return false;
    }
    printf("ok\n");
    //plot2csv(P, adj);
}
if (testtri) printf("all tests passed\n");
return true;
}

int main() {
    test(1, 100, true);
    return 0;
}

/*
 * http://wcipeg.com/problem/c3c08s2p6
 */
/*
int main() {
    int n;
    while(scanf("%d", &n) != EOF) {
        vector<point> P(n);
        for(int i = 0; i < n; i++) {
            scanf("%lf %lf", &P[i].x, &P[i].y);
        }
        Delaunay::compute(P);
        double ans = 0;
        set<triple> tri = Delaunay::getTriangles();
        for(set<triple>::iterator it = tri.begin(); it != tri.end(); it++) {
            int u = it->a, v = it->b, w = it->c;
            circle c = circumcircle(P[u], P[v], P[w]);
            if (c.r > ans) ans = c.r;
        }
        printf("%.8f\n", ans);
    }
    return 0;
}
*/

```

8.11 Giao của đa giác

Polygon intersection.

8.12 Minimum Enclosing Circle

Đường tròn bao nhỏ nhất.

8.13 Giao nửa mặt phẳng

Half-plane intersection.

Mã nguồn mẫu: Computational Geometry/halfplaneintersect.cpp

```
#include <cstdio>
#include <cmath>
#include <vector>
#include <deque>
#include <algorithm>
#define EPS 1e-9
#define INF 0x3f3f3f3f
using namespace std;

/*
 * Point 2D
 */

struct point {
    double x, y;
    point() { x = y = 0.0; }
    point(double _x, double _y) : x(_x), y(_y) {}
    double norm() {
        return hypot(x, y);
    }
    point normalized() {
        return point(x,y)*(1.0/norm());
    }
    double angle() { return atan2(y, x); }
    double polarAngle() {
        double a = atan2(y, x);
        return a < 0 ? a + 2*acos(-1.0) : a;
    }
    bool operator < (point other) const {
        if (fabs(x - other.x) > EPS) return x < other.x;
        else return y < other.y;
    }
    bool operator == (point other) const {
        return (fabs(x - other.x) < EPS && (fabs(y - other.y) < EPS));
    }
    point operator +(point other) const {
        return point(x + other.x, y + other.y);
    }
    point operator -(point other) const {
        return point(x - other.x, y - other.y);
    }
    point operator *(double k) const {
        return point(x*k, y*k);
    }
};

double inner(point p1, point p2) {
    return p1.x*p2.x + p1.y*p2.y;
}

double cross(point p1, point p2) {
    return p1.x*p2.y - p1.y*p2.x;
}

point rotate(point p, double rad) {
    return point(p.x * cos(rad) - p.y * sin(rad),
        p.x * sin(rad) + p.y * cos(rad));
}

point lineIntersectSeg(point p, point q, point A, point B) {
    double c = cross(A-B, p-q);
    double a = cross(A, B);
    double b = cross(p, q);
```

```

    return ((p-q)*(a/c)) - ((A-B)*(b/c));
}

bool parallel(point a, point b) {
    return fabs(cross(a, b)) < EPS;
}

/*
 * Polygon 2D
 */

typedef vector<point> polygon;

double signedArea(polygon &P) {
    double result = 0.0;
    int n = P.size();
    for (int i = 0; i < n; i++) {
        result += cross(P[i], P[(i+1)%n]);
    }
    return result / 2.0;
}

double area(polygon &P) {
    return fabs(signedArea(P));
}

int leftmostIndex(vector<point> &P) {
    int ans = 0;
    for(int i=1; i<(int)P.size(); i++) {
        if (P[i] < P[ans]) ans = i;
    }
    return ans;
}

polygon make_polygon(vector<point> P) {
    if (signedArea(P) < 0.0) reverse(P.begin(), P.end());
    int li = leftmostIndex(P);
    rotate(P.begin(), P.begin()+li, P.end());
    return P;
}

polygon cutPolygon(polygon &P, point a, point b) {
    vector<point> R;
    double left1, left2;
    int n = P.size();
    for (int i = 0; i < n; i++) {
        left1 = cross(b-a, P[i]-a);
        left2 = cross(b-a, P[(i+1)%n]-a);
        if (left1 > -EPS) R.push_back(P[i]);
        if (left1 * left2 < -EPS)
            R.push_back(lineIntersectSeg(P[i], P[(i+1)%n], a, b));
    }
    return make_polygon(R);
}

/*
 * Halfplane 2D
 */

typedef pair<point, point> halfplane;

point dir(halfplane h) {
    return h.second - h.first;
}

bool belongs(halfplane h, point a) {
    return cross(dir(h), a - h.first) > EPS;
}

```



```

bool hpcomp(halfplane ha, halfplane hb) {
    point a = dir(ha), b = dir(hb);
    if (parallel(a, b) && inner(a, b) > EPS)
        return cross(b, ha.first - hb.first) < -EPS;
    if (b.y*a.y > EPS) return cross(a, b) > EPS;
    else if (fabs(b.y) < EPS && b.x > EPS) return false;
    else if (fabs(a.y) < EPS && a.x > EPS) return true;
    else return b.y < a.y;
}

polygon intersect(vector<halfplane> H, double W = INF) {
    H.push_back(halfplane(point(-W,-W), point(W,-W)));
    H.push_back(halfplane(point(W,-W), point(W,W)));
    H.push_back(halfplane(point(W,W), point(-W,W)));
    H.push_back(halfplane(point(-W,W), point(-W,-W)));
    sort(H.begin(), H.end(), hpcomp);
    int i = 0;
    while(parallel(dir(H[0]), dir(H[i]))) i++;
    deque<point> P;
    deque<halfplane> S;
    S.push_back(H[i-1]);
    for(; i < (int)H.size(); i++) {
        while(!P.empty() && !belongs(H[i], P.back()))
            P.pop_back(), S.pop_back();
        point df = dir(S.front()), di = dir(H[i]);
        if (P.empty() && cross(df, di) < EPS)
            return polygon();
        P.push_back(lineIntersectSeg(H[i].first, H[i].second,
            S.back().first, S.back().second));
        S.push_back(H[i]);
    }
    while(!belongs(S.back(), P.front()) ||
        !belongs(S.front(), P.back())) {
        while(!belongs(S.back(), P.front()))
            P.pop_front(), S.pop_front();
        while(!belongs(S.front(), P.back()))
            P.pop_back(), S.pop_back();
    }
    P.push_back(lineIntersectSeg(S.front().first,
        S.front().second, S.back().first, S.back().second));
    return polygon(P.begin(), P.end());
}

/*
 * TEST MATRIX
 */

point rndpoint(int W) {
    int x = (rand()%(2*W)) - W;
    int y = (rand()%(2*W)) - W;
    return point(x, y);
}

/*bool test(int ntests, int n, int W, bool nonempty) {
    for(int test = 1; test <= ntests; test++) {
        vector<halfplane> H;
        for(int i = 0; i < n; i++) {
            halfplane h;
            do {
                h.first = rndpoint(W);
                h.second = rndpoint(W);
            } while(nonempty && !belongs(h, point(0, 0)));
            H.push_back(h);
        }
        polygon A1 = intersect(H, W);
        polygon A2;
        A2.push_back(point(-W,-W));
        A2.push_back(point(W,-W));
        A2.push_back(point(W,W));
    }
}

```

```

    A2.push_back(point(-W,W));
    for(int i = 0; i < n; i++) {
        A2 = cutPolygon(A2, H[i].first, H[i].second);
    }
    if (A1.size() != A2.size() || fabs(area(A1) - area(A2)) > EPS) {
        printf("failed on test %d:\n", test);
        printf("H:\n");
        for(int i = 0; i < n; i++)
            printf(" (%.0f,%.0f)-(%.0f,%.0f)\n", H[i].first.x, H[i].first.y, H[i].second.x, H[i].second.y);
        printf("A1 (%u):\n", A1.size());
        for(int i = 0; i < (int)A1.size(); i++) {
            printf(" (%.3f,%.3f)\n", A1[i].x, A1[i].y);
        }
        printf("A2 (%u):\n", A2.size());
        for(int i = 0; i < (int)A2.size(); i++) {
            printf(" (%.3f,%.3f)\n", A2[i].x, A2[i].y);
        }
        return false;
    }
}
printf("all tests passed\n");
return true;
}

int main() {
    test(10000, 100, 100, false);
    test(10000, 100, 100, true);
}*/

/*
 * ICPC Summer School 2019, Day 3, problem D
 */

/*point read() {
    double x, y;
    scanf("%lf %lf", &x, &y);
    return point(x, y);
}

int main() {
    int n;
    scanf("%d", &n);
    vector<halfplane> H;
    for(int i = 0; i < n; i++) {
        point a = read();
        point b = read();
        point c = read();
        H.push_back(halfplane(a, b));
        H.push_back(halfplane(b, c));
        H.push_back(halfplane(c, a));
    }
    polygon ans = intersect(H, 1000);
    printf("%.20f\n", area(ans));
    return 0;
}*/

/*
 * ICPC Summer School 2019, Day 2, problem K
 */

bool check(const polygon &I, const polygon &O, double r) {
    int n = O.size(), m = I.size();
    vector<halfplane> H;
    for(int i = 0; i < n; i++) {
        point v = (O[(i+1)%n] - O[i]).normalized();
        point ref = O[i] + rotate(v, acos(-1.0)/2.0)*r;
        double trans = -INF;
        for(int j = 0; j < m; j++) {
            double cur = cross(v, I[j] - ref);

```

```

        trans = max(trans, -cur);
    }
    point a = rotate(v, acos(-1.0)/2.0)*trans;
    point b = a + v;
    H.push_back(halfplane(a, b));
}
return !intersect(H, 10000).empty();
}

point read() {
    double x, y;
    scanf("%lf %lf", &x, &y);
    return point(x, y);
}

int main() {
    int n;
    polygon O, I;
    scanf("%d", &n);
    for(int i = 0; i < n; i++) I.push_back(read());
    scanf("%d", &n);
    for(int i = 0; i < n; i++) O.push_back(read());
    if (area(O) < area(I)) I.swap(O);
    double lo = 0;
    double hi = 1000;
    for(int it = 0; it < 100; it++) {
        double mid = (hi + lo) / 2;
        if (check(I, O, mid)) lo = mid;
        else hi = mid;
    }
    printf("%.20f\n", lo/2);
    return 0;
}

```

8.14 Điểm 3D

Point/vector 3D.

Mã nguồn mẫu: Computational Geometry/point3d.cpp

```

#include <cstdio>
#include <cmath>
#define EPS 1e-9

struct point {
    double x, y, z;
    point() { x = y = z = 0.0; }
    point(double _x, double _y, double _z) : x(_x), y(_y), z(_z) {}
    double norm() {
        return sqrt(x*x + y*y + z*z);
    }
    point normalized() {
        return point(x,y,z)*(1.0/norm());
    }
    bool operator < (point other) const {
        if (fabs(x - other.x) > EPS) return x < other.x;
        else if (fabs(y - other.y) > EPS) return y < other.y;
        else return z < other.z;
    }
    bool operator == (point other) const {
        return (fabs(x - other.x) < EPS && fabs(y - other.y) < EPS && fabs(z - other.z) < EPS);
    }
    point operator +(point other) const{
        return point(x + other.x, y + other.y, z + other.z);
    }
    point operator -(point other) const{
        return point(x - other.x, y - other.y, z - other.z);
    }
}

```

```

    }
    point operator *(double k) const{
        return point(x*k, y*k, z*k);
    }
};

double dist(point p1, point p2) {
    return (p1-p2).norm();
}

double inner(point p1, point p2) {
    return p1.x*p2.x + p1.y*p2.y + p1.z*p2.z;
}

point cross(point p1, point p2) {
    point ans;
    ans.x = p1.y*p2.z - p1.z*p2.y;
    ans.y = p1.z*p2.x - p1.x*p2.z;
    ans.z = p1.x*p2.y - p1.y*p2.x;
    return ans;
}

bool collinear(point p, point q, point r) {
    return cross(p-q, r-p).norm() < EPS;
}

double angle(point a, point o, point b) {
    return acos(inner(a-o, b-o) / (dist(o,a)*dist(o,b)));
}

point proj(point u, point v) {
    return v*(inner(u,v)/inner(v,v));
}

```

8.15 Tam giác 3D

Triangle 3D.

Mã nguồn mẫu: Computational Geometry/triangle3d.cpp

```

struct triangle{
    point a, b, c;
    triangle() { a = b = c = point(); }
    triangle(point _a, point _b, point _c) : a(_a), b(_b), c(_c) {}
    double perimeter() { return dist(a,b) + dist(b,c) + dist(c,a); }
    double semiPerimeter() { return perimeter()/2.0; }
    double area() {
        double s = semiPerimeter(), ab = dist(a,b),
            bc = dist(b,c), ca = dist(c,a);
        return sqrt(s*(s-ab)*(s-bc)*(s-ca));
    }
    double rInCircle() {
        return area()/semiPerimeter();
    }
    double rCircumCircle() {
        return dist(a,b)*dist(b,c)*dist(c,a)/(4.0*area());
    }
    point normalVector() {
        return cross(y-x, z-x).normalized();
    }
    int isInside(point p) {
        point n = normalVector();
        double u = proj(cross(b-a,p-a), n).normalized()*proj(cross(b-a,c-a), n).normalized();
        double v = proj(cross(c-b,p-b), n).normalized()*proj(cross(c-b,a-b), n).normalized();
        double w = proj(cross(a-c,p-c), n).normalized()*proj(cross(a-c,b-c), n).normalized();
        if (u > 0.0 && v > 0.0 && w > 0.0) return 0;
        else if (u < 0.0 || v < 0.0 || w < 0.0) return 2;
    }
};

```

```

        else return 1;
    } //0 = inside/ 1 = border/ 2 = outside
    int isProjInside(point p) {
        return isInside(p + proj(a-p, normalVector()));
    } //0 = inside/ 1 = border/ 2 = outside
};

double rInCircle(point a, point b, point c) {
    return triangle(a,b,c).rInCircle();
}

double rCircumCircle(point a, point b, point c) {
    return triangle(a,b,c).rCircumCircle();
}

int isProjInsideTriangle(point a, point b, point c, point p) {
    return triangle(a,b,c).isProjInside(p);
} //0 = inside/ 1 = border/ 2 = outside

```

8.16 Đường thẳng 3D

Line 3D.

Mã nguồn mẫu: Computational Geometry/line3d.cpp

```

#include <cstdio>
#include <cmath>
#define EPS 1e-9

struct point {
    double x, y, z;
    point() { x = y = z = 0.0; }
    point(double _x, double _y, double _z) : x(_x), y(_y), z(_z) {}
    double norm() {
        return hypot(x, y, z);
    }
    point normalized() {
        return point(x, y, z)*(1.0 / norm());
    }
    bool operator < (point other) const {
        if (fabs(x - other.x) > EPS) return x < other.x;
        else if (fabs(y - other.y) > EPS) return y < other.y;
        else return z < other.z;
    }
    bool operator == (point other) const {
        return (fabs(x - other.x) < EPS && fabs(y - other.y) < EPS && fabs(z - other.z) < EPS);
    }
    point operator +(point other) const {
        return point(x + other.x, y + other.y, z + other.z);
    }
    point operator -(point other) const {
        return point(x - other.x, y - other.y, z - other.z);
    }
    point operator *(double k) const {
        return point(x*k, y*k, z*k);
    }
};

double dist(point p1, point p2) {
    return (p1 - p2).norm();
}

double inner(point p1, point p2) {
    return p1.x*p2.x + p1.y*p2.y + p1.z*p2.z;
}

point cross(point p1, point p2) {

```

```

    point ans;
    ans.x = p1.y*p2.z - p1.z*p2.y;
    ans.y = p1.z*p2.x - p1.x*p2.z;
    ans.z = p1.x*p2.y - p1.y*p2.x;
    return ans;
}

bool collinear(point p, point q, point r) {
    return cross(p - q, r - p).norm() < EPS;
}

double angle(point a, point o, point b) {
    return acos(inner(a - o, b - o) / (dist(o, a)*dist(o, b)));
}

point proj(point u, point v) {
    return v*(inner(u, v) / inner(v, v));
}

struct line {
    point r;
    point v;
    line(point _r, point _v) {
        v = _v; r = _r;
    }
    bool operator == (line other) const {
        return fabs(cross(r - other.r, v).norm()) < EPS && fabs(cross(r - other.r, other.v).norm()) < EPS;
    }
};

point distVector(line l, point p) {
    point dr = p - l.r;
    return dr - proj(dr, l.v);
}

point distVectorBasePoint(line l, point p) {
    return proj(p - l.r, l.v) + l.r;
}

point distVectorEndPoint(line l, point p) {
    return p;
}

point distVector(line a, line b) {
    point dr = b.r - a.r;
    point n = cross(a.v, b.v);
    if (n.norm() < EPS) {
        return dr - proj(dr, a.v);
    }
    else return proj(dr, n);
}

double dist(line a, line b) {
    return distVector(a, b).norm();
}

point distVectorBasePoint(line a, line b) {
    if (cross(a.v, b.v).norm() < EPS) return a.r;
    point d = distVector(a, b);
    double lambda;
    if (fabs(b.v.x*a.v.y - a.v.x*b.v.y) > EPS)
        lambda = (b.v.x*(b.r.y - a.r.y - d.y) - b.v.y*(b.r.x - a.r.x - d.x)) / (b.v.x*a.v.y - a.v.x*b.v.y);
    else if (fabs(b.v.x*a.v.z - a.v.x*b.v.z) > EPS)
        lambda = (b.v.x*(b.r.z - a.r.z - d.z) - b.v.z*(b.r.x - a.r.x - d.x)) / (b.v.x*a.v.z - a.v.x*b.v.z);
    else if (fabs(b.v.z*a.v.y - a.v.z*b.v.y) > EPS)
        lambda = (b.v.z*(b.r.y - a.r.y - d.y) - b.v.y*(b.r.z - a.r.z - d.z)) / (b.v.z*a.v.y - a.v.z*b.v.y);
    return a.r + (a.v*lambda);
}

```

```
point distVectorEndPoint(line a, line b) {
    return distVectorBasePoint(a, b) + distVector(a, b);
}
```

8.17 Hình học giải tích

Công thức đường thẳng, mặt phẳng, khoảng cách và góc trong hình học giải tích.

8.18 Tọa độ cực, trụ và cầu

Đổi hệ tọa độ cực/trụ/cầu.

8.19 Giải tích vector 2D

Gradient, divergence, curl và các công thức vector 2D.

8.20 Giải tích vector 3D

Công thức vector 3D.

8.21 Vấn đề độ chính xác, tổng ổn định và công thức Bhaskara

Tránh trừ hai số gần nhau, dùng tổng ổn định và nghiệm phương trình bậc hai ổn định.

Mã nguồn mẫu: Computational Geometry/precision.cpp

```
#include <cmath>
#include <algorithm>
using namespace std;
#define EPS 1e-9

/*
 * Auxiliary geometry objects
 */

struct point {
    double x, y;
    point() { x = y = 0.0; }
    point(double _x, double _y) : x(_x), y(_y) {}
    double norm() {
        return hypot(x, y);
    }
    point normalized() {
        return point(x,y)*(1.0/norm());
    }
    double angle() { return atan2(y, x); }
    bool operator < (point other) const {
        if (fabs(x - other.x) > EPS) return x < other.x;
        else return y < other.y;
    }
    bool operator == (point other) const {
        return (fabs(x - other.x) < EPS && (fabs(y - other.y) < EPS));
    }
    point operator +(point other) const{
        return point(x + other.x, y + other.y);
    }
    point operator -(point other) const{
        return point(x - other.x, y - other.y);
    }
    point operator *(double k) const{
```

```

        return point(x*k, y*k);
    }
};

double dist(point p1, point p2) {
    return hypot(p1.x - p2.x, p1.y - p2.y);
}

double cross(point p1, point p2) {
    return p1.x*p2.y - p1.y*p2.x;
}

double inner(point p1, point p2) {
    return p1.x*p2.x + p1.y*p2.y;
}

point rotate(point p, double rad) {
    return point(p.x * cos(rad) - p.y * sin(rad),
        p.x * sin(rad) + p.y * cos(rad));
}

struct circle{
    point c;
    double r;
    circle() { c = point(); r = 0; }
    circle(point _c, double _r) : c(_c), r(_r) {}
    double chord(double rad) { return 2*r*sin(rad/2.0); }
    bool intersects(circle other) {
        return dist(c, other.c) < r + other.r;
    }
    bool contains(point p) { return dist(c, p) <= r + EPS; }
    pair<point, point> getTangentPoint(point p) {
        double d1 = dist(p, c), theta = asin(r/d1);
        point p1 = rotate(c-p, -theta);
        point p2 = rotate(c-p, theta);
        p1 = p1*(sqrt(d1*d1-r*r)/d1)+p;
        p2 = p2*(sqrt(d1*d1-r*r)/d1)+p;
        return make_pair(p1,p2);
    }
};

/*
 * Precision algorithms
 */
#include <vector>

struct StableSum {
    int cnt = 0;
    vector<double> v, pref(1, 0);
    void operator += (double a) { // a >= 0
        for (int s = ++cnt; s % 2 == 0; s >= 1) {
            a += v.back();
            v.pop_back(), pref.pop_back();
        }
        v.push_back(a);
        pref.push_back(pref.back() + a);
    }
    double val() { return pref.back(); }
};

bool circleCircle(circle c1, circle c2, pair<point, point> & out) {
    double d = dist(c2.c, c1.c);
    double co = (d*d + c1.r*c1.r - c2.r*c2.r)/(2*d*c1.r);
    if (fabs(co) > 1.0) return false;
    double alpha = acos(co);
    point rad = (c2.c-c1.c)*(1.0/d*c1.r);
    out = {c1.c + rotate(rad, -alpha), c1.c + rotate(rad, alpha)};
    return true;
}

```



```

int quadRoots(double a, double b, double c, pair<double, double> &out) {
    if (fabs(a) < EPS) return 0;
    double delta = b*b - 4*a*c;
    if (delta < 0) return 0;
    double sum = (b >= 0) ? -b-sqrt(delta) : -b+sqrt(delta);
    out = {sum/(2*a), fabs(sum) > EPS ? 0 : (2*c)/sum};
    return 1 + (delta > 0);
}

/*
 * TEST MATRIX
 */
#include <cstdio>

int main() {
    double a, b, c;
    while(scanf("%lf %lf %lf", &a, &b, &c) != EOF) {
        pair<double, double> ans;
        int d = quadRoots(a, b, c, ans);
        printf("%d solutions for %.3fx^2 + %.3fx + %.3f:", d, a, b, c);
        if (d > 0) printf(" x = %.3f", ans.first);
        if (d > 1) printf(" x = %.3f", ans.second);
        printf("\n");
    }
    return 0;
}

```

9 Linh tinh

9.1 Thuật toán Mo

Mo's algorithm.

Mã nguồn mẫu: Miscellaneous/mo.cpp

```
/*
 * Mo's Algorithm  $O(n^{3/2})$ 
 */

#include <algorithm>
using namespace std;
#define SQ 500

int curans; // current ans
void add(int v) { /* add value */ }
void rem(int v) { /* remove value */ }

struct query {
    int i, l, r, ans;
};

bool qcomp(query a, query b) {
    return a.l/SQ == b.l/SQ ? a.r < b.r : a.l < b.l;
}
bool icomp(query a, query b) { return a.i < b.i; }

void mo(int v[], query qs[], int Q) {
    int l = 0, r = -1;
    sort(qs, qs + Q, qcomp);
    for (int i = 0; i < Q; i++) {
        query & q = qs[i];
        while (r < q.r) add(v[++r]);
        while (r > q.r) rem(v[r--]);
        while (l < q.l) rem(v[l++]);
        while (l > q.l) add(v[--l]);
        q.ans = curans;
    }
    sort(qs, qs + Q, icomp);
}

/*
 * COMPILATION TEST
 */

int main() {}
```

9.2 Lịch Gregory

Cấu trúc ngày tháng.

Mã nguồn mẫu: Miscellaneous/date.cpp

```
/*
 * Date data structure
 */

int mnt[13] = {0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334, 365};

struct Date {
    int d, m, y;
    Date() : d(1), m(1), y(1) {}
    Date(int d, int m, int y) : d(d), m(m), y(y) {}
    Date(int days) : d(1), m(1), y(1) { advance(days); }
```

```

    bool leap() { return (y%4 == 0 && y%100) || (y%400 == 0); }
    int mdays() { return mnt[m] - mnt[m-1] + (m == 2)*leap(); }
    int ydays() { return 365 + leap(); }
    int count() { // dist to 01/01/01
        return (d-1) + mnt[m-1] + (m > 2)*leap() +
            365*(y-1) + (y-1)/4 - (y-1)/100 + (y-1)/400;
    }
    int weekday() { return (count()+1) % 7; }
    void advance(int days) {
        days += count();
        d = m = 1, y = 1 + days/366;
        days -= count();
        while(days >= ydays()) days -= ydays(), y++;
        while(days >= mdays()) days -= mdays(), m++;
        d += days;
    }
};

/*
 * TEST MATRIX
 */

#include <cstdio>
#include <cassert>
#include <algorithm>
using namespace std;

Date advance1(Date date) {
    int d = date.d;
    int m = date.m;
    int y = date.y;
    int w = date.weekday();
    d++;
    int mcnt = mnt[m] - mnt[m-1];
    if (m == 2 && date.leap()) mcnt++;
    if (d > mcnt) {
        m++; d = 1;
    }
    if (m == 13) {
        y++; m = 1;
    }
    date.advance(1);
    assert(d == date.d);
    assert(m == date.m);
    assert(y == date.y);
    w = (w+1)%7;
    assert(w == date.weekday());
    return date;
}

void test() {
    Date date;
    for(int i = 0; i < 1000000; i++) {
        Date aux(i);
        assert(aux.d == date.d);
        assert(aux.m == date.m);
        assert(aux.y == date.y);
        date = advance1(date);
    }
    for(int t = 1; t <= 1000; t++) {
        int d1 = rand()%100000;
        int d2 = rand()%100000;
        if (d1 > d2) swap(d1, d2);
        Date date1(d1), date2(d2);
        Date aux = date1;
        for(int i = 0; i < d2-d1; i++) aux = advance1(aux);
        date1.advance(d2 - d1);
        assert(date1.d == date2.d);
        assert(date1.m == date2.m);
    }
}

```

```

        assert(date1.y == date2.y);
        assert(aux.d == date2.d);
        assert(aux.m == date2.m);
        assert(aux.y == date2.y);
    }
}

int main() {
    test();
    int d, m, y;
    while (scanf("%d %d %d", &d, &m, &y) != EOF) {
        Date date(d, m, y);
        int lastw;
        for(int i = 0; i < 10; i++) {
            int w = date.weekday();
            printf("%02d/%02d/%04d %s\n", date.d, date.m, date.y,
                w == 0 ? "Domingo" :
                w == 1 ? "Segunda-feira" :
                w == 2 ? "Terca-feira" :
                w == 3 ? "Quarta-feira" :
                w == 4 ? "Quinta-feira" :
                w == 5 ? "Sexta-feira" :
                w == 6 ? "Sabado" : "undefined"
            );
            if (i > 0) assert(w == (lastw+1)%7);
            lastw = w;
            date.advance(1);
        }
    }
    return 0;
}

```

9.3 Simplex

Quy hoạch tuyến tính bằng simplex.

Mã nguồn mẫu: Miscellaneous/simplex.cpp

```

#include <bits/stdc++.h>

//#include <ext/pb_ds/assoc_container.hpp> // Common file
//#include <ext/pb_ds/tree_policy.hpp> // Including tree_order_statistics_node_update

#define pb(x) push_back(x)
#define sz(x) x.size()

#define eps 1e-9
#define Max 300005
#define INF 2000000000
#define mod 1000000007
#define pii pair<int,int>
#define _mp make_pair

typedef long long LL;
typedef unsigned long long ULL;

using namespace std;

#define MAXN 109
#define MAXM 10009
#define EPS 1e-9

#define MINIMIZE -1
#define MAXIMIZE +1
#define LESSEQ -1
#define EQUAL 0
#define GREATER 1

```

```

#define INFEASIBLE -1
#define UNBOUNDED 999

/****
Problem example: https://www.hackerrank.com/contests/zenhacks/challenges/circles-1

Expected complexity  $O(N*N*M)$ 
Worst case exponential complexity
1. Simplex Algorithm for Linear Programming
2. Maximize or minimize  $f_0*x_0 + f_1*x_1 + f_2*x_2 + \dots + f_{n-1}*x_{n-1}$  subject to some constraints
3. Constraints are of the form,  $c_0x_0 + c_1x_1 + c_2x_2 + \dots + c_{n-1}x_{n-1}$  ( $\leq$  or  $\geq$  or  $=$ )  $lim$ 
4.  $m$  is the number of constraints indexed from 1 to  $m$ , and  $n$  is the number of variables indexed from 0 to  $n-1$ 
5.  $ar[0]$  contains the objective function  $f$ , and  $ar[1]$  to  $ar[m]$  contains the constraints,  $ar[i][n] = lim_i$ 
6. It is assumed that all variables satisfies non-negativity constraint, i.e,  $x_i \geq 0$ 
7. If non-negativity constraint is not desired for a variable  $x$ , replace each occurrence
   by difference of two new variables  $r_1$  and  $r_2$  (where  $r_1 \geq 0$  and  $r_2 \geq 0$ , handled automatically by simplex).
   That is, replace every  $x$  by  $r_1 - r_2$  (Number of variables increases by one,  $-x$ ,  $+r_1$ ,  $+r_2$ )
8.  $solution\_flag = INFEASIBLE$  if no solution is possible and  $UNBOUNDED$  if no finite solution is possible
9. Returns the maximum/minimum value of the linear equation satisfying all constraints otherwise
10. After successful completion,  $val[]$  contains the values of  $x_0, x_1 \dots x_n$  for the optimal value returned

*** If  $ABS(X) \leq M$  in constraints, Replace with  $X \leq M$  and  $-X \leq M$ 

*** Fractional LP:

max/min
   $3x_1 + 2x_2 + 4x_3 + 6$ 
  -----
   $3x_1 + 3x_2 + 2x_3 + 5$ 

s.t.  $2x_1 + 3x_2 + 5x_3 \geq 23$ 
      $3x_2 + 5x_2 + 4x_3 \leq 30$ 
      $x_1, x_2, x_3 \geq 0$ 

Replace with:

max/min
   $3y_1 + 2y_2 + 4y_3 + 6t$ 

s.t.  $3y_1 + 3y_2 + 2y_3 + 5t = 1$ 
      $2y_1 + 3y_2 + 5t - 23t \geq 0$ 
      $3y_1 + 5y_2 + 4y_3 - 30t \leq 0$ 
      $y_1, y_2, y_3, t \geq 0$ 

****/

namespace lp {
double val[MAXN], ar[MAXM][MAXN];
int m, n, solution_flag, minmax_flag, basis[MAXM], index[MAXN];

void init(int nvars, double* f, int flag) {
  solution_flag = 0;
  ar[0][nvars] = 0.0;
  m = 0, n = nvars, minmax_flag = flag;
  for (int i = 0; i < n; i++)
    ar[0][i] = f[i] * minmax_flag;
}

void add_constraint(double* C, double lim, int cmp) {
  m++, cmp *= -1;
  if (cmp == EQUAL) {
    for (int i = 0; i < n; i++) ar[m][i] = C[i];
    ar[m+1][n] = lim;
    for (int i = 0; i < n; i++) ar[m][i] = -C[i];
    ar[m][n] = -lim;
  }
  else {
    for (int i = 0; i < n; i++) ar[m][i] = C[i] * cmp;
    ar[m][n] = lim * cmp;
  }
}

```

```

    }
}

void init() {
    for (int i = 0; i <= m; i++) basis[i] = -i;
    for (int j = 0; j <= n; j++)
        ar[0][j] = -ar[0][j], index[j] = j, val[j] = 0;
}

inline void pivot(int m, int n, int a, int b) {
    for (int i = 0; i <= m; i++)
        for (int j = 0; i != a && j <= n; j++)
            if (j != b) ar[i][j] -= (ar[i][b] * ar[a][j]) / ar[a][b];
    for (int j = 0; j <= n; j++)
        if (j != b) ar[a][j] /= ar[a][b];
    for (int i = 0; i <= m; i++)
        if (i != a) ar[i][b] = -ar[i][b] / ar[a][b];
    ar[a][b] = 1.0 / ar[a][b];
    swap(basis[a], index[b]);
}

inline double solve() {
    init();
    int i, j, k, l;
    while (true) {
        for (i = 1, k = 1; i <= m; i++)
            if ((ar[i][n] < ar[k][n]) || (ar[i][n] == ar[k][n] && basis[i] < basis[k] && (rand() & 1))) k = i;
        if (ar[k][n] >= -EPS) break;
        for (j = 0, l = 0; j < n; j++)
            if ((ar[k][j] < (ar[k][l] - EPS)) || (ar[k][j] < (ar[k][l] - EPS) && index[i] < index[j] && (rand() & 1))) l = j;
        if (ar[k][l] >= -EPS) {
            solution_flag = INFEASIBLE;
            return -1.0;
        }
        pivot(m, n, k, l);
    }
    while (true) {
        for (j = 0, l = 0; j < n; j++)
            if ((ar[0][j] < ar[0][l]) || (ar[0][j] == ar[0][l] && index[j] < index[l] && (rand() & 1))) l = j;
        if (ar[0][l] > -EPS) break;
        for (i = 1, k = 0; i <= m; i++)
            if (ar[i][l] > EPS && (!k || ar[i][n] / ar[i][l] < ar[k][n] / ar[k][l] - EPS || (ar[i][n] / ar[i][l] < ar[k][n] / ar[k][l] + EPS && basis[i] < basis[k]))) k = i;
        if (ar[k][l] <= EPS) {
            solution_flag = UNBOUNDED;
            return -999.0;
        }
        pivot(m, n, k, l);
    }
    for (i = 1; i <= m; i++)
        if (basis[i] >= 0) val[basis[i]] = ar[i][n];
    solution_flag = 1;
    return (ar[0][n] * minmax_flag);
}

double obj[109], cons[109];
int x[109], y[109];

int main()
{
    srand(time(NULL));
    int n;
    cin >> n;
    for (int i = 0; i < n; i++) {
        obj[i] = 1;
    }
    obj[n] = 0;

```

```

lp::init(n,obj,MAXIMIZE);

for (int i = 0; i < n; i++) {
    scanf("%d %d", &x[i], &y[i]);
    for (int j = 0; j < i; j++) {
        for (int k = 0; k < n; k++) {
            cons[k] = 0;
        }
        cons[i] = 1;
        cons[j] = 1;
        lp::add_constraint(cons, hypot(x[i]-x[j], y[i]-y[j]),LESSEQ);
    }
}

double ret = lp::solve();

printf("%.8lf\n", ret*2*acos(-1.0));
return 0;
}

```

9.4 Duyệt polyomino

Sinh/duyet polyomino.

Mã nguồn mẫu: Miscellaneous/polyomino_it.cpp

```

#define MAXN 59
#include <cstdio>

/*
 * Polyomino iteration
 */

int N, M;
int num[MAXN][MAXN], qi[MAXN*MAXN], qj[MAXN*MAXN];
int field[MAXN][MAXN], par[MAXN][MAXN];
int di[4] = {0, 1, 0, -1};
int dj[4] = {-1, 0, 1, 0};
int cnt;

void assign(int i, int j, int k) {
    qi[k] = i; qj[k] = j; num[i][j] = k;
}

#define valid(i, j) (i >= 0 && i < N && j >= 0 && j < M)

int iterate(int k, int h) {
    if (h == 0) return 0;
    int i = qi[k], j = qj[k], ni, nj;
    for(int d = 0; d < 4; d++) {
        ni = i + di[d]; nj = j + dj[d];
        if (!valid(ni, nj)) continue;
        if (num[ni][nj] == 0) {
            par[ni][nj] = k;
            assign(ni, nj, ++cnt);
        }
    }
    int ans = 0, cur;
    for(int t = k+1; t <= cnt; t++) {
        cur = iterate(t, h-1);
        if (cur > ans) ans = cur;
    }
    for(int d = 0; d < 4; d++) {
        ni = i + di[d]; nj = j + dj[d];
        if (!valid(ni, nj)) continue;
        if (par[ni][nj] == k) {
            par[ni][nj] = 0; cnt--;
        }
    }
}

```

```

        assign(ni, nj, 0);
    }
}
return ans + field[i][j];
}

/*
 * URI 1712
 */

#include <stdio>
#include <cstring>

int main() {
    int K;
    while(scanf("%d %d", &N, &K)!=EOF) {
        M = N;
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < M; j++) {
                scanf("%d", &field[i][j]);
            }
        }
        int ans = 0;
        memset(&num, 0, sizeof num);
        memset(&par, 0, sizeof par);
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < M; j++) {
                cnt = 0;
                assign(i, j, ++cnt);
                int cur = iterate(1, K);
                if (cur > ans) ans = cur;
                num[i][j] = -1;
            }
        }
        printf("%d\n", ans);
    }
    return 0;
}

```

9.5 Ma phương bậc lẻ

Sinh ma trận $n \times n$, n lẻ, có tổng hàng/cột/chéo bằng nhau.

Mã nguồn mẫu: Miscellaneous/magicsquare.cpp

```

#include <stdio>
#include <cstring>
#define MAXN 1009

/*
 * Magic Square
 */

int mat[MAXN][MAXN], n;    //0-indexed

void magicsquare() {
    int i=n-1, j=n/2;
    memset(&mat, 0, sizeof mat);
    for(int k = 1; k <= n*n; k++) {
        mat[i][j] = k;
        if (mat[(i+1)%n][(j+1)%n] > 0) {
            i = (i-1+n)%n;
        }
        else {
            i = (i+1)%n;
            j = (j+1)%n;
        }
    }
}

```



```

    }
}

/*
 * TEST MATRIX
 */

int main() {
    scanf("%d", &n);
    magicsquare();
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            if (j > 0) printf(" ");
            printf("%d", mat[i][j]);
        }
        printf("\n");
    }
    return 0;
}

```

9.6 Chu trình trong dãy: thuật toán Floyd

Cycle finding.

Mã nguồn mẫu: Math/floyd.cpp

```

#include <iostream>
using namespace std;

typedef pair<int, int> ii;

int f(int x) {
    return (x+1)%5;
}

ii floydCycleFinding(int x0) {
    // 1st part: finding k*start, hare's speed is 2x tortoise's
    int tortoise = f(x0), hare = f(f(x0)); // f(x0) is the node next to x0
    while (tortoise != hare) { tortoise = f(tortoise); hare = f(f(hare)); }
    // 2nd part: finding start, hare and tortoise move at the same speed
    int start = 0; hare = x0;
    while (tortoise != hare) { tortoise = f(tortoise); hare = f(hare); start++; }
    // 3rd part: finding period, hare moves, tortoise stays
    int period = 1; hare = f(tortoise);
    while (tortoise != hare) { hare = f(hare); period++; }
    return ii(start, period);
}

```

9.7 Biểu thức ngoặc sang Ba Lan

Chuyển biểu thức dạng ngoặc sang dạng Ba Lan.

Mã nguồn mẫu: Miscellaneous/paren2polish.cpp

```

#include <cstdio>
#include <map>
#include <stack>
using namespace std;

/*
 * Parenthetic to polish expression conversion
 */

inline bool isOp(char c) {
    return c=='+' || c=='-' || c=='*' || c=='/' || c=='^';
}

```

```

}

inline bool isCarac(char c) {
    return (c>='a' && c<='z') || (c>='A' && c<='Z') || (c>='0' && c<='9');
}

int paren2polish(char* paren, char* polish) {
    map<char, int> prec;
    prec['('] = 0;
    prec['+'] = prec['-'] = 1;
    prec['*'] = prec['/'] = 2;
    prec['^'] = 3;
    int len = 0;
    stack<char> op;
    for (int i = 0; paren[i]; i++) {
        if (isOp(paren[i])) {
            while (!op.empty() && prec[op.top()] >= prec[paren[i]]) {
                polish[len++] = op.top(); op.pop();
            }
            op.push(paren[i]);
        }
        else if (paren[i]=='(') op.push('(');
        else if (paren[i]==')') {
            for (; op.top()!='('; op.pop())
                polish[len++] = op.top();
            op.pop();
        }
        else if (isCarac(paren[i]))
            polish[len++] = paren[i];
    }
    for(; !op.empty(); op.pop())
        polish[len++] = op.top();
    polish[len] = 0;
    return len;
}

/*
 * TEST MATRIX
 */

int main() {
    int N, len;
    char polish[400], paren[400];
    scanf("%d", &N);
    for (int j=0; j<N; j++) {
        scanf(" %s", paren);
        paren2polish(paren, polish);
        printf("%s\n", polish);
    }
    return 0;
}

```

9.8 Bài toán Josephus

Vòng n người, bỏ mỗi k người: $f(n,k)=(f(n-1,k)+k)\%n$, $f(1,k)=0$.

9.9 Bài toán histogram

Hình chữ nhật lớn nhất trong histogram trong $O(n)$.

Mã nguồn mẫu: `Miscellaneous/histogram.cpp`

```

#include <cstdio>
#include <stack>
using namespace std;

```

```

typedef long long ll;

#define MAXN 100009

/*
 * Solution to the histogram problem in O(n)
 */

ll histogram(ll vet[], int n) {
    stack<ll> s;
    ll ans = 0, tp, cur;
    int i = 0;
    while(i < n || !s.empty()) {
        if (i < n && (s.empty() || vet[s.top()] <= vet[i])) s.push(i++);
        else {
            tp = s.top();
            s.pop();
            cur = vet[tp] * (s.empty() ? i : i - s.top() - 1);
            if (ans < cur) ans = cur;
        }
    }
    return ans;
}

/*
 * URI 1683
 */

ll vet[MAXN];

int main() {
    int n;
    while (scanf("%d", &n), n) {
        for (int i = 0; i < n; i++) {
            scanf("%lld", vet + i);
        }
        printf("%lld\n", histogram(vet, n));
    }
    return 0;
}

```

9.10 Bài toán hôn nhân ổn định

Gale-Shapley $O(n^2)$.

Mã nguồn mẫu: Miscellaneous/stablemarriage.cpp

```

#include <stdio>
#include <cstring>
#define MAXN 1009

int m, n; //número de homens e de mulheres
int p[MAXN];
int L[MAXN][MAXN]; //L[i]: preferências do homem i (melhor primeiro)
int R[MAXN][MAXN]; //R[j][i]: nota dada pela mulher j ao homem i
int R2L[MAXN]; //R2L[i]==j: mulher i casada com homem j
int L2R[MAXN]; //L2R[i]==j: homem i casado com mulher j

void stableMarriage() {
    memset(R2L, -1, sizeof(R2L));
    memset(p, 0, sizeof(p));
    for(int i = 0, wom, hubby; i < m; i++) {
        for (int man = i; man >= 0; ) {
            while(true) {
                wom = L[man][p[man]++];
                if( R2L[wom] < 0 || R[wom][man] > R[wom][R2L[wom]] ) break;
            }
        }
    }
}

```

```

        hubby = R2L[wom];
        R2L[L2R[man] = wom] = man;
        man = hubby;
    }
}

int main() {
    int t, ma, wo, rank;
    scanf("%d", &t);
    while(t--) {
        scanf("%d", &n);
        m = n;
        for(int i=0; i<n; i++) {
            scanf("%d", &wo);
            rank = n;
            for(int j=0; j<n; j++, rank--) {
                scanf("%d", &ma);
                R[wo-1][ma-1] = rank;
            }
        }
        for(int i=0; i<n; i++) {
            scanf("%d", &ma);
            for(int j=0; j<n; j++) {
                scanf("%d", &L[ma-1][j]);
                L[ma-1][j]--;
            }
        }
        stableMarriage();
        for(int i=0; i<n; i++) {
            printf("%d %d\n", i+1, L2R[i]+1);
        }
    }
    return 0;
}

```

9.11 Mã Huffman

Chi phí cây Huffman.

Mã nguồn mẫu: Trees/huffman.cpp

```

/*
 * Huffman Cost - O(nlogn)
 */

#include <queue>
using namespace std;

typedef long long ll;

ll huffman(ll* a, int n) {
    ll ans = 0, u, v;
    priority_queue<ll> pq;
    for(int i=0; i<n; i++) pq.push(-a[i]);
    while(pq.size() > 1) {
        u = -pq.top(); pq.pop();
        v = -pq.top(); pq.pop();
        pq.push(-u-v);
        ans += u + v;
    }
    return ans;
}

/*
 * Codeforces 101128C
 */

```

```

#include <stdio>
#define MAXN 100009

int main() {
    int T, N;
    ll a[MAXN];
    scanf("%d", &T);
    while(T-->0) {
        scanf("%d", &N);
        for(int i=0; i<N; i++) scanf("%I64d", &a[i]);
        printf("%I64d\n", huffman(a, N));
    }
    return 0;
}

```

9.12 Bài toán quân mã

Tìm đường Hamilton của quân mã bằng backtracking và heuristic Warnsdorff.

9.13 Giao matroid

Tập độc lập lớn nhất trong giao của hai matroid; ví dụ cây khung lớn nhất với màu cạnh khác nhau.

Mã nguồn mẫu: Graphs/colorfulspanningtree.cpp

```

#include <vector>
#include <set>
#include <queue>
using namespace std;
#define MAXN 109
#define MAXM 10009

typedef pair<int,int> ii;
vector<ii> adjList[MAXN];
vector<int> Q, A[MAXM];
ii edges[MAXM];
vector<bool> T, Fcolors;
set<int> F;
int c[MAXM], N, M, C;

class UnionFind {
private:
    vector<int> parent, rank;
public:
    UnionFind(int N) {
        rank.assign(N+1, 0);
        parent.assign(N+1, 0);
        for (int i = 0; i <= N; i++) parent[i] = i;
    }
    int find(int i) {
        while(i != parent[i]) i = parent[i];
        return i;
    }
    bool isSameSet(int i, int j) {
        return find(i) == find(j);
    }
    void unionSet (int i, int j) {
        if (isSameSet(i, j)) return;
        int x = find(i), y = find(j);
        if (rank[x] > rank[y]) parent[y] = x;
        else {
            parent[x] = y;
            if (rank[x] == rank[y]) rank[y]++;
        }
    }
}

```

```

};

vector<int> findCycle(int s, int t) {
    vector<int> ans; vector<ii> prev;
    queue<int> q;
    prev.assign(N, ii(-1, -1));
    q.push(s); prev[s] = ii(s, -1);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (u == t) break;
        for (int i = 0; i < (int)adjList[u].size(); i++) {
            int v = adjList[u][i].first;
            if (prev[v].first == -1) {
                prev[v] = ii(u, adjList[u][i].second);
                q.push(v);
            }
        }
    }
    while (t != s) {
        ans.push_back(prev[t].second);
        t = prev[t].first;
    }
    return ans;
}

void start() {
    T.assign(M, false); Q.clear();
    for (int i=0; i<N; i++) adjList[i].clear();
    for (int i=0; i<M; i++) A[i].clear();
}

void greedyAugmentation(UnionFind &UF) {
    int e, u, v;
    Fcolors.assign(C, false);
    for (set<int>::iterator it = F.begin(); it != F.end(); it++) {
        e = *it;
        u = edges[e].first, v = edges[e].second;
        UF.unionSet(u, v);
        adjList[u].push_back(ii(v, e));
        adjList[v].push_back(ii(u, e));
        Fcolors[c[e]] = true;
    }
    for (int i = 0; i < M; i++) {
        u = edges[i].first, v = edges[i].second;
        if (!UF.isSameSet(u, v) && !Fcolors[c[i]]) {
            UF.unionSet(u, v);
            adjList[u].push_back(ii(v, i));
            adjList[v].push_back(ii(u, i));
            Fcolors[c[i]] = true;
            F.insert(i);
        }
    }
}

void digraphConstruction(UnionFind &UF) {
    vector<int> cycle;
    for (int i = 0; i < M; i++) {
        if (F.count(i)) continue;
        if (!Fcolors[c[i]]) Q.push_back(i);
        else for (set<int>::iterator it = F.begin(); it != F.end(); it++) {
            if (c[*it] == c[i]) A[*it].push_back(i);
        }
        if (!UF.isSameSet(edges[i].first, edges[i].second)) T[i] = true;
        else {
            cycle = findCycle(edges[i].first, edges[i].second);
            for (int j = 0; j < (int)cycle.size(); j++)
                A[i].push_back(cycle[j]);
        }
    }
}

```

```

}

bool augmentation() {
    queue<int> q;
    vector<int> prev;
    prev.assign(M, -1);
    for (int i = 0; i < (int)Q.size(); i++) {
        q.push(Q[i]);
        prev[Q[i]] = Q[i];
    }
    int u, v, found = -1;
    while (!q.empty()) {
        u = q.front(); q.pop();
        if (T[u]) {
            found = u; break;
        }
        for (int i = 0; i < (int)A[u].size(); i++) {
            v = A[u][i];
            if (prev[v] == -1) {
                prev[v] = u; q.push(v);
            }
        }
    }
    if (found == -1) return false;
    u = found;
    while (true) {
        if (F.count(u)) F.erase(u);
        else F.insert(u);
        if (u == prev[u]) break;
        u = prev[u];
    }
    return true;
}

void buildSpanningTree() {
    F.clear();
    while (true) {
        UnionFind UF(M);
        start();
        greedyAugmentation(UF);
        digraphConstruction(UF);
        if (!augmentation()) break;
    }
}

/*
 * URI 2128
 */

#include <cstdio>

int main() {
    int u,v,c1,t=1;
    while (scanf("%d%d",&N,&M,&C)!=EOF) {
        for (int i=0;i<M;i++) {
            scanf("%d%d",&u,&v,&c1);
            u--, v--, c1--;
            edges[i] = ii(u, v);
            c[i] = c1;
        }
        buildSpanningTree();
        if (F.size()!=N-1) {
            printf("Instancia %d\nnao\n\n",t++);
        }else{
            printf("Instancia %d\nsim\n\n",t++);
        }
    }
}

```